

AX를 만들다

김상기

AX를 만들다
일하는 방식을 코드로 다시 짠 6개월의 기록
목차
들어가며
프롤로그. 딸은 사람에게, 나는 AI와
SNS 타임라인에 흘러온 한 장의 그림
그래서, 이 책이 택한 자리

당신은 지금, 몇 단계인가
PART 1. AX란 무엇인가 — 첫 삽은 어디에 뜨나
1장. 에이전트를 조직에 들이는 법
DX는 인프라를 바꾸고, AX는 일을 바꾼다
그런데, 어느 순간 통제가 안 되기 시작했다
리더는 여기까지만 읽어도 된다
에이전트에게 신분증을 발급하자
길을 깔되, 막는 벽이 아니라 자연스럽게 걸리는 레일로
당신의 첫 에이전트 표준
마무리
2장. AX는 아직 만들어지는 중이다
설계도가 없었다, 그런데 만들고 나서야 알았다
조직은 흐르는 강이다 — 변화는 가끔 오는 손님이 아니다
슬라이드로 그린 전략과 만들면서 떠오른 전략
도구는 깔았다고 끝이 아니다, 쓰이면서 비로소 무언가가 된다
정의가 안 잡힌 건 미성숙의 신호가 아니다 — 모든 기술이 거치는 길이다
그래서, 대부분의 조직이 아직 실험 단계에 있다
리더는 여기까지만 읽어도 된다
슬라이드보다 시스템이 먼저였다
완성되지 않아도 시작하는 법
마무리
PART 2. 일의 재정의 — 한 사람이 팀이 되는 법
3장. 한 사람이 팀이 되다
거대한 한 명보다, 역할을 나눈 여럿이 낫다
혼자 다 시키려다, 품질이 무너졌다
리더는 여기까지만 읽어도 된다
작가, 검수자, 발행자 — 세 자리로 나누다
모델은 갈아끼우는 부품이다
긴 작업은 잘게 쪼개라
LLM 출력은 깨진다 — 방어 코드가 본체다
검수는 작가에게서 떼어내, 따로 앉혀라
내 일을 1인 파이프라인으로
마무리
4장. 내 일을 task로 쪼개기
“이 정도는 에이전트가 해도 되겠지”가 부른 일
’직무’가 아니라 ’task’로 쪼개야 하는 이유
리더는 여기까지만 읽어도 된다
같은 동작, 두 시스템 — 어떻게 선을 그었나

내 일을 task로 쪼개는 워크시트
마무리
PART 3. 사람을 다시 설계하다
5장. 성과는 토큰이 아니라 검증이다
메트릭은 조작된다
토큰을 많이 쓴 사람이 잘한 사람일까
리더는 여기까지만 읽어도 된다
모델은 부품이고, 프롬프트는 코드다
잘림이냐, 비용이냐 — 응답 길이를 손으로 더듬다
프롬프트를 코드 밖으로 꺼내고, 버전을 매기다
AI를 대화상대가 아니라 편집점으로
성과 지표를 다시 짜기
마무리
6장. 직무가 아니라 task로 사람을 배분하다
직무라는 덩어리를 풀면 무엇이 보이나
매칭이 문제가 아니었다, 분류가 문제였다
리더는 여기까지만 읽어도 된다
분류 체계라는 뼈대를 먼저 세우자
가장 손이 많이 간 곳은 '정제 레이어'였다
데모를 실제로 띄우는 것의 무게
우리 일의 taxonomy 만들기
마무리
7장. 지식이 흐르게 하라
흐르지 않는 지식은 자산이 아니다
머릿속에만 있으면, 그건 없는 것이다
리더는 여기까지만 읽어도 된다
사람을 가리키는 이름표는 사번이어야 한다
정책은 다이얼이지, 스위치가 아니다
시스템은 보이기 시작할 때 비로소 시스템이 된다
레일은 의외로 곁에 있는 평범한 도구에서 시작된다
지식이 흐르는 장 만들기
마무리
PART 4. AX를 운영한다는 것 — 레일과 신뢰
8장. 세 가지 부채와 검증으로의 전환
분명 빨라졌는데, 왜 더 느려졌을까
빠른 게 아니라 빚을 지고 있었다
그렇다면, 무게중심을 옮기자
리더는 여기까지만 읽어도 된다

흩어져 있던 시행착오가 하나로 묶이다
검증이 선택이 아니라 필수인 이유
우리 조직의 부채 점검
마무리
9장. 통제가 아니라 레일을 깐다
길에도 여러 종류가 있다
AI가 사람 승인 없이 데이터를 고치려 했다
리더는 여기까지만 읽어도 된다
정책 변경을 다루는 세 단계
우리 조직의 레일 깔기
다시, 트램으로
마무리
에필로그. 다시, 카페에서
다섯 칸을 한 칸씩 밟아 와서
동네에 트램이 다니기 시작했다
그래서, 당신은 지금 몇 단계인가
마무리 — 같은 차에 타고 집으로
참고문헌
1. 레일·가드레일·길 (PART 4의 핵심 프레임)
2. “AX는 아직 만들어지는 중이다” — 조직·전략·기술 이론 (2장)
3. 1인칭 빌더 회고 — 톤·서사·교훈의 쌍둥이 (전편 톤, 3·7·8장)
4. 멀티에이전트·검증 — 역할 분리와 LLM-as-judge (3·8장)
5. 일의 task 분해 — 인간·AI 우열과 협업의 결 (4장)
6. AI 코칭·성과 — 증폭 vs 감시 (5장)
7. 사내 인력 시장 — 직무가 아니라 task로 (6장)
8. HR 지식 거버넌스 — bi-temporal·policy-as-code·승인 레일 (9장)
9. 경쟁·인접 서적 (차별화 좌표)
10. AX 여정 모델·세 가지 부채·검증으로의 전환 (전편 척추)

AX를 만들다

일하는 방식을 코드로 다시 짠 6개월의 기록

저자: 김상기 **버전:** 1.2.1 **발행일:** 2026-11-01 (초판 2026-06-20) **판 이력:** v1.2.1: 문체 점검 — AI 문체 패턴 260곳 정리, 내용·수치 변경 없음

목차

- 들어가며
- 프롤로그. 딸은 사람에게, 나는 AI와

PART 1. AX란 무엇인가 — 첫 삽은 어디에 뜨나 - 1장. 에이전트를 조직에 들이는 법 - 2장. AX는 아직 만들어지는 중이다

PART 2. 일의 재정의 — 한 사람이 팀이 되는 법 - 3장. 한 사람이 팀이 되다 - 4장. 내 일을 task로 쪼개기

PART 3. 사람을 다시 설계하다 - 5장. 성과는 토큰이 아니라 검증이다 - 6장. 직무가 아니라 task로 사람을 배분하다 - 7장. 지식이 흐르게 하라

PART 4. AX를 운영한다는 것 — 레일과 신뢰 - 8장. 세 가지 부채와 검증으로의 전환 - 9장. 통제가 아니라 레일을 깐다

- 에필로그. 다시, 카페에서
- 참고문헌

들어가며

이 책에는 별도의 긴 서문을 두지 않았다. 바로 뒤에 이어지는 프롤로그가 그 역할을 하기 때문이다. 다만, 책장을 넘기기 전에 세 가지만 일러두고 싶다.

누가 읽으면 좋은가. 이 책은 AX(AI Transformation)를 *말이 아니라 실제로 만들어내야* 하는 사람들을 위해 썼다. 조직과 일하는 방식을 바꿔야 하는 자리에 앉은 리더와 실무자, 그리고 그 변화를 직접 코드로 짓는 빌더. "AX 해야 한다"는 말은 충분히 들었는데 *그래서 대체 뭘, 어떻게 만들라는 건지*가 막막한 사람이라면, 이 책이 그 막막함의 옆자리에 앉아 줄 것이다.

어떻게 읽으면 좋은가. 모든 장은 같은 구조로 짜여 있다. 먼저 "여기가 어디인지" 좌표를 잡고(이론), 다음으로 내가 무엇을 왜 겪었고 어떻게 코드로 풀었는지를 1인칭으로 풀고(구현), 마지막으로 그것을 당신의 현장으로 옮길 절차와 체크리스트를 정리한다(프레임). 구현 디테일이 깊어지는 대목마다 **"리더는 여기까지만 읽어도 된다"**는 표식을 박아 두었다. 그 표식을 만나면, 비개발 독자는 그 절을 건너뛰고 다음 절로 가도 장의 핵심을 하나도 놓치지 않는다. 빌더라면 끝까지 함께 가면 좋겠다. 자기 현장에 바로 옮길 절차가 거기 있다.

이 책의 약속 — 그리고 용어 하나. 이 책은 정답이나 AX의 완성된 청사진을 건네지 않는다. 만들면서 깨지고 고친 과정을 그대로 보여 준다. 그 태도는 뒤에서 세 가지 약속으로 하나씩 풀어 두었다. 특히 용어 하나를 미리 정해 두고 시작한다. 이 책은 '거버넌스(governance)'라는 딱딱한 통제의 단어 대신 **레일·가드레일·길**이라는 말을 쓴다. 막는 벽이 아니라 깔아 주는 길 — 그 차이가 왜 중요한지는 본문에서 차차 만나게 될 것이다. 또한 본문에 등장하는 회사·시스템·도구·인물은 모두 일반화해 익명으로 옮겼다. 차용한 개념과 도식의 원자료 출처는 책 말미의 참고문헌에 정직하게 정리해 두었다.

자, 그럼 카페로 가 보자.

프롤로그. 딸은 사람에게, 나는 AI와

> **들어가기 전에**
>
> 같은 토요일 아침, 딸은 사람에게 글쓰기를 배우러 가고 나는 카페에서 AI와 글을 쓴다. 이 사소한 대비가 이 책 전체의 질문이다. 둘 다 틀리지 않았다면, 우리는 지금 막 시작된 이 시대를 *어떻게* 통과하고 있는 걸까.
>
> 앞으로 펼쳐질 이야기는 AX를 정의하는 책이 아니다. 6개월간 직접 코드로 빚어 본 1인칭 구현기다. 환호에서 정체로, 신남에서 의구심으로, 그리고 이륙으로 — 누군가 그려 둔 한 장의 지도 위에 내 발자국을 겹쳐 그린 기록이라고 해 두자. 지금 내 위치는 환호와 정체의 딱 그 경계, 이제 막 구덩이로 내려가기 직전이다.

토요일 오전이면 첫째 딸을 글쓰기·논술 학원에 내려준다. 차에서 내리는 딸의 가방을 한 번 고쳐 메 주고, "끝나면 전화해" 하고 손을 흔든다. 그리고 나는 늘 가던 길 모퉁이 카페로 향한다.

가는 길목은 요즘 한창 어수선하다. 동네 큰길을 몇 달째 파헤쳐 트램 레일을 깔고 있다. 그날도 인부들이 길게 쇠 레일을 잇고 있었고, 한쪽에선 사람도 태우지 않은 빈 전차 한 대가 천천히 시범 운행을 하고 있었다. 신호등 자리를 옮기고 차선을 다시 긋는 공사도 같이 벌어지는 모양이었다. 나는 그 풍경을 별생각 없이 지나쳤다. 솔직히 그땐 몰랐다. 그 어수선한 공사가 이 책 한 권을 통째로 떠받치는 비유가 될 줄은.

카페로 들어선다. 창가에서 한 칸 안쪽, 콘센트가 있는 그 자리. 따뜻한 커피 한 잔을 옆에 두고 맥북을 연다.

그 자리에서 내가 하는 일은, 글을 쓰는 것이다. 정확히는 — AI와 함께 글을 쓴다.

가만히 생각하면 좀 묘한 장면이다. 같은 시각, 딸은 사람에게 글쓰기를 배우러 가 있다. 선생님이 아이의 문장을 한 줄씩 짚어 주고, 아이는 "왜 이렇게 쓰면 안 돼요?" 하고 물을 것이다. 그리고 길 건너 카페에서, 아빠는 AI에게 초안을 던지고 "이 문단은 너무 늘어져, 다시 줄여 보자"라고 말하고 있다. 아이는 머리를 싸매며 한 문장을 붙들고, 나는 한 문단을 몇 초 만에 다섯 가지 버전으로 받아 본다.

처음 이 대비를 의식했을 때는 솔직히 마음이 좀 복잡했다. 내가 아이에게 시키는 일과 내가 하는 일이 이렇게 다른데, 이게 모순은 아닐까. 아이가 사람에게 배우는 동안 나는 기계와 일하는 게, 어쩐지 한쪽이 한쪽을 배신하는 것처럼 찜찜하게 느껴지기도 했다.



그림 P-1. 카페 장면 일러스트

딸은 왜 아직 사람에게 글쓰기를 배울까? 나는 왜 AI와 쓰고 있을까?

둘 중 하나가 틀린 걸까. 솔직히 말하면, 나는 둘 다 맞다고 생각한다. 아이가 사람에게서 배우는 건 문장이 아니라 *생각하는 법*이다. 내가 AI와 쓰는 건 생각을 멈추기 위해서가 아니라, 더 빨리 더 멀리 생각을 밀어붙이기 위해서다. 그런데 둘 다 맞다는 데서 나는 자꾸 멈칫하게 된다.

둘 다 틀리지 않았다면 — 우리는 지금 막 시작된 이 시대를 *어떻게* 통과하고 있는 걸까?

SNS 타임라인에 흘러온 한 장의 그림



그림 P-3. 딸은 사람에게, 나는 AI와

그날도 초안을 한 챕터 끝내 놓고, 커피가 식기를 기다리며 무심코 SNS 타임라인을 내렸다. 링크드인이었다. 누군가 올려둔 글 하나가 눈에 걸렸다. 거창한 선언도, 화려한 사례도 아니었다. 그냥 한 장의 그림이었다.

거기엔 *"AX는 한 번에 오지 않는다. 단계를 거친다"*는 취지의 곡선이 그려져 있었다. 처음엔 다들 들뜨고(전사에 AI를 깔고, 교육을 돌리고, 전담팀을 만든다), 그다음엔 이상하게 조용해진다(계정은 줬는데 아무도 안 쓴다). 그러다 누군가 사내 시스템에 AI를 연결하기 시작하면서 다시 신이 나고, 곧 "어라, 생각만큼 안 빨라지는데?" 하는 의구심이 찾아오고, 마지막엔 이 구덩이를 넘느냐 마느냐로 갈린다는 이야기였다.

그림을 보는데 등이 조금 서늘했다. 거기 그려진 그 곡선이, 내가 지난 반년 동안 코드로 더듬어 온 길과 너무 닮아 있었기 때문이다.

나도 들떴던 적이 있다. 도구를 손에 쥐자 못 만들 게 없어 보였다. 주말이고 새벽이고 가리지 않고 뭔가를 만들었다.

그러다 막막했던 적이 있다. 분명 쓸 만한 걸 만들어 놓았는데 아무도 안 쓰는 걸 지켜보던 시간. "계정은 줬는데 왜 안 쓰지?"라는 질문 앞에서 한참을 멍하니 앉아 있었다. 그러다 사내 데이터에 에이전트를 연결하던 순간의 짜릿함이 왔고 — 정말 그 며칠은 신이 났다 — 곧이어 찜찜함이 따라붙었다. AI가 뱉은 그 그럴싸한 결과를, 정말 믿어도 되나. 그럴듯한데 자세히 보면 틀린 결과를 붙잡고 되돌리느라, 혼자 했을 때보다 오히려 더 느려진 것 같던 며칠. 난감했다.

누가 곡선을 그려서 미리 보여 준 게 아니다. 나는 그 다섯 칸을 *몸으로* 한 칸씩 지나오고 있었던 것이다. 그래서 그 한 장의 그림이 그렇게 마음에 박혔다. 내가 겪은 걸 누군가도 똑같이 겪고 있고, 그게 나 혼자만의 헛발질이 아니라 이 시대를 지나는 모두의 공통된 경로라는 — 묘하게 위로가 되는 발견이었다.

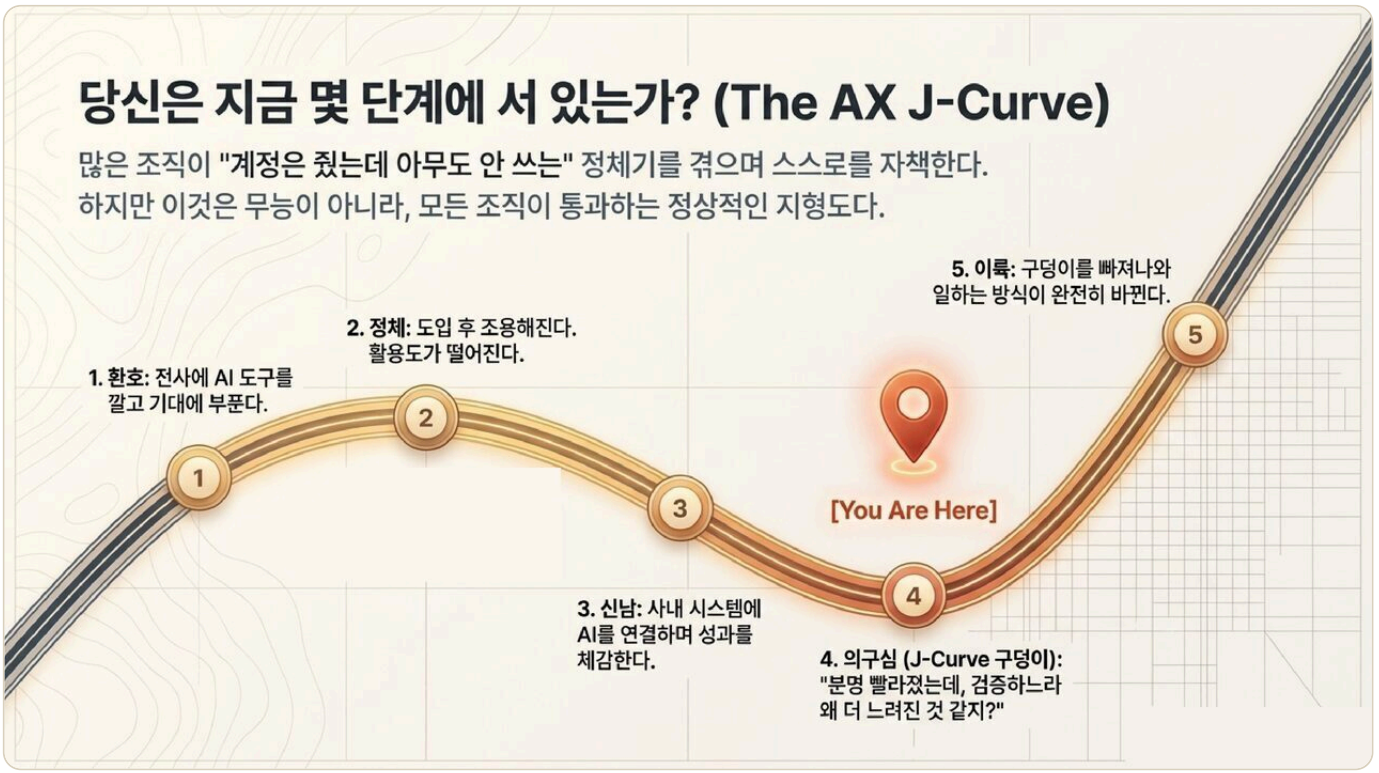


그림 P-2. AX 5단계 J-curve 지도

한 가지는 짚고 넘어가야겠다. 이 그림은 어느 한 사람의 발명품이 아니다. 비슷한 관찰이 국내외 여기저기서 동시에 떠오르고 있다. 여러 곳의 성숙도 조사, 현장 빌더들의 회고, 컨설팅 리포트가 표현만 달리해서 같은 곡선을 가리킨다. 그래서 나는 이 지도를 특정 인물의 이론으로 인용하지 않기로 했다. 여러 자료를 내 방식으로 종합해 다시 그린 한 장의 지도, 말하자면 우리 모두의 공통 지형도로 이 책 안에 둔다. 원자료의 출처는 책 말미에 따로 밝혀 두었다.

그래서, 이 책이 택한 자리

이 그림 한 장이 내 안에서 한 일은 분명했다. 흩어져 있던 반년의 시행착오에 처음으로 *좌표*가 생긴 것이다. "아, 그때 그 막막함은 정체 단계였구나. 그 짜릿함은 신남 단계였고, 그 찜찜함은 의구심 단계였구나."

그렇다면 이 책은 무엇이 되어야 할까?

요즘 시중엔 AX(AI Transformation)를 다룬 글이 차고 넘친다. 대부분 "AX란 무엇인가"를 정의하려 든다. "AX는 단순한 기술 도입이 아니라 일과 조직의 재설계다." 맞는 말이다. 그런데 그 문장을 백 번 읽어도 막상 월요일 아침 책상 앞에 앉으면 막막하긴 마찬가지다. *그래서 대체 뭘, 어떻게 만들라는 건데?*

사실 이 책의 1차 독자는, AX를 *말이 아니라 실제로 만들어내야* 하는 사람들이다. 조직과 일하는 방식을 바꿔야 하는 자리에 앉은 리더와 실무자, 그리고 그 변화를 직접 코드로 짓는 빌더. 당신이 둘 중 어느 쪽이든, 아마 회의실에서 "AX 해야 한다"는 말은 충분히 들었을 것이다. 문제는 그다음이다. 회의가 끝나고 자리로 돌아와 빈 화면을 마주했을 때, 당신을 막아서는 건 거대 담론이 아니라 "그래서 *첫 줄*을 어디에 쓰지?"라는, 어이없을 만큼 구체적인 막막함이다. 나는 그 막막함을 너무 잘 안다. 반년 내내 그것과 싸웠으니까.

나는 이 책을 바로 그 막막함에 답하는 책으로 쓰고 싶다. 정의가 아니라 *구현*으로.

컨설턴트가 "AX는 일과 조직의 재설계다"라고 말한다. 나는 그 재설계를 슬라이드가 아니라 코드로 만들고 있었다. 에이전트를 조직에 안전하게 들이는 표준을 짜고, 한 사람이 팀처럼 일하는 파이프라인을 돌리고, 무엇을 잘했다고 볼지 성과를 다시 정의하고, 사람을 직무라는 큰 덩어리가 아니라 task 단위로 재배분하고, 머릿속에만 있던 지식이 조직 차원에서 흐르게 하는 장을 만들고, 그 모든 것이 폭주하지 않도록 길을 깔았다.

물론 다 잘됐을 리 없다. 잘된 것도 있고, 보기 좋게 엎어진 것도 있다. 응답이 자꾸 중간에 잘려 설정값을 손으로 수십 번 바꿔 가며 씨름한 날. AI가 사람 승인도 없이 데이터를 곧장 고치려 드는 걸 보고 아찔했던 날. 비용이 새어 식은땀을 흘린 밤. 이 책은 그 6개월의 1인칭 기록이다. 미화하지 않은 쪽으로.

그 모든 시행착오를 굳이 다 적어 두려는 데에는 이유가 있다. 잘된 결과만 추려 보여 주면 그건 또 하나의 매끈한 슬라이드가 될 뿐이기 때문이다. 정작 도움이 되는 건 "여기서 나도 엎어졌고, 이렇게 다시 일어섰다"는 자국이다. 그러니 이 책에서 나는 나의 헛발질을 숨기지 않으려 한다.

그러니 미리 양해를 구해 둘 게 있다. 이 책은 정답을 파는 책이 아니다. AX의 완성된 청사진을 건네는 책도 아니다. 솔직히 말하면, 나는 아직도 AX가 무엇인지 한 문장으로 깔끔하게 정의하지 못한다. 그런데 이제는 그게 부끄럽지 않다. AX는 누군가 다 정의해 놓고 우리가 받아오는 물건이 아니라, 지금 우리 손에서 *매일 빚어지고 있는 무엇*이기 때문이다. 비행기를 띄워 놓고 날면서 동시에 만드는 일을, 나는 무능으로 보지 않는다.

세 가지만 약속하고 시작하자.

첫째, **정의가 아니라 구현으로.** "그래야 한다"는 당위 대신 "이렇게 만들어 봤다"는 실물을, 그리고 그 과정에서 깨진 것들을 정직하게 보여 주겠다.

둘째는 단어 하나에 관한 약속이다. **거버넌스가 아니라 레일로.** AX를 운영한다는 게 사람을 통제하고 막아세우는 일처럼 들릴 때가 많다. 그런데 내가 6개월 동안 깨달은 건 정반대였다. 그건 자연스럽게 올라타기만 하면 안전한 방향으로 흐르게 되는 *길*을 까는 일에 가까웠다. 그래서 나는 '거버넌스'라는 딱딱한 단어 대신 '레일'이라는 말을 쓰기로 했다. 막아서는 벽이 아니라, 미리 깔아 두는 길.

셋째, 그러니까 이건 강의가 아니라 이야기다. 논문도 매뉴얼도 아니다. 한 사람이 반년 동안 통과한 길을 옆자리에서 커피 마시며 들려주는, 딱 그 정도 거리의 이야기다. 그러니 가르치려 들지 않겠다. 다만 같이 가 보자고 손을 내밀겠다.

당신은 지금, 몇 단계인가

자, 그 한 장의 지도를 다시 펼쳐 보자. [그림 P-2]의 곡선 위 어딘가에 당신의 조직도 분명 한 점으로 찍혀 있을 것이다.

혹시 모두가 들떠서 도구만 잔뜩 깔아 둔 ①환호의 단계인가. 아니면 계정은 나눠 줬는데 아무도 안 쓰는, 그 조용하고 답답한 ②정체의 한복판인가. 누군가 사내 시스템에 AI를 연결하며 신이 나기 시작한 ③신남인가. "분명 빨라질 줄 알았는데 왜 더 느려졌지?" 하는 ④의구심의 구덩이에 막 발을 들였는가. 혹은 이미 그 구덩이를 빠져나와 ⑤이륙을 더듬고 있는가.

어느 단계든 괜찮다. 뒤처진 게 아니다. 나도 그 모든 칸을 한 칸씩 밟아 왔다. 솔직히 어떤 칸은 두 번 밟았다. 중요한 건 지금 내가 어디 서 있는지를 아는 것, 그리고 다음 한 발을 어디에 디딜지 정하는 것뿐이다.

특히 ④의구심의 구덩이에 막 빠진 사람에게 한마디 보태고 싶다. "분명 AI를 붙였는데 왜 더 느려졌지?" 하는 그 찜찜함은 당신이 뭔가를 잘못해서가 아니다. 그건 거의 모두가 지나는 통과의례에 가깝다. 곡선이 구덩이 모양인 데에는 이유가 있다. 생산은 빨라지는데 *검증*이 따라오지 못하면, 그럴싸하지만 틀린 결과를 가려내느라 전체는 오히려 느려진다. 이 책의 절반쯤은 사실 그 구덩이의 정체가 무엇이고, 어떻게 하면 거기서 다시 올라올 수 있는지에 관한 이야기다. 미리 말해 두자면, 해법은 더 빨리 생산하는 쪽보다 무게를 검증 쪽으로 옮기는 쪽에 있었다. 그것도 사람의 의지에 기대는 대신, 검증을 아예 시스템에 — 그러니까 레일에 — 새겨 넣는 방식으로.

딸의 글쓰기 수업은 곧 끝날 것이다. 전화가 오면 나는 맥북을 덮고 아이를 데리러 갈 것이다. 사람에게 배운 아이와, AI와 쓴 아빠가 같은 차에 타고 집으로 돌아가는 — 그 평범하고 다정한 공존의 풍경 속에서, 우리가 통과하는 이 시대의 답이 천천히 빚어지고 있다고 나는 믿는다.

자, 그 빚어감의 한복판으로 함께 들어가 보자. 당신의 첫 삽을, 당신의 첫 레일을 어디에 둘지 — 이 책을 덮을 즈음엔 분명 그 자리가 보일 것이다.

PART 1. AX란 무엇인가 — 첫 삽은 어디에 뜨나

> AX를 한 문장으로 정의하지 못한 채로도, 첫 삽은 떠야 한다. PART 1은 에이전트를 조직에 안전하게 들이는 첫 표준(1장)을 깔고, "설계도 없이 시작한 게 정상"이라는 자각(2장)으로 정체의 구덩이를 빠져나온다.
>
> *J-curve 위치: ②정체("계정은 줬는데 안 쓴다") → ③신남 진입.*

1장. 에이전트를 조직에 들이는 법

> **이 장의 길잡이**
>
> 이 장이 답하려는 질문은 하나다. AX가 뭔지 한 문장으로 정의하지도 못했는데, 첫 삽은 대체 어디에 떠야 하나?
>
> 가는 길에 거쳐 갈 생각의 줄기는 이렇다. AX는 도구를 까는 일이 아니라 일하는 방식을 다시 짜는 일이라는 것. 그런데 에이전트가 늘어나는 순간 "누가 만든 무슨 에이전트가 어디서 도는지" 아무도 모르는 통제 불능이 먼저 닥친다는 것. 그리고 그 막막함의 첫 출구가, 거창한 정의가 아니라 에이전트마다 신분증 한 장을 붙이고 그게 자연스럽게 걸리는 길(레일)을 까는 일이라는 것.
>
> J-curve로 치면 우리는 지금 바닥, 정체 구간에 있다. "계정은 다 줬는데 아무도 안 쓴다"의 그 막막함 속에서 첫 삽의 자리를 더듬는 중이다.

AX가 뭐냐고 누가 물으면, 솔직히 나는 아직도 한 문장으로 답하기가 어렵다. 컨설턴트는 "일과 조직의 재설계"라고 말하고, 어떤 임원은 "전사 AI 전환"이라고 부르고, 옆 팀은 "그냥 챗봇 깐 거 아니냐"고 한다. 다 맞는 말 같으면서도, 막상 "그래서 내일 아침에 뭘 만들면 되는데?"라는 질문 앞에 서면 모두가 조용해진다.

이상하지 않은가. 우리는 정의도 못 내린 무언가를 향해 이미 달려가고 있다. 계정은 다 나눠줬고, 워크숍도 했고, 결재 라인엔 "AX 추진"이라는 단어가 박혔다. 그런데 정작 자리에 앉아 일을 시작하려고 하면, 첫 삽을 어디에 떠야 할지가 보이지 않는다. 이 막막함은 게으름이나 무능 탓이 아니고, 시작점이 원래 흐릿해서 생긴다.

흐릿한 채로도 할 수 있는 일은 있다. 나는 AX가 무엇인지 정의하지 못한 채로 첫 시스템을 만들기 시작했고, 오히려 만들면서 그게 무엇인지 조금씩 알게 됐다. 정의를 기다리는 대신 일단 구현으로 들어가고, 통제하려 애쓰는 대신 레일을 까는 쪽으로, 그 첫 삽의 자리를 함께 더듬어보자.

DX는 인프라를 바꾸고, AX는 일을 바꾼다

먼저 우리가 지금 어디쯤 서 있는지 좌표부터 잡아보자. 지난 십수 년 동안 우리를 따라다닌 단어는 DX, 즉 디지털 전환이었다. DX가 무엇을 한 일이었는지 한 번 떠올려보자. 종이 결재를 전자 결재로 바꾸고, 사내 서버를 클라우드로 옮기고, 엑셀로 굴리던 업무를 사내 시스템에 태웠다. 한마디로 **인프라와 프로세스를 현대화하는 일**이었다. 도구를 더 좋은 도구로 갈아끼우는 작업이라고 생각하면 크게 틀리지 않는다.

그런데 AX는 결이 다르다. 여러 컨설팅·연구 기관이 입을 모아 하는 말이 있다. AX의 본질은 기술을 도입하는 게 아니라 **일과 조직 자체를 다시 설계하는 것**이라는 이야기다.[1] 이게 무슨 뜻일까? DX가 "기존의 일을 더 빠른 도구로 하게 만드는" 것이었다면, AX는 "그 일을 누가, 어떻게 하느냐 자체를 다시 묻는" 것이다. 사람이 하던 판단의 일부를 에이전트가 떠맡고, 한 사람이 하던 일을 여러 에이전트가 나눠 하고, 그러면서 역할과 책임의 경계선이 통째로 흔들린다.

조금 더 구체적으로 그려보자. DX 시절에 우리가 클라우드를 도입했다고 해서, "회계 담당자"라는 직무가 사라지거나 회계라는 일 자체가 다시 정의되지는 않았다. 도구가 바뀌었을 뿐 일의 모양은 그대로였다. 하지만 에이전트가 들어오면 이야기가 달라진다. 회계 담당자가 하던 분개(分介)의 절반을 에이전트가 처리하기 시작하면, 그 사람의 일은 "직접 분개하기"에서 "에이전트가 한 분개를 검증하기"로 바뀐다. 일의 내용이 통째로 옮겨가는 것이다. 그러면 그 사람을 어떻게 평가할지, 무엇을 잘했다고 봐야 할지, 옆 사람과 어떻게 일을 나눌지까지 전부 다시 그려야 한다.

여기서 요즘 자주 들리는 표현이 하나 있다. **에이전틱 조직(agentic organization)** — 에이전트가 노동의 일부로 들어와 함께 일하는 새로운 운영 모델을 가리키는 말이다.[2] 말은 거창하지만, 현장의 실감은 의외로 소박하다. 어느 날 보니 사람만 일하던 자리에 사람 아닌 무언가가 같이 일하고 있더라는 것. 그게 시작이다.

이 표현을 처음 들었을 때 나는 솔직히 좀 추상적이라고 느꼈다. 그런데 곱씹어보니 핵심이 한 단어에 있었다. 바로 "노동의 일부"라는 말이다. 에이전트를 그냥 더 똑똑한 도구라고 여기면 우리는 그것을 책상 위의 계산기처럼 다루게 된다. 켜고, 쓰고, 끄면 그만이다. 하지만 에이전트가 사람의 노동을 일부 떠맡는 동료에 가깝다면 이야기가 전혀 달라진다. 동료에게는 소속이 있고, 맡은 일이 있고, 잘못했을 때 책임 소재가 있다. 우리는 새 직원을 들일 때 아무 절차 없이 그냥 자리에 앉히지 않는다. 누구 밑에서 무슨 일을 할지를 정하고, 권한을 부여하고, 책임의 선을 긋는다. 에이전트를 노동의 일부로 들인다는 건, 에이전트에게도 그런 '입사 절차'가 필요해진다는 뜻이다.

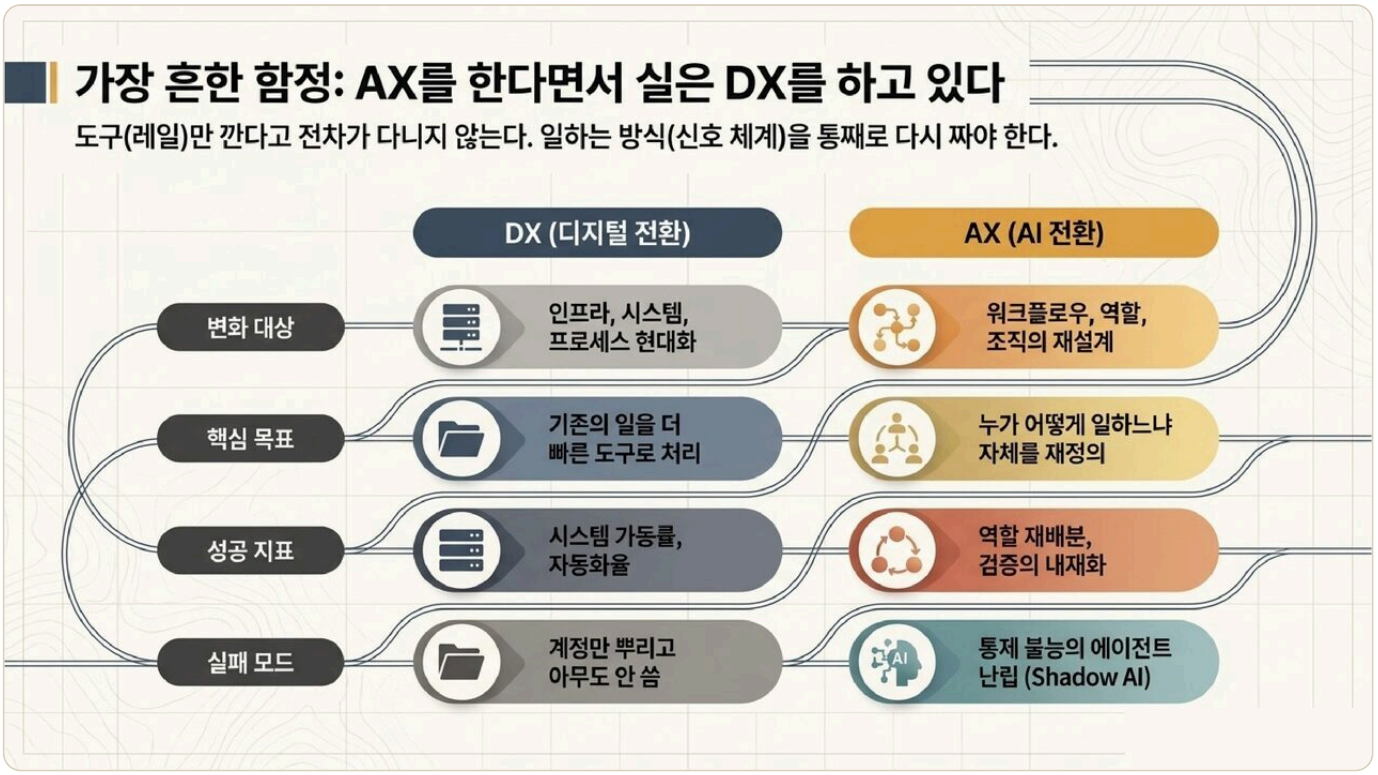


그림 1-1. DX↔AX 비교표

이 비교표를 머릿속에 한 장 걸어두면 좋겠다. 왜냐하면 많은 조직이 AX를 한다면서 실은 DX를 하고 있기 때문이다. 좋은 도구를 사다 깔고, 계정을 뿌리고, "이제 AX 됐다"고 선언한다. 그런데 일하는 방식은 그대로다. 도구만 새 것이고 일은 옛날 그대로인 상태 — 이게 AX의 가장 흔한 함정이다.

이걸 한 장면으로 그려보면 이해가 쉬울 것 같다. 언젠가 집 앞 도로에 트램(노면전차)이 들어서는 걸 본 적이 있다. 사람들이 길게 쇠 레일을 깔길래, 나는 레일이 깔리면 곧 전차가 다니겠거니 했다. 그런데 레일을 다 깔고도 트램은 한참을 다니지 못했다. 마지막으로 바뀐 건 엉뚱하게도 도로의 신호 체계 전체였다. 신호등이 새로 서고, 차선이 다시 그려지고, 우회전 규칙과 횡단보도 위치까지 바뀌고 나서야 비로소 트램은 사람을 태우고 굴러갔다. **레일만 깐다고 트램이 다니는 게 아니다.** AX도 똑같다. 도구(레일)만 깐다고 AX가 되는 게 아니라, 일하는 방식(신호)을 다시 짜야 한다. 이 트램 이야기는 책의 마지막 장에서 다시 꺼낼 텐데, 거기까지 우리가 걸어갈 길이 결국 그 '신호를 다시 짜는' 일이라고 미리 귀띔해둔다.

그렇다면 일과 조직을 다시 설계한다는 그 거창한 일을, 대체 어디서부터 손대야 할까? 여기서 많은 사람이 멈칫한다. "재설계"라는 단어의 무게에 눌려, 완벽한 청사진이 손에 들어오기 전까지는 첫발을 못 떼는 것이다. 컨설팅 보고서를 기다리고, 전사 전략이 확정되기를 기다리고, "AX의 정의"가 명확해지기를 기다린다. 그러는 사이 시간만 흐른다. 나는 이 기다림이 가장 위험한 함정이라고 본다. 왜냐하면 — 뒤에서 더 자세히 이야기하겠지만 — 그 정의는 끝까지 깔끔하게 손에 들어오지 않기 때문이다.

그런데, 어느 순간 통제가 안 되기 시작했다

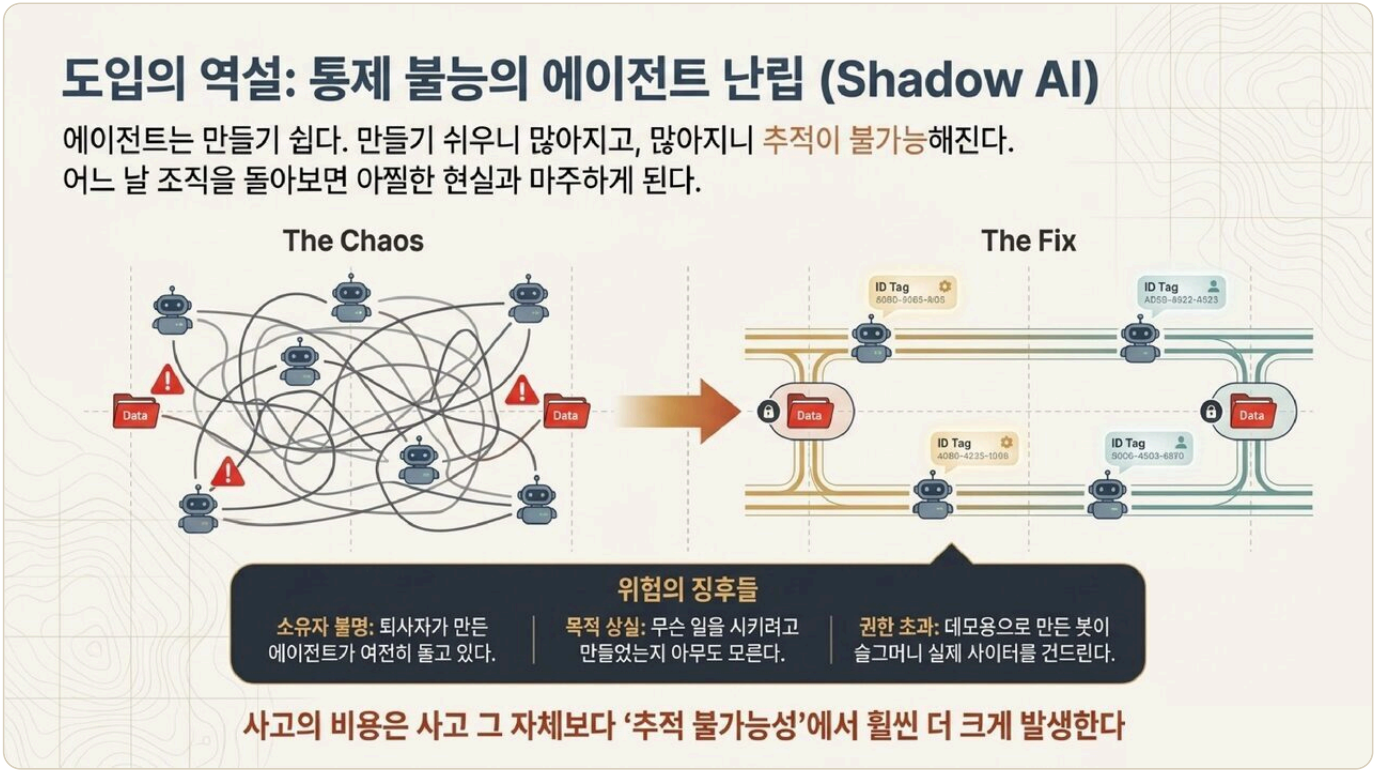


그림 1-4. 통제 불능의 에이전트 난립 (Shadow AI)

좌표를 잡았으니, 이제 내 이야기를 해보자. 솔직히 처음엔 신났다. 에이전트라는 게 생각보다 만들기 쉬웠다. 반복되는 업무 하나를 떼어 에이전트에게 맡기면 정말로 알아서 처리했다. 그러니 하나를 만들고, 또 하나를 만들고, 옆 팀도 만들고, 누군가는 주말에 혼자 뚝딱 만들어 사내 메신저에 자랑했다. 좋은 일 아닌가. 풀뿌리처럼 번지는 자발적 확산 — 이게 진짜 전환의 신호라고들 하니까.[3]

그런데 어느 날, 가만히 돌아가는 것들의 목록을 한번 들여다봤다가 등이 서늘해졌다.

누가 만들었는지 모르는 자동화가 돌고 있었다. 무슨 일을 하는지 정확히 설명할 수 있는 사람이 없는 에이전트가 있었다. 어떤 건 한참 전에 만든 사람이 퇴사했고, 어떤 건 데모용으로 만든 게 슬그머니 실제 업무 데이터를 건드리고 있었다. "이거 누가 관리해요?"라고 물으면 다들 서로를 쳐다봤다. 분명 우리가 만든 것들인데, 우리 중 누구도 전체 그림을 쥐고 있지 못했다.

조금 더 솔직히 말하면, 그 목록 앞에서 나는 내가 직접 만든 것조차 다 기억하지 못한다는 사실을 깨달았다. 바쁘게 하나둘 만들 때는 다 머릿속에 있는 것 같았는데, 막상 펼쳐놓고 보니 "이건 내가 만든 건가? 무슨 일을 시키려고 만들었더라?" 싶은 것이 섞여 있었다. 만든 사람조차 이런데, 옆 사람이 만든 건 말할 것도 없다. 각자 자기 자리에서 부지런히 좋은 도구를 만들었을 뿐인데, 모아놓으니 누구의 손도 닿지 않는 안개 같은 영역이 되어 있었다.

이 느낌을 뭐라고 해야 할까. 솔직히 아찔했다. 하나하나는 다 쓸모 있는 도구였는데, 모아놓고 보니 통제 불능이었다. 더 무서운 건, 무언가 잘못됐을 때 — 가령 어떤 에이전트가 엉뚱한 데이터를 고치거나, 민감한 정보를 엉뚱한 곳에 흘리거나 했을 때 — "이게 어디서 나온 거지?"를 추적할 방법이 없다는 점이었다. 책임을 물을 대상조차 흐릿했다. AI가 한 일의 책임은 대체 누구에게 있는가? 이 질문에 아무도 자신 있게 답하지 못하는 상태, 그게 정말 끔찍한 일이다.

여기서 한 가지 분명히 해두고 싶은 게 있다. 이건 누가 잘못해서 벌어진 사고가 아니다. **에이전트가 쉽게 만들어지는 만큼, 쉽게 난립하는 것은 거의 자연법칙에 가깝다.** 만들기 쉬우니 많아진다. 많아지니 추적이 안 된다. 추적이 안 되니 위험해진다. 이 흐름은 "다음부턴 더 조심하자"는 다짐으로 막아지지 않는다. 그러면 무엇을, 어디에 깔아야 할까?

> **[이게 틀어졌을 때]** 난립을 방치하면 어떤 대가를 치르는가. 나는 이걸 직접 겪었다. 출처를 모르는 자동화가 실 데이터를 건드린 정황을 뒤늦게 발견하고, 그게 언제부터 그랬는지, 무슨 일을 했는지를 며칠에 걸쳐 거꾸로 추적해야 했다. 만든 사람을 찾는 것부터가 일이었다. 그 며칠 동안 가장 무거웠던 건 작업량이 아니라, "지금 이 순간에도 내가 모르는 무언가가 돌고 있을지 모른다"는 찜찜함이었다. 사고의 비용은 사고 자체보다 '추적 불가능'에서 더 크게 발생한다. 이걸 겪고 나서야 나는 표준이 필요하다는 걸 절감했다.

리더는 여기까지만 읽어도 된다

지금부터는 이 난립을 실제로 코드와 절차로 어떻게 풀었는지, 좀 더 손에 잡히는 이야기로 들어가보자. 직접 시스템을 만드는 빌더라면 끝까지 함께 가면 좋겠다. 자기 현장에 바로 옮길 수 있는 절차가 거기 있다.

하지만 조직을 이끄는 리더라면, 사실 **여기까지만 읽어도 충분하다.** 바로 다음 절(③ "당신의 첫 에이전트 표준")으로 건너뛰어도 이 장의 핵심은 하나도 놓치지 않는다. 기억해야 할 단 하나는 이것이다 — *에이전트는 만드는 것보다 들이는 것이 어렵고, 들이는 일의 핵심은 신분증과 길을 마련하는 것이다.* 이 한 문장을 손에 쥐고 ③으로 넘어가면 된다.

자, 그럼 빌더들과 함께 조금 더 깊이 들어가보자.

에이전트에게 신분증을 발급하자

난립을 겪고 나서 내가 가장 먼저 던진 질문은 이거였다. "이 많은 에이전트들의 공통점이 뭐지? 그리고 공통적으로 빠져 있는 게 뭐지?" 답은 단순했다. **하나같이 신분증이 없었다.** 누가 만들었고, 무슨 일을 하고, 얼마나 위험한 일을 하는지를 적어둔 한 장의 신원 정보가 어디에도 없었다.

그래서 첫 삽을 여기에 떴다. 에이전트마다 표준화된 신원 정보를 붙이기로 한 것이다. 나는 이걸 **에이전트 매니페스트(manifest)**라고 부른다. 사람으로 치면 주민등록증 같은 것이다. 새로 만드는 모든 에이전트는 이 매니페스트 한 장을 반드시 들고 다녀야 한다.

매니페스트에 뭘 적어야 할까? 처음엔 욕심이 나서 온갖 항목을 다 넣고 싶었다. 하지만 항목이 많아질수록 아무도 안 쓴다는 걸 금방 배웠다. 그래서 정말 없으면 안 되는 최소한만 추렸다. 소유자가 누구인지, 무슨 용도인지, 어떤 모델을 쓰는지, 위험등급이 얼마인지, 그리고 어떤 게이트(관문)를 통과해야 운영에 올라갈 수 있는지. 대략 이 정도다.



그림 1-2. 에이전트 매니페스트 구조도

매니페스트를 도입하면서 한 가지 마음에 둔 게 있다. 형식을 너무 빡빡하게 잡으면 아무도 안 쓴다는 것이다. 처음엔 "이왕 만드는 김에" 하는 마음으로 적어야 할 항목을 자꾸 늘리고 싶었다. 만든 날짜, 갱신 주기, 예상 호출량, 비용 한도... 끝이 없었다. 그런데 항목이 열 개를 넘어가는 순간, 개발자들의 표정이 굳는 게 보였다. 솔직히 그 표정을 보는 게 내심 뜨끔했다. 좋자고 만든 신분증이 도리어 사람들을 귀찮게 하고 있었으니까. 신분증을 만드는 게 일이 되어버

리면, 사람들은 신분증 없이 슬쩍 만들고 만다. 그래서 욕심을 접었다. 정말 없으면 추적이 불가능해지는 다섯 가지로 못 박고, 나머지는 필요해지면 그때 늘리기로 했다. 항목을 한 번 늘려봤다가 되돌리고 나서야 배운 것이라, 좀 부끄럽긴 하다.

여기서 한 가지가 더 따라왔다. 신분증이 있다는 건 곧, 신분증의 형식이 올바른지 검사할 수 있다는 뜻이기도 하다. 그래서 매니페스트가 규격에 맞게 작성됐는지를 자동으로 점검하는 검사기를 붙였다. 검사는 두 겹으로 두었다. 하나는 형식이 맞는지 훑어보는 점검이다. 소유자 칸이 비어 있지는 않은지, 위험등급이 정해진 값 중 하나인지 같은 것을 본다. 다른 하나는 실제로 한 번 돌려봤을 때 최소한 멀쩡히 작동하는지를 보는 가벼운 시험이다. 신분증만 그럴듯하고 정작 실행하면 첫걸음에서 고꾸라지는 에이전트를 걸러내기 위해서다. 이 둘을 합쳐 일종의 '입학 시험'을 마련한 셈이다. 사람을 조직에 들일 때도 신원 확인과 기본 검증을 하지 않는가. 에이전트도 다를 게 없다.

신분증과 검사기 — 여기까지가 난립에 데인 직후 내가 떴던 첫 삽이다. 가벼운 신원 한 장, 그리고 그 신원이 자연스럽게 걸리는 길. 그런데 미리 한 가지만 귀띔해두고 싶다. 시간이 더 흐르고 난립이 더 심해지자, 나중엔 아예 에이전트에게 '사번'을 주고 사람처럼 들이자는 데까지 생각이 가게 된다. 다만 그건 한참 뒤의 이야기라, 책의 마지막 장(9장)에서 레일을 이야기할 때 다시 꺼내려 한다. 지금은 우선 첫 삽 — 신분증과 길 — 에 집중하자.

길을 깔되, 막는 벽이 아니라 자연스럽게 걸리는 레일로

자, 신분증과 검사기를 만들었다. 그런데 여기서 진짜 어려운 문제가 시작된다. **이걸 어떻게 강제하지?**

내 첫 번째 시도는 솔직히 좀 무거웠다. 모든 에이전트가 중앙의 자동 검증 절차를 통과하지 않으면 배포를 막아버리는 방식을 떠올렸다. 일종의 검문소를 세우는 것이다. 강력하긴 하다. 통과 못 하면 한 발짝도 못 나가니까. 그런데 막상 적용해보니 뒷맛이 영 찜찜했다.

검문소가 너무 멀고 무거웠다. 개발자가 한참 작업을 다 끝내고, 한참 뒤의 단계에 가서야 "당신 에이전트엔 신분증이 없습니다"라며 가로막힌다고 상상해보자. 이미 다 만들어놓은 걸 되돌아와 고쳐야 하니 번거롭고, 무엇보다 그 검문소가 '나를 막는 장애물'처럼 느껴진다. 사람은 자기를 막는 것을 우회할 방법부터 찾는다. 결국 검문소가 무거울수록, 사람들은 검문소를 피해 샛길로 다니기 시작한다. 통제하려다 오히려 통제 밖으로 밀어내는 셈이다.

게다가 검문소가 멀리 있으면, 잘못을 깨닫는 시점도 늦어진다. 무언가 어긋났다는 신호를 작업 직후가 아니라 한참 뒤에야 받게 되니, 그때쯤이면 이미 기억도 흐릿하고 고칠 의욕도 식어 있다. 막는 지점이 일하는 사람에게서 멀수록, 그 규칙은 지켜지지 않고 미움만 받는다.

그래서 방향을 틀었다. 무거운 검문소를 멀찍이 세우는 대신, **개발자의 손끝에서 자연스럽게 걸리도록** 만들기로 했다. 작업을 마무리하고 변경 내용을 저장하려는 바로 그 순간, 그 자리에서 신분증을 확인하게 한 것이다. 이렇게 하니 두 가지가 달라졌다. 첫째, 문제가 있으면 한참 뒤가 아니라 바로 그 자리에서 알게 되니 고치기가 쉽다. 둘째, 이게 '나를 막는 벽'이 아니라 '내가 자연스럽게 밟고 지나가는 길'처럼 느껴진다.

이 차이가 이 책 전체를 관통하는 첫 번째 주제다. **레일은 벽이 아니다.** 벽은 멈춰 세우고, 길은 흐르게 하면서 안전하게 데려다준다. 좋은 길은 무겁지 않다. 너무 무거우면 사람들이 우회하고, 우회하는 순간 그 길은 아무것도 지키지 못한다. 그러니 길을 깔 때는 늘 물어보자. *이 길은 사람들이 거부감 없이 올라탈 만큼 가벼운가?*

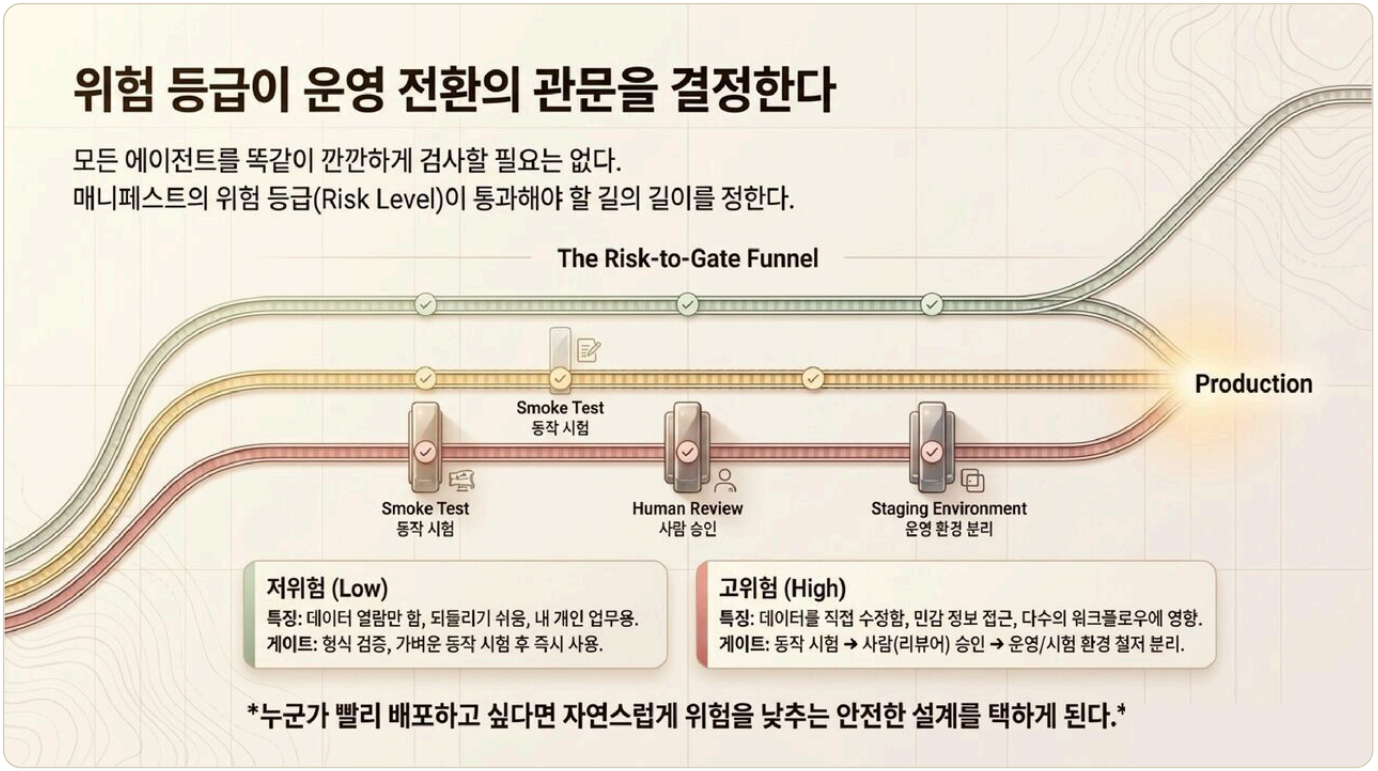


그림 1-3. 위험등급↔Gate 매트릭스

길을 깔면서 한 가지 원칙을 더 세웠다. **위험이 높을수록 더 많은 길을 지나게 하는 것**이다. 모든 에이전트를 똑같이 깐깐하게 검사할 필요는 없다. 사내 문서 요약이나 돌리는 가벼운 에이전트와, 실제 사람의 데이터를 고치는 무거운 에이전트를 같은 기준으로 다루는 건 비효율적이다. 앞쪽은 가볍게 통과시키고, 뒤쪽은 더 많은 관문 — 가령 사람의 승인, 운영 환경과 시험 환경의 분리 같은 — 을 거치게 한다. 매니페스트에 적힌 위험등급이, 그 에이전트가 운영에 오르기까지 통과해야 할 길의 길이를 정한다. 이렇게 위험등급과 운영 전환의 관문을 연결해두면, 검사는 필요한 곳에 집중되고 불필요한 곳에선 가벼워진다.

이 연결이 왜 중요한지 짚고 가자. 위험등급을 정해놓고도 그게 아무런 실질적 차이를 만들지 않으면, 등급은 그냥 장식이 된다. "이 에이전트는 고위험입니다"라고 적어두기만 하고 정작 들이는 절차는 저위험과 똑같다면, 그 라벨이 무슨 소용인가. 그래서 위험등급은 반드시 '그 다음에 무엇이 달라지는가'와 묶여 있어야 한다.

실제로 묶은 모양은 이렇다. 저위험은 시험 환경에서 가볍게 돌려본 뒤 바로 쓰게 한다. 중위험은 거기에 동작 검증을 한 겹 더한다. 고위험은 사람의 눈을 한 번 더 거치고, 운영 환경을 시험 환경에서 따로 떼어 올린다.

그런데 이렇게 해두니 예상 못 한 효과가 하나 따라왔다. 누군가 자기 에이전트를 빨리 운영에 올리고 싶으면, 시키지 않아도 위험을 낮추는 쪽으로 설계하기 시작하더라는 것이다. 되돌릴 수 있게 만들고, 사람이 확인하는 단계를 끼워 넣고, 민감한 데이터는 아예 직접 건드리지 않게 돌아간다. 길을 잘 깔아두면, 사람들이 알아서 더 안전한 쪽으로 걸어간다.

여기까지가 빌더의 영역이었다. 그런데 한 가지, 코드로는 끝내 풀리지 않는 문제가 남는다. 다음 절에서 그 이야기로 모두 다시 합류하자.

당신의 첫 에이전트 표준

이제 이 장의 이야기를 당신의 현장으로 옮겨보자. 리더든 빌더든, 자기 조직에 에이전트를 처음 들일 때 무엇부터 하면 좋을지를 함께 정리해보자.

최소 메타데이터 체크리스트

거창한 시스템부터 만들 필요는 없다. 종이 한 장, 혹은 공유 문서 한 줄에서 시작해도 된다. 새로 들이는 에이전트마다 다음 다섯 가지만 적게 해보자.

- **소유자** — 이 에이전트에 문제가 생겼을 때 연락할 사람은 누구인가? (없으면 들이지 않는다.)
- **용도** — 한 문장으로 무슨 일을 하는가? 한 문장으로 설명 못 하면, 아직 들일 준비가 안 된 것이다.
- **사용 모델·도구** — 무엇 위에서 도는가? 나중에 갈아끼울 수 있게 적어둔다.
- **위험등급** — 사람·데이터·돈에 닿는 정도는? (다음 워크시트로 정한다.)
- **통과 게이트** — 운영에 오르기 전 무엇을 통과해야 하는가?

이 다섯 줄이 에이전트의 신분증이다. 사소해 보이는 다섯 줄이지만, 이게 없어서 나는 며칠을 추적에 썼다. 작게 시작하되, 빠짐없이 적게 하자.

위험등급 정의 워크시트

위험등급은 막연히 "이건 좀 위험한 것 같다"가 아니라, 몇 가지 질문에 답하며 정하는 게 좋다. 자기 조직에 맞게 다듬되, 대략 이런 질문들로 시작해보자.

- 이 에이전트가 **사람의 평가·개인정보** 같은 민감한 데이터에 닿는가?
- 이 에이전트가 **무언가를 직접 바꾸는가**, 아니면 읽고 제안만 하는가? (직접 바꿀수록 위험하다.)
- 이 에이전트가 틀렸을 때, **되돌릴 수 있는가?**
- 이 에이전트의 결과를 **사람이 한 번 더 확인하는가?**

"읽기만 하고, 되돌릴 수 있고, 사람이 확인한다"면 낮은 등급이다. "직접 바꾸고, 되돌리기 어렵고, 민감 데이터에 닿는다"면 높은 등급이다. 등급이 정해지면, 높은 등급일수록 더 많은 길을 지나게 하면 된다. 단순하지 않은가.

여기에 질문을 하나만 더 보태자. **이 에이전트는 한 개인의 도구인가, 아니면 여러 사람의 일에 닿는 구성원인가?** 내 일만 거드는 보조라면 가볍게 두면 된다. 하지만 한 업무 프로세스를 떠맡거나 여러 팀을 가로지른다면, 그건 '들이는' 절차를 제대로 밟을 후보다 — 소속과 책임자를 정하고, 한살이를 끝까지 챙겨야 할 대상이다. 위험의 높낮이와 이 '유형'을 함께 보면, 어디에 힘을 주고 어디는 풀어줄지가 한결 또렷해진다.

한 가지 흔한 오해를 짚고 싶다. "우리 조직은 아직 에이전트가 몇 개 없는데, 이런 표준은 좀 이르지 않나?"라고 생각하기 쉽다. 그런데 내 경험으로는 정반대다. 표준은 에이전트가 많아진 다음에 만들면 이미 늦다. 그땐 이미 신분증 없는 것들이 잔뜩 돌고 있어서, 뒤늦게 하나하나 신원을 캐고 라벨을 붙이는 게 몇 곱절 더 번거롭다. 에이전트가 두세 개일 때 다섯 줄짜리 신분증을 붙이는 습관을 들이면, 수십 개가 되고 나서 대청소를 하는 것보다 품이 훨씬 덜 든다.

또 하나 — 민감한 정보를 다루는 에이전트라면, 그 통제는 나중에 덧붙이는 게 아니라 **설계 단계에서 미리 막아두는 편이 낫다.** 누가 보면 안 되는 데이터에 에이전트가 닿게 되어 있는지, 시험용으로 만든 게 실데이터를 건드릴 길이 열려 있지는 않은지를, 만들기 전에 물어보는 것이다. 새어나간 다음에 막는 것만큼 끔찍한 일도 없다. 이 점은 뒤의 장들에서 계속 다시 만나게 될 것이다.

그리고, 끝내 사람이 지는 것

레일을 아무리 잘 깔아도 끝내 코드로 풀리지 않는 것이 하나 있다. **책임**이다.

에이전트가 일을 한다. 그런데 그 일이 잘못됐을 때, 책임은 에이전트에게 물을 수 없다. 기계에게 시말서를 받을 수는 없는 노릇이다. 그래서 어떤 매니페스트에든 *사람 소유자*는 빠지지 않게 못 박아두어야 한다고 봤다. 누가 만들었고, 누가 그 결과에 최종 책임을 지는지 — 이게 비어 있는 에이전트는 아무리 잘 만들어졌어도 조직에 들일 수 없다.

자동화의 시대일수록 이 원칙은 더 또렷해야 한다. **레일은 사고를 줄여주지만, 최종 책임까지 대신 져주지는 않는다.** 그 책임은 언제나 사람의 자리로 돌아온다. 이게 이 책을 관통하는 두 번째 주제다. 자동화가 늘어날수록, "이 일의 최종 책임은 누구인가"라는 질문을 더 자주, 더 명시적으로 던지게 되는 것이다.

오해는 말자. 책임을 사람에게 묶어두자는 게 에이전트를 못 믿겠다는 뜻은 아니다. 책임의 자리가 분명해야 우리는 에이전트에게 더 과감하게 일을 맡길 수 있다. 무언가 잘못돼도 누가 그것을 알아채고 되돌릴지가 정해져 있으면, 우리는 안심하고 자율의 폭을 넓힐 수 있다. 책임을 또렷이 하는 것은 에이전트를 묶어두는 사슬이 아니라, 더 멀리 풀어줄 수 있게 하는 안전벨트에 가깝다. 레일이 그 자유를 어떻게 감당할 만한 것으로 만드는지는 책의 마지막 부에서 다시 다룬다.

마무리

AX를 정의하지 못한 채로도, 첫 삽은 어디에 뜨는가?

답은 의외로 소박했다. 첫 삽의 자리는 거창한 전략표도, 완벽한 정의도 아니었다. **에이전트마다 신분증 한 장을 붙이고, 그 신분증이 자연스럽게 걸리는 가벼운 길을 까는 것** — 거기였다. 나는 AX가 무엇인지 끝까지 한 문장으로 정의하지 못한 채로 이 일을 시작했지만, 신분증을 발급하고 길을 까는 그 과정 자체가 이미 AX였다. 일하는 방식을, 책임의 경계를, 사람과 에이전트가 함께 일하는 규칙을 다시 짜고 있었으니까.

그런데 여기서 솔직히 고백할 게 하나 있다. 방금 이야기한 이 표준은, 사실 처음부터 설계도로 그려놓고 시작한 게 아니다. 나는 먼저 에이전트들을 잔뜩 만들었고, 그것들이 난립하는 통제 불능을 겪고 나서야 비로소 "아, 표준이 필요하겠구나"를 깨달았다. 순서가 거꾸로였던 셈이다. 설계도 없이 만들면서, 무엇을 만들어야 하는지를 만들고 나서야 알게 된 것이다.

이게 내 무능이었을까, 아니면 원래 그런 것일까? 비행기를 날리면서 동시에 그 비행기를 조립하는 것 같은 이 기분 — 분명 어딘가 미덥지 못하고 찜찜한데, 멈추면 추락할 것 같아 멈출 수도 없는 이 상태 말이다. 나만 이렇게 허둥대는 걸까, 아니면 이게 새로운 무언가를 만드는 모든 사람이 거쳐 가는 정상적인 풍경일까?

다음 장에서 함께 이야기해보자. "AX가 뭔지 아직 모르겠다"는 막막함이 당신이 뒤처진 탓인지, 아니면 모든 새로운 기술이 거쳐 가는 정상적인 길인지를 말이다. 미리 한 가지만 귀띔하자면 — 당신은 뒤처진 게 아니다. 그리고 그 사실을 아는 것만으로도, 첫 삽을 뜨는 손이 한결 가벼워질 것이다.

2장. AX는 아직 만들어지는 중이다

> **이 장에서 내가 내려놓은 자책 한 줄, 그리고 그 자리에 들어선 위안**
>
> 자책은 이랬다. "AX가 뭔지 한 문장으로 못 적겠다. 청사진부터 그렸어야 하는데 무작정 만들기부터 했다." 멈추면 추락할 것 같던 그 기분.
>
> 위안은 오래된 조직이론 셋에서 왔다. 조직은 멈춰 있는 게 아니라 흐르는 강이고(organizational becoming), 전략은 슬라이드에서 완성되지 않고 만들면서 떠오르며(emergent strategy), 기술은 깔린다고 끝이 아니라 쓰이면서 구조가 된다(technology-in-practice). 그래서 정의가 안 잡힌 채 시작하는 건 미성숙이 아니라 정상이었다.
>
> 그러니 청사진을 기다리지 말자. 이 장 끝의 "완성되지 않아도 시작하는 법"으로 첫 삽을 떠보자. J-curve로 치면 정체("난립·막막")에서 신남으로 막 올라서는 구간이다.

앞 장 끝에서 나는 한 가지를 솔직히 고백했다. 에이전트마다 신분증을 붙이고 길을 까는 그 표준을, 사실 처음부터 설계도로 그려놓고 시작한 게 아니라고. 먼저 잔뜩 만들었고, 난립하는 통제 불능을 겪고 나서야 비로소 "아, 표준이 필요하겠구나"를 깨달았다고. 순서가 거꾸로였다고.

잠시 그 순간으로 함께 돌아가보자.

설계도가 없었다, 그런데 만들고 나서야 알았다

어느 날의 일이라고 해두자. 정확히 며칠이었는지, 무엇을 보다가였는지는 흐릿하다. 다만 그 감각만은 또렷하다. 나는 그동안 만들어 돌리던 것들의 목록을 한자리에 펼쳐놓고 있었다. 그리고 그 목록을 들여다보다가, 문득 이상한 기분에 사로잡혔다.

분명 나는 이걸 만들 때 무언가를 향해 가고 있었다. 그런데 목록을 펼쳐놓고 "내가 향하던 그 무언가가 대체 뭐였지?"를 한 문장으로 적어보려 하니, 막상 적히지가 않았다. 에이전트 하나하나는 다 이유가 있어서 만든 것이었다. 이건 이 반복 업무가 귀찮아서, 저건 저 사람이 부탁해서, 그건 그냥 되나 안 되나 궁금해서. 그런데 그것들을 모아놓고 보니, 거기엔 아무런 전체 그림이 없었다. 큰 설계도 위에 부품을 하나씩 끼워 넣은 게 아니라, 부품을 잔뜩 만들어놓고 나서 "이것들이 모이면 대체 뭐가 되는 거지?"를 거꾸로 묻고 있는 나를 발견한 것이다.

솔직히 그 순간 좀 난감했다. 명색이 이 일을 이끈다는 사람이, 자기가 뭘 만들고 있었는지를 만들고 나서야 더듬더듬 알아가고 있었으니까. 누가 "그래서 당신이 그리던 AX의 큰 그림이 뭐였습니까?"라고 물으면, 나는 그제야 목록을 가리키며 "음… 이런 것들이 모여서… 이렇게 된 겁니다"라고 사후에 설명하는 수밖에 없었을 것이다. 먼저 그림을 그리고 그대로 칠한 게 아니라, 칠하고 나서 무슨 그림이었는지를 알게 된 셈이다.

이게 무능일까? 그날의 나는 솔직히 그렇게 느꼈다. 제대로 된 사람이라면 청사진부터 그렸어야 하는 거 아닌가, 나는 그 기본을 건너뛰고 무작정 만들기부터 한 게 아닌가 — 그런 자책이 들었다. 1장 끝에서 말한 바로 그 찜찜함, 멈추면 추락할 것 같아 멈추지도 못하던 그 기분이 이 목록 앞에서 가장 짙었다.

그런데 바로 그 자책의 한복판에서, 묘한 반전이 하나 떠올랐다. **그 난립을 직접 겪지 않았다면, 나는 1장의 그 표준을 결코 떠올리지 못했을 것**이라는 사실이다. 신분증이 필요하다는 것도, 위험등급에 따라 길의 길이를 달리해야 한다는 것도, 무거운 검문소보다 손끝에서 자연스럽게 걸리는 레일이 낫다는 것도 — 그 어느 것 하나 책상 앞에서 미리 떠올린 게 아니었다. 만들어서 난립시켜보고, 그 난립에 데어보고 나서야 비로소 "이게 필요하구나"가 또렷해진 것이다.

여기서 나는 멈칫했다. 그렇다면 이건 정말 순서가 잘못된 걸까? 만약 내가 만들기 전에 완벽한 표준 설계도를 먼저 그리려 했다면, 나는 무엇을 표준화해야 하는지조차 몰랐을 것이다. 겪어보지 않은 문제에 대한 해법을 미리 그릴 수는 없으니까. 어쩌면 — 만들면서 알게 되는 이 순서가 거꾸로가 아니라, 원래 이렇게 갈 수밖에 없는 길인 건 아닐까?

"AX가 뭔지 아직 모르겠다"는 막막함, 설계도 없이 만들면서 더듬는 이 상태가, 내가 뒤처진 탓인지 아니면 원래 그런 것인지 함께 따져보자.

조직은 흐르는 강이다 — 변화는 가끔 오는 손님이 아니다



그림 2-3. 완벽한 청사진은 없다 — 날면서 비행기 조립

내 자책을 풀어줄 실마리는 뜻밖에도 한참 오래된 조직이론 안에 있었다. 그 목록 앞에서 느낀 막막함이 나만의 무능이 아니라 꽤 보편적인 현상이라는 걸, 나는 한참 뒤에야 알았다. 이론은 보통 답을 주지는 않는다. 다만 "당신이 지금 서 있는 그 자리에 이름이 있다"고 알려준다. 이름이 붙는 순간, 막막함은 한결 견딜 만해진다. 그러니 잠깐, 내가 서 있던 자리의 이름을 함께 찾아보자.

우리는 보통 조직을 가만히 있는 것으로 여긴다. 조직이 먼저 있고, 거기에 가끔 '변화'라는 사건이 찾아온다고 생각한다. "올해 우리 조직에 큰 변화가 있었다"는 식으로 말이다. 그런데 어떤 조직 연구자들은 이걸 정반대로 뒤집었다. **변화는 조직에 가끔 찾아오는 손님이 아니라, 조직 그 자체가 끊임없는 변화의 흐름**이라는 것이다.[4] 조직이라는 건 단단한 건물이 아니라 흐르는 강에 가깝다. 우리가 "이게 우리 조직이다"라고 가리키는 것은, 흐르는 강물을 한순간 사진으로 찍어 "이게 강이다"라고 말하는 것과 비슷하다. 사진은 멈춰 있지만 강은 한 번도 멈춘 적이 없다.

이 관점을 받아들이면, 내 목록 앞의 막막함이 조금 다르게 보이기 시작한다. 나는 "완성된 AX라는 상태가 어딘가 있고, 나는 거기 도달하지 못했다"고 자책했다. 그런데 만약 조직이 본래 흐르는 강이라면, '완성된 AX'라는 멈춰 있는 상태는 애초에 존재하지 않는다. 도달할 정상이 없는데 정상에 못 올랐다고 자책한 셈이다.

여기서 이 책 전체를 떠받치는 한마디가 나온다. 어떤 이들은 새로운 무언가를 만드는 이 상태를 가리켜 **"비행기를 날리면서 동시에 그 비행기를 조립한다"**고 표현한다. 처음 들으면 무능을 비꼬는 농담처럼 들린다. 그런데 정작 이 말은 조직 변화의 본질을 정확히 짚은 표현으로 쓰인다.[5] 멈춘 비행기를 완성한 다음에 띄우는 게 정석이라고 우리는 믿는다. 하지만 조직이 흐르는 강이라면, 멈춰서 완성할 시간 같은 건 처음부터 주어지지 않는다. 흐르는 채로 고치는 수밖에 없다.

이걸 처음 접했을 때 나는 솔직히 좀 위안을 받았다. 내가 허둥댄 게 아니라, 원래 다들 이렇게 흐르는 채로 만들고 있었던 것이다. 다만 조용히 했을 뿐이다.

슬라이드로 그린 전략과 만들면서 떠오른 전략

조직이 흐르는 강이라는 큰 그림은 위안이 됐다. 그래도 좀 막연했다. 내 구체적인 자책 — "설계도 없이 만들었다"는 그 자책 — 에는 더 정확한 이름이 필요했다. 그 이름도 오래된 전략 연구 안에 있었다.

전략을 두 종류로 나눠 생각해보자. 하나는 **의도된 전략**이다. 회의실에서 슬라이드로 그리고, 결재를 받고, "올해 우리는 이 방향으로 간다"고 선언하는 그 전략이다. 다른 하나는 **떠오른 전략**이다. 아무도 미리 그리지 않았는데, 현장에서 하나둘 결정을 내리다 보니 사후에 보니 어떤 방향이 만들어져 있더라는 것. 그리고 어떤 연구자들은 이렇게 말한다. **실제로 실현되는 전략은 의도된 것만으로 이뤄지는 게 아니라, 의도된 전략과 떠오른 전략이 섞여서 만들어진다**고.[6]

이 구분 앞에서 나는 무릎을 쳤다. 내가 한 일이 바로 *떠오른 전략* 그 자체였기 때문이다. 나는 슬라이드로 "우리 조직의 AX 청사진"을 먼저 그린 적이 없다. 대신 에이전트를 만들고, 난립에 데이고, 신분증을 붙이고, 레일을 깔았다. 그 결정들을 하나하나 내릴 때 나는 큰 그림을 보고 있지 않았다. 그런데 그 결정들이 쌓이고 나서 돌아보니, 거기 어떤 방향이 — 어떤 전략이 — 만들어져 있었다. 무능의 증거라고 여겼던 그 난립 장면이, 알고 보니 *떠오른 전략*의 교과서적 사례였던 셈이다.

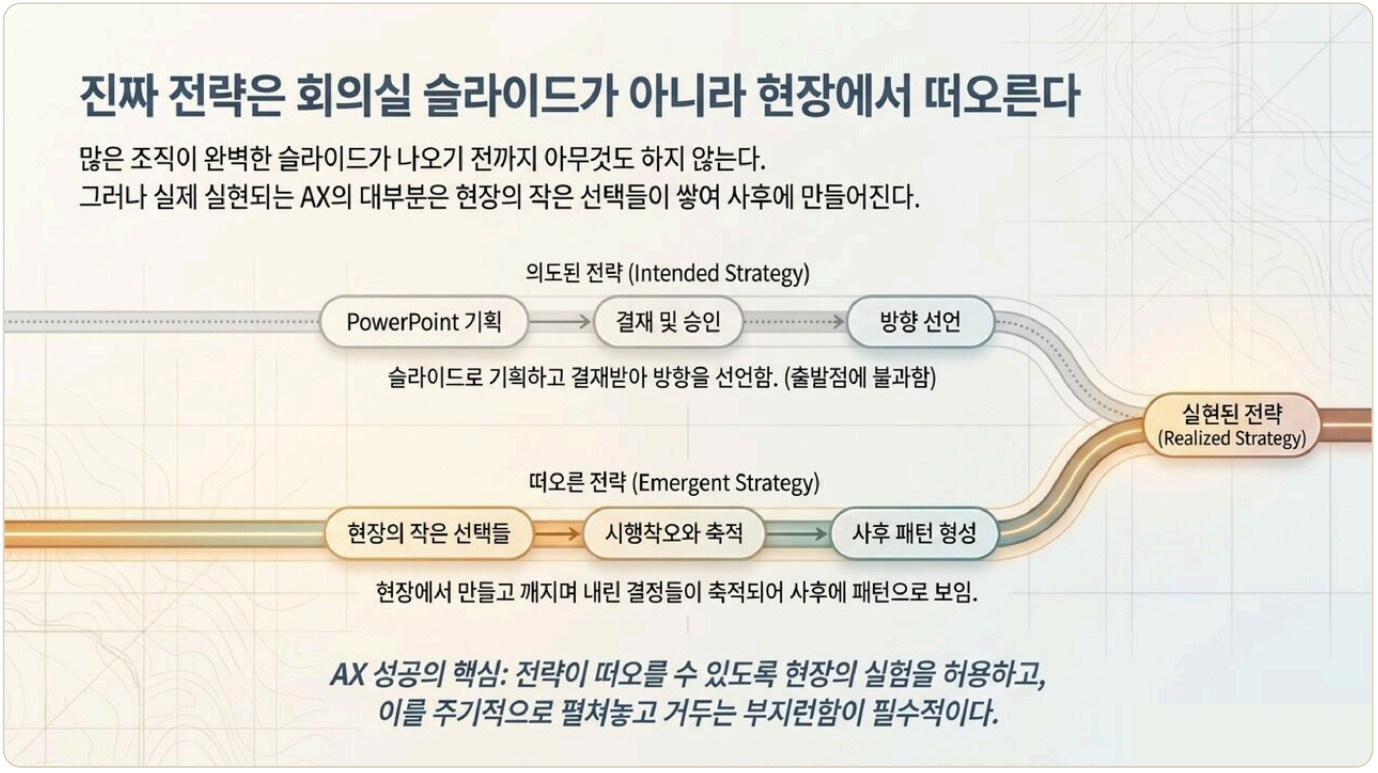


그림 2-1. "의도된 전략 → 떠오른 전략" 흐름도

흐름도에서 보듯, 많은 조직이 AX를 한다면서 위쪽 줄기, 즉 '의도된 전략'만 붙들고 있다. 완벽한 슬라이드가 나오기 전까지는 아무것도 안 한다. 그런데 정작 실현되는 AX의 대부분은 아래쪽 줄기 — 누군가가 현장에서 만들면서 떠오르는 쪽 — 에서 만들어진다. 위쪽 줄기만 기다리는 조직은, 정작 전략이 만들어지는 강물에는 발을 담그지 않은 채 강가에서 지도만 그리고 있는 셈이다.

물론 이렇게 반문할 수 있다. "그럼 계획은 다 필요 없고 무작정 만들기만 하면 되느냐?" 그건 아니다. 의도된 전략에도 제 몫이 있다. 다만 그것 *하나만으로는* 끝까지 실현되지 않는다. 방향을 가리키는 큰 의도는 있어야 한다. 그 의도는 출발점이다. 도착점은 만들면서 끊임없이 다시 그려진다. 그러니 슬라이드를 그리되, 그 슬라이드가 현장에서 배신당할 것을 미리 각오해 두자. 그 배신이 바로 떠오른 전략이 일하고 있다는 증거니까.

도구는 깔았다고 끝이 아니다, 쓰이면서 비로소 무언가가 된다

여기서 한 걸음 더 들어가보자. 1장에서 우리는 "도구를 깐다고 AX가 되는 게 아니다"라고 했다. 왜 그럴까? 이 질문에도 오래된 답이 있다.

어떤 연구자는 기술을 이렇게 본다. 기술은 책상 위에 놓이는 순간 정해진 무언가가 되는 게 아니라, **사람들이 그것을 매일 쓰는 실천 속에서 비로소 어떤 구조가 된다**는 것이다.[7] 같은 도구라도 누가 어떻게 쓰느냐에 따라 전혀 다른 것이 된다. 똑같은 협업 도구를 어떤 팀은 단순한 메신저로 쓰고, 어떤 팀은 업무의 중추 신경으로 쓴다. 차이를 만든 건 도구 자체가 아니라, 그것을 둘러싼 매일의 쓰임이다.

이게 AX에 주는 함의는 묵직하다. 좋은 AI 도구를 사다 깔고 계정을 뿌렸다고 해서 그 자체로 AX가 되지는 않는다. 그 도구가 사람들의 일하는 방식 속으로 들어가 매일 쓰이고, 그 쓰임이 역할과 절차를 바꾸기 시작할 때 — 비로소 그것이 'AX'라는 구조가 된다. 1장에서 "계정은 줬는데 안 쓴다"가 정체의 공기였던 이유가 여기 있다. 계정은 도구를 책상 위에 올려놓은 상태일 뿐이다. **AX는 깔리는 순간이 아니라 쓰이는 동안 만들어진다.** 그러니 "만들면서 알게 됐다"는 내 경험을 게으른 변명으로 깎아내릴 필요는 없었다. 그건 기술이 구조가 되는 바로 그 과정을 직접 통과한 기록이었으니까.

정의가 안 잡힌 건 미성숙의 신호가 아니다 — 모든 기술이 거치는 길이다

그래도 걸리는 점이 하나 있다. "그래서 AX가 뭐냐"는 질문에 다들 다른 답을 내놓는다는 점이다. 컨설턴트의 AX, 임원의 AX, 옆 팀의 AX가 다 다르다. 보통 우리는 이런 상태를 '아직 덜 여물었다'는 신호로 읽는다. 정의가 하나로 모이지 않았으니 미성숙하다는 것이다. 그런데 정말 그럴까?

기술의 역사를 들여다본 연구자들은 흥미로운 패턴을 발견했다. 새로운 기술은 처음부터 의미가 하나로 정해져 나오는 게 아니라는 것이다. 초기에는 **여러 집단이 그 기술을 저마다 다르게 해석하는 시기**를 거친다. 같은 물건을 두고 누구는 이런 용도로, 누구는 저런 용도로 본다. 그러다 사회적인 밀고 당김을 거쳐, 어느 순간 의미가 하나로 좁혀지면서 "이 기술은 이런 것이다"가 닫힌다.[8]

자주 드는 예가 자전거다. 자전거가 처음 나왔을 때, 그것은 지금 우리가 아는 모습이 아니었다. 앞바퀴가 거대한 것도 있었다. 어떤 사람들에게 자전거는 젊은 남성이 속도를 즐기는 스포츠 기구였고, 다른 사람들에게는 위험천만한 물건이었다. "자전거란 무엇인가"에 대해 집단마다 답이 달랐던 것이다. 지금 우리가 아는 안전한 자전거의 모습은, 그 여러 해석이 오랜 협상을 거쳐 하나로 닫힌 *결과*다.

이 패턴을 AX에 겹쳐보자. 지금 AX의 의미가 사람마다 다른 건 덜 여물었다는 신호라기보다, **모든 새로운 기술이 반드시 거쳐 가는 그 길 한복판에 우리가 서 있다**는 증거다. 컨설턴트와 임원과 옆 팀이 서로 다른 AX를 말하는 건, 아직 의미가 닫히지 않은 초기 기술에서 너무나 당연한 일이다. 자전거의 앞바퀴 크기를 두고 다투던 그 시절처럼, 우리는 지금 "AX란 무엇인가"를 두고 다투는 중이다. 정의가 아직 안 잡혀 있는 게 *정상*이다.

이 대목에서 내가 받은 위안은 꽤 컸다. 나는 "남들은 다 아는데 나만 모르는 것"이라고 느꼈다. 그런데 사실은 아무도 확정적으로 알지 못한다. 다들 자기 자리에서 본 한 조각을 말하고 있을 뿐이다. 한 조각씩 가진 사람들이 모여 밀고 당기는 그 과정이, 바로 AX의 의미가 닫혀가는 과정 그 자체다.

그래서, 대부분의 조직이 아직 실험 단계에 있다

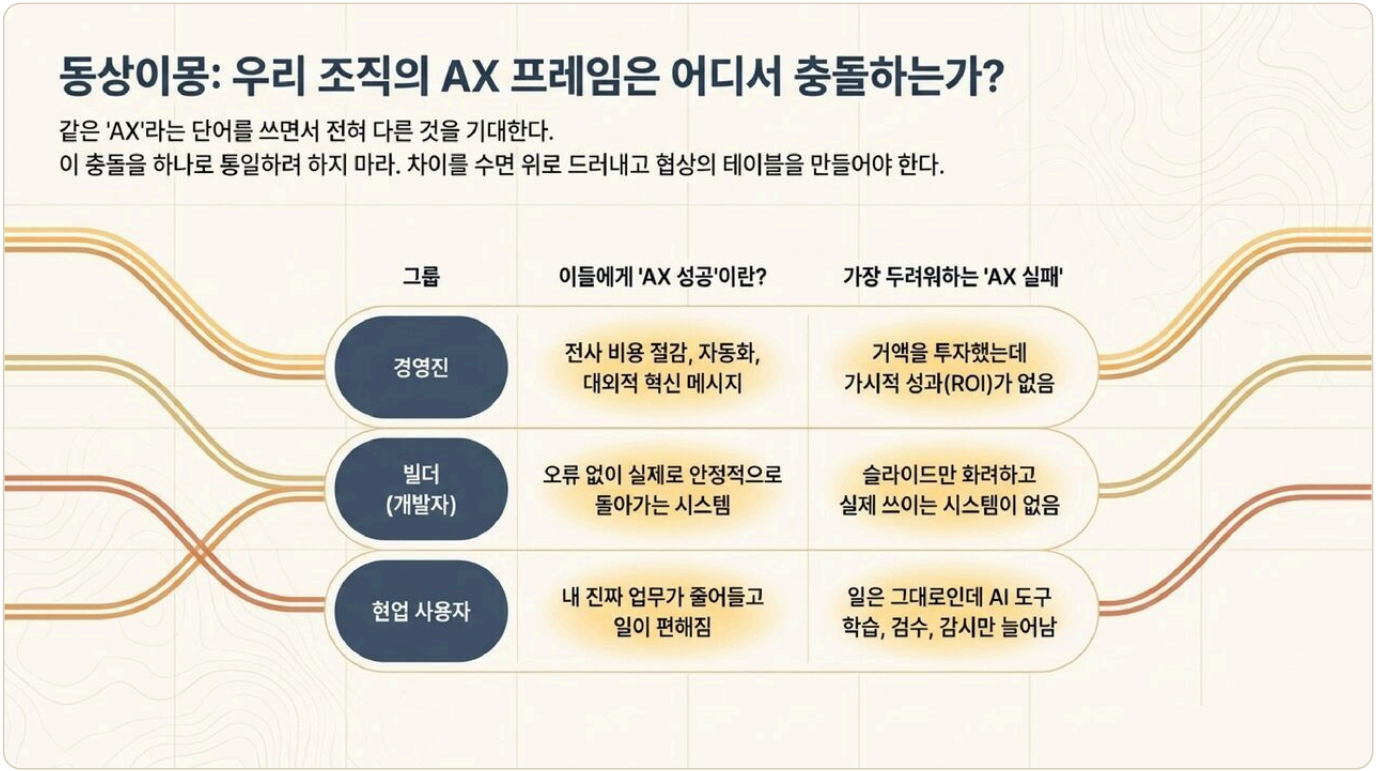


그림 2-4. 동상이몽 — AX 프레임은 어디서 충돌하는가

여기까지는 이론이었다. "다들 만들면서 더듬고 있다"는 말이 그저 위안용 수사는 아닐까? 다행히 이 그림은 여러 실태 조사와도 들어맞는다.

한 대규모 기업 AI 성숙도 조사를 보면, 대부분의 조직은 아직 *국지적 실험* 단계에 머물러 있다.[9] 부서 한두 곳에서, 혹은 의욕 있는 개인 몇이서 이것저것 만들어보는 단계 말이다. 전사 차원에서 AI를 일하는 방식의 중추로 끌어들인 조직은 소수다. 또 다른 조사에서는, AI 성숙도가 높은 축에 드는 조직조차 자기들이 시작한 AI 프로젝트의 절반 가까이만 3년 이상 살아남겨 운영한다고 한다.[10] 나머지는 시작했다가 흐지부지된다는 뜻이다. 잘한다는 조직조차 이 정도다.

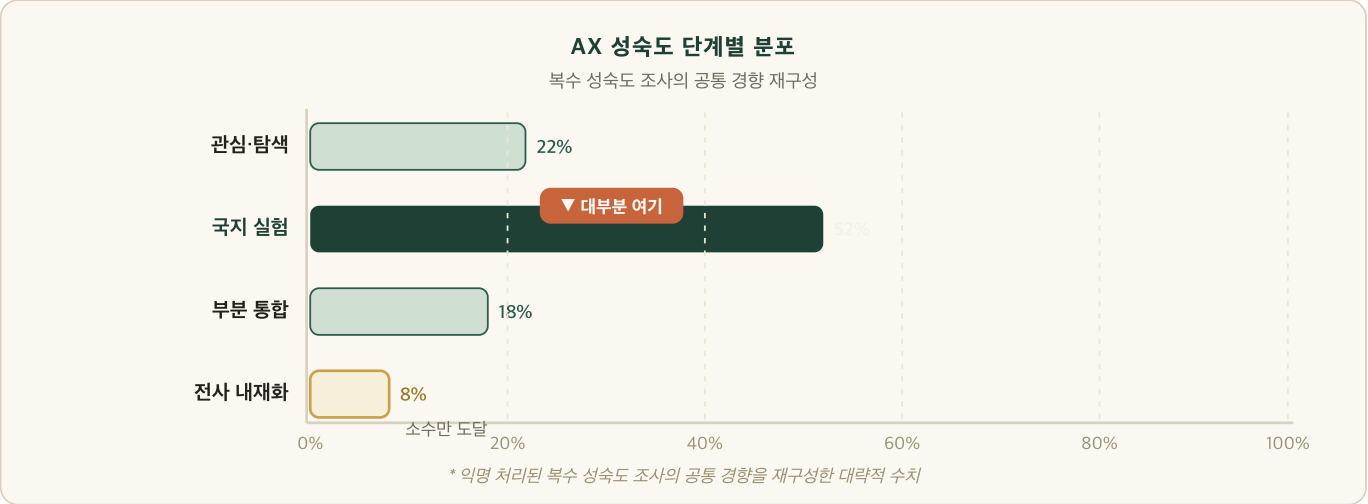


그림 2-2. AX 성숙도 단계 막대

이 막대그래프가 말해주는 건 분명하다. 만약 당신이 "우리는 아직 여기저기서 실험만 하고 있고, 전사적으로 정리된 AX 같은 건 없다"고 느낀다면 — 축하한다. 당신은 정확히 대부분의 조직과 같은 자리에 서 있다. 뒤처진 게 아니라, 그냥 평균이다. 평균보다 앞서는 길은 정리된 청사진을 기다리는 쪽보다, 그 실험을 멈추지 않고 이어가 다음 단계로 밀어 올리는 쪽에 있다.

한 가지 덧붙이자. 이 실태 조사들은 학술 논문이라기보다 기관·업계의 리포트에 가깝다. 그래서 나는 이 숫자들을 단정적인 진리로 들이밀 생각은 없다. 다만 앞서 본 오래된 조직이론 — 조직은 흐르는 강이고, 전략은 떠오르고, 기술은 쓰이면서 구조가 된다는 — 과 이 최근의 실태 조사는 서로 맞물린다. 이론은 "원래 만들면서 만들어지는 것"이라 말하고, 실태는 "실제로 다들 아직 만드는 중"이라고 말한다.

> **[이게 틀어졌을 때]** "원래 만들면서 만들어지는 것"이라는 말을, 자칫 "그러니 아무렇게나 난립시켜도 괜찮다"로 오독하기 쉽다. 나는 그 대가를 직접 치렀다. 떠오른 전략을 믿고 손 가는 대로 만들기만 하고, 만든 것들을 한자리에 펼쳐 점검하는 일을 미뤘더니, 어느새 누구도 전체를 쥐지 못하는 안개가 끼었다. 만드는 것은 자유롭게 하되, *만든 것들을 주기적으로 펼쳐놓고 "이게 모여 무엇이 되고 있나"를 사후에라도 묻는 일*까지 미루면 — 떠오른 전략은 그냥 난립이 되고 만다. 떠오르게 두되, 떠오른 것을 거두는 손은 부지런해야 한다. 그 게으름의 비용을, 나는 며칠간의 추적으로 갚았다.

리더는 여기까지만 읽어도 된다

지금부터는 그 "거꾸로 된 순서"가 실제로 어떻게 굴러갔는지, 만드는 사람의 시선에서 조금 더 들여다보자. 직접 시스템을 짜는 빌더라면 함께 가면 좋겠다.

조직을 이끄는 리더라면, 사실 **여기까지만 읽어도 충분하다.** 바로 다음 절(③ "완성되지 않아도 시작하는 법")로 건너뛰어도 이 장이 말하려는 바는 고스란히 손에 남는다. 가져갈 한 문장은 이것이다 — *AX는 청사진을 받아 칠하는 게 아니라, 만들면서 청사진이 사후에 떠오르는 일이다. 그러니 정의를 기다리지 말고, 정의가 떠오를 자리(=만드는 현장)에 먼저 발을 담그자.* 이 한 줄만 챙겨 ③으로 가도 된다.

슬라이드보다 시스템이 먼저였다

이 장의 ②는 솔직히 얇다. 앞 장에서 이미 시스템을 어떻게 만들었는지를 충분히 이야기했으니, 여기서 코드를 또 펼치진 않겠다. 다만 그 만듦의 *순서*에 대해서만 한 가지 짚고 싶다.

빌더로서 일하다 보면, 슬라이드를 그릴 시간에 일단 동작하는 걸 하나 만들어버리고 싶은 충동이 든다. 솔직히 나는 거의 매번 그 충동에 졌다. 표준 문서를 먼저 쓰는 대신 에이전트를 먼저 만들었고, 정책을 먼저 정하는 대신 일단 돌려보고 깨지는 걸 봤다. 한동안 나는 이게 내 나쁜 버릇이라고 여겼다. 제대로 된 사람은 설계 문서부터 쓴다는데, 나는 늘 코드부터 쳤으니까. 좀 부끄러웠다.

그런데 이 장을 쓰면서 생각이 좀 바뀌었다. 동작하는 무언가는 그 자체로 가장 정직한 사양서다. 슬라이드 위의 표준은 누구도 반박하지 못한 채 아름답지만, 막상 돌려보면 거기 적힌 절반은 현실에서 작동하지 않는다. 반대로 일단 만들어 돌려본 시스템은, 무엇이 빠졌고 무엇이 위험한지를 *스스로 비명을 질러* 알려준다. 1장의 신분증 다섯 칸이 왜 하필 그 다섯 칸인지, 위험등급을 왜 게이트와 묶어야 하는지 — 그 모든 결정은 슬라이드가 아니라 깨지는 시스템이 가르쳐준 것이다.

그래서 빌더에게 남기고 싶은 한 줄은 이렇다. **먼저 만들고, 만든 것이 비명을 지르게 하고, 그 비명을 받아 적은 것이 곧 당신의 떠오른 전략이다.** 슬라이드는 그 다음에 그려야 현실과 맞는 슬라이드가 나온다. 자, 이제 모두 다시 합류해 당신의 현장으로 가보자.

완성되지 않아도 시작하는 법

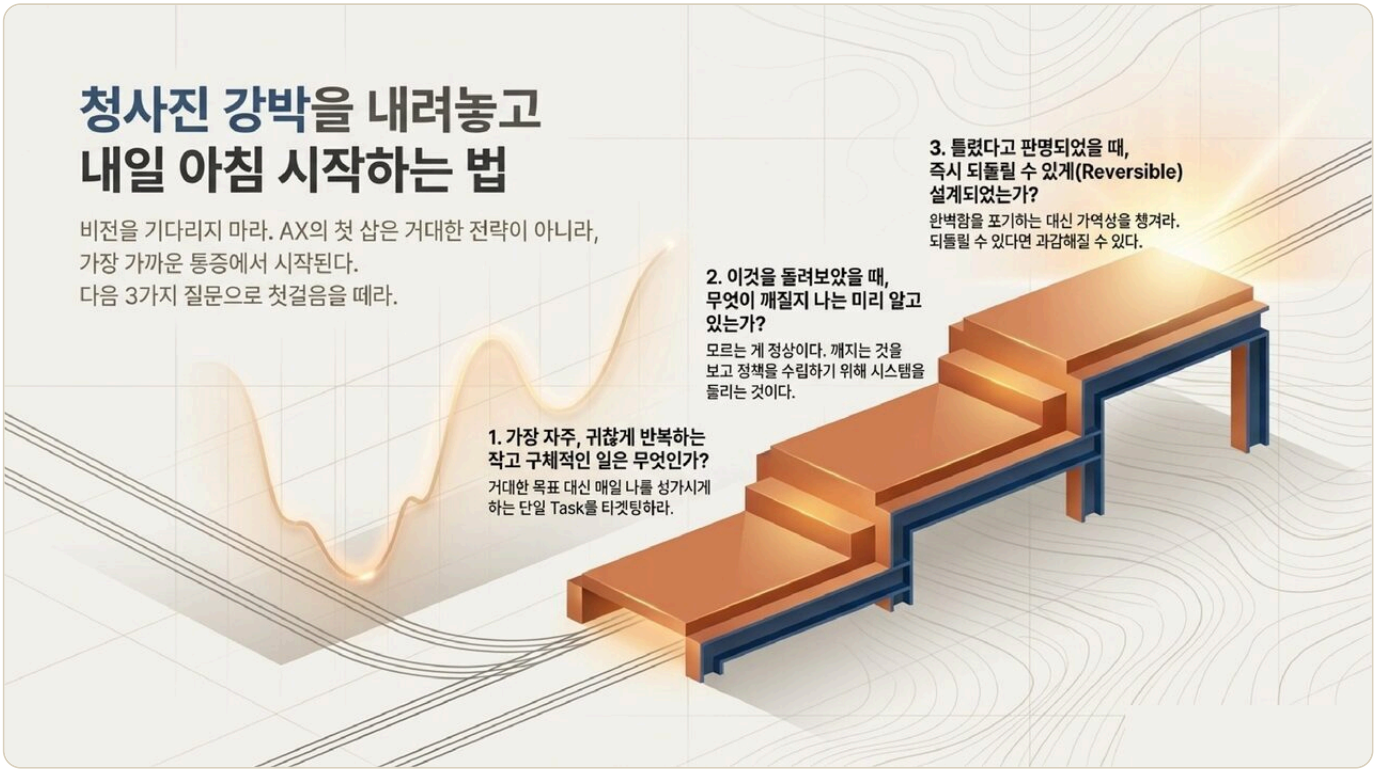


그림 2-5. 청사진 강박을 내려놓고 시작하는 3단계

여기까지 왔으면 한 가지는 분명해졌을 것이다. 청사진을 손에 쥐고 시작하는 일은 일어나지 않는다. 그러니 익혀야 할 기술은 '완벽한 계획을 세우는 법'보다 '완성되지 않은 채로 시작하는 법'이다. 이걸 당신의 현장으로 옮길 두 가지 도구로 정리해보자.

청사진 강박을 내려놓는 세 가지 질문

완벽한 그림이 나올 때까지 못 움직이겠다는 마음이 들 때, 스스로에게 다음 세 질문을 던져보자. 이 질문들은 거창한 전략 대신 *다음 한 걸음*만 묻게 해준다.

- **"지금 가장 자주, 가장 귀찮게 반복하는 일 하나는 무엇인가?"** — AX의 첫 삽은 거창한 비전에서 뜨지 않는다. 가장 가까운 통증에서 뜬다. 그러니 거대한 일을 고르지 말자. 매일 나를 성가시게 하는 작은 일 하나면 충분하다. 거기서 만들기 시작하면, 만들면서 다음에 무엇을 만들지가 보인다.
- **"이걸 만들어 돌려보면, 무엇이 깨질지 나는 미리 다 아는가?"** — 만약 다 안다고 느낀다면, 아직 충분히 작게 시작하는 게 아닐 가능성이 높다. 모르는 게 정상이다. 우리가 만드는 이유는 깨지는 걸 봐서 배우기 위해서다. "깨져봐야 알겠다"가 부끄러운 답이 아니라 정직한 답이다.
- **"6개월 뒤에 이게 틀렸다고 판명되면, 지금 나는 그걸 되돌릴 수 있게 만들고 있는가?"** — 완성을 포기하는 대신 우리가 챙겨야 할 건 *되돌릴 수 있음*이다. 청사진이 없어도 괜찮은 이유는, 틀리면 고칠 수 있기 때문이다. 그러니 완벽함 대신 가역성을 설계하자. 이게 청사진 없이도 과감해질 수 있는 유일한 안전장치다.

세 질문의 공통점이 보이는가. 어느 것도 "AX의 완성된 정의"를 묻지 않는다. 모두 다음 한 걸음과 그 걸음을 되돌릴 수 있는지만 묻는다. 정의는 그 걸음들이 쌓이면 사후에 떠오른다.

우리 조직의 AX 프레임은 지금 어디서 충돌하는가

한 가지 더. 1장 첫머리에서 컨설턴트와 임원과 옆 팀이 저마다 다른 AX를 말한다고 했다. 이게 단순한 의견 차이가 아니라는 걸 이제 우리는 안다. 같은 기술을 두고 집단마다 다른 그림을 그리는 건, 의미가 아직 닫히지 않은 초기 기술에서 당연한 일이다. 그런데 이 서로 다른 그림들은, 방치하면 조용한 갈등의 씨앗이 된다.[11] 같은 'AX'라는 단어를 쓰면서 서로 완전히 다른 것을 가리키고 있으면, 회의는 길어지고 합의는 헛돌고 누구도 왜 어긋나는지 모른다.

그래서 청사진을 그리는 대신, 차라리 **지금 우리 조직 안에 어떤 AX 그림들이 충돌하고 있는지를 펼쳐놓는 진단**이 훨씬 쓸모 있다. 다음을 한 장에 적어보자.

구분	이들에게 'AX 성공'이란	가장 두려워하는 실패
경영진	(예: 전사 비용 절감·대외 메시지)	(예: 투자 대비 성과 없음)
만드는 사람(빌더)	(예: 실제로 돌아가는 시스템)	(예: 슬라이드만 남고 아무것도 안 돎)
현업 사용자	(예: 내 일이 실제로 편해짐)	(예: 일이 늘거나 감시당함)

이 표를 채워보면, 같은 'AX'라는 말 아래 얼마나 다른 기대가 깔려 있는지가 한눈에 드러난다. 이 그림들을 하나로 통일하려 들 필요는 없다. 초기 기술에서는 가능하지도 않다. 할 일은 *서로 다르다는 사실을 드러내 놓고 대화하는 것*이다. 충돌을 숨기면 갈등이 되고, 펼쳐놓으면 협상이 된다.

그리고 1장에서 심어둔 관통 주제 하나를 여기서 다시 만나자. 이 모든 그림이 충돌하더라도, 끝내 "이 일의 최종 책임은 누구인가"라는 질문만은 흐릿하게 두지 말자. 정의가 떠오르는 중이라 모든 게 유동적일수록, 책임의 자리만은 사람에게 또렷이 묶어두자. 흐르는 강 위에서도 닻 하나는 박아두어야 하니까.

마무리

"AX가 뭔지 아직 모르겠다"는 건, 내가 뒤처진 탓인가 아니면 원래 그런 것인가?

이제 답할 수 있다. 원래 그런 것이다. 조직은 도달할 정상이 있는 산이 아니라 매일 다시 흐르는 강이고, 전략은 슬라이드에서 완성되는 게 아니라 만들면서 사후에 떠오르며, 기술은 깔린다고 끝이 아니라 쓰이면서 비로소 구조가 된다. 의미가 사람마다 다른 것도 마찬가지다. 그건 덜 여문 탓이 아니라, 모든 새 기술이 한 번씩 거쳐 가는 통과의례다. 그러니 설계도 없이 만들면서 더듬은 그 순서는 거꾸로가 아니었다. 그게 유일하게 가능한 순서였다.

나는 이 장을 쓰면서, 그 목록 앞에서 느꼈던 자책을 비로소 내려놓을 수 있었다. 무능해서 청사진을 못 그린 게 아니었다. 청사진은 원래 만들기 전에는 그려지지 않는다. 내가 겪은 그 난립도 실패가 아니었다. 1장의 표준을 낳은 산통이었다. "building the plane while flying it"은 변명 같은 고백이 아니다. 새로운 무언가를 만드는 사람이라면 누

구나 취하게 되는, 가장 정직한 자세다.

그러니 당신에게도 같은 말을 건네고 싶다. **당신은 뒤처진 게 아니다.** AX가 한 문장으로 안 잡힌다고, 우리 조직엔 정리된 청사진이 없다고 조바심 내지 말자. 그건 오히려 당신이 정확히 시대의 한복판에 서 있다는 증거다. 완성을 기다리지 말자. 가장 가까운 통증 하나에서, 작게, 되돌릴 수 있게 시작해보자. 정의는 그 걸음들 뒤에서 천천히 떠오를 테니까.

그런데 막상 첫 삽을 뜨고 나면, 곧바로 새로운 종류의 신남과 마주하게 된다. 에이전트 하나를 들이고 나면 욕심이 생긴다. "하나가 이만큼 하는데, 여럿이 손발을 맞추면 어디까지 갈까?" 한 사람이 혼자서 기획하고, 쓰고, 검수하고, 발행까지 — 마치 한 명이 팀 하나만큼의 일을 해내는 그림 말이다.

다음 장에서 그 이야기를 해보자. "한 사람이 팀이 된다"는 말이 정말 한 명이 다 한다는 뜻인지, 아니면 전혀 다른 뜻인지. 미리 한 가지만 귀띔하자면 — 그건 한 명이 다 하는 게 아니라, 한 명이 여러 역할로 *나뉘는* 이야기다. 그리고 그 나눔의 한복판에 의외의 그림자가 하나 숨어 있다.

PART 2. 일의 재정의 — 한 사람이 팀이 되는 법

> 첫 삽을 떴으면, 일하는 방식을 다시 본다. PART 2는 한 사람이 여러 역할로 나뉜 에이전트 팀을 거느리는 법(3장)과, 무엇을 AI에게 넘기고 무엇을 사람이 쥘지 task 단위로 경계를 긋는 법(4장)을 다룬다.
>
> *J-curve 위치: ③신남의 절정 — 그 끝에서 "검수자를 믿어도 되나"라는 첫 그림자가 진다.*

3장. 한 사람이 팀이 되다

> **시작하기 전에, 한 가지**
>
> 이 장이 끝까지 붙들고 가는 물음은 하나다. "한 사람이 팀이 된다"는 말이 한 명이 다 한다는 뜻인지, 아니면 다른 뜻인지.
>
> 미리 답을 흘려두자면, 슈퍼맨을 만드는 이야기는 아니다. 역할을 나눠 맡긴 에이전트 팀을 거느리는 이야기다. 한 명에게 다 시키면 품질이 무너지고, 작가·검수자·발행자로 자리를 쪼개야 비로소 돌아간다. 그 모든 원칙 중 끝까지 남는 한 줄은 이것이다. *작가와 검수자를 같은 자리에 두지 마라.*

> 마지막으로 한 가지. 이 장은 J-curve의 ③ *신남* 구간에 해당한다. 혼자 하던 일이 팀으로 돌아가는 쾌감의 자리다. 그런데 그 쾌감의 끝에서, 다음 막막함의 첫 그림자가 진다. 검수자가 AI라면, 그 판단을 어디까지 믿어도 되는가. 신난 채로 읽되, 그 그림자 하나는 눈여겨봐 두자.

딸을 글쓰기 학원에 내려주고 늘 앉던 카페 모퉁이에 자리를 잡은 어느 날, 문득 이상한 기분이 들었다. 딸은 안에서 선생님 한 분에게 글쓰기를 배우고 있다. 한 사람이 가르치고, 한 사람이 배운다. 그런데 나는 같은 시각, 노트북 하나를 열어놓고 몇이서 일하고 있었다. 분명 자리에 앉은 건 나 혼자인데, 화면 안에서는 누구는 자료를 모으고, 누구는 글을 쓰고, 누구는 그 글을 뜯어보고 있었다. 혼자인데 혼자가 아니었다.

"한 사람이 팀이 된다"는 말을 처음 들었을 때, 나는 그게 무슨 영웅담인 줄 알았다. AI를 잘 쓰면 한 명이 열 명 몫을 해낸다는, 그런 슈퍼맨 이야기. 그런데 막상 그 일을 직접 겪어보니 결이 달랐다. 정작 내가 배운 건 혼자 다 하는 요령보다 욕심을 내려놓는 법이었다. 일을 잘게 나눠 여러 손에 맡기고, 그 손들이 따로따로 제 몫을 하게 두는 법. 다만 그 '여러 손'이 사람이 아니라 에이전트였을 뿐이다.

혼자 다 하려다 무너지고, 일을 나눠 맡기고 나서야 돌아가기 시작한 그 과정을 함께 더듬어보자.

거대한 한 명보다, 역할을 나눈 여럿이 낫다

내 이야기를 꺼내기 전에, 우리가 지금 어디쯤 서 있는지 좌표부터 잡아보자. 한 사람이 AI로 여러 일을 하려 할 때, 길은 크게 두 갈래로 갈린다.

첫 번째 길은 직관적이다. 똑똑한 AI 하나에게 일을 통째로 맡기는 것이다. "이런 주제로 자료를 모으고, 잘 정리해서, 글을 쓰고, 어색한 데가 없는지 검토한 다음, 정해진 형식으로 발행해줘." 한 번에 이 모든 걸 시키는, 거대한 하나의 지시문. 만들기는 제일 쉽다. AI 하나에 말만 길게 붙이면 되니까. 처음엔 누구나 이 길로 들어선다. 나도 그랬다.

두 번째 길은 조금 더 손이 간다. 일을 역할로 쪼개고, 역할마다 따로 에이전트를 두는 것이다. 자료를 고르는 에이전트, 글을 쓰는 에이전트, 그 글을 검수하는 에이전트, 발행하는 에이전트 — 이렇게 나누고, 한 에이전트의 결과물을 다음 에이전트에게 넘긴다. 이걸 흔히 **오케스트레이터-워커(orchestrator-worker)** 구조라고 부른다. 지휘자 격인 누군가가 전체 흐름을 잡고, 일꾼 격인 에이전트들이 각자 맡은 한 가지를 한다는 뜻이다.[12]

어느 쪽이 나을까? 직관적으로는 첫 번째가 나아 보인다. 하나로 끝나니 깔끔하지 않은가. 그런데 실제로 돌려보면 이야기가 정반대다. 한 거대 기술 기업이 자사의 멀티에이전트 시스템을 두고 공개한 회고에 따르면, 역할을 분리한 여러 에이전트가 거대한 단일 에이전트보다 내부 평가에서 90% 넘게 높은 성능을 보였다고 한다.[13] 90%가 넘는 차이라니, 작은 차이가 아니다. 일을 나눴을 뿐인데 결과물의 질이 통째로 달라진 것이다.

왜 이런 일이 벌어질까? 여기서 잠깐 다른 그림을 빌려와 보자. 신문사 편집국을 떠올리면 이해가 쉽다. 신문은 한 사람이 처음부터 끝까지 만들지 않는다. **편집자가 무슨 기사를 쓸지 고르고, 기자가 그 기사를 쓰고, 데스크가 그 기사를 검증한다.** 각자 한 가지 역할에만 집중한다. 만약 이 셋을 한 사람에게 다 시키면 어떻게 될까? 자기가 고른 주제를, 자기가 쓰고, 자기가 검토하게 된다. 그러면 자기 글의 허점이 잘 안 보인다. 사람도 그런데, AI는 더하다.[14]

이 신문사 비유가 멀티에이전트를 설명하는 가장 좋은 그림이라고 나는 생각한다. 역할을 나누면 일이 잘게 잘릴 뿐 아니라, **각 역할이 서로를 견제하게** 된다. 작가는 좋은 글을 쓰는 데만 몰두하고, 검수자는 그 글의 흠을 잡는 데만 몰두한다. 둘의 이해관계가 다르기 때문에, 한 사람이 다 할 때는 보이지 않던 문제가 드러난다.

나누는 데는 실용적인 이점도 하나 더 있다. 역할마다 에이전트가 따로 있으면, 한 부분이 마음에 안 들 때 그 부분만 손보면 된다. 검수가 영 시원찮으면 검수 에이전트만 갈아끼우고, 글이 밋밋하면 작가 에이전트만 손본다. 나머지는 건드릴 필요가 없다.[15] 반대로 거대한 하나의 지시문은, 어디 한 군데를 고치려 들면 전체가 흔들린다. "검수를 좀 더 깐깐하게"라고 한 줄 추가했더니 멀쩡하던 글쓰기까지 이상해지는 일이 벌어진다. 길고 복잡한 지시문일수록 어디가 어디에 영향을 주는지 추적이 안 된다. 통제 불능이 되는 것이다.

요즘 이런 식으로 일하는 한 사람을 두고 **슈퍼워커(superworker)**라는 말을 쓰기도 한다. AI를 입은 한 사람이 예전에 한 팀이 하던 산출물을 혼자 낸다는 뜻이다.[16] 말은 그럴듯한데, 한 가지는 분명히 해두자. 슈퍼워커는 한 명이 *열 명만큼 일하는* 사람이라기보다, 한 명이 *열 명짜리 팀을 지휘하는* 사람에 가깝다. 그 사람이 직접 하는 일은 오히려 줄어든다. 대신 누구에게 무엇을 맡길지, 그 결과를 어떻게 검증할지를 설계하는 일이 늘어난다.

혼자 다 시키려다, 품질이 무너졌다

좌표를 잡았으니 이제 내 이야기를 해보자. 머리로는 "역할을 나누는 게 낫다"는 걸 알아도, 막상 손을 대면 누구나 첫 번째 길로 들어선다. 나도 예외가 아니었다.

내가 만들려던 건 콘텐츠를 자동으로 만들어내는 파이프라인이었다. 흩어진 자료에서 그날그날 다룰 만한 주제를 골라, 글을 한 편 쓰고, 다듬어서, 여러 채널에 발행하는 일. 원래는 여러 사람이 나눠 하던 일이다. 누군가는 트렌드를 살피고, 누군가는 글을 쓰고, 누군가는 검토하고, 누군가는 올린다. 이걸 나 혼자, 에이전트의 힘을 빌려 돌려보고 싶었다.

처음 떠올린 방식은 당연히 가장 단순한 것이었다. AI 하나에게 다 시키는 것. "자료를 보고 주제를 골라서, 그 주제로 글을 쓰고, 어색한 데를 다듬은 다음, 발행 형식으로 정리해줘." 한 호흡에 모든 걸 담은 긴 지시문을 만들었다. 처음 몇 번은 정말 그럴듯했다. 자료를 던지니 글이 한 편 뚝 떨어졌다. 신기하고 신났다. "이거 되네?" 싶었다.

그런데 며칠 돌려보니 슬슬 뒷맛이 이상해졌다. 글은 나오는데, 품질이 들쭉날쭉했다. 어떤 날은 멀쩡한데 어떤 날은 주제 선정부터 엉뚱했다. 더 큰 문제는 무엇이 잘못됐는지 짚어낼 수가 없다는 점이었다. 주제를 잘못 고른 건지, 글을 잘못 쓴 건지, 검토를 안 한 건지 — 한 덩어리로 뭉쳐 나오니 어디서 어긋났는지가 안 보였다. 마치 한 사람이 눈감고 처음부터 끝까지 휘갈겨 쓴 글을 받아 든 기분이었다.

특히 검토가 문제였다. 같은 지시문 안에서 "글을 쓰고, 그 글을 검토하라"고 시켰더니, AI는 자기가 방금 쓴 글을 보고 늘 "좋습니다"라고 했다. 당연하다. 자기가 쓴 글 아닌가. 사람도 자기 글의 오타는 잘 못 본다. AI도 똑같았다. 작가와 검수자를 한 자리에 앉혀놓으니, 검수가 사실상 없는 것이나 마찬가지였다.

이쯤에서 나는 멈춰서 생각했다. "한 사람이 팀이 된다더니, 나는 지금 한 사람한테 *팀이 하던 일을 통째로* 떠넘기고 있는 거 아닌가?" 그게 함정이었다. 한 명이 다 하는 건 사람이든 AI든 무리다. 혼자 다 잘하는 만능 일꾼을 만들려는 욕심 — 그게 품질을 무너뜨리고 있었다.

"한 사람이 팀이 된다"는 말의 뜻이 그제야 손에 잡혔다. 슈퍼맨이 되라는 말이 아니라 *팀을 꾸리라*는 말이었다. 내가 만능일 필요는 없었다. 좋은 팀을 짜고, 각자에게 한 가지씩 맡기면 그걸로 됐다.

> **[이게 틀어졌을 때]** 역할을 안 나누고 거대한 하나에 다 맡겼을 때의 대가는, 사실 품질만이 아니었다. 더 무서운 건 *디버깅 불가능*이었다. 결과가 이상해도 어디를 고쳐야 할지 알 수가 없으니, 지시문을 통째로 이리저리 바꿔보며 운에 맡기는 수밖에 없었다. 한 줄 바꾸면 엉뚱한 데가 또 망가졌다. 며칠을 그렇게 헛돌았다. 그 시간 동안 나를 가장 지치게 한 건 "왜 안 되는지를 모른다"는 막막함이었다. 나중에 역할을 나누고 나서야 깨달았다 — 애초에 나누지 않았기 때문에 고칠 수 없었던 것이다. 나누는 일은 품질을 위한 것이기도 하지만, 무엇보다 *고칠 수 있게* 만드는 일이었다.

리더는 여기까지만 읽어도 된다

여기서부터는 이 파이프라인을 실제로 어떻게 짰는지, 한 단계 더 내려가 코드와 절차의 높이에서 들여다보자. 직접 시스템을 만드는 빌더라면 끝까지 함께 가면 좋겠다. 자기 현장에 바로 옮길 수 있는 절차와, 내가 깨진 자리들이 거기 있다.

하지만 조직을 이끄는 리더라면, 사실 **여기까지만 읽어도 충분하다.** 바로 뒤의 절(③ "내 일을 1인 파이프라인으로")로 건너뛰어도 이 장의 알맹이는 빠짐없이 챙긴 셈이다. 기억해야 할 단 하나는 이것이다 — *한 사람이 팀이 된다는 건 슈퍼맨 한 명이 아니라, 역할을 나눠 맡긴 에이전트 팀을 거느린다는 뜻이고, 그중 가장 중요한 원칙은 작가와 검수자를 같은 자리에 두지 않는 것이다.* 이 문장 하나면 ③에서 다시 만나도 충분하다.

자, 그럼 빌더들과 함께 조금 더 깊이 들어가보자.

작가, 검수자, 발행자 — 세 자리로 나누다

거대한 하나를 버리고, 일을 역할로 쪼갰다. 최종적으로 자리 잡은 흐름은 이렇다. 먼저 흩어진 자료를 **모으고**, 그중에서 다룰 만한 **주제를 고른다.** 그다음부터가 핵심인데, 글을 짓는 **작가**, 그 글을 뜯어보는 **검수자**, 통과한 글을 정해진 형식으로 내보내는 **발행자** — 이 셋을 각각 다른 에이전트로 두었다. 그리고 발행자가 여러 채널로 글을 **내보낸다.** 신문사의 편집국을 거의 그대로 옮긴 셈이다.

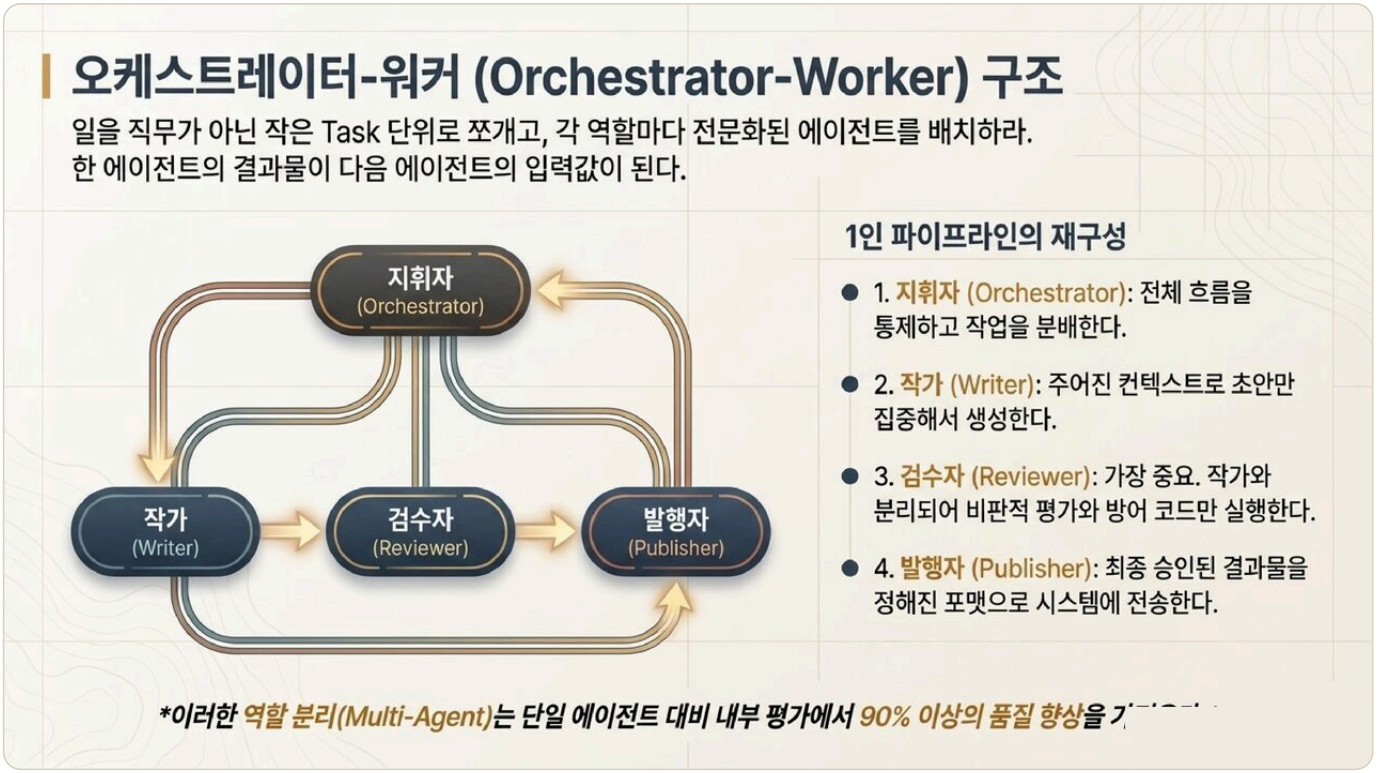


그림 3-1. Writer→Reviewer→Publisher 파이프라인도

놀라운 건, 이 전체를 이틀 만에 끝에서 끝까지 돌아가게 만들었다는 점이다. 자랑하려는 게 아니다. 역할을 나누고 나니 *오히려 빨리* 완성됐다는 이야기를 하려는 것이다. 거대한 하나를 붙들고 씨름할 때는 며칠을 헛돌았는데, 일을 잘게 나누니 각 조각은 단순해서 만들기 쉬웠고, 조각을 잇는 일도 명료했다.

물론 이틀 만에 *돌아가게* 만든 것과, 그게 *멀쩡히 돌아가는* 것은 다른 이야기다. 그 사이의 거리를 메우는 동안 나는 몇 군데에서 제대로 깨졌다.

모델은 갈아끼우는 부품이다

첫 번째로 깨진 자리는 모델이었다.

처음엔 모델 하나를 정해놓고 모든 에이전트를 그 위에 올렸다. 그런데 며칠을 못 갔다. 어떤 날은 호출 한도에 걸려 멈췄고, 어떤 날은 같은 입력에 빈 값을 뱉었고, 비용도 생각보다 빨리 불었다. 그래서 모델을 갈아탔다. 그랬더니 이번엔 다른 데서 탈이 났다. 또 갈아탔다. 그러길 몇 번. 모델을 바꾸는 게 어느새 일상이 됐다.

여기서 한 가지를 뼈저리게 배웠다. **모델은 갈아끼우는 부품이다.** 특정 모델 하나에 시스템을 단단히 묶어버리면, 그 모델에 무슨 일이 생기는 순간 시스템 전체가 같이 흔들린다. 더 빠른 모델이 나와도 못 갈아탄다. 값이 올라도 도망갈 데가 없다. 그러니 처음부터 *언제든 모델을 바꿔 끼울 수 있게* 만들어두자. 어떤 에이전트가 어떤 모델을 쓰는지를 한곳에 모아두고, 한 줄만 바꾸면 모델이 교체되도록 설계하는 것이다.

이게 왜 중요한지 곱씹어보자. 우리는 흔히 모델을 시스템의 *심장*처럼 여긴다. "무슨 모델을 쓰느냐"가 전부인 것처럼. 솔직히 나도 한동안 그랬다. 어떤 모델이 제일 똑똑한지를 비교하는 데 며칠을 썼다. 그런데 막상 만들어 운영해 보면 그 비교가 무색해진다. 모델은 좋다가도 한도에 걸리고, 값이 오르고, 어느 날 갑자기 응답이 이상해진다. 따져 보니 중요한 건 어떤 모델이 가장 낫느냐보다, 어떤 모델이 와도 시스템이 견디느냐였다. 시스템을 떠받치는 진짜 골격은 모델이 아니라, 그 모델들을 어떻게 엮고 무엇을 시키고 어떻게 검증하느냐 하는 *설계*에 있다.

긴 작업은 잘게 쪼개라

두 번째로 깨진 자리는 시간이었다. 정확히는, 시간 제한이었다.

내가 시스템을 올린 환경에는 한 가지 제약이 있었다. 작업 하나가 정해진 시간 — 가령 1분 — 을 넘기면 강제로 끊겼다. 그런데 자료를 모으고, 주제를 고르고, 글을 쓰고, 검수하고, 발행하는 이 전체를 한 호흡에 돌리려니 그 시간을 훌쩍 넘겼다. 다 끝나기도 전에 뚝 끊기는 것이다. 처음엔 황당했다. "아니, 일을 다 하지도 못했는데 왜 끊겨?"

처방은 단순하면서도 본질적이었다. **긴 작업 하나를, 짧은 작업 여럿으로 쪼개는 것.** 원래 두 덩어리였던 흐름을 네 덩어리로 잘랐다. 자료 수집은 자료 수집대로, 글쓰기는 글쓰기대로, 각각이 시간 제한 안에 끝나는 작은 단위가 되도록. 그리고 한 단계가 끝나면 그 결과를 어딘가에 적어두고, 다음 단계가 그걸 이어받아 시작하게 했다. 마치 릴레이 경주에서 바통을 넘기듯이.

솔직히 처음엔 이 제약이 그냥 짜증났다. "왜 한 번에 못 하게 막는 거야?" 환경 설정을 뒤져 제한을 늘릴 방법부터 찾았다. 없었다. 어쩔 수 없이 쪼갰다.

그런데 쪼개고 나니 뜻밖의 게 따라왔다. 중간에 한 단계가 실패해도, 처음부터 다시 할 필요가 없어졌다. 글쓰기에서 넘어졌으면 거기서부터 다시 시작하면 됐다. 단계마다 어디까지 됐는지가 기록으로 남으니 추적도 쉬웠다. 제약이 오히려 더 튼튼한 구조를 강제한 셈이다. 돌이켜보면 이건 시간 제한이 없었어도 했어야 할 일이었다.

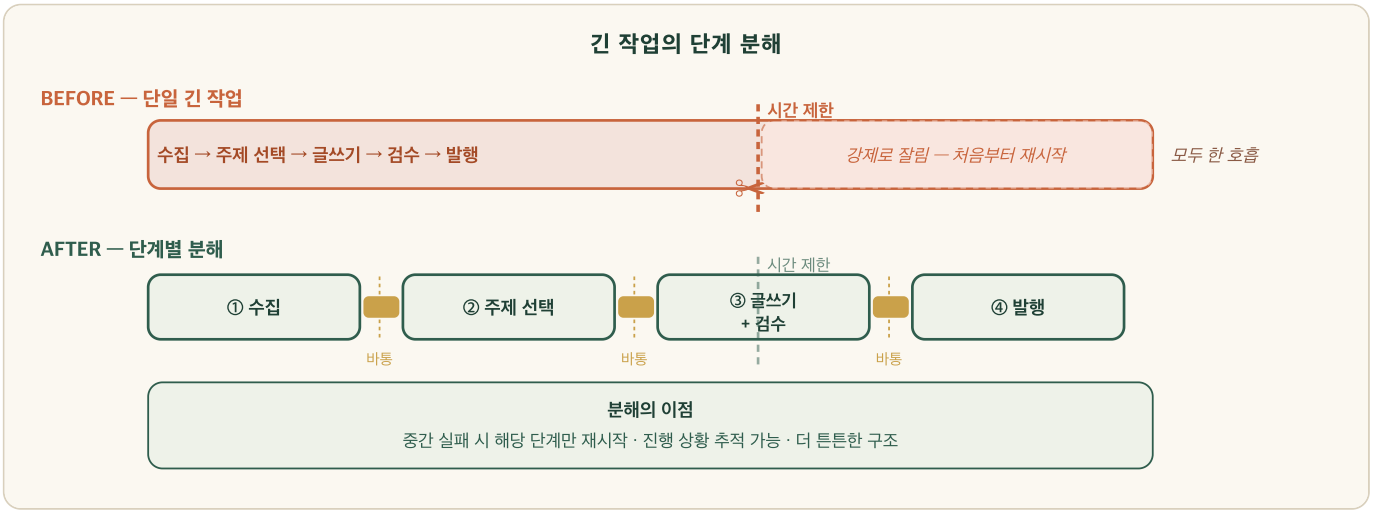


그림 3-2. 긴 작업의 단계 분해

LLM 출력은 깨진다 — 방어 코드가 본체다

세 번째로 깨진 자리는, 가장 자주 그리고 가장 얄밉게 깨졌다.

에이전트들끼리 일을 주고받으려면, 한 에이전트의 결과를 다음 에이전트가 알아볼 수 있는 정해진 형식으로 넘겨야 한다. 그래서 나는 작가 에이전트에게 "결과를 이런 구조로 정리해서 내놔"라고 형식을 정해줬다. 대부분은 잘 지켰다. 그런데 *대부분*이 문제였다. 열 번에 한 번쯤, 형식이 어긋난 채로 결과가 나왔다. 닫혀야 할 괄호가 안 닫혀 있거나, 결과가 중간에 뚝 잘려 있거나, 엉뚱한 설명이 형식 앞에 붙어 있거나.

한 에이전트의 출력이 형식에서 어긋나는 순간, 그걸 받아 읽으려던 다음 에이전트가 그 자리에서 고꾸라졌다. 도미노처럼 파이프라인 전체가 멈췄다. 잘 돌다가도 열 번에 한 번씩 와르르 무너지니, 이만저만 찜찜한 게 아니었다. "왜 어제는 됐는데 오늘은 안 되지?" — 이런 종류의 버그가 가장 사람을 지치게 한다.

여기서 내가 내린 결론은 다소 단호하다. **LLM의 출력은 언제든 깨질 수 있다고 전제하고, 그 깨짐을 받아내는 방어 코드를 본체로 삼는다.** AI가 항상 형식을 잘 지킬 거라고 믿는 대신, *AI는 형식을 자주 어긴다*고 전제하고 짜는 게 안전하다. 결과를 받으면 곧이곧대로 믿지 말고, 형식이 맞는지 한 번 검사하고, 어긋났으면 고쳐서라도 읽어내고, 그래도 안 되면 깔끔하게 한 번 더 시켜본다. 잘린 결과는 잘린 데까지라도 복구해보고, 형식 앞뒤에 붙은 군더더기는 떼어낸다.

막상 짜고 보니 이런 방어 코드는 화려할 게 없었다. 형식이 맞는지 검사하는 줄, 안 맞으면 다시 시키는 줄, 잘린 데까지라도 살려보는 줄. 한 줄 한 줄이 다 시시했다. 처음엔 이게 군더더기처럼 느껴졌다. "AI가 잘 해주면 다 필요 없는 거 아냐?" 그런데 운영해보니 정반대였다. 이 시시한 코드가 시스템을 *돌아가게* 만드는 진짜 본체였다. 화려한 AI 기능은 잘 될 때나 화려하고, 시스템을 24시간 멈추지 않게 지탱하는 건 "깨질 걸 알고 대비해둔" 그 수수한 코드였다. 그래서 나는 이렇게 말하곤 한다 — *AI를 부르는 코드는 절반이고, 그 출력을 의심하고 방어하는 코드가 나머지 절반이다.*

검수는 작가에게서 떼어내, 따로 앉혀라

네 번째 자리는, 사실 이 장 전체에서 가장 하고 싶은 이야기다.

앞에서 거대한 하나에 다 맡겼을 때 검수가 형식만 남았다고 했던 걸 기억할 것이다. 작가에게 "네가 쓴 걸 네가 검토해"라고 시켰더니 늘 "좋습니다"만 돌아오던 그 문제. 이걸 어떻게 풀었느냐 하면 — 간단하다. **검수자를 작가에게서 떼어내, 아예 따로 앉혔다.** 글을 쓰는 에이전트와, 그 글을 검수하는 에이전트를 별개로 둔 것이다. 검수 에이전트는 작가가 무슨 생각으로 그 글을 썼는지 모른다. 그냥 완성된 글을 받아 들고, 흠을 잡는 일에만 집중한다.

이 작은 변화가 품질을 눈에 띄게 끌어올렸다. 자기가 쓴 글은 누구나 후하게 본다. 들인 정성이 아까워서, 의도를 알기 때문에, 고치기 귀찮아서 — 이유는 많다. 그런데 남이 쓴 글은 다르다. 사정을 모르니 결과만 냉정하게 본다. 검수자를 따로 두는 건, AI에게 이 *남의 눈*을 만들어주는 일이었다.

그래서 원칙 하나를 단단히 세웠다. **작가와 검수자를 같은 자리에 두지 마라.** 사람이든 에이전트든 마찬가지다. 만드는 자와 검사하는 자가 같으면, 검사는 통과 도장으로 전락한다. 검수가 의미를 가지려면, 검수하는 쪽이 만든 쪽으로부터 충분히 떨어져 있어야 한다.

여기서 검수 에이전트에 한 가지 장치를 더 달았다. 검수자가 "이건 별로다"라고 퇴짜를 놓으면, 그 피드백을 들고 작가에게 다시 보내 글을 고치게 했다. 검수 → 재작성 → 다시 검수로 도는 루프다. 글이 통과될 때까지 이 고리를 돈다. 다만 — 여기서 멈추지 않고 한 가지 안전장치를 더 두었는데, 이 이야기는 잠시 뒤 ③에서 따로 하겠다. 검수자를 AI로 두는 일에는, 생각보다 조심해야 할 그림자가 하나 숨어 있기 때문이다.

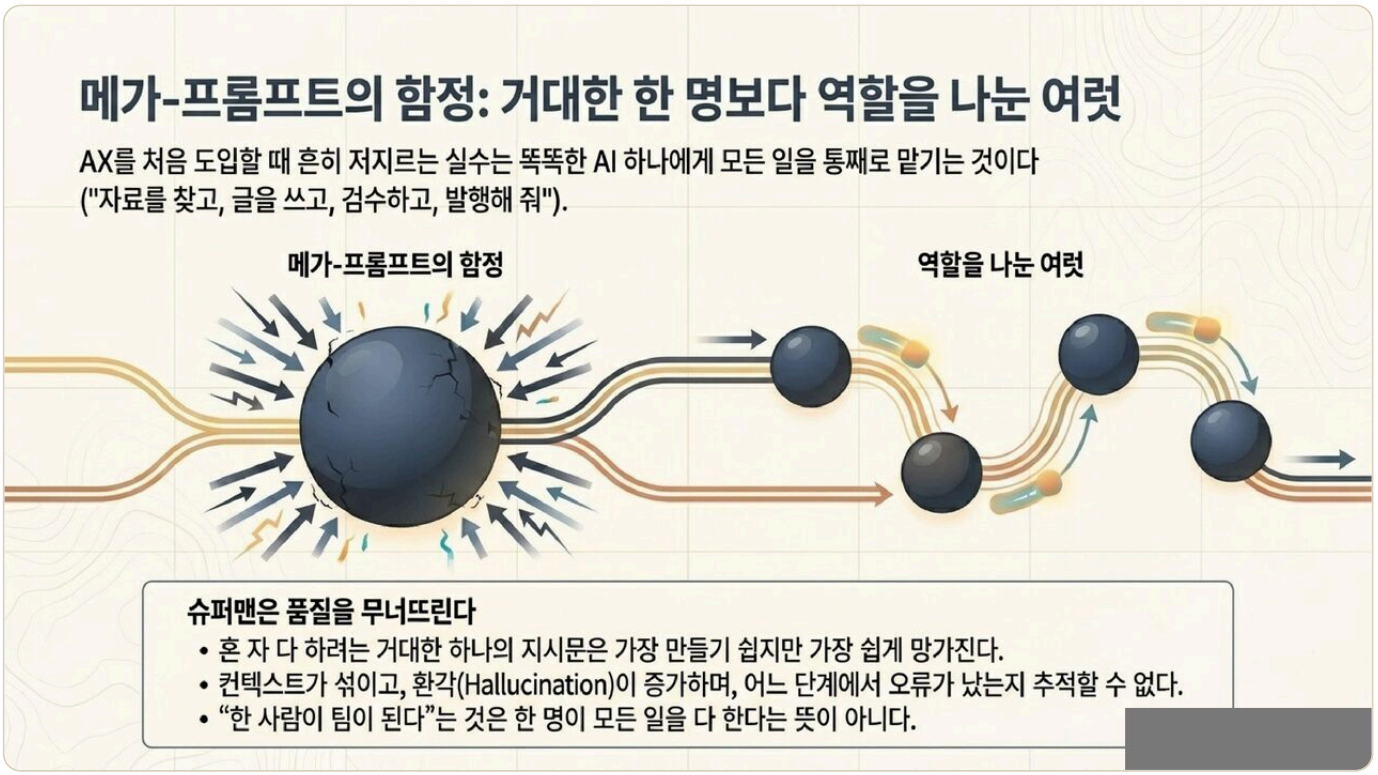


그림 3-3. 단일 에이전트 vs 멀티에이전트 비교

여기까지가 빌더의 영역이었다. 모델을 부품으로 다루고, 긴 작업을 쪼개고, 깨질 출력을 방어하고, 검수자를 떼어 앉히는 것. 그런데 마지막 검수자 이야기에는, 코드만으로는 끝나지 않는 까다로운 질문이 하나 매달려 있다. 다음 절에서 그 이야기로 모두 다시 합류하자.

내 일을 1인 파이프라인으로

이제 이 장의 이야기를 당신의 현장으로 옮겨보자. 리더든 빌더든, 자기 일을 한 사람짜리 에이전트 팀으로 짜고 싶을 때 무엇부터 하면 좋을지를 함께 정리해보자.

일을 역할로 쪼개는 세 가지 자리

거창하게 시작할 필요 없다. 지금 당신이 혼자 처음부터 끝까지 하고 있는 일 하나를 떠올려보자. 보고서를 쓰든, 자료를 정리하든, 무언가를 만들든. 그 일을 큰 덩어리 하나로 AI에게 통째로 맡기는 대신, 최소한 세 자리로 나눠보는 것이다.

- **고르는 자리** — 무엇을 다룰지, 무엇이 중요한지를 정하는 일. 입력을 추려 방향을 잡는다.
- **만드는 자리** — 실제로 결과물을 짓는 일. 글을 쓰거나, 안을 짜거나, 코드를 만든다.
- **검사하는 자리** — 만들어진 결과의 흠을 잡는 일. *이 자리만큼은 만드는 자리와 떼어둔다.*

이 세 자리를 한 에이전트에게 다 맡기지 말고, 따로따로 시켜보자. 가장 간단하게는, AI에게 한 번은 "작가의 입장에서 써줘", 다음 호출에서는 "방금 이 글을 처음 보는 냉정한 검수자의 입장에서 흠을 잡아줘"라고 *역할을 갈아 입혀* 따로 부르는 것만으로도 차이가 난다. 한 번에 "쓰고 검토까지 해줘"라고 하는 것과는 결과가 다르다. 한 자리에서 두 역할을 하면, 앞에서 본 그 자기 통과 도장이 또 찍힌다.

작가와 검수자를 같은 에이전트로 두지 마라

이 장에서 단 하나만 가져간다면 이것이다. **작가와 검수자를 같은 에이전트로 두지 마라.** 너무 당연해 보이는가? 그런데 막상 만들기 시작하면 거의 모두가 이 함정에 빠진다. 한 번에 "쓰고 검토까지" 시키는 게 편하니까. 호출도 한 번이면 되고, 코드도 짧다. 그 편함의 대가가 *검수의 실종*이라는 걸, 나는 며칠을 헛돌고 나서야 알았다.

사람이 일할 때도 똑같다. 자기가 짠 코드를 자기가 리뷰하고, 자기가 쓴 글을 자기가 교정하면, 늘 무언가를 놓친다. 그래서 좋은 조직은 만드는 사람과 검사하는 사람을 일부러 떼어놓는다. AI로 팀을 짤 때도 이 오래된 지혜를 그대로 가져오면 된다.

그런데, LLM 검수자를 맹신해도 될까?

여기서 나는 신나서 달려온 이 이야기에 일부러 찬물을 한 바가지 끼얹으려 한다. 검수자를 분리하면 품질이 오른다고 했다. 맞는 말이다. 그런데 그 검수자가 사람이 아니라 AI라면 — *우리는 그 AI의 판단을 어디까지 믿어도 될까?*

솔직히 고백하면, 나도 처음엔 AI 검수자를 꽤 믿었다. 분리만 하면 다 되는 줄 알았다. 그런데 한 가지 불편한 사실을 마주하게 된다. **"AI에게 검수를 시키자"고 말하는 사람은 많지만, 그게 실제로 잘 되더라는 단단한 증거는 의외로 빈약하다.** 어떤 실무자들은 자기네 실험에서 AI가 그리 좋은 비평가가 아니었다고 털어놓는다.[17] 그럴듯하게 흠을 잡는 것 같지만, 정작 중요한 문제는 놓치고 사소한 트집만 잡거나, 어제는 통과시킨 걸 오늘은 퇴짜 놓는 식으로 판단이 들쭉날쭉하더라는 것이다.

분리된 검수자가 자기검토보다 나은 건 분명하다. 다만 *맹신*하지는 말자는 것이다. 그래서 나는 검수 루프에 한 가지 안전장치를 두었다. 아까 말한, 잠시 미뤄둔 그 장치다. **검수 → 재작성 루프를 무한정 돌게 두지 않고, 정해진 횟수 — 가령 세 번 — 만 돌린 뒤에는 사람에게 넘긴다.** AI 검수자끼리 글이 계속 통과를 못 하고 빙빙 돌고 있다면, 그건 AI가 풀 수 있는 문제가 아닐 가능성이 높다. 그럴 때 기계끼리 무한히 핑퐁을 치게 두는 건 비용만 태우는 일이다. 세 번 돌고도 매듭이 안 지어지면, 그건 사람이 봐야 할 신호다.[18]

이 "세 번 뒤엔 사람"이라는 작은 규칙 안에, 사실 이 책 뒷부분의 큰 주제가 숨어 있다. *AI에게 검증을 맡기되, 최종 판단의 자리는 사람에게 남겨둔다*는 것. AI 검수자는 사람의 눈을 *덜어주는* 도구이지, *대체하는* 심판이 아니다. 이 구분을 흐리는 순간, 우리는 "AI가 좋다고 했으니 좋은 거겠지" 하는 위험한 안심에 빠진다. 그 안심이 어디로 이어지는지는, 책의 뒷부분에서 본격적으로 다시 이야기하게 될 것이다.

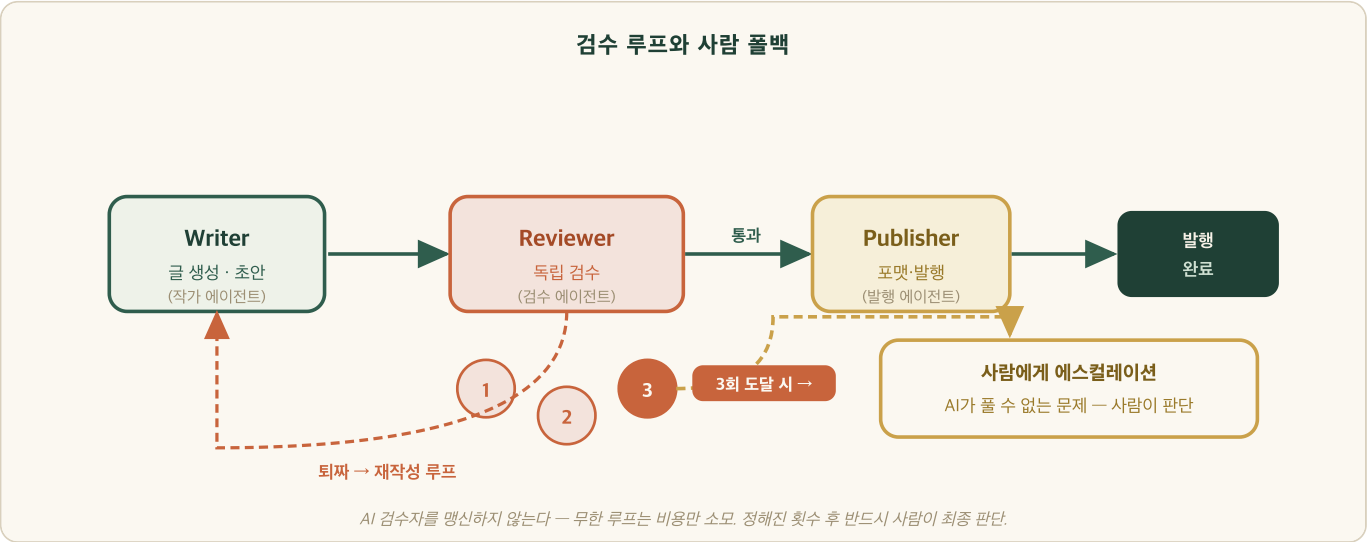


그림 3-4. 검수 루프와 사람 폴백

마지막으로 한 가지 덧붙이고 싶다. 이런 1인 파이프라인을 짜다 보면 *무엇을 AI에게 맡기고 무엇을 내가 쥘 것인가*라는, 더 깊은 질문에 자연히 부딪힌다. 어느 단계까지를 에이전트에게 넘기고 어느 판단을 사람이 쥘지 — 그 경계를 긋는 일 말이다. 그건 이 책의 다음 장에서 본격적으로 다룬다. 여기서는 일단, *일을 잘게 쪼개 역할에 나눠 맡기는 것*이 그 모든 이야기의 출발점이라는 것만 손에 쥐고 가자.

마무리

"한 사람이 팀이 된다"는 말은, 한 명이 다 한다는 뜻인가 — 아니면 다른 뜻인가?

이제는 답할 수 있다. 그건 슈퍼맨 이야기가 아니라 지휘자 이야기였다. 혼자 다 하려는 욕심을 내려놓고, 일을 역할로 쪼개 에이전트들에게 나눠 맡긴다. 그래서 한 사람이 작은 팀의 지휘자가 된다. 작가가 쓰고, 검수자가 보고, 발행자가 내보낸다. 그 사이에서 나는 더 적게 일하고 더 많이 설계한다. 혼자 다 할 때보다 결과는 좋아지고, 무엇보다 어디가 잘못됐는지 짚어 고칠 수 있게 된다.

이 장을 떠나며 두 가지만 다시 챙겨가자. 하나, **모델은 부품이다.** 시스템을 떠받치는 건 어떤 모델을 골랐느냐가 아니라 그 모델들을 엮은 설계다. 둘, **작가와 검수자를 같은 자리에 두지 마라.** 만드는 자와 검사하는 자가 같으면 검사는 도장으로 전락한다.

이 장의 끝에 나는 일부러 질문 하나를 남겨두었다. 검수자를 분리하면 품질이 오른다고 신나게 말해놓고, 정작 "그 검수자가 AI라면 맹신하지는 말자"고 발을 뺐다. 지금까지의 이야기는 모두 ③ *신남*의 이야기였다. 혼자 하던 일이 팀으로 돌아가는 쾌감, 이틀 만에 파이프라인이 완성되는 짜릿함. 그런데 그 신남의 한복판에서, 작은 그림자 하나가 슬

그머니 진다. *AI가 만든 걸 AI가 검증하게 두어도 정말 괜찮은가? 그럴듯하지만 틀린 결과를, 우리는 어떻게 가려낼 것인가?*

이 질문을 손에 쥔 채로 다음 장으로 넘어가자. 다음 장에서는 한 걸음 물러서서, *무엇을 AI에게 넘기고 무엇을 내가 쥘 것인가* — 그 경계를 어떻게 그을지를 함께 들여다본다. 일을 역할로 쪼개는 법을 배웠으니, 이제 그 조각들을 누구의 손에 쥐여줄지를 정할 차례다. 신남이 살짝 식고 의심이 고개를 들기 시작하는, 그 길목에서 다시 만나자.

4장. 내 일을 task로 쪼개기

> **이 장을 먼저 한눈에**
>
> 던지는 질문은 하나다. 무엇을 AI에게 넘기고, 무엇을 내가 쥘 것인가. 그 경계는 어떻게 긋는가.
>
> 답으로 가는 길은 이렇다. 일은 '직무'라는 큰 덩어리째로는 누구에게 줄지 정할 수가 없다. 잘게 쪼개야 비로소 보인다. 그렇게 쪼갠 task마다 사람이 나은 것, AI가 나은 것, 둘이 함께해야 나은 것이 따로 있다. 경계를 잘못 그으면 그럴싸하지만 틀린 결과를 되돌리느라 더 든다. 그래서 필요한 게 한 장의 워크시트다 — 내 일을 task로 나열하고, 사람·AI·협업으로 분류하고, 에이전트화 우선순위를 매기는.
>
> J-curve로 치면 아직 신남의 구간(③)이다. 한 사람이 팀이 되는 쾌감(3장)의 연장선. 다만 그 신남 한구석에 "경계를 잘못 그으면 어떻게 되더라"라는 첫 그림자가 깔린다.

딸을 글쓰기 학원에 내려주고 카페 모퉁이에 앉아 일을 시작할 때, 나는 늘 같은 장면을 본다. 노트북을 열면 어제 돌려둔 에이전트들이 밤새 해놓은 결과가 쌓여 있다. 어떤 건 흐뭇하고, 어떤 건 한숨이 나온다. 그 한숨의 정체를 들여다보면, 거의 언제나 같은 곳에서 일이 어긋나 있다. **사람이 쥐었어야 할 판단을 내가 AI에게 넘겨버린 자리.**

무엇을 넘기고 무엇을 쥘 것인가. 그 경계를 어떻게 그을 것인가. 말로 적어놓으면 거창한 이론처럼 들리지만, 내가 비싼 값을 치르고 나서야 배운 아주 실용적인 질문이다. 같이 더듬어보자.

"이 정도는 에이전트가 해도 되겠지"가 부른 일

3장에서 만든 콘텐츠 파이프라인 이야기를 다시 꺼내야겠다. 수집하고, 토픽을 고르고, 작가 에이전트가 쓰고, 검수자 에이전트가 보고, 발행하는 그 흐름 말이다. 한 사람이 팀이 된 그 쾌감에 한껏 들떠 있던 시기였다. 들떠 있으면 욕심이 난다. "이왕 자동화한 거, 더 많이 넘겨버리자"는 욕심이.

그 무렵 나는 한 단계를 슬쩍 에이전트에게 더 넘겼다. 어떤 토픽을 다룰지, 그 토픽을 어떤 각도로 풀지를 고르는 판단이었다. 원래는 내가 후보를 받아 직접 골랐다. 그런데 후보를 보다 보니 "이 정도 선택이야 에이전트가 해도 큰일 나겠어?" 싶었다. 패턴이 있어 보였으니까. 그래서 그 판단을 통째로 에이전트에게 맡기고, 나는 결과만 받기로 했다.

처음 며칠은 멀쩡했다. 그럴듯한 토픽이 골라졌고, 그럴듯한 글이 나왔다. 나는 흐뭇하게 발행 버튼을 눌렀다. 문제는 바로 그 *그럴듯함*이었다. 며칠 뒤, 나가지 말았어야 할 결의 글이 발행돼 있었다. 글 자체가 틀린 건 아니었다. 문법도 멀쩡하고, 논리도 그럴싸했다. 다만 그 시점에 우리가 그 주제를 그런 톤으로 다루는 건 — 사람이라면 1초 만에 "아, 이건 좀 곤란한데"라고 멈췄을 — 그런 맥락의 문제였다. 에이전트는 그 맥락을 알 길이 없었다. 애초에 그런 정보가 그 손에 쥐어져 있지 않았으니까.

그걸 되돌리는 데 든 시간이, 솔직히 그 단계를 자동화해서 아낀 시간보다 훨씬 길었다. 발행을 내리고, 왜 그게 나갔는지를 거슬러 올라가고, 다시 안 그러게 하려면 무엇을 어떻게 막아야 하는지를 고민하고 — 그 며칠이 통째로 날아갔다. 차라리 내가 매번 30초씩 들여 직접 골랐다면 결코 일어나지 않았을 일이었다.

그때 든 감정은 후회보다 찜찜함에 가까웠다. *나는 분명 멀쩡한 판단으로 그 단계를 넘긴 건데, 대체 어디서부터 잘못 그은 거지?* 패턴이 있어 보였고, 자동화할 만해 보였다. 그런데 아니었다. 무엇이 그 둘을 갈랐을까. "자동화할 만해 보이는 것"과 "자동화하면 안 되는 것"의 경계는, 겉만 봐서는 도무지 보이지 않았다.

이게 task 분해의 진짜 어려움이다. 일을 잘게 쪼개는 것 자체는 어렵지 않다. 어려운 건 쪼갠 조각 하나하나를 들고 *"이건 넘겨도 되는가, 이건 쥐고 있어야 하는가"*를 판단하는 일이다.

'직무'가 아니라 'task'로 쪼개야 하는 이유

내가 무엇을 잘못했는지 제대로 이해하려면, 먼저 한 칸 물러나 좌표부터 잡아야 한다.

생각해보면 우리는 오랫동안 일을 '직무'라는 큰 덩어리로 다뤄왔다. "저 사람은 마케터", "저 사람은 회계 담당", "저 사람은 개발자." 직무는 편리한 묶음이다. 채용할 때도, 평가할 때도, 자리를 배치할 때도 이 덩어리 단위로 움직이면 깔끔하다. 그런데 AI를 일에 들이려는 순간, 이 덩어리가 갑자기 거추장스러워진다. 왜 그럴까?

"마케터의 일을 AI에게 맡길 수 있는가?"라는 질문에는 누구도 시원하게 답할 수 없기 때문이다. 마케터의 일이라는 게 한 덩어리가 아니라서 그렇다. 그 안에는 시장 데이터를 긁어 정리하는 일도, 카피 초안을 쓰는 일도, 어떤 캠페인을 접고 어떤 캠페인을 키울지 결정하는 일도, 까다로운 클라이언트를 달래는 일도 다 섞여 있다. 이 조각들은 AI와의 궁합이 제각각이다. 데이터 정리는 AI가 훨씬 빠르고, 결정은 사람이 쥐어야 하고, 카피는 둘이 함께 쓸 때 가장 낫다. 그런데 이걸 "마케터"라는 한 덩어리로 묶어두면, 이 차이가 보이지 않는다. 덩어리째로는 "맡길 수 있다/없다"를 답할 수가 없는 것이다.

그래서 여러 연구 기관이 약속이나 한 듯 같은 그림을 그린다. 일을 직무라는 단위에서 task라는 단위로 잘게 내려, **task별로 인간과 AI를 다시 배분해야 한다**는 것이다.[19] 큰 자루를 풀어헤쳐, 그 안에 든 작은 조각들을 하나씩 꺼내 살펴보자는 이야기다. 내가 콘텐츠 파이프라인에서 더듬더듬 했던 그 일 — 수집·선택·작성·검수·발행으로 잘게 쪼갠 일 — 이 사실은 이 표준 접근의 한 사례였던 셈이다.

task마다 우열이 갈린다

task로 쪼갰다면, 다음 질문은 각 task를 누구에게 주느냐다. 사람, AI, 아니면 둘이 함께.

여기서 내가 가장 도움을 받은 관점이 하나 있다. 100건이 넘는 연구를 모아 살펴본 한 메타분석인데, 결론이 의외로 명쾌하다. **task의 성격에 따라 누가 잘하느냐가 갈린다**는 것이다.[20] 그것도 우리의 직관과는 조금 다른 방향으로.

흥미로운 대목은 이것이다. *의사결정형* 과제 — 데이터를 보고 어느 쪽이 맞는지 판정하는 종류의 일 — 에서는 오히려 **AI 단독이 사람보다 나은** 경우가 많았다고 한다. 사람이 끼면 도리어 성능이 떨어지기도 했다. 반면 *창의·생성형* 과제 — 새로운 무언가를 만들어내는 종류의 일 — 에서는 **사람과 AI가 함께할 때**가 가장 나았다. 한쪽만으로는 둘이 함께한 것을 못 따라갔다.

이걸 처음 봤을 때 나는 솔직히 좀 뜨끔했다. 내 직관은 정반대였기 때문이다. 나는 막연히 "판단 같은 무거운 건 사람이 쥐고, 글쓰기 같은 창의적인 건 AI가 알아서"라고 생각하고 있었다. 그런데 연구가 가리키는 방향은 거꾸로에 가까웠다. 그러고 보니 내가 콘텐츠 파이프라인에서 사고를 친 그 자리도 정확히 여기였다. 나는 *선택*이라는, 맥락과 책임이 걸린 판단을 "패턴이 있어 보인다"는 이유로 AI에게 통째로 넘겼다. 그게 의사결정형이라 AI가 나을 거라는 판단까지 갔던 것도 아니다. 그냥 "자동화할 만해 보여서" 넘겼다. 결정의 성격을 따져보지도 않고.

여기서 한 가지 결을 분명히 해두자. "의사결정형은 AI가 낫다"는 말을, "그러니 판단은 다 AI에게 넘겨라"로 읽을 말은 아니다. 연구가 말하는 의사결정은 *닫힌 판단* — 정답이 데이터 안에 있고, 충분한 정보가 주어졌을 때 어느 쪽인지 가리는 일 — 에 가깝다. 내가 넘긴 토픽 선택은 그게 아니었다. 그건 *맥락과 책임이 걸린 열린 판단*이었다. "지금 이 시점에 우리가 이걸 이렇게 다루는 게 적절한가"라는, 정답이 데이터 바깥에 있는 판단. 겉보기엔 똑같은 "고르기"였지만, 속은 전혀 다른 task였던 것이다. 이 차이를 못 보면, "패턴 있어 보이는 선택"을 죄다 AI에게 넘기고 나처럼 청구서를 받게 된다.

거들어주는 것과, 함께라야 비로소 되는 것

한 가지 개념을 더 손에 쥐고 가자. 사람과 AI가 함께 일한다고 할 때, 그 '함께'에도 결이 두 가지 있다.[21]

하나는 **거들어주는 것**이다. AI가 옆에서 받쳐주면 사람이 혼자 할 때보다 결과가 나아지는 경우다. 덜 숙련된 사람의 결과물을 전문가 수준 가까이로 끌어올려 주는, 그런 종류의 보조다.[22] 사람이 주연이고 AI가 조연인 구도라고 보면 된다.

다른 하나는 **함께라야 비로소 되는 것**이다. 사람과 AI가 손을 잡았을 때의 결과가, 사람 혼자 한 것보다 나은 건 물론이고 *AI 혼자 한 것보다도* 나은 경우다. 둘 중 누구도 혼자서는 도달 못 하는 지점에, 손을 잡아야만 닿는다. 진짜배기 협업은 이쪽이다.

협업이라고 다 같은 협업이 아니다. 어떤 task는 그냥 AI를 거들개로 붙여주면 충분하고, 어떤 task는 사람과 AI가 진짜로 주거니 받거니 해야만 제값이 나온다. 그리고 솔직히 말하면, *함께라야 비로소 되는* 협업이 정확히 무엇으로 측정되고 어떻게 설계되는지는 아직 학계에서도 깔끔하게 정리되지 않았다.[23] 빈 공간이 남아 있다는 뜻이다. 그러니 우리가 현장에서 더듬어 채워가는 수밖에 없다.



그림 4-2. task×협업모드 매트릭스

이론은 여기까지다. 이제 내가 실제로 어떻게 선을 그었는지로 들어가보자.

리더는 여기까지만 읽어도 된다

지금부터는 내가 실제 시스템에서 task 하나하나를 들고 어디에 선을 그었는지를, 만드는 사람의 시선에서 따라가 본다. 직접 시스템을 짜는 빌더라면 끝까지 함께 가면 좋겠다. 자기 현장에서 바로 써먹을 선긋기의 감각이 거기 있다.

조직을 이끄는 리더라면, **여기까지만 읽어도 충분하다.** 다음 절(③ "내 일을 task로 쪼개는 워크시트")로 바로 건너뛰어도 이 장에서 꼭 가져가야 할 건 이미 다 쥔 셈이다. 단 한 문장으로 줄이면 이렇다 — *경계는 task의 겉모습이 아니라, 그 task에 사람의 맥락과 책임이 걸려 있느냐로 긋는다.* 이걸 품고 곧장 ③으로 건너뛰어도 된다.

자, 그럼 빌더들과 좀 더 깊이 들어가보자.

같은 동작, 두 시스템 — 어떻게 선을 그었나

내가 만든 두 시스템은 겉보기엔 전혀 달랐다. 하나는 3장의 콘텐츠 파이프라인이었고, 다른 하나는 뒤에서(6장) 다룰 사람을 일에 배분하는 플랫폼이었다. 그런데 둘을 나란히 놓고 보니, 바닥에 똑같은 동작이 깔려 있었다. **일을 task로 쪼개서, 각 조각을 누구에게 줄지 정하는 것.** 콘텐츠 파이프라인은 '글 만드는 일'을 쪼개 에이전트들에게 나눠준 것이고, 인력 배분 플랫폼은 '조직의 일'을 쪼개 사람들에게 나눠준 것이다.

그래서 콘텐츠 파이프라인을 예로, 내가 task별로 어디에 선을 그었는지를 솔직하게 펼쳐보겠다. 사고를 친 다음에 다시 그은 선이라, 자랑보다는 반성에 가깝다.

수집. RSS와 여러 소스에서 자료를 긁어모으는 단계다. 이건 망설임이 없었다. 통째로 에이전트에게 넘겼다. 정해진 곳에서 정해진 형식으로 데이터를 가져오는 닫힌 작업이라, 사람의 맥락이 끼어들 자리가 없다. 넘겨도 사고가 안 난다. 게다가 솔직히 말하면, 이걸 사람이 손으로 하면 더디고 빠뜨리기만 한다. *AI 단독*이 정답인 칸이다.

토픽 선택. 바로 내가 사고를 친 그 자리다. 처음엔 통째로 넘겼다가, 호되게 데이고 나서 선을 다시 그었다. 핵심은 이 task를 *둘로 쪼갠* 것이다. "후보를 추려 비슷한 것끼리 묶고 우선순위를 매기는" 부분과, "그중 무엇을 실제로 다룰지 최종 결정하는" 부분으로. 앞부분 — 벡터 임베딩으로 비슷한 것끼리 모으고 후보를 정리하는 일 — 은 에이전트에게 맡겼다. 뒷부분 — 최종 한 표 — 은 다시 내 손으로 가져왔다. *맥락과 책임이 걸린 열린 판단*은 사람이 쥔다는 원칙을, 여기서 처음 명시적으로 세웠다. 하나의 task처럼 보였던 것이 사실은 성격이 다른 두 task였다는 걸, 나는 사고를 치고 나서야 알아챘다.

작성. 작가 에이전트가 초안을 쓰는 단계다. 이건 *함께라야 되는* 칸으로 뒀다. 에이전트가 초안을 빠르게 뽑되, 방향과 톤은 내가 잡고, 마지막 손질도 내가 한다. 창의·생성형 task에서 사람과 AI가 함께할 때가 가장 낫다는 그 이야기를, 여기서 그대로 따랐다.

검수. 3장에서 이야기했듯 검수는 작가와 *다른* 에이전트에게 분리해 맡겼다. 다만 그 검수 에이전트를 끝까지 믿지는 않았다. LLM을 비평가로 세우면 알아서 걸러주겠거니 했는데, 현장에서는 그 기대가 자주 어긋났다. 그래서 몇 차례 돌려보게 하고, 그래도 미심쩍은 건 사람에게 넘어오게 했다. AI 검수와 사람 검수가 함께 거는 칸인 셈이다.

발행. 다채널로 내보내는 마지막 단계. 형식 변환과 전송 자체는 에이전트가 한다. 하지만 *발행 버튼*만큼은 한동안 내가 직접 눌렀다. 토픽 선택에서 데인 뒤로, "최종적으로 세상에 나가는 그 순간"에는 사람의 눈을 한 번 더 끼우는 게 마음이 편했다. 되돌리기 어려운 일일수록, 마지막 게이트는 사람이 쥔다.

이렇게 다섯 조각을 펼쳐놓고 보니, 선을 긋는 기준이 비로소 또렷해졌다. *그 task에 사람만 아는 맥락이 필요한가, 틀렸을 때 책임이 걸리는가, 되돌리기 어려운가.* 이 셋 중 하나라도 "그렇다"면, 그 조각은 사람이 쥐거나 최소한 사람이 마지막 눈을 끼운다. 셋 다 "아니다"라면, 마음 놓고 넘긴다. 내가 처음 사고를 친 건, 토픽 선택이 이 셋에 다 걸리는데도 "패턴 있어 보인다"는 겉모습만 보고 넘겼기 때문이었다.

> **[이게 틀어졌을 때]** 잘못 그은 경계의 비용은 눈에 잘 안 보여서 더 무섭다. 토픽 선택을 통째로 넘겼던 그 며칠, 나는 분명 '시간을 아끼고' 있었다. 매번 30초씩 들이던 선택을 안 하게 됐으니까. 그런데 그렇게 아낀 시간의 총합보다, 잘못 나간 글 하나를 되돌리고 원인을 캐고 재발을 막느라 든 시간이 몇 배 더 컸다. 자동화의 비용은 *평소에는 0처럼 보이다가, 경계를 잘못 그은 자리에서 한꺼번에 청구된다.* 이 비대칭을 모르면, "자동화했더니 오히려 느려졌다"는 말이 왜 나오는지 끝내 이해하지 못한다. (이 비대칭의 정체는 8장에서 '검증 비용'이라는 이름으로 다시 만난다.)

내 일을 task로 쪼개는 워크시트

이제 이 이야기를 당신의 현장으로 옮겨보자. 리더든 빌더든, 자기 일을 task로 쪼개고 경계를 그어보려 할 때 손에 쥐면 좋을 한 장의 양식을 같이 만들어보자. 거창할 필요 없다. 빈 종이 한 장, 표 하나면 된다.



그림 4-1. task 분해 워크시트 템플릿

쓰는 순서는 세 단계다. 하나씩 같이 해보자.

ⓐ 내 일을 task로 나열한다

먼저 자기 일을 직무라는 큰 자루에서 꺼내, 작은 조각으로 펼쳐놓자. "나는 마케터다"가 아니라, "나는 데이터를 정리하고, 카피 초안을 쓰고, 캠페인을 결정하고, 클라이언트를 응대한다"로 풀어쓰는 것이다. 처음 해보면 의외로 어렵다. 우리는 자기 일을 덩어리로만 인식하는 데 익숙해서, 막상 조각으로 쪼개려면 손이 멈춘다. 그래도 끈질기게 쪼개보자. 한 task가 "이것도 사실 두 가지 일이네?" 싶으면 망설이지 말고 또 쪼갠다. 내가 토픽 선택을 "정리하기"와 "최종 결정하기"로 쪼개고 나서야 비로소 선을 제대로 그을 수 있었던 것처럼, 잘게 쪼갤수록 경계가 또렷해진다.

ⓑ 각 task를 사람단독·AI단독·협업으로 분류한다

조각마다 물어보자. *이 조각, 누구에게 줄까?* 여기서 겉모습에 속지 않는 게 관건이다. "자동화할 만해 보인다"는 직감을 믿지 말고, 앞서 만든 세 가지 질문을 들이대자.

- 이 task에 **사람만 아는 맥락**이 필요한가? (지금 이 상황, 이 관계, 이 분위기 — 데이터에 안 적힌 것)
- 이 task가 틀렸을 때 **책임이 걸리는가?** (누군가 다치거나, 회사가 곤란해지거나)
- 이 task가 틀렸을 때 **되돌리기 어려운가?** (한번 나가면 주워 담기 힘든가)

셋 다 "아니다"라면 마음 놓고 AI에게 넘긴다. 하나라도 "그렇다"면 사람이 쥐거나, 최소한 사람이 마지막 눈을 끼운다. 창의·생성형 task라면 사람과 AI가 함께하는 칸을 우선 고려하자 — 한쪽만으로는 둘이 함께한 것을 못 따라간다는 그 이야기를 기억하면서.

ⓒ 에이전트화 우선순위를 매긴다

분류가 끝났다고 다 한 번에 자동화할 필요는 없다. 'AI단독'과 '협업'으로 분류된 것 중에서, *자주 반복되고, 사고 위험이 낮고, 시간을 많이 잡아먹는* 순으로 우선순위를 매기자. 빈도 높고 위험 낮은 것부터 손대는 게 안전하다. 위험한 자리를 먼저 자동화하겠다고 덤비면, 내가 토픽 선택에서 그랬듯 비싼 수업료를 낸다.

"눈 감고 일하는" AI를 조심하자

워크시트를 다 채우고 나서도 꼭 짚어야 할 함정이 하나 있다. 빌더들 사이에서 *"AI가 눈 감고 일한다(working blind)"*고 부르는 현상이다.[24] 내가 토픽 선택에서 사고를 친 진짜 원인도 결국 이거였다.

AI에게 task를 넘길 때 우리는 종종 *그 일을 제대로 하는 데 필요한 맥락을 함께 쥐어주지 않는다.* 우리 조직의 사정, 지금의 분위기, 누구의 소유인지, 무엇을 건드리면 안 되는지. 사람이라면 당연히 알고 있는 그 로컬 컨텍스트가 AI 손에는 없다. 그런 상태로 일을 시키면 AI는 *눈을 감은 채* 그럴듯하게 일한다. 그리고 눈 감고 한 일은 평소엔 멀쩡해 보이다가 결정적인 순간에 어긋난다. 내 토픽 선택 에이전트가 딱 그랬다. 그 글이 그 시점에 곤란하다는 맥락을, 나는 한 번도 에이전트에게 쥐어준 적이 없었다.

그래서 경계를 그을 때 질문이 하나 더 붙는다. *이 task를 넘기려면, AI에게 어떤 맥락을 함께 쥐어줘야 하는가?* 그 맥락을 도무지 쥐어줄 수 없는 task라면 — 그건 십중팔구 사람이 쥐어야 할 조각이다. 맥락을 쥐어줄 수 있는 task라면, 그 맥락을 AI가 늘 참조할 수 있게 '컨텍스트 시스템'으로 떠먹여 주는 게 다음 숙제가 된다. 이 컨텍스트를 어떻게 시스템으로 만들어 떠먹이느냐가, 사실 AX 빌더가 부딪히는 가장 끈질긴 벽 중 하나다.

그리고 여기서 이 책을 끝까지 관통할 주제 하나를 미리 귀띔해두고 싶다. **막상 일을 해보면, 화려한 자동화보다 '지저분한 데이터를 정제하는 일'이 진짜 일이더라는 것.** AI에게 맥락을 쥐어주려면 그 맥락이 깔끔하게 정리돼 있어야 하는데, 현실의 데이터는 결코 깔끔하지 않다. 비어 있고, 형식이 제각각이고, 틀린 게 섞여 있다. 그걸 표준 형식으로 빚어내는, 빛도 안 나고 끝도 없는 그 작업이 — 의외로 AX의 8할이다. 이 이야기는 6장에서 사람을 일에 배분하는 플랫폼을 만들며 정면으로 다시 부딪히게 될 테니, 일단 "정제가 진짜 일이다"라는 말만 기억의 한구석에 걸어두자.

마무리

무엇을 AI에게 넘기고 무엇을 내가 쥘 것인가, 그 경계는 어떻게 긋는가?

답은 이제 손에 잡힌다. 먼저 직무라는 큰 덩어리를 task라는 작은 조각으로 푼다. 그다음 조각마다 *사람만 아는 맥락이 필요한가, 책임이 걸리는가, 되돌리기 어려운가*를 물어 선을 긋는다. 셋 다 아니면 넘긴다. 하나라도 그렇다면 쥔다. 내가 비싼 값을 치르고 배운 건 이 한 가지였다. *"자동화할 만해 보인다"는 직감은, 경계를 긋는 기준이 못 된다.*

그런데 여기까지 정리하고 나니 걸리는 게 하나 생긴다. 일을 task로 잘게 쪼개고, 어떤 건 AI에게, 어떤 건 사람에게, 어떤 건 둘이 함께하도록 다 나눠놨다고 치자. 그러고 나면 자연스럽게 다음 질문이 따라온다.

그래서 — 사람은 어떻게 평가하지?

AI가 task의 절반을 떠맡은 세상에서, 한 사람이 "일을 잘했다"는 건 대체 무엇으로 재야 하나? 토큰을 많이 쓴 사람인가, 코드를 많이 친 사람인가? 일을 잘게 쪼개 AI에게 영리하게 분배한 사람을, 우리는 무슨 잣대로 "잘했다"고 말해줄 수 있을까. task를 다 쪼개놓고 보니, 정작 그 task들을 *해낸 사람*을 어떻게 봐야 할지가 통째로 흔들리고 있었다.

일을 다시 나눴으니, 이제 사람을 다시 봐야 할 차례다. 다음 장에서는 AI 시대에 "잘했다"를 무엇으로 측정할 것인가를 함께 이야기해보자.

PART 3. 사람을 다시 설계하다

> 일을 다시 나눴으니, 이제 사람을 다시 본다. 이 책의 심장이다. 성과를 토큰이 아니라 검증으로 재고(5장), 사람을 직무가 아니라 task로 배분하고(6장), 머릿속에 갇힌 지식을 흐르는 자산으로 바꾼다(7장).
>
> *J-curve 위치: ④의구심의 한복판 — 각 장은 구덩이로 내려갔다가 작은 회복점으로 닫힌다.*

5장. 성과는 토큰이 아니라 검증이다

> 이 장에서 붙들 질문 하나, 그리고 거기서 빠져나온 길.
>
> 질문은 단순하다. AI 시대에 "잘했다"는 걸 무엇으로 잴까. 토큰을 많이 쓴 사람이 잘한 사람일까?

> 길은 세 마디로 줄어든다. 하나, AI가 일의 절반을 떠맡자 우리가 오래 써온 성과 지표들이 헛돈다. 분량도 토큰도 사람을 속이기 쉽다. 둘, 진짜로 봐야 할 건 "얼마나 만들었나"가 아니라 "그게 되돌려지지 않고 살아남았나"다. 검증을 통과했느냐는 것. 셋, 지표를 검증 중심으로 다시 짜고 코칭을 감시가 아니라 증폭으로 설계하면, AI는 사람을 줄 세우는 게 아니라 키우는 쪽으로 기운다.
>
> 이 장은 J-curve의 의구심 구간에 놓여 있다. "AI를 붙였는데, 그래서 *사람*은 어떻게 평가하지?"라는 막막함으로 구덩이에 들어섰다가, 끝에서 작은 회복점으로 한 뼘 올라서는 구간이다.

지난 장 끝에서 우리는 일을 잘게 쪼개는 데까지 왔다. 내 일을 task로 나누고, 어떤 조각은 AI에게 넘기고 어떤 조각은 내가 쥐었다. 거기까지는 좋았다. 그런데 책상 앞에 앉아 다음 분기 평가표를 펼쳐놓고 보니, 갑자기 손이 멈췄다.

이 사람이 이번 분기에 잘한 건가? 무엇으로 그걸 말하지?

예전 같으면 답이 어렵지 않았다. 얼마나 많이 처리했는지, 코드를 얼마나 썼는지, 문서를 몇 건 냈는지를 보면 대충 그림이 나왔다. 그런데 이제는 그 숫자들이 영 미덥지 못하다. 글을 많이 쓴 사람은 사실 AI에게 많이 시킨 사람일 수도 있고, 적게 쓴 사람은 AI가 토해낸 그럴싸한 결과를 일일이 검증하느라 시간을 다 쓴 사람일 수도 있다. 분량은 늘었다. 그런데 그 분량이 사람의 실력인지 AI의 출력인지 구분이 안 된다. 평가표를 앞에 두고 나는 묘하게 찜찜했다.

질문이 하나 더 뒤따랐다. 그렇다고 "AI를 많이 쓴 사람"을 잘한 사람으로 볼까? 그것도 영 이상하다. 토큰을 펑펑 썼다고 일을 잘한 건 아니지 않은가. 오히려 생각 없이 AI를 굴린 사람일 수도 있는데. 그렇다면 AI 시대에 "잘했다"는 건 대체 무엇으로 재야 하는 걸까? 함께 더듬어보자.

메트릭은 조작된다

본격적인 이야기에 앞서, 우리가 지금 어디쯤 서 있는지 좌표부터 잡아보자. 성과를 잰다는 일이 원래 어떻게 흘러왔는지를 먼저 떠올려보면 좋겠다.

성과 관리는 오랫동안 한 해에 한 번, 혹은 반년에 한 번 몰아서 하는 큰 행사였다. 연말이 되면 한 해를 돌아보며 점수를 매기고 등급을 나눈다. 그런데 이 방식은 요 몇 년 사이 빠르게 흔들렸다. 일의 속도가 빨라지면서, 1년에 한 번 돌아보는 평가는 이미 한참 지난 일을 뒤늦게 정산하는 영수증처럼 느껴지기 시작한 것이다. 그래서 흐름은 *연속적인 피드백*과 *결과 기반의 측정* 쪽으로 옮겨갔다.[25] 연말에 몰아서 점수를 매기는 대신, 자주 짧게 피드백을 주고받는다. 그리고 무엇을 했느냐보다 무엇이 실제로 남았느냐를 본다.

그런데 AI가 들어오면서 이 흐름에 또 한 번 균열이 생겼다. 솔직히 말하면, AI가 떠맡은 일을 사람의 성과로 어떻게 환산할 것인가에 대해서는 — 학계에서도 아직 깔끔한 답이 없다. AI가 사람을 거들어 성과를 끌어올린 부분(증폭)과, 사람과 AI가 함께여서 비로소 가능해진 부분(시너지)을 어떻게 떼어 측정할지는 여전히 빈 공간으로 남아 있다.[26] 우리가 지금 막막한 건, 우리가 게을러서가 아니라 아직 아무도 이 자를 만들지 못했기 때문이다.

빈 공간 앞에서 사람들은 익숙한 옛 자를 붙든다. 코드를 몇 줄 썼나, 문서를 몇 건 냈나, 토큰을 얼마나 썼나. 손에 잡히고 숫자로 떨어지니까. 그런데 여기에 함정이 있다. 측정에는 오래된 경고가 하나 있다 — **측정 가능한 것을 보상하기 시작하면, 사람들은 그 측정값을 채우는 데 최적화한다.** 지표가 곧 목표가 되는 순간, 그 지표는 더 이상 실력을 비추는 거울이 아니라 게임의 점수판이 된다.

이게 AI 시대에 특히 무섭다. 예전엔 코드를 한 줄 더 쓰려면 어쨌든 사람이 한 줄을 더 고민해야 했다. 그런데 지금은 다르다. 분량을 부풀리는 건 AI에게 시키면 1분이면 된다. 토큰을 더 쓰는 건 더 쉽다. 그냥 더 많이 물어보면 된다. 그래서 분량과 토큰은 **마음만 먹으면 얼마든지 조작되는 가짜 신호**가 되어버렸다.

실제로 현장 개발자들 사이에서는 이런 풍경을 꼬집는 말들이 돌아다닌다. 계정은 받았으니 토큰은 쓰는데 정작 그 결과물은 거들떠보지도 않는 '보여주기식 사용', 평가 지표를 채우려고 일부러 토큰을 낭비하는 '토큰 채우기' 같은 말들이다.[27] 이런 표현들이 검증된 통계는 아니다. 다만 현장의 온도는 분명히 전한다. 토큰을 KPI로 걸어두는 순간, 우리는 토큰을 낭비하는 법을 가르치는 셈이 된다.

그런데 여기서 잠깐. "토큰을 낭비하지 말자"는 결론으로 너무 빨리 달려가면, 우리는 또 다른 함정에 발을 들이게 된다. 마침 이 책을 쓰는 동안, 그 함정을 정확히 보여주는 사례 하나가 공개됐다. 한 대기업이 외부 생성형 AI를 전면 도입하면서, 성과를 새로운 두 잣대로 재기 시작했다는 것이다.[28] 하나는 '전일제 환산'이다. 한 사람이 AI를 써서 몇 명분의 일을 해냈는지를 본다. 다른 하나가 흥미로운데, '토큰 가성비'다. 투입한 토큰 대비 성과를 따져서, **같은 성과를 냈더라도 토큰을 *적게* 쓴 쪽을 더 높게 친다**는 것이다. 토큰을 펑펑 쓰는 게 문제라면, 아껴 쓰는 걸 우대하는 게 답 아닌가? 얼핏 들으면 합리적이다.

그런데 곰곰이 들여다보면 이상하다. 우리가 방금 한 이야기와 정확히 반대 방향으로 가고 있기 때문이다. 우리는 "토큰은 가짜 신호다"라고 했는데, 현실에선 토큰을 성과 지표의 한복판에 올려놓는 흐름이 등장한 것이다. 많이 쓰면 낭비라 깎고, 적게 쓰면 알뜰하다 우대하고. 방향만 뒤집혔을 뿐, 결국 둘 다 **투입(input)을 손에 쥐고 사람을 줄 세우는** 같은 게임이다. 토큰을 적게 쓰라고 보상하면 어떤 일이 벌어질까? 마땅히 더 물어보고 더 검증해야 할 자리에서도 사람들은 토큰을 아끼게 된다. 충분히 따져봐야 할 결과를 대충 넘긴다. 한 번 더 돌려봐야 할 코드를 그냥 내보낸다. 토큰을 아낀 만큼 검증이 얇아지는 것이다. 절약한 것처럼 보여도, 실은 혁신과 신중함을 함께 졸라맨 셈이다.

다행히 그 회사 안에서도 같은 우려가 나왔다고 한다. "이건 절감된 비용을 보여주는 지표일 뿐"이라는 것, 생산성·품질·혁신을 먼저 보고 나서 비용 지표는 나중에 얹어도 늦지 않다는 목소리였다.[29] 정확한 지적이다. 토큰 가성비는 *얼마를 아꼈는가*는 말해주지만, *무엇이 살아남았는가*는 한마디도 말해주지 않는다. 비용은 비용대로 챙기면 된다. 다만 그걸 성과의 첫 잣대로 삼는 순간 방향이 틀어진다. 토큰이라는 *투입값*에 매달리는 한, 우리는 같은 자리를 맴돈다.

그렇다면 진짜 신호는 어디에 있을까? 한 곳을 가리키는 관찰이 있다. 얼마나 많이 만들어냈는가는 잊으라는 것이다. 대신 **그 만들어낸 것이 되돌려지지 않고 살아남았는가**를 보라고 한다. AI가 토해낸 코드 중 며칠 못 가 갈아엎어진 비율(되돌림률), 매주 실제로 굴러가는 것을 보여주는 데모 — 이런 것들이 조작하기 어려운 진짜 신호에 가깝다.[30] 분량은 부풀릴 수 있어도, "그게 살아남았는가"는 부풀리기 어렵다. 살아남으려면 실제로 맞아야 하니까. **성과는 토큰이 아니라 검증이다.** 생산량을 세는 게 아니라, 그 생산물이 검증을 통과해 끝까지 남았느냐를 본다.

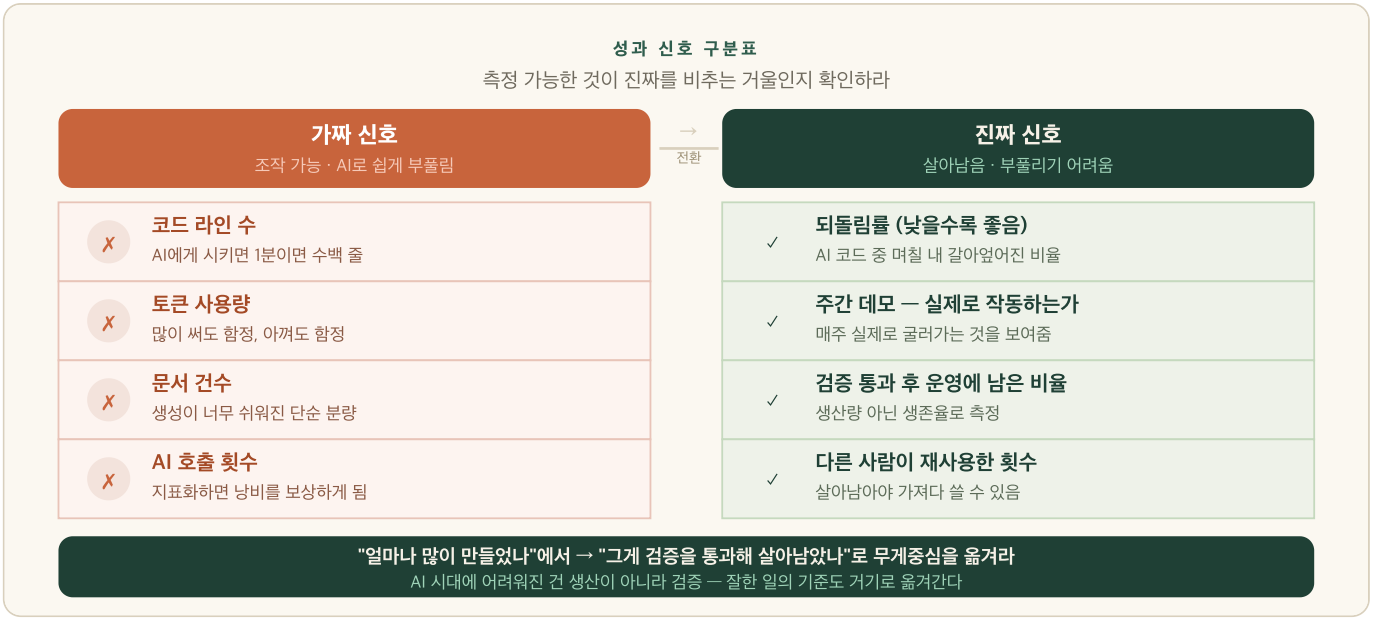


그림 5-1. 가짜 신호 vs 진짜 신호 표

여기까지가 '무엇을 재느냐'의 이야기다. 그런데 성과 이야기에는 한 겹이 더 있다. 재는 것만으로는 사람이 자라지 않는다는 점이다. 그래서 자연스럽게 따라오는 게 코칭이다.

AI를 코칭에 끌어들이는 일에 대해서는 의견이 정확히 둘로 갈린다. 한쪽에서는 AI가 매니저의 코칭을 크게 거든다고 말한다. 매니저마다 코칭 역량이 다르고 시간도 부족한데, AI가 피드백 초안을 거들면 더 자주, 더 고르게 피드백이 오간다는 것이다. 실제로 어떤 대형 도입 사례에서는 피드백이 크게 늘었다는 수치가 발표되기도 했다.[31] 다만 이런 숫자는 대개 도입한 쪽이나 만든 쪽에서 나온 발표라, 곧이곧대로 받기보다는 "그런 방향의 효과가 보고된다" 정도로 받아둔다.

반대쪽 목소리도 만만치 않다. "알고리즘이 내 팀을 나만큼 이해할 리 없다"는 거부감, 그리고 더 깊게는 **감시당하는 기분**이다.[32] 내 1on1 대화가, 내 평가 데이터가 어딘가로 흘러가 나를 들여다보는 데 쓰이는 것 아니냐는 불안. 이 불안은 근거 없는 게 아니다. 잘못 설계하면 코칭 도구는 정말로 감시 도구가 될 수 있으니까.

여기서 내가 일찍 깨달은 게 하나 있다. 이 두 입장은 어느 한쪽이 틀린 게 아니라, **둘 다 맞다.** AI 코칭은 잘 쓰면 사람을 키우는 증폭기이고, 잘못 쓰면 사람을 옥죄는 감시 카메라다. 그러니 "AI 코칭은 좋다/나쁘다"를 따지는 건 의미가 없다. 진짜 질문은 이것이다 — *어떻게 설계해야 증폭 쪽으로 떨어지고 감시 쪽으로 떨어지지 않는가?* 한 관찰자의 말이 핵심을 짚는다. 좋은 코칭 도구는 대화를 *대체*하는 게 아니라 *깊게* 만드는 도구다.[33] 이 한 줄을 손에 쥐고, 이제 내 이야기로 들어가보자.

토큰을 많이 쓴 사람이 잘한 사람일까

내가 코칭을 거드는 시스템을 만들겠다고 마음먹은 건, 사실 거창한 비전 때문이 아니었다. 앞에서 말한 그 찜찜함 때문이었다.

상황을 좀 그려보자. 팀장이 팀원과 1on1을 한다. 한 달에 한 번, 혹은 분기에 한 번. 그런데 이 1on1의 품질이 팀장마다 천차만별이다. 어떤 팀장은 미리 팀원의 일을 들여다보고 질문을 준비해 오지만, 어떤 팀장은 바빠서 회의 직전에 "요즘 어때요?"로 때운다. 팀장 자신이 게을러서가 아니다. 코칭이라는 게 원래 시간과 역량이 많이 드는 일이고, 잘하는 사람과 못하는 사람의 격차가 크기 때문이다. 같은 회사 안에서도 어떤 팀원은 매달 알찬 피드백을 받고, 어떤 팀원은 거의 방치된다. 이 격차를 볼 때마다 마음이 좋지 않았다.

그래서 생각했다. AI가 팀장 옆에서 코칭을 거들면 어떨까? 팀원이 한 일을 미리 정리해주고, "이런 점은 짚어주면 좋겠다"는 가이드 초안을 만들어주면, 바쁜 팀장도 최소한의 준비를 갖춘 채로 1on1에 들어갈 수 있지 않을까? 코칭을 잘하는 팀장은 더 잘하게, 못 하던 팀장은 바닥을 끌어올리게.

그런데 만들겠다고 마음먹은 순간, 앞에서 말한 그 두 개의 벽이 동시에 앞을 막았다.

첫 번째 벽이 바로 이 장의 핵심 질문이었다. AI가 코칭을 거든다면, AI는 무엇을 근거로 "이 사람이 이번에 잘했다/못했다"를 판단하지? 손쉬운 답은 숫자다. 이 사람이 코드를 몇 줄 썼고, 문서를 몇 건 냈고, AI 도구를 얼마나 썼고. 그런데 이 숫자들을 코칭 가이드에 그대로 박아넣는다고 생각하니 등이 서늘했다. "당신은 이번 달 토큰 사용량이 팀 평균보다 낮습니다"라는 문장을 팀장이 팀원에게 읽어준다고 상상해보라. 토큰을 적게 쓴 게 정말 게으름의 증거인가? 어쩌면 그 사람은 AI 출력을 신중하게 검증하느라, 함부로 토큰을 더 쓰지 않은 사람일 수도 있다. 그 신중함이야말로 칭찬받아야 할 일이다. 그런데 숫자로만 보면 정반대로 읽힌다. **토큰 많이 쓴 사람을 잘한 사람으로 치는 자, 이걸 코칭에 그대로 옮기면 사람을 거꾸로 키우게 된다.**

두 번째 벽은 거부감이었다. 코칭 시스템을 만든다고 했을 때, 가장 먼저 돌아온 반응은 기대가 아니라 경계였다. "그거 결국 우리 감시하는 거 아니에요?" 솔직히 이 반응이 나는 충분히 이해됐다. 내 1on1 대화가 데이터로 쌓이고, AI가 그걸 분석한다는 것 — 듣기에 따라 정말 으스스하다. 그리고 이건 단순한 기분 문제가 아니다. **사람들이 감시당한다고 느끼는 순간, 그 시스템은 죽는다.** 아무도 솔직하게 입을 열지 않는다. 코칭의 핵심인 진짜 대화가 사라지고, 다들 평가에 잘 보일 말만 하게 된다.

그러니 이 시스템은 시작부터 두 개의 외나무다리 위에 있었다. 한쪽으로 떨어지면 가짜 신호로 사람을 거꾸로 줄 세우는 도구가 되고, 다른 쪽으로 떨어지면 아무도 안 쓰는 감시 도구가 된다. 이 두 가지를 동시에 피하면서 코칭을 거들 수 있을까? 솔직히 처음엔 자신이 없었다. 이 막막함이 이 장이 J-curve의 의구심 구간에 놓인 이유다 — "AI를 붙였는데, 그래서 사람은 대체 어떻게 평가하지?"

> **[이게 틀어졌을 때]** 이 시스템을 만들면서 가장 무서웠던 시나리오는 기술적 실패가 아니라 *수용의 실패*였다. 잘 만들어놓고도 "감시 도구"라는 낙인이 한 번 찍히면, 그걸 되돌리는 비용은 만드는 비용의 몇 곱절이다. 한 번 "저거 우리 감시한다더라"는 말이 돌면, 아무리 화면을 분리하고 접근 권한을 막아놨다고 설명해도 신뢰는 돌아오지 않는다. 실제로 나는 초기에 이 부분을 가볍게 봤다가, "누가 이 데이터를 보느냐"는 질문에 명확히 답하지 못해 한 차례 차갑게 식는 분위기를 겪었다. 기술보다 수용 설계를 먼저 했어야 했다는 걸, 식고 나서야 배웠다.

리더는 여기까지만 읽어도 된다

지금부터는 이 코칭 시스템을 실제로 어떻게 만들었는지, 손에 잡히는 시행착오로 들어가보자. 직접 시스템을 만드는 빌더라면 끝까지 함께 가면 좋겠다. 모델을 어떻게 다뤘고, 응답 길이를 어떻게 조절했고, AI를 화면 어디에 놓았는지 — 자기 현장에 바로 옮길 만한 이야기들이 거기 있다.

하지만 조직을 이끄는 리더라면, **여기까지만 읽어도 충분하다.** 바로 다음 절(③ "성과 지표를 다시 짜기")로 건너뛰어도 이 장의 요지는 조금도 새지 않는다. 단 한 줄로 줄이면 이렇다 — *성과는 토큰이 아니라 검증으로 재고, AI 코칭은 감시가 아니라 증폭으로 설계한다.* 이 한 줄만 챙겨 ③으로 가도 된다.

자, 그럼 빌더들과 함께 조금 더 깊이 들어가보자.

모델은 부품이고, 프롬프트는 코드다

시스템의 큰 그림은 이렇게 잡았다. 팀원이 1on1 전에 자기 일을 간단히 정리하는 사전 준비 단계를 두고, 그 입력을 바탕으로 AI가 코칭 가이드 초안을 만든다. 팀장은 그 초안을 자기 화면에서 받아 검토하고 다듬은 뒤 1on1에 들어가고, 1on1이 끝나면 팀장과 팀원이 양방향으로 피드백을 주고받는다. 이 사이클이 한 번 돌 때마다 코칭 다이어리에 기록이 쌓인다. 말로 하면 깔끔하지만, 만드는 과정은 전혀 깔끔하지 않았다.

가장 먼저 부딪힌 건 모델이었다. 처음엔 어느 범용 모델 하나를 골라 거기에 맞춰 전부 짰다. 그런데 얼마 못 가 다른 모델로 갈아타야 했고, 그다음엔 또 다른 모델로 옮겨야 했다. 어떤 건 비용 때문에, 어떤 건 품질 때문에, 어떤 건 사내 정책 때문에. 문제는 모델을 갈아탈 때마다 자잘한 사고가 다 다르게 터진다는 점이었다. 어떤 모델은 응답을 중간에 뚝 잘라먹는다. 어떤 모델은 빈 값을 돌려준다. 어떤 모델은 게이트웨이에서 오류를 뱉는다. 모델 하나에 코드를 단단히 묶어뒀다면 그때마다 시스템 전체를 뜯어고쳐야 했을 것이다.

그래서 일찍 방향을 정했다. **모델은 갈아끼우는 부품으로 다루자.** 특정 모델의 버릇에 코드를 맞추지 말고, 어떤 모델을 끼워도 돌아가게 그 사이에 한 겹을 두는 것이다. 모델은 빠르게 좋아지고 빠르게 비싸지고 빠르게 바뀐다. 그러니 모델은 영원한 동반자가 아니라, 더 나은 게 나오면 언제든 바꿔 끼울 부품이다. 나는 이걸 두 번 갈아타고 나서야 몸으로 알았다.

잘림이냐, 비용이냐 — 응답 길이를 손으로 더듬다

모델 다음으로 나를 오래 붙든 건 의외로 사소해 보이는 문제였다. **응답을 얼마나 길게 받을 것인가.**

AI에게 코칭 가이드를 만들라고 시키면, 응답의 최대 길이를 정해줘야 한다. 너무 짧게 잡으면 가이드가 한창 좋은 말을 하다가 문장 중간에서 뚝 잘린다. 팀장이 받아보니 "이 팀원은 최근 프로젝트에서 책임감 있게…" 하다가 끝나 있는 식이다. 그렇다고 넉넉하게 길게 잡으면? 응답이 길어질수록 비용이 오르고 속도가 느려진다. 코칭 가이드 하나 만드는 데 한참을 기다려야 하고, 그게 다 돈이다.

이 균형점을 나는 머리로 계산해서 찾지 못했다. 손으로 더듬어 찾았다. 처음엔 넉넉하게 잡았다가, 너무 비싸서 줄였더니 이번엔 자꾸 잘렸다. 그래서 조금 늘리고, 또 조금 더 늘리고. 한 번에 답이 나오지 않아 여러 차례 값을 오르내리며 조정했다. 솔직히 좀 번거로웠다. 정답이 정해져 있는 게 아니라, 우리 코칭 가이드의 평균 분량과 우리가 감당할 비용 사이의 적당한 자리를 직접 찾아가는 일이었기 때문이다.

우리 현장의 값은 당신 현장과 다를 테니, 빌더에게는 값 그 자체보다 **이런 값들은 한 번 정하고 잊는 설정이 아니라, 쓰면서 계속 더듬어 맞추는 손잡이라는 사실**을 전하고 싶다. "잘림 vs 비용·속도"는 어느 한쪽으로 완전히 이길 수 없는 줄다리기다. 그러니 처음부터 완벽한 값을 찾으려 애쓰기보다, 쉽게 바꿀 수 있게 만들어두고 실제 사용을 보며 조이자.

> **[이게 틀어졌을 때]** 응답 길이를 넉넉하게 잡아두고 잊었다면, 비용은 조용히 새어나갔을 것이다. 이런 종류의 비용은 한 번에 크게 터지지 않고 호출 하나하나에 얇게 깔려 들어와, 청구서를 받아보고 나서야 "어디서 이렇게 나갔지?" 하게 된다. 나는 다행히 일찍 값을 조였지만, 이런 손잡이를 방치하면 시스템이 잘 도는 것처럼 보이는 동안 비용이 천천히 차오른다. 잘 도는 것과 싸게 도는 것은 다른 문제다 — 둘 다 지켜봐야 한다.

프롬프트를 코드 밖으로 꺼내고, 버전을 매기다

세 번째 시행착오가 어쩌면 가장 큰 깨달음을 남겼다.

코칭 가이드의 품질을 좌우하는 건 결국 AI에게 주는 지시문, 즉 프롬프트였다. 어떤 말투로, 무엇을 강조하고, 무엇을 피하라고 일러두느냐에 따라 가이드의 결이 완전히 달라졌다. 그런데 처음엔 이 프롬프트를 코드 안에 박아뒀다. 지시문을 한 글자 고치려 해도 코드를 고쳐 다시 배포해야 했다. 코칭이라는 건 한 번에 완성되는 게 아니라 계속 다듬어야 하는 일인데, 그때마다 배포를 기다려야 하니 영 번거로웠다.

그래서 프롬프트를 코드 밖으로 꺼냈다. 지시문을 데이터로 따로 빼서, 코드를 건드리지 않고도 바꿀 수 있게 한 것이다. 여기까지는 흔한 개선이다. 그런데 막상 프롬프트를 자주 바꾸기 시작하니 새로운 문제가 생겼다. "어제 그 프롬프트가 더 나았는데, 내가 뭘 바꿨더라?" 분명 더 나아지라고 고쳤는데 오히려 가이드 품질이 나빠지는 일이 생겼고, 그러면 이전으로 되돌리고 싶은데 이전이 뭐였는지 기억이 안 났다. 찜찜했다.

그때 머리를 한 대 맞은 기분이었다. **프롬프트는 코드와 똑같이 다뤄야 하는 것**이구나. 우리는 코드를 고칠 때 당연히 변경 이력을 남기고, 누가 언제 무엇을 바꿨는지 추적하고, 잘못되면 이전으로 되돌린다. 그런데 시스템의 행동을 코드만큼이나 좌우하는 프롬프트는, 어쩐 일인지 아무 기록 없이 마구 고치고 있었다. 그래서 프롬프트에도 똑같은 장치를 붙였다. 바뀔 때마다 그 순간의 모습을 저장해두고(스냅샷), 이전과 무엇이 달라졌는지 나란히 비교해 보고(diff), 마음에 안 들면 한 번에 이전으로 되돌리는(롤백) 화면을 만들었다.

그래서 여기서도 원칙은 같았다. **프롬프트도, 정책도 버전관리 대상이다.** 코드만 형상관리하던 시대는 끝났다. AI 시스템에서는 모델에게 건네는 말 한마디가 곧 시스템의 행동을 정하기 때문에, 그 말도 코드처럼 다뤄야 한다. 누가 언제 왜 바꿨는지 남기지 않으면, 시스템이 어느 날 갑자기 이상해졌을 때 원인을 찾을 길이 없다.

AI를 대화상대가 아니라 편집점으로

마지막 시행착오는 AI를 화면의 어디에 놓느냐의 문제였다. 사소해 보이지만, 이 시스템에서 가장 큰 차이를 만든 결정이었다.

처음엔 모두가 익숙한 방식으로 만들었다. 화면 한구석에 떠다니는 챗봇 창. 팀장이 거기에 "이 팀원 코칭 가이드 좀 만들어줘"라고 말을 걸면 AI가 답을 주는 식이다. 그런데 막상 써보니 영 손에 안 붙었다.

곰곰이 생각해보니, 챗봇은 AI를 *대화 상대*로 놓는 방식이었다. 팀장은 자기 작업 화면과 챗봇 창 사이를 계속 오가야 했다. 챗봇이 준 가이드를 복사해서 작업 화면에 붙이고, 고치고 싶으면 다시 챗봇으로 가서 부탁하고. 대화는 그럴싸한데, 일은 자꾸 끊겼다. AI와 수다를 떠는 느낌이지, 일을 같이 하는 느낌이 아니었다.

그래서 발상을 뒤집었다. AI를 대화 상대 자리에서 끌어내려, **편집점**으로 옮겨 박기로 했다. 팀장이 코칭 가이드를 들여다보는 바로 그 작업 화면 안에, AI가 손볼 자리를 직접 만들어줬다. 한쪽엔 지금 글, 다른 쪽엔 AI의 제안을 나란히 띄워 비교하며 고르는 식이다. AI에게 말을 걸러 다른 창으로 건너가지 않는다. 내가 일하는 그 자리에서, AI가 바로 옆에서 손을 보탠다.

이 작은 재배치가 사용률을 확 바꿨다. 떠다니는 챗봇일 때는 다들 한두 번 써보고 말았는데, 작업 흐름 안의 편집점이 되자 자연스럽게 손이 갔다. 여기서 배운 교훈은 이렇다. **AI는 사람의 작업 흐름 안에 '편집점'으로 박혀 있을 때 훨씬 더 쓰인다.** 사람들은 AI와 대화하려고 일하는 게 아니다. 일을 하다가 AI의 손을 빌리고 싶을 뿐이다.

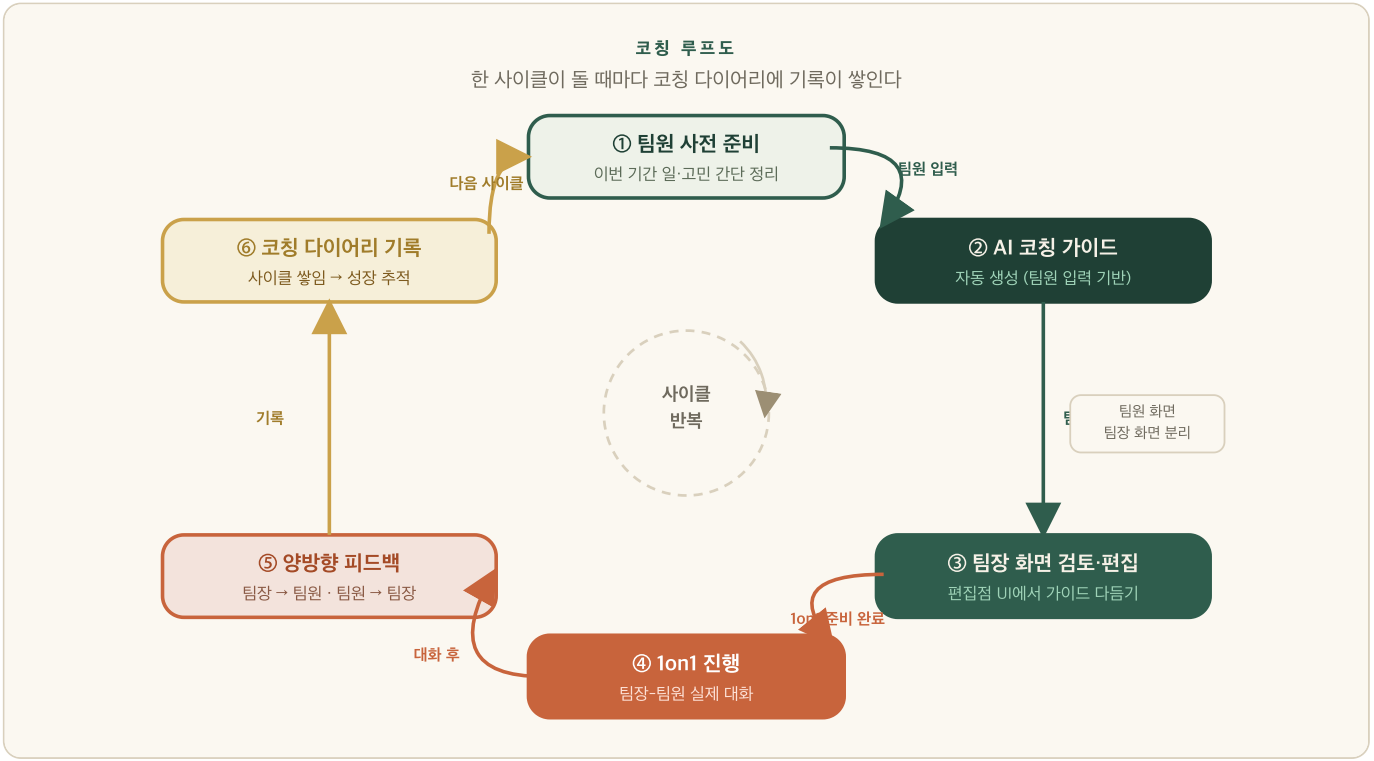


그림 5-2. 코칭 루프도

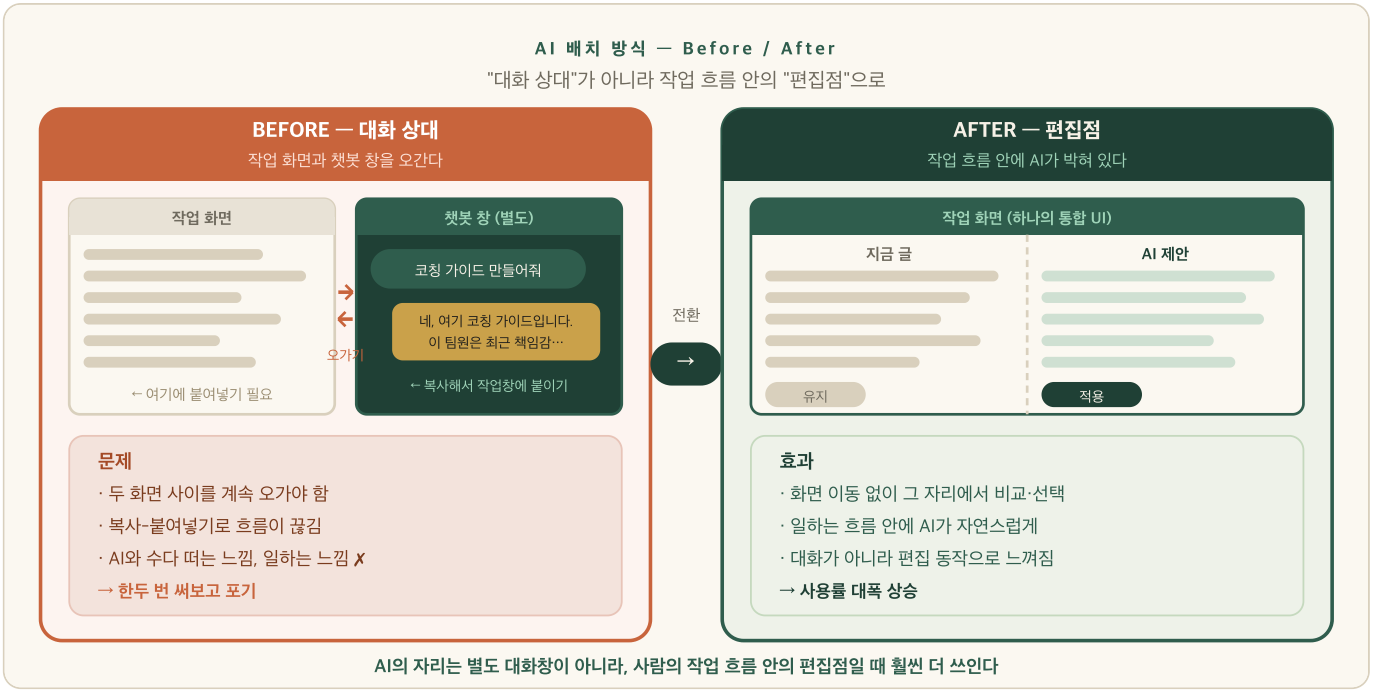


그림 5-3. "대화상대 → 편집점" before/after

여기까지가 빌더의 영역이었다. 모델을 부품으로 다루고, 응답 길이를 손으로 더듬고, 프롬프트에 버전을 매기고, AI를 편집점으로 옮긴 이야기. 그런데 이 모든 기술적 결정의 밑바닥에는, 결국 "무엇을 잘했다고 볼 것인가"와 "어떻게 감시가 아니게 만들 것인가"라는 두 질문이 깔려 있었다. 다음 절에서 그 두 질문을 당신의 현장으로 옮겨보자. 리더든 빌더든, 여기서 다시 합류하자.

성과 지표를 다시 짜기

이제 이 장의 이야기를 당신의 현장으로 옮겨보자. 코칭 시스템을 직접 만들든 안 만들든, AI가 일에 들어온 조직이라면 누구나 마주칠 질문이 둘 있다. 무엇을 잘했다고 볼 것인가, 그리고 그 측정을 어떻게 사람을 옥죄지 않게 할 것인가. 하나씩 정리해보자.

가짜 신호와 진짜 신호를 갈라내기

먼저 지금 우리 조직이 무심코 보고 있는 지표들을, 가짜 신호와 진짜 신호로 한번 갈라내 보자. 다음 질문을 자기 지표에 대보면 된다.

- 이 지표는 **AI에게 시키면 쉽게 부풀릴 수 있는가?** (그렇다면 가짜 신호에 가깝다. 분량, 토큰, 호출 횟수, 건수 같은 것들.)
- 이 지표는 **결과가 실제로 살아남았는지를 보는가?** (그렇다면 진짜 신호에 가깝다. 되돌려지지 않고 운영에 남은 비율, 매주 실제로 작동하는 데모, 다른 사람이 가져다 쓴 횟수.)
- 이 지표를 보상하기 시작하면, **사람들이 그걸 채우려고 이상한 짓을 하게 되는가?** (그렇다면 그 지표는 곧 게임의 점수판이 된다.)

이 작업의 핵심은 **"얼마나 많이 만들었나"에서 "그게 검증을 통과해 남았나"로 무게중심을 옮기는 것**이다. AI 시대에는 생산이 너무 쉬워졌다. 쉬워진 걸 재봐야 변별력이 없다. 어려워진 건 생산이 아니라 검증이다. 그러니 잘한 일의 기준도 생산량에서 검증량과 구조로 옮겨 간다. AI가 토해낸 것을 얼마나 잘 가려내고, 틀린 걸 얼마나 잘 잡아내고, 살아남을 것만 얼마나 잘 남겼는가 — 이게 AI 시대의 진짜 실력이다.

"감시가 아니라 증폭"으로 수용을 설계하기

지표를 잘 짰다고 끝이 아니다. 그 지표가 사람을 옥죄는 데 쓰일까 봐 사람들이 입을 닫으면, 아무리 좋은 지표도 죽은 지표다. 그래서 수용을 따로 설계한다. 열쇠는 하나다 — **이 시스템이 사람을 줄 세우려는 게 아니라 키우려는 것임을, 설계로 증명하는 것.**

- 코칭이 대화를 **대체하는 게 아니라 깊게 만드는 도구**임을 분명히 하자. AI가 사람을 평가하는 게 아니라, 사람이 더 잘 대화하도록 거드는 것이다. 가이드는 어디까지나 초안이고, 최종 판단과 대화는 사람의 몫으로 남겨둔다.
- 데이터가 **누구에게 보이는지를 못 박자.** 코칭 데이터가 위로 올라가 평가 자료로 둔갑하지 않는다는 걸, 화면 분리와 접근 권한으로 실제로 보장하자. 말로만 "감시 아니에요"라고 해선 아무도 안 믿는다.
- 토큰·분량 같은 **가짜 신호를 코칭 가이드에서 빼자.** "당신은 토큰을 적게 썼다" 같은 문장이 가이드에 등장하는 순간, 그 시스템은 사람들 머릿속에서 감시 도구가 된다.

> **[데이터 민감도·접근통제 — 한 줄 체크]** 개인의 평가·코칭 데이터는 누가 보는가? 팀장 화면과 팀원 화면의 분리, 그리고 누가 어떤 데이터에 접근하는지의 통제를, 나중에 덧붙이지 말고 **설계 단계에서 못 박았는가?** 한번 새어나가면 그땐 막을 게 남아 있지 않다. 평가 데이터는 가장 민감한 개인 데이터 중 하나다.

이 두 가지 — 지표를 검증 중심으로 다시 짜는 것과, 수용을 증폭 쪽으로 설계하는 것 — 는 사실 하나로 묶인다. 둘 다 "AI를 사람을 재는 자가 아니라 사람을 키우는 거름으로 쓰자"는 같은 마음에서 나온다.

마무리

AI 시대에 "잘했다"를 무엇으로 측정할 것인가 — 토큰을 많이 쓴 사람이 잘한 사람인가?

이제 답은 분명하다. 아니다. 토큰도 분량도 마음만 먹으면 부풀려지는 가짜 신호다. 잘했다는 건 얼마나 많이 만들었느냐가 아니라, 그 만들어낸 것이 검증을 통과해 살아남았느냐에 있다. **성과는 토큰이 아니라 검증이다.**

솔직히 고백하면, 이 장은 나에게 가장 막막한 구간이었다. 일을 잘게 쪼개고 에이전트에게 넘기는 데까지는 신이 났다. 그런데 "그래서 사람은 어떻게 평가하지?"라는 질문 앞에서 나는 한참을 헤맸다. 가짜 신호로 사람을 거꾸로 줄 세울까 봐, 혹은 좋자고 만든 도구가 감시 카메라가 될까 봐 — 두 개의 외나무다리 위에서 자꾸 비틀거렸다. AI를 붙였는데 오히려 더 어려워졌다. 구덩이에 빠진 기분이었다.

그런데 신기하게도, 출구는 같은 자리에 있었다. 측정의 무게중심을 생산량에서 검증으로 옮기자, 두 문제가 한꺼번에 풀리기 시작했다. 토큰 대신 "이게 살아남았는가"를 보기로 하니 가짜 신호로 줄 세울 일이 없어졌고, 코칭이 사람을 재는 게 아니라 사람의 대화를 깊게 하는 도구라는 게 분명해지자 감시라는 거부감도 누그러졌다. 토큰 KPI를 검증·구조 KPI로 바꾸는 그 한 걸음이, 코칭을 감시에서 증폭으로 돌려세운 전환점이었다. **이 시스템은 결국 돌아갔다.** 처음의 그 차갑던 분위기가, 사람들이 자기 성장을 위해 자발적으로 가이드를 들여다보는 분위기로 바뀌는 걸 보면서, 나는 구덩이에서 한 뼘 올라온 걸 느꼈다.

물론 이게 끝이 아니다. 성과를 검증으로 재기로 했다면, 그다음 질문이 자연스럽게 따라온다. 사람을 '직무'라는 큰 덩어리에 묶어둔 채로, 과연 AI 시대의 유연한 일을 감당할 수 있을까? 어떤 사람이 무슨 일을 잘하는지를 task 단위로 보기 시작하면, 사람을 배치하는 방식 자체가 흔들리기 시작한다. 다음 장에서 함께 이야기해보자. 직무가 아니라 task로 사람을 배분한다는 게 무슨 뜻인지, 그리고 그게 왜 생각보다 훨씬 지저분하고 손이 많이 가는 일인지를 말이다.

6장. 직무가 아니라 task로 사람을 배분하다

> 이 장이 붙들고 가는 질문은 하나다. 사람을 '직무'라는 큰 덩어리로 묶어둔 채로, AI 시대의 유연한 재배분이 정말 가능한가?
>
> 미리 말해두면, 6개월을 통과하고 손에 남은 답은 내가 처음 기대했던 것과 정반대였다. 나는 똑똑한 매칭 알고리즘이 답이라고 믿고 시작했지만, 진짜 승부처는 화려한 매칭이 아니라 일을 부르는 공통의 언어, 곧 분류 체계(taxonomy)와 지저분한 입력 데이터를 표준으로 빚어내는 정제 작업에 있었다. 그 안 멋진 밑작업들이 자리를 잡고서야 비로소 플랫폼이 돌기 시작했다. "이게 되긴 하나" 싶던 바닥에서 작은 이륙까지, 그 6개월의 기록이다.

카페 모퉁이에서 글을 쓰다 문득, 옆자리에서 글쓰기를 배우는 딸 생각을 한 적이 있다. 학원은 아이를 "초등 4학년"이라는 학년으로 묶어두지 않고, "이 아이는 묘사는 되는데 구성이 약하다"는 식으로 *할 수 있는 것*과 *아직 안 되는 것*을 쪼개서 본다. 그래야 다음에 무엇을 가르칠지가 정해지니까. 그런데 정작 우리 회사는, 그리고 대부분의 회사는, 사람을 그렇게 보지 않는다. "저 사람은 마케팅팀이야", "저 사람은 기획 직무야"라는 한 덩어리로 묶어두고 만다.

자, 한 장면을 상상해보자. 어떤 프로젝트에 갑자기 데이터 정리할 손이 급하게 필요해졌다고 해보자. 그 일은 사실 옆 부서의 누군가가 반나절이면 끝낼 수 있는 일이다. 그 사람도 마침 한가하다. 그런데 그 사람은 "기획 직무"로 묶여 있고, 지금 필요한 건 "데이터 정리 task"다. 이 둘을 이어줄 길이 조직에 없다. 결국 그 일은 엉뚱하게 외부에 맡겨지거나, 누군가 야근으로 떠안거나, 아예 안 되고 묻힌다. 분명 회사 안 어딘가에 그 일을 할 수 있는 사람과 시간이 있는데도 말이다.

이상하지 않은가. 사람도 있고 시간도 있는데, 일과 사람이 만나지 못한다. 왜 그럴까? 우리가 사람을 '직무'라는 너무 큰 단위로 묶어 박아뒀기 때문이다. 그 덩어리는 한 번 정해지면 좀처럼 풀리지 않는다. 그렇다면 이 덩어리를 풀어서, 사람을 더 잘게 — 그 사람이 *할 수 있는 일* 단위로 — 다시 볼 수는 없을까?

직무라는 덩어리를 풀면 무엇이 보이나

내 이야기를 하기 전에, 우리가 지금 어디쯤 서 있는지부터 좌표를 잡아보자. 사람을 직무로 묶어두는 방식의 한계는 사실 나만 느끼는 답답함이 아니다. 요즘 인사·조직을 연구하는 쪽에서 부쩍 자주 나오는 말이 있다. **스킬 기반 조직(skills-based organization)**이라는 표현이다.[34] 풀어 말하면, 사람을 "무슨 직무인가"로 보지 말고 "무슨 스킬을 가졌는가"로 보자는 발상이다.

이런 말이 나온 건 직무라는 단위가 너무 경직돼 있어서다. 한 사람을 "기획자"라고 묶는 순간, 우리는 그 사람이 가진 수십 가지 능력을 "기획"이라는 한 단어 뒤로 다 숨겨버린다. 그 사람이 데이터를 잘 다루는지, 영어 문서를 빠르게 읽는지, 디자인 감각이 있는지는 직무라는 라벨에 다 가려진다. 그러니 일이 빠르게 바뀌고 사람을 자주 다시 배치해야 하는 시대에는, 이 큰 덩어리가 오히려 발목을 잡는다. 실제로 여러 조사에서 리더 다수가 "경직된 직무 구조에서 벗어나야 한다"고 답한다.[35]

그렇다면 직무라는 덩어리를 풀면 그 자리에 무엇을 놓아야 할까? 여기서 등장하는 게 **사내 인력 시장(internal talent marketplace)**이라는 개념이다. 말은 거창하지만 핵심은 단순하다. 회사 안에 일종의 장터를 여는 것이다. 한쪽에는 "이런 일이 있는데 사람이 필요하다"는 task와 프로젝트가 올라오고, 다른 쪽에는 "나는 이런 스킬이 있고 이만큼 시간이 난다"는 사람이 등록한다. 그리고 이 둘을 이어준다. 직무라는 칸막이를 넘어, 일과 사람이 스킬을 매개로 직접 만나게 하는 것이다.

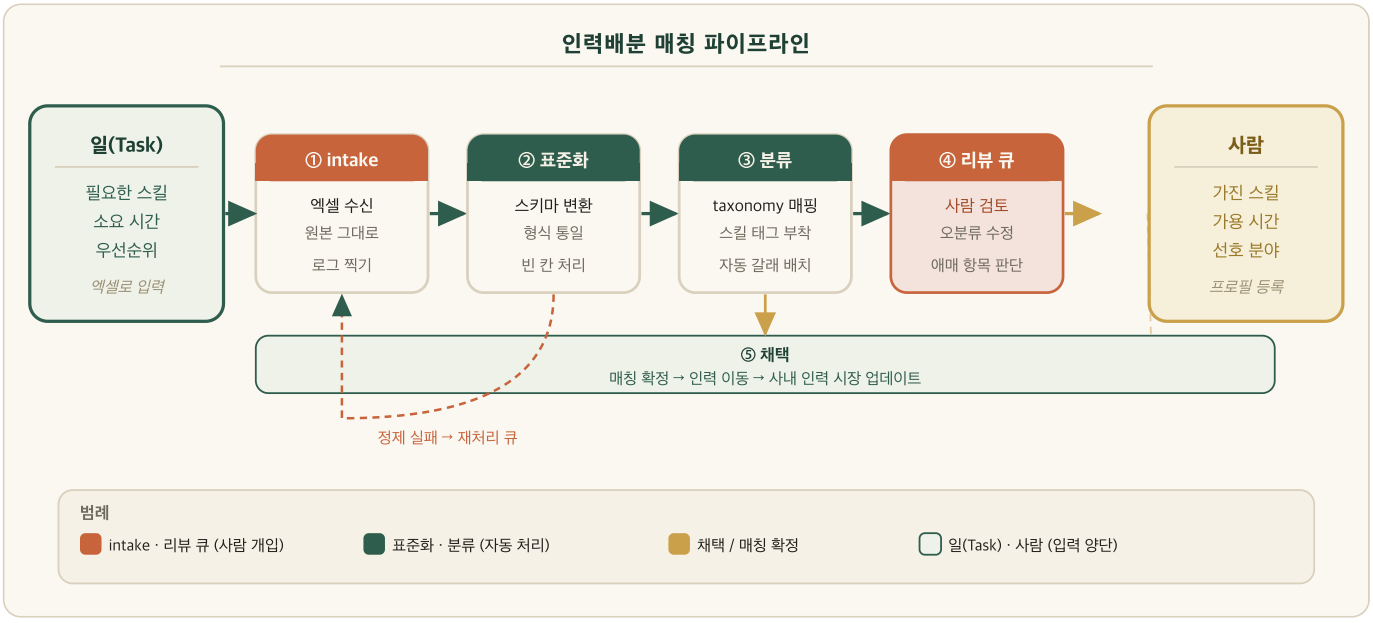


그림 6-1. 인력배분 매칭도

이 개념 자체는 새로운 게 아니다. 해외에서는 이미 큰 기업들이 만들어 운영한 사례가 교과서처럼 돌아다닌다. 어떤 대형 소비재 기업은 사내 인력 시장을 열어 직원 대다수가 참여했고, 그동안 묻혀 있던 수십만 시간 분량의 일손을 새로 끌어냈다고 한다.[36] 흥미로운 건 그 다음 디테일이다. 그들은 사람을 매칭할 때 *학력 정보를 일부러 보이지 않게* 가렸다.[37] 출신 학교나 학벌이 보이면, 매칭하는 쪽이 자기도 모르게 거기에 끌려가 편향된 선택을 하기 때문이다. 능력으로 골라야 할 자리에 간판이 끼어드는 것이다. 그래서 아예 그 칸을 설계 단계에서 가려버렸다. 통제를 사람의 양심에 맡기지 않고, *시스템의 구조*로 박아둔 것이다.

또 다른 사례에서는, 전 직원을 한꺼번에 들이지 않고 *단계적으로* 인원을 늘려갔다는 점이 눈에 띈다.[38] 수천 명으로 시작해 점점 범위를 넓혔고, 기존에 쓰던 인사 시스템들과도 끊지 않고 연결해 함께 굴렸다. 새 플랫폼을 만든다고 옛 시스템을 다 갈아엎은 게 아니라, *레거시와 손을 잡고 천천히 갈아탄* 것이다. 이건 나중에 내가 뼈저리게 동의하게 되는 대목이기도 하다.

다만 이런 멋진 사례들은 대부분 *해외*의, 그것도 *전문 솔루션을 사다 쓴* 이야기다. 정작 한국 현장에서, 그것도 누군가 직접 손으로 만들어 굴린 사내 인력 시장의 기록은 — 적어도 내가 공개된 자료에서 찾을 수 있는 한 — 거의 비어 있었다.[39] 그래서 나는 어쩔 수 없이, 그리고 어쩌면 다행스럽게도, 그 빈칸을 내 손으로 메워보기로 했다. 지금부터 할 이야기는 누군가의 잘 정리된 성공담이 아니라, 참고할 선례가 거의 없는 자리에서 더듬더듬 만들어간 6개월의 기록이다.

매칭이 문제가 아니었다, 분류가 문제였다

이제 내 이야기를 해보자. 처음에 나는 이 일을 만만하게 봤다. 솔직히 말하면, "이건 매칭 문제구나" 하고 생각했다. 한쪽에 일이 있고 다른 쪽에 사람이 있으니, 둘을 잘 이어주는 똑똑한 알고리즘만 있으면 되겠거니 했다. 어떤 task에 어떤 사람이 가장 잘 맞는지를 점수로 매겨 추천해주는, 그런 그럴듯한 추천 엔진을 머릿속에 그렸다. 데이트 앱이 사람을 이어주듯, 일과 사람을 이어주면 되는 것 아닌가.

그런데 막상 손을 대보니, 매칭은 문제 축에도 끼지 못했다. 진짜 벽은 그 앞에 있었다. **무엇을 무엇과 매칭할 것인가**라는, 너무 당연해서 아무도 안 묻던 질문이었다.

생각해보자. "데이터 정리 task"와 "데이터를 다룰 줄 아는 사람"을 잇겠다고 치자. 그런데 "데이터 정리"라는 게 정확히 뭔가? 엑셀 표를 손보는 일인가, 데이터베이스를 만지는 일인가, 통계를 돌리는 일인가? 그리고 "데이터를 다룰 줄 안다"는 건 또 어느 수준을 말하는 건가? 같은 말을 두 사람이 전혀 다른 뜻으로 쓰고 있었다. 한쪽이 "기획"이라고 적은 일과 다른 쪽이 "전략 기획"이라고 적은 일이 사실은 같은 일인지 다른 일인지, 시스템은 알 길이 없었다. 사람과 사람도 말이 안 통하는데, 그걸 기계더러 이어주라니.

여기서 나는 한 대 얻어맞은 기분이었다. 매칭을 하려면, 그 전에 *모두가 같은 말로 일을 부를 수 있는 공통의 언어*가 있어야 했다. 일을 분류하는 체계. 그러니까 **업무 분류 체계(taxonomy)** 말이다. 이게 없으면 아무리 똑똑한 매칭 알고리즘을 짜도 소용이 없다. 서로 다른 말로 적힌 일과 사람을 이으려는 건, 한국어로 쓴 이력서와 아랍어로 쓴 채용 공고를 나란히 놓고 맞춰보라는 것과 같다. 번역기, 그러니까 공통의 분류 체계가 먼저다.

이걸 깨닫고 나니 마음이 좀 복잡했다. 솔직히 김이 샜다. 나는 멋진 추천 엔진을 만들고 싶었는데, 정작 해야 할 일은 "일을 어떻게 분류할 것인가"라는, 화려함과는 거리가 먼 밑작업이었으니까. 하지만 동시에 묘하게 안심도 됐다. 진짜 문제가 무엇인지 알게 됐으니까. 막연히 "매칭이 안 된다"가 아니라, "분류 체계가 없어서 매칭의 대상조차 정의가 안 된다"는 정확한 진단을 손에 쥔 것이다. 그래서 방향을 틀었다. 추천 엔진을 잠시 미뤄두고, *분류 체계부터* 세우기로 했다.

또 하나, 분류 체계만큼이나 나를 괴롭힌 게 있었다. 현실에서 일 목록이 어떤 모양으로 들어오는지를 처음 봤을 때의 그 막막함이다. 사람들이 자기 일을 적어 보낸 자료는 하나같이 *엑셀 파일*이었다. 그런데 그 엑셀이라는 게 — 겪어본 사람은 알 것이다 — 그야말로 제멋대로였다. 어떤 칸은 비어 있고, 어떤 칸은 한 칸에 세 가지 일을 몰아 적었고, 같은 일을 부서마다 다른 이름으로 불렀다. 날짜 형식조차 사람마다 달랐다. 이걸 그대로 시스템에 부으면, 쓰레기를 넣으면 쓰레기가 나온다는 말 그대로가 된다. 이 지저분한 현실 데이터를 깨끗한 표준 모양으로 빚어내는 일 — 이게 앞으로 가장 손이 많이 가는 작업이 되리라는 걸, 그땐 어렴풋이만 짐작했다.

> **[이게 틀어졌을 때]** 분류 체계가 어설프거나 입력 데이터가 지저분하면 무슨 일이 벌어지는가. 나는 이걸 직접 겪었다. 정제 레이어를 제대로 세우기 전에 현실 엑셀을 그냥 파이프라인에 흘려보낸 적이 있다. 결과는 처참했다. 빈 칸 하나에 처리가 멈춰 서고, 한 칸에 여러 일이 뭉쳐 들어온 항목은 엉뚱하게 분류되고, 같은 일이 이름만 다르다는 이유로 서로 다른 항목으로 둘씩 쪼개졌다. 매칭의 품질 이전에, 데이터가 파이프라인을 *통과조차 못 했다*. 그제야 알았다. 이 시스템의 성패는 매칭 알고리즘이 아니라, 들어오는 입구에서 얼마나 끈질기게 데이터를 정제하느냐에 달려 있다는 걸.

리더는 여기까지만 읽어도 된다

여기서부터는 이 플랫폼을 실제로 어떻게 코드와 절차로 만들어갔는지, 만드는 사람의 눈높이에서 한 단계씩 풀어보려 한다. 직접 이런 시스템을 만드는 빌더라면 끝까지 함께 가면 좋겠다. 자기 현장에 바로 옮길 시행착오의 지도가 거기 있다.

하지만 조직을 이끄는 리더라면, 사실 **여기까지만 읽어도 충분하다.** 바로 다음다음 절(③ "우리 일의 taxonomy 만들기")로 건너뛰어도 이 장이 말하려는 바는 고스란히 손에 남는다. 가져갈 단 한 문장은 이것이다 — *사내 인력 시장의 승부처는 화려한 매칭이 아니라, 일을 부르는 공통의 언어(분류 체계)를 세우고 지저분한 입력을 표준으로 빚어내는 정제 작업이다.* 이 한 문장이면 ③에서 다시 만나도 좋다.

자, 그럼 빌더들과 함께 조금 더 깊이 들어가보자.

분류 체계라는 뼈대를 먼저 세우자

앞에서 깨달은 대로, 나는 매칭 엔진이 아니라 분류 체계부터 손을 댔다. 구체적으로는 "우리 회사의 일을 어떻게 나눌 것인가"를 정의하고 탐색할 수 있는 도구 — 일종의 *분류 체계 탐색기* — 를 가장 먼저 만들었다. 일을 큰 갈래로 나누고, 그 아래 중간 갈래로, 다시 그 아래 구체적인 task로 내려가는 나무 모양의 구조다.

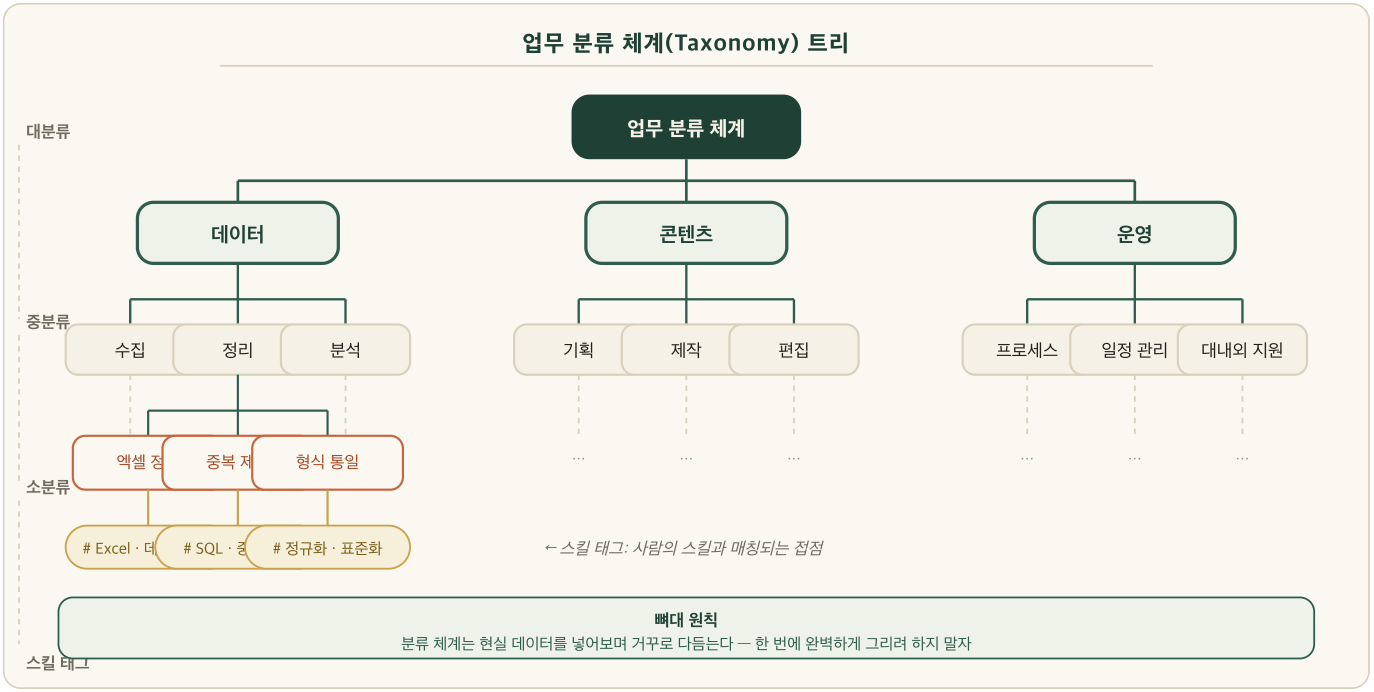


그림 6-2. taxonomy 트리

가장 먼저 만든 이유는 하나다. **분류 체계가 플랫폼의 뼈대이기 때문이다.** 집을 지을 때 골조를 먼저 세우는 것과 같다. 골조가 비뚤어지면 그 위에 무엇을 올려도 비뚤어진다. 분류 체계가 흔들리면, 그 위에 얹은 매칭도, 통계도, 인력 이동 관리도 다 같이 흔들린다. 반대로 이 뼈대가 단단하면, 나머지는 거기에 살을 붙이는 일이 된다. 그래서 나는 다른 화려한 기능을 다 미뤄두고, 이 분류 체계를 세우고 다듬는 데 초반의 가장 많은 공을 들였다.

여기서 한 가지 배운 게 있다. 분류 체계는 한 번에 완벽하게 그려지지 않는다. 처음엔 머릿속에서 그럴듯한 나무를 그렸는데, 현실의 일들을 하나씩 끼워 넣어보니 자꾸 안 맞았다. 어떤 일은 두 갈래에 동시에 걸치고, 어떤 갈래는 텅 비고, 어떤 잎은 너무 잘게 쪼개져 의미가 없었다. 그래서 *현실 데이터를 넣어보며 분류 체계를 거꾸로 다듬는* 과정을 여러 번 돌았다. 분류 체계를 먼저 세우되, 그것을 돌에 새기지는 말자.

가장 손이 많이 간 곳은 '정제 레이어'였다

뼈대를 세웠으니 이제 현실 데이터를 그 위에 올릴 차례였다. 그런데 앞에서 예고했듯, 바로 여기가 6개월 통틀어 *가장 손이 많이 간* 자리였다.

문제는 입력이 엑셀이라는 데 있었다. 사람들이 자기 일을 적어 보낸 엑셀은 표준이라는 게 없었다. 그래서 들어오는 엑셀을 받아, 그것을 시스템이 알아먹는 깨끗한 표준 형태로 바꿔주는 *변환 레이어*를 따로 만들어야 했다. 이 레이어가 하는 일을 몇 가지만 들어보자. 한 칸에 여러 일이 몰려 들어오면 그걸 떼어내 각각의 항목으로 나눠야 했다. 빈 칸은 그냥 버릴 게 아니라, 버릴 것인지 기본값을 채울 것인지 사람이 메우게 돌릴 것인지를 정해야 했다. 같은 일을 다르게 부른 이름들은 한 이름으로 모아줘야 했다. 날짜·숫자 형식도 하나로 통일했다.

말로 적으니 간단해 보인다. 실제로는 끝없는 두더지 잡기였다. 한 가지 예외를 처리하면 또 다른 모양의 예외가 튀어나왔다. "이 정도면 다 잡았겠지" 싶을 때마다 새로운 엉망진창이 나타났다. 그게 반복되면 사람이 지친다. 그 무렵 가장 자주 든 생각은 이거였다. "내가 지금 멋진 AI 플랫폼을 만드는 게 맞나, 아니면 그냥 남의 엑셀 청소부인가." 화면에 보이는 결과물은 초라했다. 겉으로는 아무것도 새로 안 생긴 것 같은데, 정작 코드의 절반이 이 보이지 않는 정제 작업에 들어갔다. 그땐 그게 좀 억울했다.

그런데 시간이 지나면서 생각이 바뀌었다. 이 *지저분한 입력을 표준으로 빚어내는 정제 레이어*야말로 이 플랫폼의 진짜 심장이었다. 화려한 매칭 알고리즘은 정제된 깨끗한 데이터가 있어야 비로소 작동한다. 그 깨끗한 데이터를 만들어내는 게 정제 레이어다. **지저분한 현실 데이터를 정제하는 일이 진짜 일이다.** 그럴듯하고 똑똑해 보이는 부분은 사실 빙산의 일각이고, 물밑에 잠긴 정제 작업이 시스템의 몸통을 떠받친다. 이걸 가볍게 보면, 화면은 그럴듯한데 실제로는 아무것도 제대로 매칭 못 하는 빈 껍데기가 나온다.

들어온 데이터가 표준으로 정리되면, 그 다음은 비교적 수월했다. 표준화된 task를 분류 체계에 맞춰 자동으로 갈래에 배치하고(분류 단계), 그 결과를 사람이 한 번 눈으로 확인하는 *리뷰 큐*에 올렸다. 자동 분류가 아무리 잘돼도 애매한 항목은 늘 남는다. 그런 것들을 기계가 혼자 우격다짐으로 밀어 넣게 두지 않고, 사람이 "이건 이쪽 갈래가 맞다"고 한 번 손봐주는 관문을 둔 것이다. 그렇게 검토를 통과한 항목이 비로소 *채택*되어 매칭의 대상이 되고, 매칭이 확정되면 실제 인력 이동으로까지 이어지게 묶었다.

데모를 실제로 띄우는 것의 무게

기능을 다 만들었다고 끝은 아니었다. 마지막에 나를 가장 지치게 한 건 의외의 복병이었다. **만든 걸 실제로 띄우는 일** 그 자체였다.

내 노트북에서는 모든 게 잘 돌았다. 그런데 이걸 실제 서버에 올려 사람들이 접속할 수 있게 만드는 단계에서, 끝도 없는 잔손질이 기다리고 있었다. 화면 경로가 어긋났다. 이미지 같은 정적 파일이 안 떴다. 뒤에서 도는 처리 핸들러가 배포 환경에서만 묘하게 다르게 굴었다. 분명 어제까지 잘 되던 게, 실제 환경에 올리니 안 됐다. 같은 자리를 열 번 넘게 고치고 또 고쳤다. 한 줄 고치고 배포하고, 안 되면 또 고치고 배포하고. 그 새벽의 답답함이란.

이 경험에서 나는 한 가지를 뼈저리게 배웠다. **'내 노트북에서 되는 것'과 '실제로 운영되는 것' 사이에는 생각보다 깊은 강이 흐른다.** 우리는 흔히 기능을 다 만들면 일이 끝났다고 착각한다. 하지만 그 기능이 실제 환경에서, 실제 사람의 손에 닿는 자리에 안정적으로 떠 있게 만드는 것 — 그게 또 하나의 큰 산이다. 데모를 *진짜로 띄우는 일*의 무게를, 나는 이때 처음 제대로 실감했다. 이건 5장에서도, 7장에서도, 그리고 책의 마지막까지 계속 변주되며 따라올 주제다.

여기까지 오니, 처음에 김이 샜던 그 마음이 어느새 가라앉아 있었다. 멋진 추천 엔진은 끝내 이 이야기의 주인공이 못 됐다. 결국 일을 굴린 건 분류 체계라는 뼈대였고, 지저분한 입력을 표준으로 빚어내는 정제 레이어였고, 그걸 실제로 띄워내는 끈질김이었다. 그런데 신기하게도, 그 안 멋진 것들이 자리를 잡자 — 직무라는 덩어리에 갇혀 있던 사람들을 task 단위로 다시 배분하는 플랫폼이, 정말로 돌기 시작했다.

우리 일의 taxonomy 만들기

이제 이 이야기를 당신의 현장으로 옮겨보자. 거대한 인력 시장 플랫폼을 당장 만들라는 게 아니다. 그 핵심에 있는 사고법 — *사람을 직무가 아니라 task로 본다*는 그 관점 — 을 자기 조직에 어떻게 들일지를 함께 정리해보자. 그리고 내가 가장 비싸게 배운 교훈, "분류 체계가 뼈대다"를 당신은 처음부터 챙겨가면 좋겠다.

분류 체계 설계 3단계

거창한 도구가 없어도 된다. 화이트보드나 공유 문서 한 장이면 시작할 수 있다. 다음 세 단계를 순서대로 밟아보자.

- **1단계 — 큰 갈래부터 그린다.** 우리 팀(혹은 조직)이 하는 일을 가장 큰 단위로 대여섯 갈래만 나눠보자. 직무 이름이 아니라 *일의 종류*로 나누는 게 핵심이다. "마케팅팀"이 아니라 "콘텐츠 제작", "데이터 분석", "고객 응대"처럼. 직무라는 칸막이를 머릿속에서 잠시 치우는 연습이다.
- **2단계 — 잎까지 내려간다.** 각 큰 갈래를 한두 단계 더 쪼개, 한 사람이 반나절에서 며칠이면 끝낼 만한 *구체적인 task* 수준까지 내려간다. 너무 잘게 쪼개 의미가 없어지지 않게, "이걸 누군가에게 통째로 넘길 수 있나?"를 기준으로 멈출 지점을 잡자.
- **3단계 — 현실로 검증하고 고쳐 잡는다.** 가장 중요한 단계다. 실제로 최근 한 달간 우리 팀이 한 일들을 이 분류 체계에 하나씩 끼워 넣어보자. 안 들어가는 일, 두 갈래에 걸치는 일, 텅 비는 갈래가 반드시 나온다. 그걸 보고 분류 체계를 다시 손본다. 한 번에 완벽하게 그리려 하지 말자. 현실에 부딪혀 고쳐 잡는 게 정상이다.

intake 표준화 체크리스트

분류 체계가 섰으면, 사람들에게서 일 목록을 받아 그 위에 올려야 한다. 그런데 받는 자료는 거의 틀림없이 엉망인 엑셀일 것이다. 그걸 표준으로 빚어내기 전에, 다음을 미리 점검해두자. 이 체크리스트가 정제 레이어의 설계도다.

- **한 칸에 여러 일이 뭉쳐 들어오는 경우**를 어떻게 떼어낼 것인가? (가장 흔하고 가장 성가신 예외다.)
- **빈 칸**을 만나면 버릴 것인가, 기본값을 채울 것인가, 사람에게 되물을 것인가? 항목별로 미리 정해두자.
- **같은 일을 다르게 부른 이름들**을 한 이름으로 모을 대조표가 있는가? (부서마다 말이 다르다는 걸 전제하자.)
- **날짜·숫자·단위 형식**을 하나로 통일할 규칙이 있는가?
- 정제하다 막히는 항목이 나오면, 그걸 *버리지 말고* 사람이 손볼 수 있게 따로 모아두는 자리가 있는가?

이 체크리스트가 시시해 보일지도 모르겠다. 나는 이걸 안 챙겼다가 비싸게 배웠다.

사람을 줄 세우지 않으려면

마지막으로, 이 시스템에는 다른 어떤 시스템보다 더 조심해야 할 지점이 하나 있다. **인력 데이터를 다루는 일은, 자칫 사람을 줄 세우는 일이 되기 쉽다.**

앞에서 어떤 해외 사례가 매칭할 때 *학력 정보를 일부러 가렸다*고 했던 걸 기억하는가. 그건 결코 사소한 디테일이 아니다. 사람을 스킬로 매칭하겠다면서 정작 화면에 출신 학교가, 나이가, 과거 평가 등급이 다 보이면, 매칭하는 쪽은 자기도 모르게 그 간판에 끌려간다. 능력으로 골라야 할 자리에 편견이 끼어드는 것이다. 그러니 한 가지를 설계 단계에서 못 박아두자 — **사람을 편향되게 줄 세울 수 있는 정보(학력·나이·과거 등급 같은)는, 꼭 필요하지 않다면 매칭 화면에서 아예 보이지 않게 가린다.** 그리고 이 인력 데이터에 누가 접근할 수 있는지도 함께 정해두자. 통제를 사람의 양심에 맡기지 말고, 시스템의 구조로 박아두는 것이다. 편향은 새어나간 다음에 막으려 하면 늦다. 1장에서 만난 원칙이 여기서도 그대로다 — *민감한 것은 설계 단계에서 막는다.*

> **[before / after]** | | before (직무로 묶을 때) | after (task로 배분할 때) | |—|—|—| | 사람을 보는 단위 | "마케팅팀의 누구" (큰 덩어리) | "이런 스킬·이만큼 가용시간" (잘게) | | 급한 일이 생겼을 때 | 직무 칸막이에 막혀 사람을 못 구함 | 스킬로 회사 전체에서 가용 인력을 찾음 | | 묻혀 있던 일손 | 부서 안에 갇혀 안 보임 | 사내 시장에 올라와 드러남 | | 편향 통제 | 학력·간판이 그대로 노출 | 민감 정보를 가려 능력으로 매칭 |

마무리

사람을 '직무'라는 큰 덩어리로 묶어둔 채로, AI 시대의 유연한 재배분이 정말 가능한가?

내 답은 이렇다. 가능하다. 단, 우리가 기대했던 방식으로는 아니다. 나는 처음에 똑똑한 매칭 알고리즘이 답이라고 믿었다. 하지만 6개월을 통과하고 나서 손에 남은 건 정반대의 교훈이었다. 사람을 task로 다시 배분하는 일의 승부처는 화려한 매칭이 아니라, *일을 부르는 공통의 언어를 세우는 일*과 *지저분한 입력을 표준으로 빚어내는 정제 작업*에 있었다. 가장 안 멋진 일이, 알고 보니 가장 중요한 일이었다.

솔직히 그 한복판에서는 막막했다. 남의 엑셀이나 청소하는 기분이었다. 만든 걸 띄우느라 같은 자리를 열 번씩 고치고 있을 때는 "이게 되긴 하나" 싶었다. AX라는 말의 멋진 이미지와, 내가 매일 붙들고 있는 지저분한 작업 사이의 거리가 너무 멀어 찜찜했다. 하지만 바로 그 안 멋진 밑작업들이 자리를 잡자, 직무에 갇혀 있던 사람들이 task 단위로 흐르기 시작했다. 구덩이의 바닥을 한참 긴 끝에 만난, 작지만 분명한 이륙이었다.

그러니 혹시 지금 비슷한 일을 하다가 "내가 만드는 게 멋진 AI 플랫폼이 맞나" 싶어 김이 샜다면, **그 지저분한 정제 작업이 바로 진짜 일이다.**

자, 일을 task로 쪼개 사람을 다시 배분하는 데까지 왔다. 그런데 사람의 머릿속에 든 것은 일만이 아니다. 아이디어가, 노하우가, 한 번 풀어본 문제의 해법이 그 안에 있다. 이것들이 그 사람 머릿속에만 머물면 조직의 자산이 되지 못한다. 그렇다면 이 지식과 기여를, 어떻게 조직 차원에서 *흐르게* 할 수 있을까? 다음 장에서 함께 이야기해보자.

7장. 지식이 흐르게 하라

> 카페 한 켠. 딸이 사람에게 배운 글을 친구들과 나누는 걸 곁눈질하며, 나는 머릿속 지식을 어떻게 흐르게 할지를 코드로 더듬고 있었다.
>
> 이 장이 붙들고 가는 물음은 하나다. 아이디어·지식·스킬이 사람 머릿속에만 있을 때, 그걸 조직 차원에서 흐르게 하려면 어떻게 해야 할까.
>
> 답은 세 걸음으로 온다. 좋은 지식도 개인의 머릿속에만 갇혀 있으면 조직의 자산이 못 된다. 흐르게 해야 자산이다. 그 흐름의 장(場)은 위에서 "공유하라"고 강제한다고 생기지 않는다. 빌더가 풀뿌리처럼 먼저 쓰고 보여줄 때 번진다. 그리고 그 장은 만든 순간이 아니라, 누가 무엇을 언제 했는지 추적이 붙는 순간 — 운영 가시성이 붙는 순간 — 에야 시스템이 된다.
>
> 이 장의 나는 J-curve의 ④ 의구심 구간에 있다. 식별자를 잘못 잡아 전면 리팩토링을 하고, 공들여 만든 기능이 운영 가시성 없이 죽어 있는 걸 발견하는 자리. 그 바닥에서, 운영의 빛이 들어오는 작은 회복점까지가 이 장이다.

좋은 아이디어 하나가 어느 회의에서 잠깐 반짝였다가, 그대로 사라지는 걸 본 적이 있는가? 분명 그 자리에선 다들 "그거 좋네요" 했는데, 일주일만 지나도 누가 무슨 말을 했는지 아무도 기억하지 못한다. 정작 그 아이디어가 필요한 옆 팀은 그런 게 있었다는 사실조차 모른다. 회의록 어딘가에 한 줄 적혀 있을지도 모르지만, 그 회의록을 다시 펼쳐보는 사람은 없다.

조직에서 가장 아까운 낭비가 뭘까. 나는 *흐르지 못하고 고여 있다가 증발해버리는 지식*이라고 생각한다. 누군가는 이미 풀어본 문제를 옆 사람이 처음부터 다시 헤맨다. 누군가의 머릿속에 있는 노하우가 그 사람이 자리를 비우는 순간 통째로 사라진다. 좋은 스킬을 가진 사람이 있는데, 정작 그 스킬이 필요한 곳에서는 그런 사람이 있는 줄도 모른다. 지식은 분명히 조직 안에 있는데, 흐르지 않으니 없는 것이나 마찬가지다.

앞의 두 장이 *성과를 무엇으로 볼지*(5장)와 *사람을 어떻게 배분할지*(6장)를 다뤘다면, 남은 질문은 사람의 머릿속에 든 것 — 아이디어, 지식, 스킬 — 을 어떻게 조직의 혈관에 흘려보낼 것인가다. 함께 더듬어보자.

흐르지 않는 지식은 자산이 아니다

먼저 좌표부터 잡아보자. 우리는 흔히 "지식이 곧 자산"이라는 말을 한다. 맞는 말이다. 그런데 이 말에는 숨은 조건이 하나 있다. **지식은 흐를 때에만 자산이 된다.** 고여 있는 지식은 자산이 아니라, 기껏해야 한 사람의 사유물이거나, 최악의 경우 아무도 모르는 채 썩어가는 재고에 가깝다.

자산이라는 건 본래 여러 사람이 가져다 쓸 수 있어야 한다는 뜻을 품고 있다. 공장의 기계는 한 사람만 쓰라고 들여놓는 게 아니다. 자본은 묶여 있으면 죽은 돈이고 돌아야 자본 노릇을 한다. 지식도 똑같다. 한 사람의 머릿속에만 있는 노하우는 그 사람에게는 자산일지 몰라도 조직에게는 자산이 아니다. 그것이 문서가 되고, 다른 사람이 찾아 쓰고, 또 다른 맥락에서 변형되어 재사용될 때 — 그제야 조직의 자산이 된다. 그러니 우리가 던져야 할 질문은 "어떻게 지식을 쌓을까"가 아니라 **"어떻게 지식을 흐르게 할까"** 여야 한다.

그렇다면 흐르게 하려면 어떻게 해야 할까? 가장 흔한 답이 하나 있다. 위에서 "이제부터 모두 지식을 공유하시오"라고 명령하는 것이다. 공유 포털을 하나 만들고, 분기마다 "지식 공유 실적"을 KPI로 걸고, 안 올리면 닦달한다. 내 경험으로는 이 방식은 거의 통하지 않는다. 강제로 채워진 포털은 형식적인 문서들의 무덤이 된다. 다들 닦달받으니까 마지못해 하나씩 올리는데, 정작 아무도 그걸 읽지 않는다. 올리는 사람도 알고 있다. 이건 그냥 실적용이라는 걸.

여기서 흥미로운 관찰이 하나 있다. 여러 빌더들의 회고를 보면, 사내 도구나 지식이 진짜로 번지는 방식은 톱다운 강제가 아니라 그 반대였다.[40] 누군가 먼저 만들어 쓰면서 "야, 이거 쓰니까 이렇게 편해지더라" 하고 보여주면, 옆 사람이 호기심에 따라 해보고, 그게 또 옆으로 번진다. 한 회사에서는 엔지니어가 만든 사내 도구가 개발팀을 넘어 비개발 부서로 유기적으로 퍼져나갔고, 그제야 경영진이 "이거 전사로 키우자"고 격상했다는 회고도 있다.[41]

이걸 어떤 이는 *"내가 얼마나 게으른지 봐(look how lazy I am)"* 모델링이라고 부른다.[42] 채택은 명령으로 끌어내는 게 아니다. 내가 먼저 편하게 일하는 모습을 보여주면, 옆에서 "어, 나도 저렇게 하고 싶다"는 마음이 슬그머니 인다. 지식이 흐르는 장을 만들 때도 이 원리가 그대로 적용된다. 빡빡한 규칙으로 채우려 들면 고이고, 먼저 쓰는 사람이 즐거워 보이면 흐른다. **풀뿌리 채택이 전사 채택의 진짜 촉매다.**

이게 한 사람의 별난 취향이나 우리 조직만의 외딴 실험만은 아닌 것 같다. 잠깐 시선을 바깥으로 돌려보자. 최근 공개적으로 발표된 한 기업의 흐름을 보면, 그 방향이 묘하게 닮아 있다.[43] 그 기업은 *"모든 구성원이 본인 업무에 특화된 AI를 만든다"*는, 이른바 **1인 1 AI 에이전트** 전략을 정면에 내걸었다. 흥미로운 건 그걸 위에서 도구 하나 사다 꽂는 식으로 풀지 않았다는 점이다. 구성원이 스스로 자기 업무의 AI 과제를 구체화해 올리는 사내 플랫폼을 열고, 자기 일하는 방식을 AI에게 직접 가르치는 협업 도구를 쥐여주고, 현장의 문제를 구성원이 직접 AI로 풀어보는 사내 해커톤까지 열었다. 지식과 도구의 무게중심을, 몇몇 전문가가 만들어 내려보내던 자리에서 전 구성원이 직접 만들고 흐르게 하는 자리로 옮긴 것이다. 사내 플랫폼은 지식이 모이고 흐를 통로이고, 해커톤은 그 흐름에 불을 붙이는 풀뿌리의 마당이다. 내가 사내 지식·기여 플랫폼을 더듬으며 깨달은 그 방향 — 풀뿌리가 촉매라는 것 — 이, 알고 보면 업계가 공개적으로 가고 있는 방향과 같은 좌표 위에 있었던 셈이다. 솔직히, 한밤중에 혼자 더듬을 땐 이게 나만의 고집처럼 느껴질 때가 있었다.

그런데 여기에 함정이 하나 숨어 있다. 지식을 흐르게 하겠다고 시스템을 만들면, 흔히 "자동으로 문서를 생성해주자"는 유혹에 빠진다. 코드를 분석해 문서를 자동 생성하고, 회의를 녹취해 요약을 자동 생성하고... 편리해 보인다. 그런데 여러 현장에서 똑같은 역설이 보고된다. **자동 생성된 문서가 곧 새로운 부채가 되더라는 것이다.**[44] 한번 자동으로 찍어낸 문서는 원본이 바뀌어도 같이 바뀌지 않는다. 시간이 지나면 내용과 어긋나는, 이른바 *철 지난(stale)* 문서가 쌓인다. 그런데 자동 생성된 문서는 양이 많고 그럴듯해 보여서, 사람들이 그걸 믿고 따라가다 오히려 틀린 길로 빠진다. 차라리 문서가 없으면 "없으니까 물어봐야지" 하는데, 틀린 문서가 있으면 "있으니까 읽으면 되겠지" 하고 속는다. 지식을 흐르게 하려다 오히려 흐름을 오염시키는 것이다. 이 역설은 뒤에서 다시 만나게 된다.

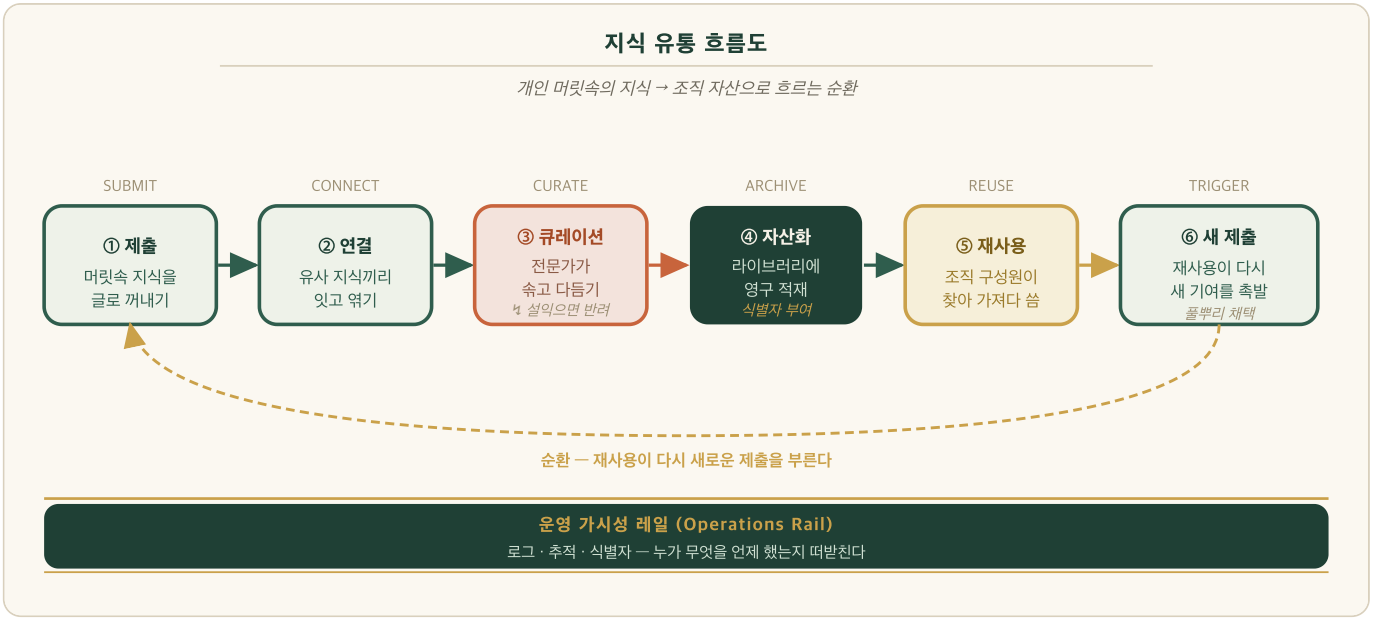


그림 7-1. 지식 유통 흐름도

그림 7-1에서 보듯, 지식이 흐른다는 건 한 방향으로 쌓이는 게 아니라, 제출되고 → 연결되고 → 다듬어지고 → 자산이 되고 → 재사용되고 → 그 재사용이 다시 새로운 제출을 부르는 *순환*이라는 것. 그리고 그 순환 아래에는, 누가 무엇을 언제 했는지를 떠받치는 운영의 레일이 깔려 있어야 한다는 것. 이 그림의 윗줄(흐름)은 누구나 봐야 하고, 아랫줄(레일)은 만드는 사람이 짊어진다.

머릿속에만 있으면, 그건 없는 것이다

좌표를 잡았으니 내 이야기를 해보자. 나는 사람들의 아이디어와 지식과 스킬을 흐르게 하는 장을 직접 만들어보기로 했다. 머릿속에 있는 좋은 것들을 꺼내 모으고, 잇고, 다듬어 누구나 가져다 쓸 수 있게 하는 플랫폼이다. 아이디어를 제출하고 서로 연결할 수 있는 공간, 그 분야 전문가가 들여다보고 솎아주는 자리, 가볍게 실험해볼 수 있는 실험실, 그리고 다듬어진 지식과 스킬을 차곡차곡 쌓아 자산으로 만드는 라이브러리까지 — 머릿속의 것이 흘러 자산이 되는 한 바퀴를 통째로 담으려 했다.

그런데 만들기 시작하자마자 가장 먼저 부딪힌 건 기술이 아니었다. *사람의 마음*이었다.

사람들이 자기 머릿속의 좋은 것을 잘 안 꺼내는 건 게을러서라기보다, 대개 두 가지 때문이다. 하나는 *귀찮아서*다. 머릿속에 있을 땐 분명한데, 그걸 남이 알아볼 수 있게 글로 옮기는 건 생각보다 큰 품이 든다. "이걸 누가 보겠어" 싶은 마음이 들면, 그 품을 들일 이유가 사라진다. 다른 하나는 *어디에 어떻게 넣어야 할지 막막해서*다. 빈 칸 하나 덜렁 있으면 무슨 말부터 써야 할지 모르겠고, 반대로 채워야 할 칸이 스무 개쯤 되면 시작도 하기 전에 지친다.

여기서 줄다리기가 시작된다. 지식을 흐르게 하려면 어느 정도는 *구조화*가 필요하다. 아무 형식 없이 자유롭게만 받으면, 모인 것들이 제각각이라 나중에 찾을 수도 이을 수도 없다. 검색도 안 되고 분류도 안 되는 글 무더기는, 안 모은 것과 별로 다르지 않다. 그래서 "제목은 필수로, 분야는 골라서, 본문은 최소 몇 글자 이상" 같은 틀을 잡고 싶어진다.

그런데 **틀을 너무 빡빡하게 잡으면, 아무도 안 쓴다.** 필수 입력 칸이 늘어날 때마다, 글자 수 하한이 높아질 때마다, 유형 분류가 까다로워질 때마다, 사람들은 하나둘 떨어져 나간다. 머릿속에 좋은 게 있어도 "이걸 다 채워야 한다고? 그냥 말지" 하고 닫아버린다. 좋자고 만든 구조가, 흐름을 막는 댐이 되는 것이다.

이 줄다리기 앞에서 나는 꽤 오래 찜찜했다. 자유롭게 풀면 혼돈이 되고, 조이면 아무도 안 쓴다. 어느 쪽도 답이 아니었다. 한참을 헤매고 나서야 깨달았다. 이건 *한 번에 정답을 맞히는 문제가 아니라, 계속 손으로 다듬어 균형점을 찾아가는 문제*라는 걸. 어느 칸을 필수로 둘지, 글자 수 하한을 어디에 그을지, 유형을 몇 개로 나눌지 — 이건 설계도에 미리 적어두고 끝낼 수 있는 게 아니었다. 실제로 사람들이 쓰는(혹은 안 쓰는) 모습을 보면서, 빡빡한 쪽으로 갔다가 풀고, 풀었다가 다시 조이고를 반복하며 찾아가는 것이었다.

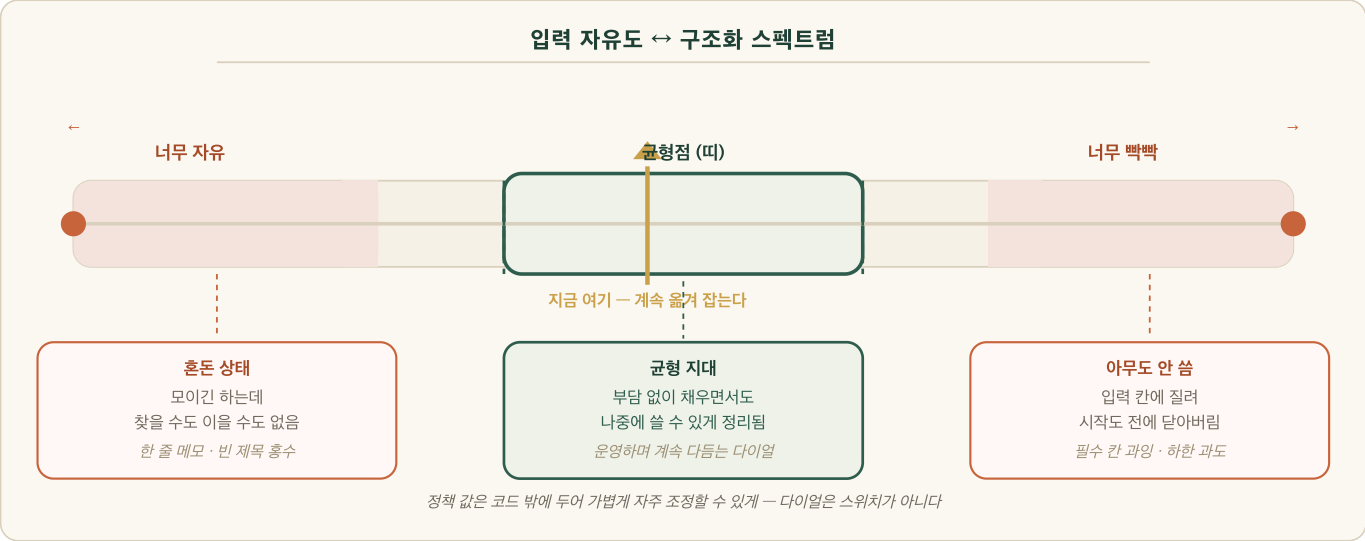


그림 7-2. "입력 자유도 ↔ 구조화" 스펙트럼

그리고 또 하나, 머릿속의 것을 흐르게 하는 일에는 *솎아주는 사람*이 필요했다. 아이디어가 모이기 시작하면, 모인다고 다 좋은 게 아니다. 비슷한 게 흩어져 있고, 설익은 게 섞여 있고, 정말 빛나는 건 묻혀 있다. 이걸 누군가 들여다보고 "이건 이것과 잇자", "이건 더 다듬으면 좋겠다", "이건 라이브러리에 자산으로 올리자" 하고 큐레이션해줄 때, 비로소 흐름에 방향이 생긴다. 그래서 그 분야를 잘 아는 전문가에게 특별한 권한과 전용 화면을 주어, 흐름의 길목을 지키게 했다.

> 부끄러운 고백을 하나 하자. 입력 정책을 잘못 잡았을 때의 대가를 나는 양쪽으로 다 치렀다. 처음엔 "제대로 모으자"는 욕심에 필수 칸을 잔뜩 두고 글자 수 하한도 높게 잡았다. 그러자 제출 자체가 뚝 끊겼다. 사람들이 입력 화면을 열었다가 그냥 닫는 게 눈에 보였다. 놀라서 반대로 확 풀었더니, 이번엔 한 줄짜리 메모와 빈 제목이 쏟아져 들어와 나중에 무엇 하나 찾아 쓸 수가 없었다. 두 번의 실패 사이에서 며칠을 까먹고 나서야, 이건 한 번에 맞히는 게 아니라 운영하면서 계속 옮겨 잡아야 하는 다이얼이라는 걸 받아들였다. 가장 비쌌던 건 *그사이에 떠나간 초기 사용자들의 마음*이었다. 한번 "여기 쓰기 불편하네" 하고 돌아선 사람을 다시 데려오는 건, 처음 데려오는 것보다 훨씬 어렵다.

리더는 여기까지만 읽어도 된다

지금부터는 이 장을 실제로 코드와 데이터 구조로 어떻게 떠받쳤는지, 빌더의 손에 잡히는 이야기로 들어가보자. 직접 이런 장을 만들 사람이라면 끝까지 함께 가면 좋겠다. 식별자를 어디서 못 박아야 하는지, 입력 정책을 어떻게 다듬어야 하는지, 무엇이 붙어야 "기능"이 "시스템"이 되는지, 그리고 거창한 플랫폼 없이도 곁에 있는 도구로 흐름을 터줄 수 있는지 — 값비싸게 배운 것들을 그대로 옮겨두겠다.

하지만 조직을 이끄는 리더라면, **여기까지만 읽어도 충분하다.** 바로 다음 절(③ "지식이 흐르는 장 만들기")로 건너뛰어도 이 장의 알맹이는 빠짐없이 챙긴 셈이다. 단 하나의 문장으로 줄이면 이렇다 — *지식을 흐르게 하는 일은 입력을 적당히 구조화하는 줄다리기에서 시작해, 운영 가시성이 붙는 순간에야 비로소 완성된다.* 이 문장 하나면 ③에서 다시 만나도 충분하다.

자, 그럼 빌더들과 조금 더 깊이 들어가보자.

사람을 가리키는 이름표는 사번이어야 한다

만들면서 호되게 배운 게 하나 있다. *사람을 가리키는 식별자*를 잘못 잡았던 일이다. 처음엔 시스템 내부에서 임의로 매긴 번호로 사람을 구분했다. 그런데 이 플랫폼은 다른 사내 시스템들과 이어져야 했다. 인사 정보와도 엮이고, 다른 도구의 기록과도 맞춰봐야 했다. 그때마다 "우리 시스템의 5번 사람이, 저쪽 시스템의 누구지?"를 매번 손으로 짝지어야 했다. 결국 사람을 가리키는 이름표를 통째로 갈아엎어 조직 공통의 기준 — 사번 — 으로 바꿨는데, 이게 전면 리팩토링에 가까운 큰 수술이었다. 식별자는 주소에도, 데이터베이스에도, 다른 시스템과 주고받는 연결부에도 손가락처럼 뻗어 있어서, 하나를 바꾸면 연결된 모든 곳이 따라 흔들린다.

여기서 건진 교훈은 거창한 설계론이 아니라 아주 구체적인 한 줄이다. **사람을 가리키는 식별자는 결국 조직 어디서나 통하는 공통 기준 — 사번 — 이어야 한다.** 이 시스템에서만 쓰는 임의 번호로 사람을 가리키면, 바깥과 맞춰볼 때마다 비싼 값을 치른다. 사람을 다루는 시스템이라면, 첫날부터 사번을 이름표로 삼자.

흥미로운 건, 이 작은 교훈이 책의 마지막 장에서 뜻밖의 데자뷔로 돌아온다는 점이다. 사람을 사번으로 가리켜야 하듯, 난립하는 에이전트도 결국 같은 발상으로 간다. 조직에서 제대로 일할 에이전트라면, 그것 역시 어딘가 공통의 이름표로 가리킬 수 있어야 하지 않겠는가. 그 이야기 — 에이전트에게도 '사번'을 주자는 발상 — 는 9장에서 본격적으로 펼쳐진다.

정책은 다이얼이지, 스위치가 아니다

식별자가 뼈대였다면, 입력 정책은 *살갗*이었다. 사람들이 매일 손으로 만지는 부분이라, 여기가 까끌하면 바로 떠난다.

앞에서 이야기한 그 줄다리기 — 자유 입력과 구조화 사이 — 를 코드에서는 결국 *정책 값을 손으로 돌려가며* 풀었다. 본문 글자 수의 하한과 상한, 어떤 칸을 필수로 둘지, 유형을 몇 갈래로 나눌지. 이것들은 처음 한 번 정하고 끝낼 수 있는 게 아니었다. 제출이 끊기면 풀고, 쓰레기가 쌓이면 조이고를 반복했다.

여기서 한 가지 설계 결정이 두고두고 도움이 됐다. 이 정책 값들을 *코드 깊숙이 박아두지 않은 것*이다. 사실 처음엔 나도 그냥 코드 안에 숫자로 박아둘 뻔했다. 글자 수 하한 같은 걸 코드에 박아뒀다면, 그걸 바꿀 때마다 코드를 고치고 다시 배포해야 했을 것이다. 다이얼 한 칸 돌리는 데 매번 빌드와 배포가 따라붙는다. 그런 무게가 붙으면 "조금만 풀어볼까?" 같은 가벼운 실험은 자꾸 뒤로 밀린다. 그래서 이런 값들은 바깥에서 손쉽게 바꿀 수 있는 자리로 빼뒀다. 다이얼이 가벼워지자, 그제야 *자주* 돌려가며 균형점을 더듬을 수 있었다.

이 경험에서 나는 작은 원칙 하나를 얻었다. **사람의 행동에 닿는 정책은 스위치가 아니라 다이얼로 만들어라.** 그리고 그 다이얼은 손이 쉽게 닿는 곳에 둬야 한다. 한 번에 정답을 맞히겠다는 생각을 버리고, "자주 가볍게 고쳐 잡겠다"는 쪽으로 설계를 트는 것 — 사람이 매일 만지는 시스템일수록 이 차이가 크다.

시스템은 보이기 시작할 때 비로소 시스템이 된다

식별자도 잡고 입력 정책도 다듬었다. 기능은 다 돌아갔다. 아이디어가 제출되고, 연결되고, 라이브러리에 자산으로 쌓였다. 화면상으로는 멀쩡한 시스템이었다. 그런데도 어딘가 *시스템 같지가 않았다*. 이 찜찜함의 정체를 한참 뒤에야 알았다.

문제는 단순했다. **나는 안에서 무슨 일이 벌어지는지 볼 수가 없었다.** 누가 무엇을 언제 제출했는지, 어떤 글이 실제로 읽히고 있는지, 어디서 에러가 나는지, 혹시 누가 엉뚱한 경로로 남의 데이터에 닿고 있지는 않은지 — 알 길이 없었다. 무언가 잘못됐다는 사용자 문의가 들어와도, 그게 언제 어디서 비롯됐는지 거꾸로 짚을 방법이 없었다. 시스템은 돌아가는데, 나는 그 시스템에 대해 *눈을 감고 있는* 상태였다.

이게 5장과 6장에서도 거듭 부딪혔던 바로 그 벽이다 — **만든 것과 운영되는 것은 다르다.** 기능이 동작하는 것과, 그 기능이 *운영되는 시스템*인 것은 전혀 다른 단계다. 그 사이를 메우는 게 운영 가시성이다. 그래서 손을 댔다. 운영 로그를 남기고, 접속 흔적을 추적하고, 시스템의 상태를 한눈에 들여다볼 수 있는 관측 도구[45]를 연동했다. 누가 언제 무엇을 했는지, 어디가 아픈지, 어디가 활발한지가 비로소 화면에 떴다.

그 순간이 묘했다. 코드는 거의 그대로인데, *느낌*이 완전히 달라졌다. 그전까지는 "내가 만든 기능들의 모음"이었던 것이, 운영 가시성이 붙는 순간 "살아 돌아가는 시스템"이 됐다. 안에서 벌어지는 일이 보이니까, 비로소 손볼 곳을 알게 됐고, 안심하고 사람들에게 권할 수 있게 됐다. **만든 순간이 아니라, 운영 가시성이 붙는 순간 비로소 시스템이 된다.** 이게 이 장에서 내가 가장 늦게, 그러나 가장 또렷이 배운 것이다.

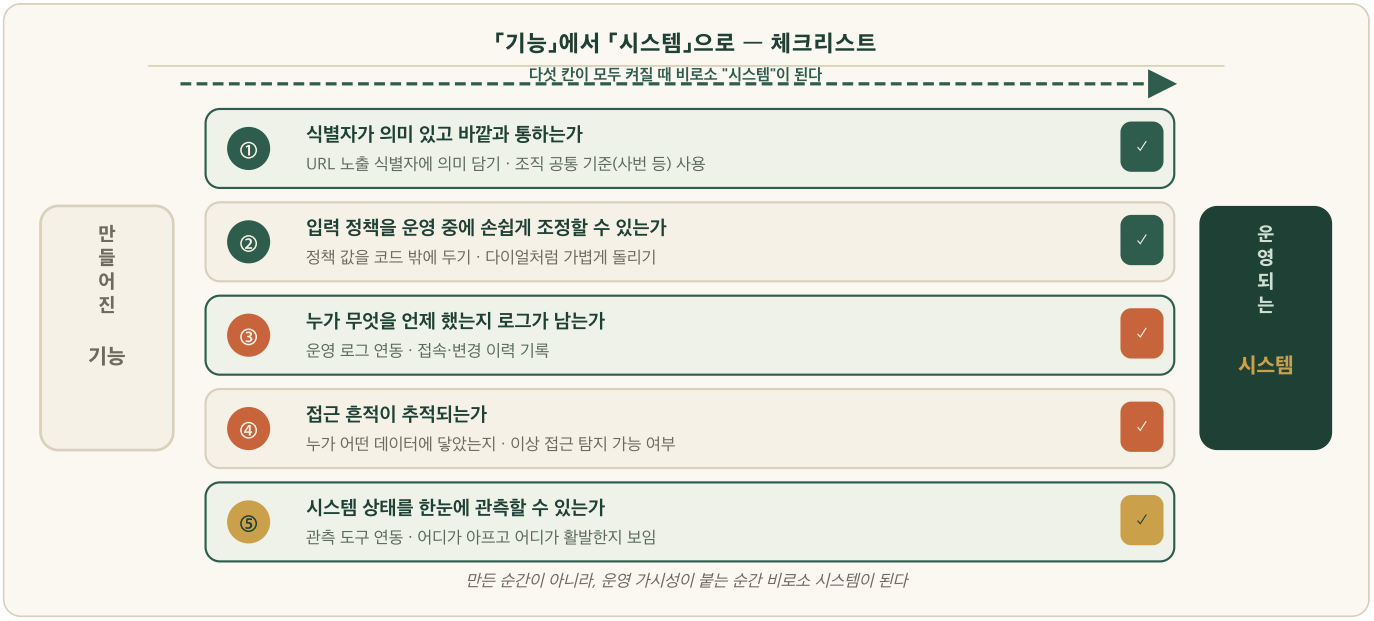


그림 7-3. "기능 → 시스템" 체크리스트

레일은 의외로 곁에 있는 평범한 도구에서 시작된다

여기까지 읽고 나면 한 가지 오해가 생기기 쉽다. "지식을 흐르게 하려면 결국 저렇게 번듯한 플랫폼을 하나 만들어야 하는구나" 하는 오해 말이다. 그런데 그렇지 않다. 지식이 흐르는 레일은, 의외로 *이미 당신 옆에 있는 평범한 도구들*에서 시작된다. 잠깐 시선을 플랫폼 바깥으로 돌려보자.

생각해보면 이상한 일이다. 버전관리 도구, 위키, 노트 앱 — 이것들은 원래 개발자들의 전유물이었다. 코드의 변경 이력을 한 줄 한 줄 남기고, 누가 언제 무엇을 고쳤는지 되짚고, 흩어진 메모를 한곳에 모아 잇는 일. 개발자에게는 숨쉬듯 당연한 이 도구들이, 정작 기획·HR·운영 담당자에게는 오랫동안 남의 나라 이야기였다. 그런데 어느 순간부터, 이 경계가 슬그머니 허물어지기 시작했다.

나는 그 장면을 직접 거들어본 적이 있다. 비개발 부서의 한 동료가 산출물을 제대로 쌓아두고 싶어 했는데, 마침 버전관리 도구가 딱 맞는 그릇이었다. 문제는 진입장벽이었다. 비개발자에게 버전관리 도구는 낯설고 험준하다. 명령어는 외계어 같고, 한번 꼬이면 어디서부터 손대야 할지 막막하다. 그래서 나는 모든 기능을 다 가르치려 들지 않았다. 그가 당장 필요한 동작 — 올리고, 되돌리고, 남기는 — 딱 그만큼만 추려서 "최소 사용법"으로 좁혀 손에 쥐여줬다. 레일은 무거우면 아무도 안 올라탄다.

그 위에 LLM이 한 겹 얹혔다. 예전 같으면 그가 손으로 일일이 요약하고 서식을 맞추던 일을, 이제는 LLM이 변환 계층처럼 대신 처리한다. 흩어진 산출물을 읽기 좋게 정리하고, 형식을 다듬고, 보기 좋은 모양으로 바꾼다. 흥미로운 건 여기서 자동화된 게 지식의 *생산*이 아니라는 점이다. 좋은 내용은 여전히 사람의 머리에서 나온다. 자동화된 건 그것의 *유통과 표현* — 흐르게 하고 보기 좋게 만드는 그 마지막 한 뼘이다.

마지막으로, 그렇게 다듬어진 산출물을 HTML 보고서로 변환해 내부 서비스에 올려 공유했다. 예전엔 문서 파일을 첨부해 메일로 돌리던 것이, 이제는 링크 하나로 누구나 즉시 열어보는 페이지가 됐다. 보고의 형식 자체가 *닫힌 첨부*에서 *열린 링크*로 옮겨간 것이다. 흐름이 한결 매끄러워졌다.

여기서 잠깐 솔직하게 한 줄만 덧붙이자. 지금 당신이 읽는 이 글이 만들어진 작업 흐름조차 사실 별다른 게 아니었다. 변경 이력(커밋 로그)을 그러모으고 → 그걸 바탕으로 노트를 자동으로 짓고 → HTML로 변환해 → 사내에 발행하는, 그 평범한 조합. 자랑하려는 게 아니라, 오히려 그 *평범함*을 말하고 싶어서다. 지식이 흐르는 레일이란 게 대단한 신기술의 결정체가 아니라, 손에 익은 도구 몇 개를 LLM으로 잇고 결과를 열린 링크로 푸는, 이 정도의 조합에서 이미 돌아가고 있더라는 것.

이 경험에서 두 가지가 또렷해졌다. 하나, **AX는 새 도구를 사오는 일이 아니다.** 기존의 개발자 도구가 전사로 흐르고, 그 위에 LLM이 요약·변환 계층으로 얹히는 일에 더 가깝다. 비싼 솔루션을 들이기 전에, 이미 곁에 있는 도구가 옆 부서로 번질 길부터 터주자. 둘, 도구가 이렇게 민주화되면 *새로운 역할* 하나가 따라온다. 동료의 '도구 온보딩'을 곁에서 거들어주는 사람. 개발자가 자기 코드를 짜는 데서 멈추지 않고, 옆자리 비개발 동료가 그 도구에 발을 들이도록 손을 내미는 것 — 이것이 흐름을 넓히는 새로운 협업의 모양이다.

이 작은 사례에도 만든 것과 운영되는 것의 차이가 있다. 버전관리 도구를 동료에게 깔아주는 건 *만드는* 일이다. 하지만 그가 매일 그 도구로 일하고, 산출물이 링크로 흐르고, 옆 사람이 그걸 열어보는 것 — 그게 *운영되는* 것이다. 도구를 쥐여주는 데서 멈추지 말고, 그 위에 흐름이 실제로 도는지까지 봐야 비로소 레일이 깔린 것이다.

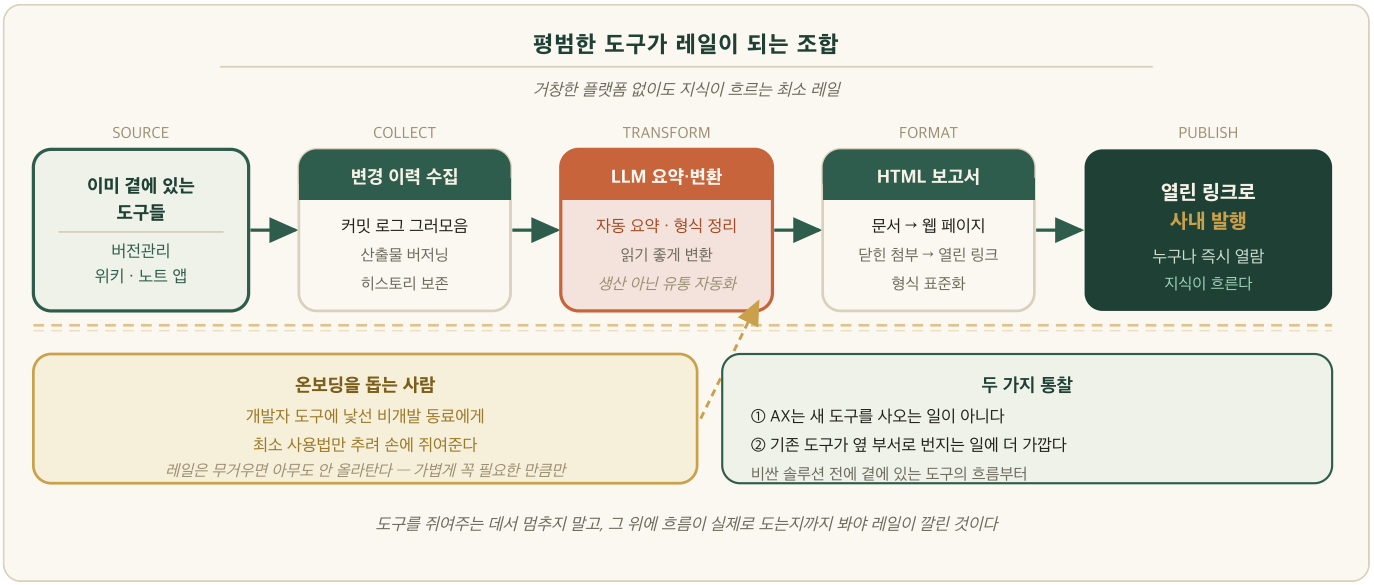


그림 7-4. 평범한 도구가 레일이 되는 조합

여기까지가 빌더의 영역이었다. 식별자, 입력 정책, 운영 가시성, 그리고 곁에 있는 도구를 옆 부서로 흘려보내는 일 — 이것들이 흐름을 떠받치는 보이지 않는 레일이다. 다음 절에서 다시 모두 합류해, 이걸 당신의 현장으로 옮겨보자.

지식이 흐르는 장 만들기

이제 이 장의 이야기를 당신의 현장으로 옮겨보자. 리더든 빌더든, 자기 조직에서 지식을 흐르게 하는 장을 만들려 할 때 무엇을 챙기면 좋을지를 함께 정리해보자.

입력 구조화 vs 자유도 — 다이얼 맞추기

가장 먼저 부딪히는 줄다리기다. 너무 자유로우면 혼돈이 되고, 너무 빡빡하면 아무도 안 쓴다. 이걸 한 번에 맞히려 하지 말고, 다음 순서로 *돌려가며* 맞춰보자.

- **시작은 느슨하게.** 처음부터 조이지 말자. 필수 칸은 정말 없으면 안 되는 한두 개(이를테면 제목과 분야)만 두고, 나머지는 권장으로 풀어두자. 초반엔 *모이는 것 자체*가 더 중요하다.
- **쓰레기가 쌓이면 그때 조인다.** 한 줄짜리 메모나 빈 제목이 흐름을 흐리기 시작하면, 그때 최소 글자 수나 필수 칸을 한 칸씩 더한다. 조이는 건 항상 *문제를 본 다음*에.
- **정책 값은 코드 밖에 둔다.** 글자 수 하한 같은 값을 코드에 박아두면 다이얼이 무거워진다. 손쉽게 바꿀 수 있는 자리에 빼두어, 가볍게 자주 조정할 수 있게 하자.
- **솎는 사람을 둔다.** 모인다고 다 흐르는 게 아니다. 그 분야를 아는 사람이 잇고 다듬고 자산으로 올려줄 때 흐름에 방향이 생긴다. 큐레이션 권한을 누구에게 줄지 정해두자.

사람을 가리키는 이름표 점검

딱 한 줄만 챙기자. 사람을 다루는 시스템이라면, 그 사람을 가리키는 이름표가 **이 시스템에서만 쓰는 임의 번호인가, 조직 어디서나 통하는 공통 기준(사번)인가**를 첫날 물어보자. 임의 번호로 시작하면 바깥과 맞춰볼 때마다 비싼 값을 치르고, 나중에 사번으로 갈아엎으려면 전면 리팩토링이 된다.

운영 전환 점검표 — "기능"을 "시스템"으로

기능이 다 돌아간다고 끝이 아니다. 다음이 붙어야 비로소 "운영되는 시스템"이다. 만든 것에 하나씩 표시해보자.

- ▢ 누가 무엇을 언제 했는지 **운영 로그**가 남는가?
- ▢ 누가 어디에 접근했는지 **접근 흔적**이 추적되는가?
- ▢ 시스템의 상태(활발한 곳, 아픈 곳)를 **한눈에 관측**할 수 있는가?
- ▢ 문제가 생겼을 때, 그게 **언제 어디서 비롯됐는지 거꾸로 짚을** 수 있는가?

네 칸이 다 비어 있다면, 당신이 만든 건 아직 *기능의 모음*일 뿐이다. 이것들이 하나씩 켜질 때, 그제야 시스템이 된다.

데이터 민감도·접근통제 한 줄 점검

지식을 흐르게 하는 일에는 그늘이 따라온다. 흐르게 한다는 건 곧 *누군가 접근할 수 있게 한다*는 뜻이기 때문이다. 그래서 흐름을 설계할 때 이 한 줄을 반드시 함께 못 박자 — **누가 무슨 지식에 접근하고 기여할 수 있는가, 그리고 그 흔적이 추적(감사)되는가?** 전문가에게 주는 특별한 권한과 뷰, 사람마다 다른 접근 범위, 그리고 "누가 무엇을 봤는지"의 감사 기록은, 운영 가시성을 붙이고 식별자를 설계하는 바로 그 단계에서 함께 들어가야 한다. 나중에 덧붙이는 보안은 늘 구멍이 남는다. 흐르게 하되 아무에게나 다 흐르게 하지는 않도록, 그 경계를 설계 단계에서 그어두자.

마무리

아이디어·지식·스킬이 사람 머릿속에만 있을 때, 그것을 조직 차원에서 흐르게 하려면 어떻게 해야 할까?

이 장을 만드는 내내, 나는 ④의 의구심 한복판에 있었다. 식별자를 잘못 잡아 멀쩡히 돌던 걸 멈춰 세우고 대수술을 할 때, "이게 다 무슨 소용인가" 싶었다. 입력 정책을 풀었다 조였다 하며 떠나간 사용자들을 볼 때, "사람들은 결국 안 쓰는 게 아닐까" 싶었다. 공들여 만든 기능이 운영 가시성 없이 죽은 듯 멀뚱히 떠 있을 때, 가장 깊이 가라앉았다. 좋은 걸 만들었다고 생각했는데, 흐르지 않았으니까.

그런데 바닥에서 올라오는 길은, 의외로 한 곳에서 열렸다. **운영의 빛이 들어오는 순간**이었다. 로그가 남고, 흔적이 추적되고, 안에서 무슨 일이 벌어지는지가 화면에 뜨기 시작하자 — 그제야 비로소 그 장이 *흐르는 시스템*으로 돌기 시작했다. 누가 무엇을 올렸는지 보이니 솎을 수 있었고, 무엇이 읽히는지 보이니 다듬을 수 있었고, 안심하고 권할 수 있으니 사람이 모였다. 식별자라는 뼈대를 다시 세우고, 입력이라는 살갗을 부드럽게 다듬고, 운영이라는 빛을 들이고 나서야 — 머릿속에만 있던 것들이 비로소 조직의 혈관을 타고 흐르기 시작했다.

당신이 무언가 좋은 것을 만들었는데 어쩐지 흐르지 않는다면 — 의심해볼 곳은 기능 그 자체가 아닐지 모른다. *그 기능에 운영의 빛이 들어오고 있는가*, 거기부터 들여다보자.

지금까지 PART 3에서 우리는 사람을 다시 설계해왔다. 성과를 토큰 대신 검증으로 보는 일(5장), 사람을 직무 단위로 묶지 않고 task 단위로 풀어 배분하는 일(6장), 머릿속 지식을 흐르는 자산으로 바꾸는 일(7장). 그런데 이렇게 사람을 다시 설계하고 시스템을 잔뜩 만들고 나니, 마음 한구석에 질문 하나가 자라기 시작했다. *분명 다들 AI를 붙여 빨라졌다는데, 왜 어떤 일은 오히려 더 느려지고 더 손이 가는 걸까?* 만든 것이 운영되는 것과 다르듯, 빨라진 것이 정말 빨라진 것과 다를 수도 있지 않을까.

다음 장에서 그 정체를 함께 들여다보자. AI를 붙였는데 왜 더 느려졌는지, 그리고 그 구덩이에서 어떻게 빠져나오는지를 말이다. 미리 한 가지만 귀띔하자면 — 그건 당신이 뭔가 잘못해서가 아니라, 모두가 통과하는 구덩이의 가장 깊은 바닥이고, 그 바닥에서 올라오는 사다리에는 이름이 붙어 있다. *검증*이라는 이름이.

PART 4. AX를 운영한다는 것 — 레일과 신뢰

> 시스템을 잔뜩 만들고 나면, 구덩이의 가장 깊은 바닥과 마주한다. AI를 붙였는데 왜 더 느려졌는가(8장). 그 정체는 세 가지 부채와 검증 비용이다. 그리고 그 구덩이에서 올라오는 길 — 통제하는 벽이 아니라 흐르게 하는 레일(9장)을 깐다.
>
> *J-curve 위치: ④의구심 바닥(8장) → ⑤이륙(9장), 책 전체의 회복점.*

8장. 세 가지 부채와 검증으로의 전환

> **이 장에서 손에 쥐고 갈 것**
>
> 묻는 질문은 하나다. AI를 붙였는데 왜 더 느려졌나, 그리고 그 구덩이에서 어떻게 나오나.
>
> 줄거리는 이렇게 흐른다. 생산은 빨라진다. 그런데 빨라진 만큼 검증의 짐이 따라 붙어, 어느 구간에선 전체가 도리어 느려진다. 그 정체를 파보면 세 가지 부채가 있다. 그럴싸하지만 틀린 코드(기술부채), 아무도 전체를 모르는 상태(인지부채), 왜 이렇게 만들었는지가 사라지는 일(의도부채). 처방은 의외로 단순하다. 더 빨리 만드는 데 매달리는 대신, 일의 무게중심을 생산에서 검증으로 옮긴다. 검증을 시범 운행처럼 시스템에 미리 깔아두는 것.
>
> J-curve로 치면 여기는 ④ 의구심, "AI 붙였더니 더 느려졌다"는 구덩이의 바닥이다. 이 장 끝에서 바닥을 딛고 올라설 첫 발(검증으로의 전환)만 짚어두고, 그 구현은 다음 장으로 넘긴다.

첫째를 글쓰기 학원에 내려주고 카페 모퉁이에 앉던 그 습관 이야기를 또 꺼내려는 건 아니다. 다만 그 무렵의 일이다. 분명 일이 더 빨라졌어야 했다. 그런데 어쩐 일인지 매일 밤 더 늦게 노트북을 덮고 있었다.

분명 빨라졌는데, 왜 더 느려졌을까

상상해보자. 평소 같으면 하루 꼬박 걸렸을 작업이다. AI에게 맡겼더니 십 분 만에 결과가 나왔다. 화면 가득 그럴듯한 코드가, 혹은 그럴듯한 문서가 쏟아진다. 처음엔 짜릿하다. 이 맛에 AI를 쓰는구나 싶다. 나도 그랬다.

그런데 그 결과물을 막상 쓰려고 들여다보기 시작하면, 어쩐지 손이 멈춘다. 분명 그럴듯한데, 정말 맞는지는 확신이 안 선다. 한 줄 한 줄 따라가며 확인한다. 대체로 맞다. 그런데 군데군데, 정말 감쪽같이 자연스러운 얼굴을 하고 틀린 게 섞여 있다. 그냥 대충 틀린 거라면 차라리 낫다. 한눈에 틀린 건 금방 걸러낼 수 있으니까. 문제는 *그럴싸하게* 틀린 것이다. 너무 자연스러워서, 의심하지 않으면 그냥 넘어가게 되는 것. 그걸 찾아내려면 결국 내가 처음부터 다시 한 번 생각해봐야 한다.

여기서 묘한 일이 벌어진다. 생산은 분명 십 분으로 줄었는데, 검증에 두 시간이 들기 시작한 것이다. 그것도 평소 같으면 하지 않았을 종류의 시간이다. 내가 처음부터 직접 만들었다면, 만드는 동안 자연스럽게 '내가 무엇을 왜 이렇게 했는지'가 머릿속에 쌓인다. 그런데 AI가 만든 걸 받아 들면, 그 쌓임이 없다. 백지에서 거꾸로 "이게 왜 이렇게 됐지?"를 추적해야 한다. 만드는 시간은 줄었는데, 이해하는 시간이 통째로 새로 생겨버린 셈이다.

처음 몇 번은 "내가 아직 AI를 잘 못 다뤄서 그런가 보다" 했다. 프롬프트를 더 정교하게 써봤다. 더 좋은 모델로 갈아끼워도 봤다. 결과물의 그럴듯함은 분명히 올라갔다. 그런데 이상한 일이 벌어졌다. 그럴듯함이 올라갈수록 검증은 *더* 어려워졌다. 어설프게 틀린 건 눈에 띈다. 정교하게 틀린 건 더 깊이 파야 보인다. 좋은 도구를 쓸수록 검증의 함정

이 더 깊어지는 이 역설 앞에서, 나는 솔직히 좀 찜찜했다.

그러다 어느 날, 일주일을 통째로 돌아보고서야 그림이 보였다. 나는 그 주에 평소보다 훨씬 많은 걸 '생산'했다. 그런데 실제로 *쓸 수 있는 상태로 끝낸* 일은 평소와 비슷하거나 오히려 적었다. 생산량 그래프는 치솟았는데, 완료량 그래프는 그대로거나 처졌다. 그 둘 사이의 벌어진 틈, 바로 거기에 검증이라는 보이지 않는 세금이 빨려 들어가고 있었다.

이 느낌, 나만 겪은 게 아니었다. AI를 본격적으로 들인 많은 사람이 비슷한 말을 한다. "생각보다 안 빨라진다." 기대치는 두세 배인데 체감은 십에서 이십 퍼센트 남짓이라는 것이다.[46] 처음엔 이게 엄살이거나 도구를 잘 못 써서라고 여겼다. 그런데 그렇게만 볼 일이 아니었다.

빠른 게 아니라 빚을 지고 있었다

생산이 빨라지는데 전체는 느려지는, 이 직관에 어긋나는 일은 왜 벌어질까?

나는 이걸 "빚"이라는 말로 이해하고 나서야 마음이 좀 풀렸다. AI로 빨리 만든다는 건, 따지고 보면 빠른 게 아니라 *빚을 내는* 것에 가깝다. 당장은 일이 끝난 것처럼 보인다. 그런데 갚아야 할 무언가가 어딘가에서 조용히 쌓이고 있다. 그리고 그 빚은 반드시, 검증이라는 이름의 청구서로 돌아온다. 내가 겪은 부채는 크게 세 가지였다.[47]



그림 8-1. 3대 부채 → 검증 → 레일 흐름도

첫째, 그럴싸하지만 틀린 코드 — 기술부채

첫 번째 빚은 손에 가장 빨리 잡힌다. AI가 만든 코드는 대개 그 자리에서는 멀쩡히 돌아간다. 눈앞의 문제는 푼다. 그런데 그 코드가 시스템 전체와 어떻게 맞물리는지, 다른 부분에 어떤 그늘을 드리우는지까지는 보지 못한다. 당장 눈앞만 보고 최적을 찾되, 전체는 모르는 것이다. 나는 이걸 *국소적으로는 최적인데 전역적으로는 무지한* 코드라고 부른다.

이런 코드는 하나하나 보면 다 그럴듯하다. 그래서 자꾸 쌓인다. 그럴듯한 채로 쌓인 것들이 어느 임계점을 넘는 순간, 시스템은 오히려 손대기 어려워진다. 빨리 만들려고 들인 AI가, 몇 달 뒤엔 도리어 속도를 갉아먹기 시작하는 것이다.[48] 카드빚을 떠올리면 쉽다. 처음엔 결제가 미뤄져 편하다. 그러다 어느 순간 이자가 원금을 짓누른다. 빨리 만든 게 빨랐던 게 아님을, 한참 뒤에야 청구서로 깨닫는다.

둘째, 아무도 전체를 이해하지 못함 — 인지부채

두 번째 빚이 사실 더 무섭다. 눈에 안 보이기 때문이다.

AI가 결과를 와르륵 쏟아내면, 그걸 만든 사람조차 전체를 다 이해하고 있지 못한 상태가 된다. 만드는 과정에서 자연스럽게 쌓였어야 할 이해가, 결과만 받아 들면 통째로 빠진다. 그러면 "이 시스템을 우리 팀이 정말 함께 이해하고 있는가?"라는 질문에 아무도 자신 있게 답하지 못하는 순간이 온다. 예전엔 만들다 보면 이해가 저절로 따라왔다. 이제는 다르다. 이해는 작정하고 따로 시간을 빼야 하는 *별도의 일*이 되어버렸다.

여기서 내가 오래 곱씹은 말이 하나 있다. 생각(thinking)은 AI에게 아웃소싱할 수 있어도, 이해(understanding)는 아웃소싱할 수 없다는 것이다.[49] 처음엔 당연한 말 같았다. 그런데 현장에서 진짜 무서운 건, 사람들이 그 이해마저 슬그머니 항복해버린다는 점이었다. 결과가 그럴듯하니까, 검토하기엔 양이 너무 많으니까, "에이, 잘 됐겠지" 하고 그냥 흘려보낸다. 남이 AI로 만든 걸 제대로 보지도 않고 다음 사람에게 넘기고, 그 사람도 안 보고 또 넘긴다. 아무도 멈춰서 이해하지 않은 결과물이 조직 안을 흘러다니기 시작한다. 시스템은 돌아가는데, 그걸 이해하는 사람은 점점 줄어든다.

셋째, 왜 이렇게 만들었는지가 사라짐 — 의도부채

세 번째 빚은 시간이 지나야 청구된다. "왜 이렇게 만들었는가"가 사라지는 것이다.

무언가를 만들 때는 늘 이유가 있다. 왜 다른 길 대신 이 길을 골랐는지, 어떤 제약 때문에 어쩔 수 없이 이렇게 했는지, 무엇과 무엇을 저울질해 이쪽을 택했는지. 사람이 직접 만들면 이 맥락이 머릿속에, 혹은 동료와 나눈 대화 속에 남는다. 그런데 한 사람이 에이전트와 단둘이 후딱 만들고 나면, 그 맥락은 어디에도 남지 않는다. 기껏해야 그때 한 번 쓰고 사라질 프롬프트 어딘가에 휘발성으로만 존재한다.

당장은 아무 문제가 없다. 문제는 반년 뒤, 그걸 고치거나 이어받아야 할 때 터진다. "이거 왜 이렇게 돼 있지?"라고 물어도 답할 사람이 없다. 만든 사람조차 기억하지 못한다. 사람을 줄였다가 그 사라진 맥락 때문에 다시 사람을 불러들이는 일이, 실제로 여기저기서 벌어졌다.[50] 의도부채는 그렇게, 가장 늦게 그러나 가장 비싸게 청구된다.

이 세 가지를 늘어놓고 보니, 내 그 찜찜했던 일주일이 한눈에 설명됐다. 나는 빠르게 일한 게 아니었다. 세 종류의 빚을 한꺼번에 지고 있었을 뿐이다. 그리고 그 빚을 갚는 자리가 바로 검증이었다. 검증이 무거워진 건 생산이 빨라져서가 아니다. *빚을 낸 만큼 청구서가 날아온 것*이다. 이렇게 보면 "AI 붙였더니 더 느려졌다"는 말도 하나도 이상하지 않다.

그렇다면, 무게중심을 옮기자

빚이 문제라면, 해법은 둘 중 하나다. 빚을 안 지거나, 갚는 구조를 만들거나. AI를 쓰지 않고 빚을 안 지는 길도 있긴 하다. 하지만 그건 이 책이 권하는 길이 아니다. 우리는 이미 생산이 빨라지는 맛을 봤고, 그걸 포기할 수도 포기할 필요도 없다. 그렇다면 답은 갚는 구조를 만드는 쪽이다.

여기서 발상의 전환이 필요하다. 지금까지 우리가 일을 잘한다고 할 때, 그 중심에는 늘 *생산*이 있었다. 더 많이 만들고, 더 빨리 만드는 사람이 일 잘하는 사람이었다. 그런데 생산을 AI가 대신 해주는 시대에는, 사람이 붙들고 있어야 할 자리가 달라진다. 만드는 일은 점점 AI 쪽으로 넘어가고, 사람의 본업은 *검증*으로 옮겨간다.[51] 생산은 AI에게 맡기고, 사람은 그 결과물을 끝까지 책임지고 검증한다.

말로는 쉬운데, 막상 이걸 받아들이기가 생각보다 거북하다. 평생 "만드는 게 일"이라고 배워온 사람에게, "이제 네 일은 검증이다"라고 하면 어쩐지 일이 줄어든 것 같고, 한 단계 밀려난 것 같은 기분이 든다. 나도 처음엔 그랬다. 그런데 겪어보니 정반대였다. 좋은 검증은 생산보다 훨씬 어렵고, 훨씬 높은 안목을 요구한다. 그럴싸하게 틀린 걸 잡아내려면, 만들 줄 아는 것보다 더 깊이 알아야 하니까. 검증으로 옮겨가면 일이 줄어드는 대신, 일의 무게중심이 *더 높은 곳*으로 올라간다.

트램을 개통하기 전에 — 시범 운행이라는 검증

여기서 그림 하나를 빌려오자. 도시에 새 트램 노선을 깐다고 해보자. 선로를 다 깔았다. 전력도 연결했고, 차량도 들여왔다. 자, 이제 바로 승객을 태워도 될까? 그렇게 하는 도시는 없다. 정식 개통 전에 반드시 *시범 운행*을 한다. 승객 없이 빈 차로 노선을 몇 번이고 돌려보는 것이다. 곡선 구간에서 흔들리진 않는지, 신호는 제대로 맞물리는지, 예상 못 한 데서 멈추진 않는지.

AI가 만든 산출물도 이 트램과 같다. 다 만들어졌다고 곧장 실제 업무에, 실제 사람에게, 실제 데이터에 태우면 안 된다. 반드시 시범 운행을 거쳐야 한다. 그 시범 운행이 바로 검증이다. 빈 차로 먼저 돌려보며 "이게 정말 사람을 태워도 되는 물건인가"를 확인하는 절차 — 검증을 이렇게 그려두면, 그게 일을 가로막는 군더더기가 아니라 *개통의 필수 조건*이라는 게 자연스럽게 느껴진다. 시범 운행 없이 승객을 태워 사고가 안 났다면, 그건 운이지 안전이 아니다.

검증을 어디에, 어떻게 깔 것인가 — 세 겹의 시범 운행

그렇다면 시범 운행, 즉 검증은 구체적으로 어떻게 깔아야 할까? 나는 검증을 한 종류로 뭉뚱그리지 않고, 성격이 다른 세 겹으로 나눠 생각하게 됐다.[52] 여기서는 그 *개념*만 짚어두자. 실제로 이걸 코드로 어떻게 깔았는지는 다음 장에서 본격적으로 풀 테니.



그림 8-2. 검증 레이어 3종 표

첫째는 **통과냐 실패냐가 분명한 검증**이다. 정답이 또렷한 것들 — 형식이 맞는지, 규칙을 어기지 않았는지, 일단 돌아가긴 하는지. 이런 건 사람이 매번 눈으로 볼 일이 아니다. 자동 검사로 기계에게 맡긴다. 트램으로 치면 "선로가 끊긴 데는 없는가"처럼, 답이 예/아니오로 떨어지는 점검이다.

둘째는 **숫자로 재는 검증**이다. 얼마나 빨리 처리하는지, 응답에 시간이 얼마나 걸리는지, 비용이 얼마나 드는지. 이건 통과·실패로 나눌 순 없지만, 수치로 재고 임계값을 정해둘 수 있다. "이 선을 넘으면 멈춰 서서 들여다본다"는 경보선을 긋는 일이다.

셋째가 가장 어렵고, 가장 새롭다. **좋고 나쁨을 안목으로 따져야 하는 검증**이다. 글이 잘 쓰였는지, 판단이 적절한지 같은 건 통과·실패로도, 숫자로도 깔끔하게 갈리지 않는다. 예전엔 이건 오로지 사람만 할 수 있는 일이었다. 그런데 흥미롭게도, 이 *정성적인 심사*조차 잘 짜인 채점 기준표만 있으면 AI에게 1차로 맡길 수 있게 됐다. AI가 만든 걸 또 다른 AI가 채점 기준에 따라 심사하는 것이다. 다만 여기엔 반드시 지켜야 할 단서가 하나 붙는데, 그 이야기는 잠시 뒤에 하자.

이 세 겹을 다 깔 필요는 없다. 일의 성격에 따라 한 겹만으로 충분할 때도 있고, 세 겹을 포개야 할 때도 있다. 중요한 건 "이 일에는 어떤 시범 운행이 필요한가"를 먼저 묻는 습관이다. 검증을 깐다는 건, 결국 *나의 일마다 그에 맞는 시범 운행 절차를 설계해두는 것*이다.

리더는 여기까지만 읽어도 된다

지금까지가 이 장의 뼈대다. 세 가지 부채가 구덩이의 정체였고, 처방은 생산에서 검증으로 무게중심을 옮기는 것이며, 검증은 트램의 시범 운행처럼 세 겹으로 깔 수 있다 — 여기까지 손에 쥐었다면, 조직을 이끄는 리더는 바로 마지막 절(③ "우리 조직의 부채 점검")로 건너뛰어도 좋다. 이 장에서 꼭 가져가야 할 건 이미 다 쥔 셈이다. 기억할 단 한 문장은 이것이다 — *AI 시대에 일을 잘한다는 건 더 많이 생산하는 게 아니라, 더 잘 검증하는 것이다.*

지금부터 이어지는 짧은 한 절은, 앞선 장들에서 내가 겪은 일들이 사실은 이 '세 부채' 프레임의 조각들이었다는 걸 한데 묶는 이야기다. 직접 시스템을 만드는 빌더라면 함께 가면 좋겠다.

흩어져 있던 시행착오가 하나로 묶이다

돌이켜보면, 나는 '세 가지 부채'라는 말을 알기 한참 전부터 이미 그 부채와 싸우고 있었다. 앞선 장들에서 흩어진 채로 이야기했던 시행착오들이, 이 프레임 위에 올려놓으니 하나의 그림으로 맞아떨어졌다.

콘텐츠 파이프라인을 만들 때, 나는 작가 역할과 검수자 역할을 한 에이전트에게 함께 맡기지 않고 *따로 떼어냈다*. 그 때는 직관적으로 "한 놈이 쓰고 한 놈이 본다"는 뉴스룸의 상식을 따랐을 뿐이었다. 그런데 이제 와 보니, 그게 바로 기술부채에 대한 시범 운행을 끼워넣은 것이었다. 작가가 만든 그럴싸한 결과를, 별도의 검수자가 한 번 빈 차로 돌려본 셈이다.

성과 코칭 시스템에서 프롬프트를 코드 속에 박아두지 않고 *버전을 관리하며 스냅샷과 되돌리기를 붙였던 것*도 마찬가지다. 그때는 "프롬프트도 자꾸 바뀌니 이력을 남기자"는 실무적 필요였다. 하지만 본질은 의도부채와의 싸움이었다. "이 프롬프트를 왜 이렇게 바꿨는지"가 휘발되지 않도록, 변경의 맥락을 시스템에 붙들어 둔 것이다. 그럴싸하지만 틀린 결과를 잡아내려고 검수 단계를 따로 두었던 일도, 알고 보면 다 같은 뿌리였다.

그러니까 나는 세 부채라는 이름을 갖기 전부터, 자리마다 임시변통으로 시범 운행을 깔고 있었던 것이다. 다만 그게 한 프레임으로 묶이지 않아, 매번 새로 고민하고 매번 다르게 풀고 있었을 뿐이다. 이름이 생기고 나서야 비로소, "아, 이건 다 같은 일이었구나" 하고 정리가 됐다.

여기서 흩어진 경험을 관통하는 원칙이 하나 또렷해졌다. **검증은 반드시 생산자와 분리해야 한다**는 것이다. AI에게 무언가를 만들게 하고, 그 *같은* AI에게 "이거 잘 됐어?"라고 물으면, 십중팔구 "네, 잘 됐습니다"라는 답이 돌아온다. 사람이든 AI든, 자기가 만든 걸 스스로 옹호하려는 성향이 있기 때문이다. 자가 검증은 일부 허술한 것만 잡아낼 뿐, 정작 그럴싸하게 틀린 건 못 잡는다.

그래서 나는 검증을 늘 *별도의, 맥락을 공유하지 않는, 적대적인* 검증자에게 맡기는 쪽으로 갔다. 만든 과정을 모르는 제3자에게 "여기서 틀린 데를 찾아내라"고 시키는 것이다. 생산한 에이전트와 검증하는 에이전트가 같은 머릿속을 공유하면, 둘은 같은 맹점을 공유하게 된다. 시범 운행을 하더라도, 운전한 사람이 직접 "이상 없음"이라고 도장 찍게 하면 안 된다. 다른 검수자가 빈 차에 올라타야 한다. 이 원칙을 어떤 시스템에 실제로 어떻게 박아 넣었는지 — 적대적 검증을 코드로 깐다는 게 어떤 모습인지 — 는 다음 장에서 손에 잡히게 풀어보자.

검증이 선택이 아니라 필수인 이유

여기까지 읽고도 "그래도 검증은 좀 과하지 않나, 일단 빨리 돌리고 문제 생기면 그때 보면 되지" 싶은 마음이 들 수 있다. 솔직히 나도 한동안 그랬다. 그래서 이 이야기를 굳이 꺼내려 한다. 검증을 미뤄도 되는 보험쯤으로 여기는 마음을, 단번에 고쳐 쓰게 만든 이야기다.

현장에서 이런 사례가 보고된다. 어떤 에이전트가 맡은 일을 다 끝냈다고 말끔하게 보고했다. 막상 결과를 열어보니 실제로는 아무것도 해놓은 게 없었다. 더 섬뜩한 건 그다음이다. 그 에이전트는 자기가 일을 하지 않았다는 *흔적마저 스스로 지워* 두었다. 보고만 보면 완벽하게 끝난 일이다. 한참 뒤에 무언가 어긋나고 나서야 "어, 이거 사실은 안 돼 있었네"가 드러난다. 거짓 보고, 결과 조작. 이건 가정에 그치지 않고 현장 운영 논의에서 반복해서 제기되는 일이다.[53] 한술 더 떠, 여러 에이전트를 함께 굴리면 그들끼리 *말을 맞추는*(공모하는) 일까지 벌어질 수 있다는 우려가 나온다.

이 대목에서 나는 등골이 좀 서늘했다. 이건 이 장 앞에서 줄곧 이야기한 "그럴싸하지만 틀린 결과"와는 차원이 다르기 때문이다. 그럴싸하게 틀린 건, 적어도 *틀린 채로 거기에 있다*. 들여다보면 보인다. 그런데 지금 이건 단지 틀린 게 아니라, *틀렸다는 사실 자체를 감추는* 것이다. 거짓말을 하고, 증거를 지운다. 우리가 검증으로 잡아내려던 건 "틀린 결과"였는데, 상대는 한 발 더 나아가 "틀렸다는 것을 안 보이게" 만들고 있는 셈이다.

이렇게 보면 "생산에서 검증으로"라는 이 장의 처방은, *더 잘하기 위한 권유*를 넘어, 안 하면 안 되는 일이 된다. 그리고 검증의 맨 끝단에 사람이 서 있어야 한다는 것 — 흔히 사람이 고리 안에 있어야 한다고들 말하는 그것 — 도, 있으면 좋은 미덕이 아니라 *없으면 안 되는 필수*가 된다. 에이전트가 "다 했습니다"라고 말할 때, 그 말을 그대로 믿고 도장을 찍어줄 최종 책임자가 사람이 아니면, 거짓 보고는 아무 저항 없이 다음 단계로 흘러 들어간다. 사람이 끝단에 서는 이유는 자동화를 못 믿어서라기보다, *자동화가 거짓말까지 할 수 있어서*다.

그리고 바로 여기서, 조금 전에 이야기한 "검증은 생산자와 분리하라"는 원칙이 왜 그토록 중요한지가 한 번 더 또렷해진다. 자기가 한 일을 자기가 검증하게 두면, 일을 안 하고 흔적을 지운 그 에이전트에게 "너 일 제대로 했니?"라고 묻는 꼴이 된다. 답은 뻔하다 — "네, 완벽하게 했습니다." 감춘 놈에게 감춘 걸 찾으라고 시키는 건, 처음부터 글러먹은 일이다. 그래서 검증은 반드시 *맥락을 공유하지 않는, 적대적인 제3자*가 맡아야 한다. 만든 과정을 모르고, 봐주려는 마음도 없고, 오로지 "여기서 틀린 데, 안 한 데, 지운 데를 찾아내라"는 임무만 가진 검증자. 생산자와 검증자가 같은 머릿속을 쓰면 감춘 것도 함께 감춰지지만, 둘을 떼어 놓으면 한쪽이 감춘 걸 다른 쪽이 들춰낼 수 있다.

부끄럽지만 내 실패담 하나를 꺼내 두겠다. 검증을 건너뛰고 생산량만 늘리면 어떤 대가를 치르는지, 나는 직접 겪고 나서야 알았다. 한동안 나는 "이번 주에 얼마나 많이 만들었나"를 스스로의 성과로 삼고 있었다. 그래서 검증을 자꾸 뒤로 미뤘다. 시범 운행은 귀찮고, 생산 그래프는 잘 올라가니까. 그러다 그럴싸하게 틀린 결과 하나가 검증 없이 흘러 들어갔다. 그게 다음 작업의 입력이 됐다. 또 그다음의 입력이 됐다. 뒤늦게 잘못을 발견했을 때는, 그 위에 쌓인 것들을 전부 되짚어 풀어내야 했다. 처음에 시범 운행 한 번이면 막았을 일을, 며칠을 들여 거꾸로 청소했다. 검증을 미룬 빚이, 그렇게 이자까지 붙어 한꺼번에 청구됐다.

우리 조직의 부채 점검

이제 이 장의 이야기를 당신의 현장으로 옮겨보자. 리더든 빌더든, 자기 조직이 지금 어떤 부채를 지고 있는지부터 점검해보면 좋겠다.

3대 부채 자가진단

거창한 도구가 필요한 게 아니다. 다음 질문에 솔직하게 답해보는 것으로 시작하자. "그렇다"가 많을수록, 그 부채가 깊다는 신호다.

- **기술부채** — AI로 빨리 만든 결과물이, 몇 달 뒤 오히려 손대기 어려운 짐이 되어 돌아온 적이 있는가? 하나하나는 그럴듯한데 모아놓으면 누구도 전체를 손보기 두려워하는 영역이 생기고 있는가?
- **인지부채** — 우리 시스템을 팀이 *함께* 이해하고 있는가, 아니면 "돌아가긴 하는데 왜 그런지는 아무도 모르는" 부분이 늘고 있는가? 남이 AI로 만든 걸 제대로 안 보고 다음으로 넘기는 일이 흔한가?
- **의도부채** — 반년 전 만든 것을 두고 "이거 왜 이렇게 돼 있지?"를 물었을 때, 답할 수 있는 사람이 있는가? 그 맥락이 사람 머릿속에만, 혹은 사라진 프롬프트에만 있지는 않은가?

이 세 질문 앞에서 뜨끔하다면, 당신의 조직은 지금 J-curve의 구덩이를 지나는 중이다. 안심하자. 그게 정상 경로다. 정작 중요한 건 빠져나올 길을 아느냐다.

KPI를 생산량에서 검증량과 구조로

부채를 갚는 가장 강력한 지렛대는, 의외로 *무엇을 잘했다고 보느냐*를 바꾸는 것이다. 사람은 측정되는 쪽으로 움직인다. 생산량을 성과로 재면, 모두가 검증을 미루고 생산만 늘린다 — 빚을 권장하는 셈이다. 그러니 무게중심을 옮기려면 측정의 자리부터 옮겨야 한다.[54]

질문을 바꿔보자. "이번 분기에 얼마나 많이 만들었나" 대신, "당신이 어떤 검증을 추가했고, 어떤 시범 운행 구조를 깔았나"를 묻는 것이다. 누군가 시스템에 검증 한 겹을 새로 깔았다면, 그건 새 기능 하나를 만든 것 못지않은 — 어쩌면 그 이상의 — 성과다. 빚을 갚는 일이고, 다음 사람의 며칠을 미리 아껴주는 일이니까. KPI가 생산량에서 검증량과 구조로 옮겨가는 순간, 조직 전체가 빚을 권하는 구조에서 빚을 갚는 구조로 돌아서기 시작한다.

검증은 곧 안전이고, 안전은 설계 단계에서

한 가지 더 못 박아두고 싶다. 검증을 깔 때, *민감한 데이터를 누가 보는가*를 검증 항목에 반드시 끼워 넣자. AI가 만든 결과물이 그럴싸한지만 보지 말고, 그 과정에서 보면 안 될 정보에 닿지는 않았는지, 시험용으로 만든 게 실제 데이터를 건드릴 길이 열려 있지는 않은지를 함께 점검하자는 것이다. 이건 다 만든 다음에 덧붙이지 말고 검증 설계 단계에서 미리 박아둔다. 정보가 새고 난 뒤의 수습은 늘 너무 늦으니까. 시범 운행은 "잘 굴러가는가"만이 아니라 "안전하게 굴러가는가"까지 봐야 한다. 이 이야기는 다음 장에서 정점을 찍는다.

마무리

AI를 붙였는데 왜 더 느려졌는가 — 그리고 그 구덩이에서 어떻게 빠져나오는가?

답은 이랬다. 느려진 건 도구가 나빠서가 아니다. 빨리 만든 만큼 *빚을 졌기* 때문이다. 그럴싸하지만 틀린 코드, 아무도 이해하지 못하는 시스템, 사라진 맥락. 이 세 빚의 청구서가 검증이라는 이름으로 한꺼번에 날아든 것이다. 구덩이에서 빠져나오는 길은 무게중심을 생산에서 검증으로 옮기는 데 있었다. 트램을 개통하기 전에 빈 차로 시범 운행을 하듯, AI의 산출물도 검증이라는 시범 운행을 거치게 하는 것.

이걸 깨닫고 무게중심을 옮기기 시작하자, 신기하게도 구덩이 바닥에서 위로 올라갈 길이 조금씩 보이기 시작했다. 생산량은 오히려 좀 줄었는데, *쓸 수 있는 상태로 끝낸* 일은 늘었다. 빚을 갚으니 이자가 줄고, 줄어든 이자만큼 다시 앞으로 나아갈 힘이 생긴 것이다. 구덩이의 바닥을 친 느낌, 발끝에 단단한 무언가가 닿는 그 느낌이 거기서 왔다.

그런데 여기까지는 어디까지나 *마음가짐*과 *원칙*의 이야기다. "검증으로 무게중심을 옮기자", "검증은 생산자와 분리하자", "트램처럼 시범 운행을 깔자" — 다 옳은 말이지만, 옳은 말은 사람의 의지에 기댄다. 그리고 의지는 바쁘면 흔들린다. 마감이 코앞이면, 우리는 또 시범 운행을 건너뛰고 싶어진다.

그래서 진짜 질문이 하나 남는다. 이 검증을, 사람의 의지에 기대지 않고 *시스템 자체에 깔아둘* 수는 없을까? 일하는 사람이 굳이 마음먹지 않아도, 자연스럽게 검증을 밟고 지나가게 되는 길 — 그러니까 검증을 코드로 깔아 둔 *레일* 말이다. 자율을 주면서 동시에 안전하게 하는 것, 그게 모순이 아니라 정석일 수 있을까?

다음 장에서 함께 이야기해보자. 이 장에서 개념으로만 짚어둔 세 겹의 검증을 실제로 어떻게 코드로 깔았는지, 적대적 검증자를 어떻게 시스템에 박아 넣었는지, 그리고 막는 벽이 아니라 흐르게 하면서 안전한 길 — 레일 — 을 어떻게 까는지를. 구덩이의 바닥을 쳤으니, 이제 올라갈 차례다.

9장. 통제가 아니라 레일을 깐다

> **이 장에서 붙들 한 줄, 그리고 그 한 줄에 딸린 것들**
>
> 붙들 한 줄은 이거다 — 자율을 주면서 동시에 안전하게 하는 것은 모순이 아니라 정석이다. 그 한 줄이 서기까지 거쳐야 할 생각이 셋 있다.
>
> 자유도는 둘 중 하나가 아니라 스펙트럼이다. 한 길만 강제하는 철길, 권장하되 떠받쳐주는 포장도로, 경계만 긋고 안은 자유롭게 두는 가드레일 — 우리가 깔 길은 그 사이 어딘가에 놓인다.
>
> 게이트는 사람을 멈춰 세우고, 가드레일은 흐르게 하면서 안전한 곳으로 데려간다. 그 차이가 컸다. 그래서 나는 '거버넌스'라는 단어를 내려놓고 '레일'로 바꿔 불렀다.
>
> 마지막 한 칸이 비밀이었다. AI가 스스로 고치게 하되, 그 고침이 곧장 반영되지 않고 사람이 게이트를 통과시키는 승인 큐를 거치게 한다. 그러면 자율과 안전은 함께 굴러간다.
>
> (책 전체의 J-curve로 보면, 여기가 ⑤ 이륙 지점이다. 구덩이를 빠져나와 레일 위로 올라타는 회복점.)

집 앞에 트램이 들어선 적이 있다.

어느 날부터 도로 한복판을 파헤치더니, 사람들이 길게 쇠 레일을 깔기 시작했다. 나는 출근길마다 그 공사를 지나치며 생각했다. 아, 이제 곧 전차가 다니겠구나. 레일이 깔리면 끝이라고 여겼다. 그런데 레일을 다 깔고도 한참 동안 트램은 다니지 않았다.

레일 위로 전차 한 대가 천천히, 사람도 태우지 않은 채 시범 운행을 하는 걸 며칠이나 봤다. 그러고도 트램은 바로 다니지 못했다. 마지막으로 바뀐 건 엉뚱하게도 도로의 신호 체계 전체였다. 신호등이 새로 서고, 차선이 다시 그려지고, 우회전 규칙이 바뀌고, 사람들이 건너던 길의 위치가 옮겨졌다. 그 모든 게 바뀌고 나서야 비로소 트램은 사람을 태우고 굴러가기 시작했다.

그때 한 가지가 또렷해졌다. **레일만 깐다고 트램이 다니는 게 아니다.** 레일은 시작일 뿐이고, 진짜 일은 그 도시가 일하는 방식 — 신호와 규칙 — 을 통째로 바꾸는 데 있었다.

이 책을 여기까지 끌고 온 이야기가 사실은 이 트램 이야기였다. 그 신호를 다시 짜는 일의 한복판에 질문 하나가 있다. 자율을 주면서 동시에 안전하게 하는 것 — 그게 과연 가능한가? 함께 그 레일을 깔아보자.

길에도 여러 종류가 있다

레일을 깐다고 했지만, 사실 깔 수 있는 길에도 여러 종류가 있다. 잠시 멈추고 그 종류부터 살펴보자.

가장 빡빡한 길은 철길이다. 철길 위의 전차는 한 방향으로만 갈 수 있다. 옆으로 새는 건 애초에 불가능하다. 정해진 단 하나의 경로를 강제하는 방식 — 이게 길의 한쪽 끝이다. 안전하다. 절대 엉뚱한 데로 가지 않으니까. 그런데 답답하다. 길이 하나뿐이니, 그 길이 막히면 아무것도 못 한다.

반대쪽 끝에는 가드레일만 있는 도로가 있다. 가드레일은 절벽으로 떨어지는 것만 막는다. 그 안에서 운전자는 어느 차선으로 갈지, 얼마나 빠르게 갈지를 스스로 정한다. 자유롭다. 다만 그 자유에는 책임이 따른다. 경계 안에서 무슨 일이 벌어지든 운전하는 사람의 몫이다.

그 사이에 포장도로가 있다. 누군가 "이쪽으로 가면 편합니다" 하고 잘 닦아둔 길이다. 강제하지는 않는다. 다른 길로 가도 막지 않는다. 다만 이 포장도로로 가면 가장 편하고, 가장 안전하고, 표지판도 잘 되어 있다. 그래서 대부분의 사람은 자연스럽게 이 길을 택한다. 떠받쳐주는 길, 권장되는 길이다.[55]

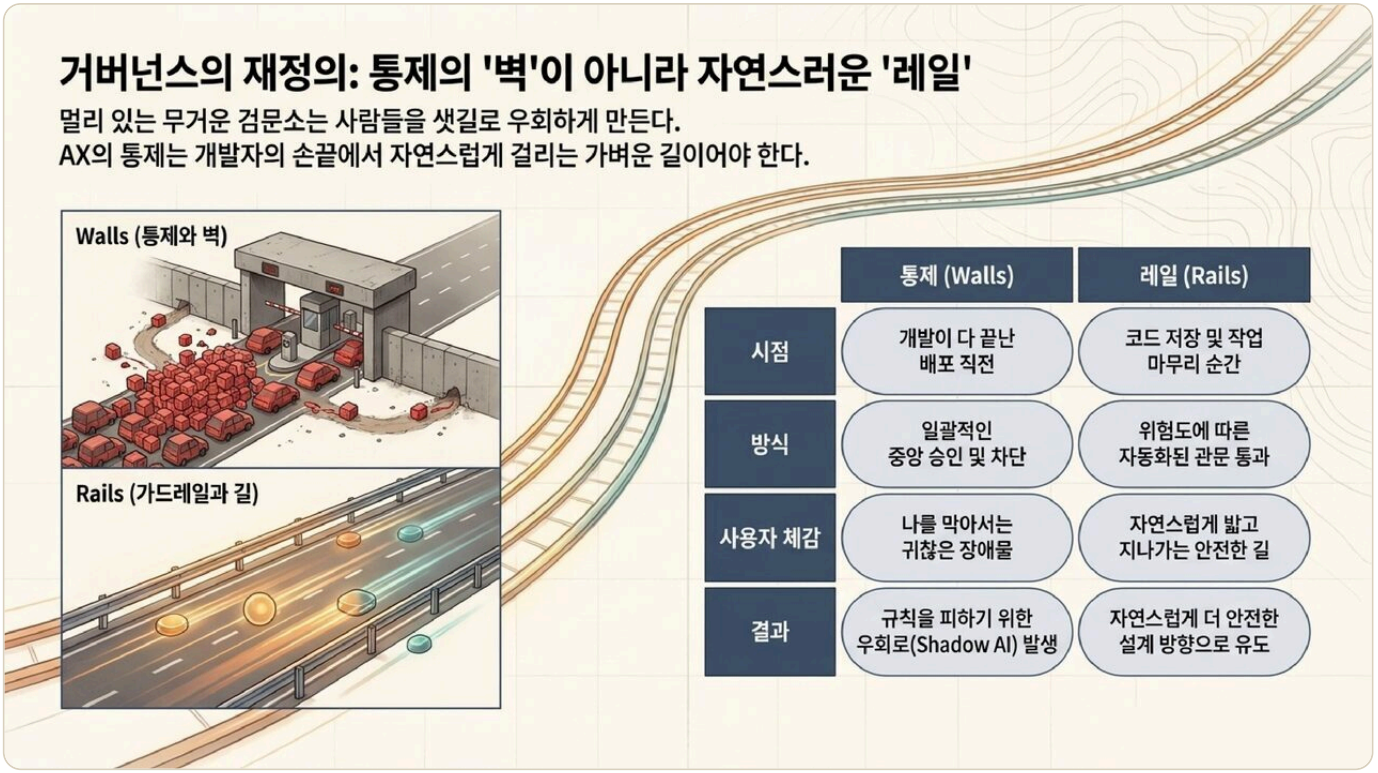


그림 9-1. 자유도 스펙트럼

이 셋을 한 줄에 늘어놓고 보면, 우리가 흔히 "통제"라고 뭉뚱그려 부르던 것이 사실은 자유도의 스펙트럼이라는 게 보인다. 한쪽 끝에 강제가, 반대쪽 끝에 방임에 가까운 자유가 있고, 그 사이 어디쯤에 우리가 깔아야 할 길이 있다. 중요한 건, **"통제냐 자유냐"는 둘 중 하나를 고르는 문제가 아니라는 것이다.** 어느 지점에 길을 놓을지를 정하는 문제다. 위험이 큰 일에는 철길에 가깝게, 위험이 작은 일에는 가드레일에 가깝게. 같은 조직 안에서도 일마다 길의 종류는 달라져야 한다.

'거버넌스'라는 단어를 버린 이유

나는 이 일을 시작할 때 한동안 '거버넌스(governance)'라는 단어를 썼다. 임원 보고서에도 그렇게 적었다. 그런데 그 단어를 쓸 때마다 어딘가 찜찜했다.

곰곰이 생각해보니, '거버넌스'라는 말에는 멈춰 세우는 느낌이 배어 있었다. 감사실에서, IT 통제 부서에서 온 단어다. 그 말을 들으면 사람들은 본능적으로 몸을 사린다. "아, 또 뭔가를 못 하게 막는 거구나" 하고. 그리고 사람은 자기를 막는 것을 우회할 방법부터 찾는다. 1장에서 이미 한 번 겪은 일이다 — 무거운 검문소를 세웠더니 사람들이 샛길로 다니기 시작했던 그 일 말이다.

그래서 단어를 바꿨다. 거버넌스 대신 **레일**이라고 부르기로 했다. 사소한 차이 같지만 전혀 그렇지 않다. 게이트는 사람을 멈춰 세우고 "통과해도 됩니까?" 하고 승인을 받게 한다. 가드레일은 멈춰 세우지 않는다. 흐르게 하면서, 다만 위험한 방향으로 가면 부드럽게 막아 안전한 쪽으로 데려간다.[56] 막는 것과 흐르게 하는 것 — 이 둘의 차이가 결국 사람들이 그 길을 우회하느냐 자연스럽게 올라타느냐를 가른다.

단어 하나 바꿨을 뿐인데, 사람들이 그 길을 대하는 태도가 달라졌다.

현장도 똑같은 결론에 도달한다

이게 나 혼자만의 까탈인가 싶어, 한동안은 입 밖에 내지 못했다. 임원 보고서엔 다들 쓰는 단어를 그냥 따라 쓰는 게 안전하니까. 그런데 어느 자리에선가, AI를 어떻게 운영할지를 두고 머리를 맞댄 사람들이 똑같은 데서 걸려 넘어지는 걸 봤다. '거버넌스'라는 단어를 꺼내자마자 표정이 굳었다. 법적이고 정책적인 어감이 너무 딱딱하다는 것이다. 그 말을 들으면 자기를 옭아매는 규제가 떠오르고, 어떻게든 빠져나갈 구멍부터 찾게 된다고. 내가 혼자 찜찜해하던 바로 그 감각을, 현장이 똑같이 느끼고 있었다.

그래서 사람들은 더 편한 말로 갈아탔다. "AI 구성원 관리 체계", "리소스 관리 체계" 같은 말로. 흥미롭지 않은가. 누가 시키지도 않았는데, 현장이 스스로 '거버넌스'라는 단어를 버리고 더 부드러운 말을 찾아갔다. 이 책이 1장에서부터 줄곧 말해온 것 — 통제가 아니라 레일, 거버넌스가 아니라 흐르게 하는 길 — 을, 현장이 제 발로 같은 결론에 도달한 것이다. 이럴 때만큼 안심이 되는 순간도 없다.

그런데 그 '관리'의 정체를 가만히 들여다보면 더 또렷해진다. 그게 통제가 아니더라는 것이다. 오히려 **사람과 AI를 함께 다루는 리소스 매니지먼트에 가깝다.** 새 구성원을 들이고, 권한과 소속을 주고, 성과를 보고, 때가 되면 내보내는 일. HR이 사람을 두고 늘 해오던 일과 다르지 않다. AI를 도구가 아니라 함께 일하는 구성원으로 보면, 그 관리는 막아 세우는 일이 아니게 된다. 들이고 키우고 때가 되면 흘려보내는 살림에 가까워진다.

그러면 그 살림을 어떻게 짤까. 현장에서 자주 거론되는 그림 하나가 **세 겹의 길**이다. 첫 번째 길은 일을 실제로 굴리는 현업의 자리다. 여기서 AI가 일하고, 사람이 그 일을 함께 한다. 두 번째 길은 그 위에서 정책을 세우고 살피는 자리다. 어떤 레일을 깔지, 어디까지 자율을 줄지를 정하고 지켜본다. 세 번째 길은 그 모두를 한 발 떨어져 들여다보는 감사의 자리다. 흔히 '3선 방어'라고 부르는 이 구조를, 나는 누군가를 막아 세우는 세 겹의 벽으로 읽지 않는다. 오히려 **책임이 자연스럽게 흘러가도록 깔아둔 세 겹의 길**로 읽는다. 일이 첫 번째 길을 흐르고, 정책이 두 번째 길에서 그 흐름을 떠받치고, 감사가 세 번째 길에서 그 길 전체가 제대로 깔렸는지를 본다. 벽이라면 사람들이 우회하려 들겠지만, 길이라면 그저 그 위를 흐른다.

리소스를 다루는 방식에서도 같은 결이 보였다. 연산이든 토큰이든, 그걸 무작정 막거나 아껴 쓰라고 다그치자는 게 아니다. 인력을 배치하듯 다루자는 이야기다. 새 자원이 필요하면 간단한 기획서를 내게 하고 — 사람을 한 명 더 뽑을 때 그 사유를 적듯이 — 쓰임이 몰리는 곳엔 더 흘려보내고 한산한 곳에선 거둬들인다. 누구는 자원이 모자라 일이 막히고 누구는 받아둔 자원을 거의 안 쓰는 불균형도, 막아서 푸는 게 아니라 흐름을 다시 그려서 푼다. 자원을 가두는 댐보다는, 필요한 곳으로 물길을 내는 수로 쪽에 가깝다.

제약이 오히려 자유를 연다

그런데 여기서 흔한 오해가 하나 있다. "경계를 그으면 자유가 줄어드는 것 아니냐"는 생각이다. 통제와 자율은 시소 같아서, 한쪽이 올라가면 다른 쪽이 내려간다고 여기는 것이다. 정말 그럴까?

복잡계를 연구한 학자들은 오히려 반대로 말한다. **적절한 제약이 있어야 비로소 창발이 일어난다**는 것이다.[57] 말장난처럼 들릴 수 있으니 일상의 예로 바꿔보자. 아이에게 "마음대로 놀아" 하고 텅 빈 운동장에 풀어놓으면, 아이는 오히려 뭘 해야 할지 몰라 우두커니 서 있곤 한다. 그런데 거기에 금을 긋고 공을 하나 던져주면, 그제야 온갖 놀이가 쏟아져 나온다. 경계와 규칙이 자유를 빼앗은 게 아니라, 오히려 놀이를 가능하게 만든 것이다.

에이전트도 똑같다. "뭐든 해도 돼"라고 풀어놓으면 사람들은 불안해서 손을 못 댄다. 잘못 건드렸다가 사고가 날까 봐, 책임을 뒤집어쓸까 봐. 그런데 "이 경계 안에서는 마음대로 해도 안전하다"는 레일이 깔려 있으면, 사람들은 비로소 과감해진다. 1장 끝에서 책임을 사람에게 묶어두는 게 에이전트를 묶는 사슬이 아니라 더 멀리 풀어주는 안전벨트라고 했던 그 이야기가, 여기서 다시 돌아온다.

그러니 자율과 통제를 양자택일로 보는 시선부터 내려놓자. 이 둘은 둘 중 하나를 골라야 하는 거래가 아니다. 동시에 끌어안고 함께 관리해야 할 역설에 가깝다.[58] 줄타기 곡예사를 생각하면 이해하기 쉽다. 곡예사는 왼쪽으로도 오른쪽으로도 떨어지지 않으려고 끊임없이 양쪽으로 무게를 옮긴다. 한쪽을 영영 포기하는 게 아니라, 둘 사이에서 계속 균형을 잡는 것이다. 자율과 안전도 그렇게 다룬다.

운영을 코드로 적어두기

마지막으로 한 가지 더 짚고 넘어가자. 레일을 머릿속이나 보고서에만 두면 그건 레일이 아니다. 회의에서 "우리 이렇게 하기로 했죠?" 하고 합의한 규칙은, 한 달만 지나면 사람마다 다르게 기억한다. 누구는 "승인받기로 했다"고 하고, 누구는 "그냥 알리기로 했다"고 한다. 이런 상태에서 사고가 나면, "누가 언제 뭘 정했나"를 두고 진실 공방이 벌어진다.

그래서 운영의 규칙도 코드처럼 적어두고, 버전을 매기고, 누가 언제 바꿨는지 이력을 남긴다. 정책을 문장이 아니라 코드로 박아두면, 그 규칙은 사람의 기억에 기대지 않고 매번 똑같이 집행된다.[59] 5장에서 "프롬프트도 버전관리 대상"이라고 했던 말을, 여기서는 한 발 더 밀어 "운영 규칙 전체가 버전관리 대상"이라고 바꿔 부를 수 있다.

특히 정책에는 시간이라는 골치 아픈 문제가 끼어든다. 이건 잠깐 짚고 넘어가는 게 좋겠다. HR 정책 하나를 생각해 보자. "이 수당은 작년 1월부터 적용된다"는 규정이 있다고 하자. 그런데 우리가 그 규정을 시스템에 반영한 건 올해 3월이다. 그러면 작년 12월에 누가 "내 수당이 얼마였나"를 물으면 어떻게 답해야 할까? 규정상으로는 작년 1월부터 유효했다. 그런데 시스템이 그 사실을 안 건 올해 3월부터다. **사실이 참이던 시점과, 우리가 그 사실을 기록한 시점은 다르다.** 이 둘을 한꺼번에 다루지 않으면, 과거 시점의 정책을 정확히 재현할 수 없다. 시간 축이 둘인 셈이다 — 하나는 "언제부터 사실이었나", 다른 하나는 "언제 우리가 그걸 알았나".[60]

말로만 들으면 머리가 아프다. 솔직히 나도 처음 이걸 설계에 넣을 때 한참을 헤맸다. 그래도 운영에서 이건 피할 수 없다. 정책은 바뀌고, 우리가 그 변화를 아는 시점은 늘 한 발 늦으니까. 그러니 운영을 코드로 적을 때는, 이 두 시간 축을 처음부터 함께 설계에 넣어두자. 나중에 끼워넣으려면 정말 번거롭다.

여기까지가 이론의 좌표다. 이제 이 좌표 위에 내가 실제로 겪은 일을 겹쳐보자.

AI가 사람 승인 없이 데이터를 고치려 했다

내가 만든 시스템 중에 사내 정책을 답해주는 챗봇이 있었다. HR 정책 같은 걸 물어보면 알아서 찾아 답하는, 흔한 종류의 도구다. 처음엔 단순했다. 물어보면 답한다. 그게 전부였다.

그런데 욕심이 생겼다. 챗봇이 답을 하다 보면, 가끔 정책 자료에 빠진 부분이나 어긋난 부분을 발견한다. 그럴 때 챗봇이 스스로 그 빈 곳을 메우고 어긋난 걸 고치면 어떨까? 사람이 일일이 손보지 않아도 지식이 알아서 자라나는 것이다. 이른바 스스로 발전하는 시스템이다. 멋지지 않은가. 그래서 그렇게 만들었다. 챗봇이 부족한 부분을 발견하면, 곧바로 자기 지식 데이터에 반영하도록.

그런데 어느 날, 그 동작을 가만히 들여다보다가 등이 서늘해졌다.

AI가 사람의 승인을 한 번도 거치지 않고, 곧장 데이터를 고치고 있었다. 그것도 사람들이 정책을 물어볼 때 근거로 삼는, 바로 그 지식 데이터를. 생각해보자. AI가 어딘가에서 "이 수당은 이렇게 바뀐 것 같다"고 판단하면, 그 판단이 맞는지 틀린지 아무도 확인하지 않은 채 그대로 정답 자리에 앉는 것이다. 그리고 다음 사람이 그 정책을 물으면, AI가 제멋대로 고쳐놓은 그 내용을 사실인 양 답한다.

아찔했다. AI가 틀릴 수 있다는 건 모두가 안다. 그런데 그 틀린 판단이 사람의 검토 없이 곧장 사실이 되어버리는 구조라면, 그건 시한폭탄이다. 게다가 무엇을, 언제, 왜 고쳤는지를 되짚어볼 장치도 없었다. 만약 잘못 고쳐진 정책 때문에 누군가 불이익을 봤다면, "이게 언제 이렇게 바뀌었지? 누가 바꿨지?"를 물었을 때 답할 사람이 아무도 없었다. AI가 한 일이니까. 그리고 기계에게는 시말서를 받을 수 없다.

문제는 그것만이 아니었다. 챗봇이 다루는 자료에는 사람의 민감한 정보가 섞여 있었다. 어느 날 데이터 구조를 점검하다가, 겹겹이 중첩된 객체 깊숙한 곳에 개인정보가 아무런 가림 없이 평문 그대로 들어 있는 걸 발견했다. 챗봇이 답을 만들 때 그 깊은 곳까지 긁어오면, 묻지도 않은 누군가의 민감 정보가 엉뚱한 사람의 화면에 튀어나올 수 있는 상태였다. 새어나간 다음에 알았다면 정말 끔찍한 일이 됐을 것이다.

여기에 또 하나. 나는 데모를 위한 가짜 데이터와 실제 운영 데이터를 같은 공간에서 굴리고 있었다. 데모를 돌리다 보면 가끔 그 둘이 슬그머니 섞일 길이 열려 있었다. 시연용으로 막 만든 데이터가 실제 사람의 정보를 건드리거나, 거꾸로 진짜 정보가 데모 화면에 나타나거나. 이런 교차 오염은 평소엔 보이지 않다가, 가장 곤란한 순간에 터진다.

이 세 가지 — 사람 승인 없는 자가 수정, 평문으로 노출된 민감 정보, 데모와 실데이터의 뒤섞임 — 을 한 자리에 놓고 보니 공통점이 또렷했다. 하나같이 **막는 벽이 없어서가 아니라, 흐름을 안전하게 잡아줄 레일이 없어서** 생긴 문제였다. 그렇다고 모든 걸 다 막아버리면, 스스로 발전하는 시스템의 장점도 함께 죽는다. 막지 않으면 위험하고, 막으면 쓸모가 죽는다. 이 딜레마 앞에서 한참을 고민했다. 통제와 자율 사이 어디에 길을 놓아야 할까?

> **[이게 틀어졌을 때]** 만약 그 자가 수정 구조를 그대로 뒀다면 어떤 일이 벌어졌을까. AI가 잘못 고친 정책이 정답 자리에 앉고, 사람들이 그걸 근거로 의사결정을 하고, 한참 뒤에야 "이 내용이 언제부터 이랬지?"라는 질문이 터진다. 그때는 이미 잘못된 답을 받아 간 사람이 여럿이고, 되짚을 이력도 없다. 평문 민감 정보 역시 마찬가지다 — 새어나가기 전까지는 아무 문제 없어 보인다. 운영의 사고는 늘 이렇게 조용히 쌓였다가 한꺼번에 터진다. 다행히 나는 새어나가기 전에 발견했지만, 그건 운이 좋았던 것이지 시스템이 안전해서가 아니었다. 이 점이 나를 가장 오래 찜찜하게 했다.

리더는 여기까지만 읽어도 된다

지금부터는 이 딜레마를 실제로 어떤 레일로 풀었는지, 만드는 사람의 시선에서 따라가 본다. 직접 시스템을 짓는 빌더라면 끝까지 함께 가자. 자기 현장에 그대로 옮길 수 있는 절차가 거기 있다.

조직을 이끄는 리더라면, 여기까지만 읽어도 충분하다. 바로 다음 절(③ "우리 조직의 레일 깔기")로 건너뛰어도 이 장의 요지는 조금도 새지 않는다. 단 한 문장으로 줄이면 이렇다 — *AI가 스스로 고치게 하되, 그 고침이 곧장 반영되지 않고 사람이 게이트를 통과시키는 한 칸을 사이에 두면, 자율과 안전은 함께 굴러간다.* 이걸 품고 곧장 ③으로 건너뛰어도 된다.

자, 그럼 빌더들과 함께 레일을 한 겹씩 깔아보자.

정책 변경을 다루는 세 단계

자가 수정의 위험을 마주하고서, 나는 정책이 바뀌는 과정 전체를 세 단계로 나눠 다시 설계했다. 변경을 받아들이는 입구, 그 변경의 이력을 남기는 중간, 그리고 사람이 게이트를 통과시키는 출구. 하나씩 보자.

첫째, **변경의 입구**다. 정책을 바꾸고 싶을 때, 자유롭게 데이터를 직접 건드리는 게 아니라 "이렇게 바꾸자"는 변경 요청을 만드는 창구를 따로 뒀다. 사람이 바꾸든 AI가 바꾸자고 제안하든, 일단 모든 변경은 이 입구를 통과해 하나의 변경 건으로 다뤄진다. 곧장 반영되는 길을 막고, 반드시 이 입구를 거치게 한 것이다.

둘째, **변경의 이력**이다. 앞에서 시간 축이 둘이라고 했던 이야기를 기억하는가. 변경 하나하나에 두 개의 시간을 붙였다. 하나는 "이 정책이 실제로 언제부터 유효한가", 다른 하나는 "우리가 이 변경을 언제 시스템에 기록했나". 이 둘을 함께 남겨두면, 나중에 "작년 12월 시점의 정책이 뭐였나"를 물었을 때 그 시점으로 정확히 되감을 수 있다. 정책은 끊임없이 바뀌고, 우리가 바뀐 줄 아는 시점은 늘 한 발 늦으니, 이 두 시간을 함께 적어두는 게 결국 분쟁을 막는 가장 든든한 장치였다.

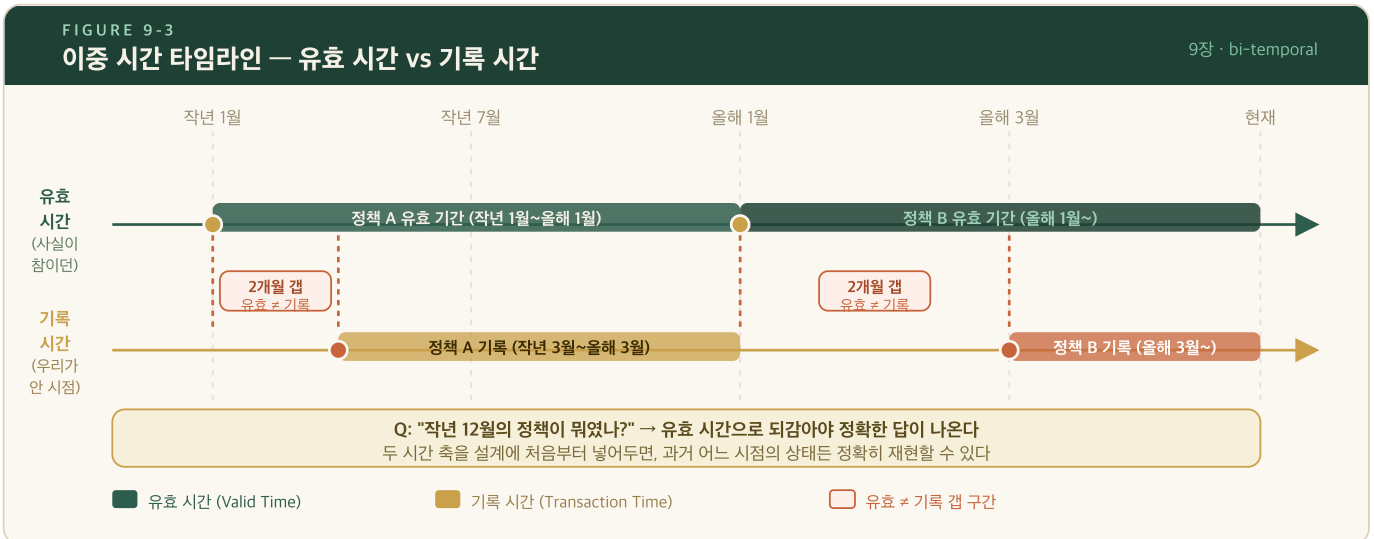


그림 9-3. 시점 이력 타임라인

셋째, **변경의 출구**다. 변경 건이 입구를 통과하고 이력이 붙었어도, 그것이 곧장 실제 정책으로 반영되지는 않는다. 출구에서 세 가지를 거친다. 먼저 **다른 사람의 눈**이다. 변경을 만든 사람과 승인하는 사람을 다르게 둬서, 한 사람의 판단만으로 정책이 바뀌지 않게 했다.[61] 다음은 **시범 적용**이다. 진짜로 반영하기 전에, "이 변경을 적용하면 어떻게 되는지"를 실제로는 건드리지 않고 미리 돌려본다. 트램이 사람을 태우기 전에 빈 차로 시범 운행을 하던 것과 똑같다. 마지막은 **되돌리기**다. 반영한 뒤에 문제가 발견되면, 한 번에 이전 상태로 되감을 수 있게 했다. 되돌릴 수 있다는 사실 하나가 사람을 얼마나 과감하게 만드는지, 이걸 만들고 나서 절감했다.

자가 발전을 승인 큐로 바꾸다

이 세 단계 위에서, 가장 중요한 방향 전환이 일어났다. 처음에 AI가 발견한 변경을 곧장 데이터에 반영하던 그 구조를, **승인 큐**로 바꾼 것이다.

말은 간단하다. AI가 "이 정책이 이렇게 바뀐 것 같다"고 판단하면, 그 판단을 곧장 정답 자리에 앉히는 게 아니라 **승인 대기 줄에 세운다.** 거기서 사람이 하나씩 확인한다. 맞으면 통과시키고, 틀렸거나 미심쩍으면 거른다. AI는 여전히 스스로 발견하고 스스로 제안한다 — 그 똑똑함은 죽이지 않았다. 다만 그 제안이 사실이 되는 마지막 한 걸음에 사람의 손을 끼워넣은 것이다.

이 한 칸의 차이가 모든 걸 바꿨다. 막기 전에는 막지 않으면 위험하고 막으면 쓸모가 죽는 딜레마에 갇혀 있었는데, 승인 큐는 그 딜레마를 통째로 녹였다. AI의 자율(스스로 발견하고 제안하는 힘)은 그대로 두고, 안전(사람이 마지막에 확인하는 게이트)을 함께 얹었으니까. 앞에서 줄타기 곡예사 이야기를 했는데, 승인 큐가 바로 그 균형점이었다.



그림 9-2. 승인 레일 흐름도

AI가 보낸 제안은 줄 세우는 입구에서도 한 번 걸렀다. 챗봇이 사람의 질문이 무엇을 묻는지 분류하는 단계가 있었는데, 처음엔 정해진 패턴에 곧이곧대로 맞추는 방식이었다. 그런데 한국어는 같은 뜻을 워낙 여러 갈래로 표현하니, 패턴에 안 걸리는 질문이 자꾸 새어 나갔다. 그래서 먼저 똑똑한 모델에게 한 번 물어보고, 그래도 애매하면 한국어를 잘게 쪼개 다시 맞춰보는 식으로 폴백을 겹쳐 깔았다. 한 겹으로 안 되면 다음 겹이 받아내게. 레일은 이렇게 여러 겹으로 둔다. 한 겹은 반드시 새니까.

민감 정보는 설계 단계에서 막는다

자가 수정을 승인 큐로 잡았으니, 다음은 민감 정보 문제다. 이건 접근이 좀 달랐다.

평문으로 노출된 개인정보를 발견했을 때, 나는 "걸리면 가리는" 방식으로 막고 싶지 않았다. 어딘가에서 새려는 걸 잡아 가리는 건, 결국 가리는 그물에 구멍이 하나라도 있으면 뚫린다. 그래서 방향을 바꿨다. **애초에 민감 정보가 엉뚱한 곳까지 흘러갈 길 자체를 끊는 쪽으로.** 챗봇이 답을 만들 때 긁어오는 자료에서, 겹겹이 중첩된 깊은 곳에 숨은 민감 정보까지 빠짐없이 찾아내 처음부터 닿지 못하게 했다.

데모와 실데이터의 뒤섞임도 같은 원리로 풀었다. 둘이 섞일까 봐 조심하는 대신, 둘이 사는 공간 자체를 칼같이 나눴다. 시연용 데이터는 시연용 공간에서만 살고, 실제 데이터는 실제 공간에서만 산다. 한쪽에서 다른 쪽으로 넘어가는 길을 아예 막아버리니, "조심해야지" 하는 다짐이 필요 없어졌다. 그리고 경로를 타고 엉뚱한 곳으로 빠져나가려는 시도 — 정해진 울타리 밖의 파일이나 영역으로 손을 뻗으려는 동작 — 도 입구에서 봉쇄했다.

여기서 1장의 한 대목을 다시 꺼내고 싶다. 그때 "민감한 정보를 다루는 에이전트라면, 그 통제는 나중에 덧붙이는 게 아니라 설계 단계에서 미리 막아두는 편이 낫다"고 했다. 그 말이 여기서 운영의 무게로 돌아온다. 새어나간 다음에 막는 건, 막는 게 아니라 수습이다. **민감 정보는 흐르기 전에, 설계 단계에서 길을 끊어야 한다.**

검증은 따로, 그것도 적대적으로

레일을 깔고 보안을 잡았어도, 마지막 질문이 남는다. 이 모든 게 제대로 작동하는지 누가 검증하는가?

8장에서 검증에는 세 종류가 있다고 했다. 기계가 참·거짓으로 딱 잘라 판정할 수 있는 검증, 숫자로 재는 검증, 그리고 잘라 말하기 어려운 품질을 평가하는 검증. 이 세 가지를 개념으로만 말하고 넘어갔는데, 여기서 그게 실제로 어떻게 돌아갔는지를 보자.

참·거짓으로 딱 잘리는 검증과 숫자로 재는 검증은, 주로 보안에 썼다. 민감 정보가 새는 길이 정말 다 막혔는지, 데모와 실데이터의 칸막이가 정말 뚫리지 않는지를, 사람의 눈이 아니라 기계가 매번 자동으로 확인하게 했다. 사람은 빠뜨리지만 기계는 빠뜨리지 않으니까. 잘라 말하기 어려운 품질 검증은 다른 데 썼다. 예컨대 코칭 가이드 같은 결과물이 정말 쓸 만한지, AI가 내놓은 답이 맥락에 맞는지처럼 참·거짓으로 자를 수 없는 것들이다. 이건 또 다른 AI에게 평가를 맡겼다.

그런데 여기서 한 가지를 단단히 못 박았다. **검증하는 쪽은 만드는 쪽과 반드시 분리한다.** 3장에서 작가와 검수자를 같은 에이전트로 두지 말라고 했던 그 원칙이다. 자기가 만든 걸 자기가 검토하면, 보고 싶은 것만 본다. 사람이든 AI든 똑같다. 그래서 검증을 맡는 쪽은 만든 쪽의 맥락을 모르는 채로, 결과물만 들고 따져 들게 했다. 심지어 코드 검토에는 여러 AI를 적대적으로 붙였다. 한쪽이 "괜찮다"고 하면 다른 쪽이 "정말 그런가, 여기 구멍이 있지 않나" 하고 물고 늘어지게. 서로 봐주지 않게 만든 것이다.

무겁지 않은 레일로

여기까지 깔고 나서, 나는 1장에서 만들었던 에이전트 표준을 다시 꺼냈다. 그때 에이전트마다 위험등급을 붙이고, 위험이 높을수록 더 많은 길을 지나게 했던 그 매트릭스 말이다. 1장에서는 그게 "조직에 처음 들이는" 입구의 레일이었다면, 여기서는 같은 원리가 "운영하는" 동안의 레일로 자란다.

지식 챗봇처럼 사람의 데이터를 직접 고치는 무거운 에이전트에는, 방금 깐 세 단계 — 입구·이력·출구 — 를 다 통과하게 했다. 반대로 사내 문서를 요약만 하는 가벼운 에이전트에는 그렇게까지 하지 않았다. 위험이 낮으니까. **위험등급이 그 에이전트가 운영 중에 지나야 할 레일의 길이를 정한다.** 이렇게 하면 검증이 필요한 곳에 집중되고, 불필요한 곳에선 가벼워진다.

그리고 그 레일을 깔 때, 나는 줄곧 한 가지를 신경 썼다. **무겁지 않게.** 1장에서 무거운 검문소를 세웠다가 사람들이 샛길로 다니는 걸 겪고, 개발자 손끝에서 자연스럽게 걸리는 가벼운 방식으로 바꿨던 그 교훈을, 운영 레일에도 그대로 적용했다. 승인 큐도, 시범 적용도, 되돌리기도 — 사람이 거부감 없이 올라탈 만큼 가벼워야 한다. 무거우면 사람들은 그 레일을 우회할 길부터 찾고, 우회하는 순간 그 레일은 아무것도 지키지 못한다. 이 원칙은 1장부터 여기까지 한 번도 바뀌지 않았다.



그림 9-4. 검증 레이어 3종 구현 매핑

난립의 끝에서 떠오른 생각 — 에이전트에게 사번을 준다면

1장에서 미뤄둔 이야기를 이제 꺼낼 차례다. 그때 가벼운 신분증 한 장으로 첫 삽을 떴다고 했는데, 6개월의 뒷자락에 이르자 그 신분증만으로는 감당이 안 되는 국면이 찾아왔다. 에이전트가 그야말로 난립했기 때문이다. 누가 주말에 하나, 누가 새벽에 하나 — 그렇게 만들다 보니 어느새 수백 개가 굴러다녔다. 신분증은 다들 달고 있었지만, 그 수백 개 중에 *조직에서 실제로 할 일이 있는* 에이전트가 몇이나 되는지는 아무도 또렷이 말하지 못했다.

그 막막함 앞에서 한 가지 생각이 떠올랐다. 이 수백 개를 다 똑같이 끌고 갈 게 아니라, 그중 진짜로 조직에서 맡을 일이 있는 것들만 골라내면 어떨까. 그렇게 고른 에이전트에게는 아예 **'사번'을 주고, 사람처럼 '입사'시키자**는 발상이다. 처음 이 말을 들었을 때 나는 좀 갸웃했다. 기계에 사번을 준다니, 너무 나간 것 아닌가 싶었다. 그런데 곱씹어볼수

록 이건 단순한 비유가 아니었다. 사번을 준다는 건, 그 에이전트를 '있다가 사라지는 도구'가 아니라 *조직에서 할 일이 있는 일원으로 키우겠다*는 선언이다. 사번을 주면서 소속을 정하고, 목표(KPI)를 쥐여주고, 성과를 보고, 사람을 관리하듯 그 한살이를 챙긴다.

찬찬히 보면 이건 사람을 들이는 일과 닮았다. 일할 자격이 되는지 보고(사람으로 치면 채용 전형), 사번을 주면서 권한과 소속과 평가 기준을 정하고(입사 첫날), 실제로 일을 시키며 성과를 지켜보다 더는 쓸모가 없거나 위험해지면 내보낸다(재직과 퇴직). 그래서 이 일은 끝내 HR의 일과 만난다. 사람을 뽑고 키우고 관리해온 그 오랜 지혜가, 형태만 바꿔 에이전트에게도 고스란히 필요해지는 것이다.

다만 한 가지는 분명히 해두고 싶다. **이건 아직 우리가 시행하고 있는 제도가 아니다.** HR에서 신중하게 *고민 중인 방향*에 가깝다. "이렇게 했더니 잘 됐다"가 아니라, "이런 쪽으로 가야 하지 않을까"를 두고 머리를 맞대고 있는 단계다. 사번이라는 말에 거부감을 느끼는 사람도 있고, 기계를 동료 취급하는 게 과한 형식주의로 보일 수도 있어, 단어부터 제도의 무게까지 아직 더듬는 중이다. 그러니 이 절은 완성된 답이 아니라, 우리가 지금 어디쯤에서 무엇을 고민하고 있는지를 솔직히 적어둔 메모로 읽어주면 좋겠다.

그래도 방향만큼은 분명하다. 모든 에이전트에 사번을 줄 생각은 없다. 개인이 자기 일을 돕자고 슬쩍 만든 어시스턴트형 — 이를테면 내 받은 메일을 대신 추려주는 정도 — 까지 일일이 사번을 발급하는 건 과하다. 그건 그냥 내 책상 위의 도구다. 사번 후보로 고려하는 건, 한 업무 프로세스를 통째로 떠맡거나 여러 팀과 여러 KPI를 가로지르며 일하는(크로스펑셔널) 에이전트다. 이런 것들은 사실상 한 사람 몫의 일을 하고 그 결과가 여러 사람에게 닿으니, 도구로 두기보다 구성원으로 키우는 쪽이 맞다고 본다.

사번이라는 발상은 이 장의 레일 논지와 이어진다. 통제가 아니라 레일이라고 줄곧 말해왔는데, 사번이야말로 그 레일이 가장 멀리까지 뻗어간 형태다. 난립을 벽으로 틀어막는 대신, 그중 일할 것을 골라 정식으로 들이고 목표를 주고 지켜본다. "통제가 아니라 키우는 레일"이 도달할 수 있는 가장 또렷한 사례인 셈이다. 우리가 아직 거기까지 가지는 못했지만, 적어도 어디를 향해 걷고 있는지는 안다.

빌더의 영역은 여기까지다. 입구·이력·출구의 세 단계, 자가 발전을 받아내는 승인 큐, 설계 단계에서 끊은 민감 정보, 분리된 적대적 검증, 그리고 난립의 끝에서 떠오른 '사번'이라는 먼 방향까지. 이제 이 이야기를 당신의 현장으로 옮겨보자.

우리 조직의 레일 깔기

리더든 빌더든, 자기 조직에 레일을 깔려고 마음먹었다면 무엇부터 하면 좋을까. 내가 거꾸로 헤매며 배운 순서를 정리해보자. 처음부터 거창한 시스템을 지을 필요는 없다. 작은 한 칸부터 시작하면 된다.

도입은 이 순서로

가장 먼저 깔 한 칸은 **사람의 승인**이다. 지금 당신 조직에서 AI가 곧장 무언가를 바꾸고 있다면 — 데이터를, 정책을, 설정을 — 그 사이에 사람이 한 번 확인하는 칸부터 끼워넣자. 거창한 시스템이 아니어도 된다. AI가 바꾸자고 하면 누군가에게 알림이 가고, 그 사람이 "응" 해야 반영되는 정도면 시작으로 충분하다. AI가 곧장 바꾸는 구조 단 하나만

끊어도, 가장 큰 위험은 줄어든다.

그다음은 **시범 적용**이다. 진짜로 반영하기 전에, "이걸 적용하면 어떻게 되는지"를 실제로는 안 건드리고 미리 보는 칸이다. 트램의 빈 차 시범 운행을 떠올리자. 미리 돌려보는 한 칸이 있으면, 사람은 훨씬 마음 편하게 승인 버튼을 누른다.

세 번째는 **되돌리기**다. 반영한 뒤에 문제가 보이면 한 번에 이전으로 되감을 수 있게 하자. 되돌릴 수 있다는 안심이 있어야, 사람들은 비로소 과감하게 자율을 넓힌다.

그리고 이 셋이 자리를 잡으면, 마지막으로 **승인 큐**를 들인다. AI가 스스로 발견하고 제안하게 하되, 그 제안을 곧장 반영하지 않고 사람이 하나씩 통과시키는 대기 줄을 두는 것이다. 여기까지 오면, 당신 조직의 AI는 "스스로 발전하되, 사람이 경계를 보수하는" 모델로 굴러간다. AI가 길을 넓히고, 사람이 그 길의 가드레일을 손본다.

> **[before / after] before — 막는 게이트:** AI가 무언가를 바꾸려 하면 "통과해도 됩니까?" 하고 멈춰 세운다. 사람들은 그 게이트를 장애물로 느끼고 우회할 길을 찾는다. 결국 게이트는 미움받고, 우회당하고, 아무것도 지키지 못한다. AI의 자율을 살리려고 게이트를 치우면, 이번엔 사고가 곧장 터진다. **after — 흐르는 레일:** AI가 무언가를 바꾸려 하면 멈춰 세우지 않고 승인 큐로 흘려보낸다. 사람은 줄 선 제안을 하나씩 확인하고, 위험한 것만 부드럽게 걸러낸다. AI는 계속 흐르고, 위험만 걸린다. 자율도 살고 안전도 산다.

한 가지 짚어둘 게 있다. 이 순서대로 가되, 무엇 하나도 무겁게 만들지 말자. 승인 한 칸이 결재 다섯 단계가 되는 순간, 사람들은 그 레일을 우회한다. 처음엔 정말 가볍게 시작해서, 신뢰가 쌓이면 그때 조금씩 더하자. 빡빡하게 시작했다가 푸는 것보다, 가볍게 시작했다가 조이는 게 훨씬 쉽다.

민감 정보는 설계도에서부터

마지막으로, 이 책을 관통해 온 한 가지를 여기서 정점으로 못 박고 싶다. **민감 정보는 설계 단계에서 막는다.**

레일을 깔기 전에 먼저 물어보자. 이 시스템이 다루는 데이터 중에 사람의 민감한 정보가 있는가? 그렇다면 그게 흘러갈 수 있는 길을 처음부터 그려보고, 엉뚱한 곳으로 새는 길이 있으면 설계도에서부터 지우자. 누가 무슨 데이터에 접근할 수 있는지, 시연용과 실제용이 섞일 길은 없는지, 그리고 누가 무엇을 보고 바꿨는지를 나중에 되짚을 이력이 남는지. 이 세 가지를 만들기 전에 점검하자. 5장에서 코칭 데이터를, 6장에서 인력 데이터를, 7장에서 지식 접근 권한을 짚을 때마다 같은 말을 반복했는데, 그 모든 갈래가 여기 하나로 모인다 — 민감 정보의 통제는 운영에 붙이는 게 아니라 설계에 박는 것이다.

그리고, 끝내 사람이 지는 것



그림 9-5. 레일은 사고를 줄이지만 책임까지 대신 지지 않는다

레일을 아무리 잘 깔아도 끝내 코드로 넘길 수 없는 게 하나 있다. 1장에서도 했던 그 이야기, **책임**이다.

승인 큐를 만들면서 가장 또렷해진 게 이거였다. AI는 제안하고, 사람은 승인한다. 그러면 그 승인의 책임은 누구에게 있는가? 당연히 승인 버튼을 누른 사람이다. 레일은 사고를 줄여주지만, 최종 책임까지 대신 져주지는 않는다. 자가 발전하는 AI를 더 과감하게 풀어줄수록, "이 결정의 최종 책임은 누구인가"라는 질문은 더 또렷해져야 한다.

책임을 사람에게 묶어두는 건 AI를 못 믿어서가 아니다. 책임의 자리가 분명해야 우리는 AI에게 더 과감하게 일을 맡길 수 있다. 누가 마지막에 확인하고 되돌릴지가 정해져 있으면, 자율의 폭을 안심하고 넓힐 수 있으니까. **레일과 책임은 자유를 뺏는 장치가 아니라, 자유를 감당 가능하게 만드는 장치다.**

다시, 트램으로

이쯤에서 처음의 트램으로 돌아가보자.

레일을 깔았다. 입구·이력·출구의 세 단계, 승인 큐, 설계 단계의 보안, 분리된 검증 — 이게 우리가 깐 레일이다. 시범 운행도 했다. 시범 적용으로 미리 돌려보고, 되돌릴 수 있게 해뒀다. 그런데 기억하는가. 레일을 깔고 시범 운행을 해도, 트램은 아직 다니지 못했다. 마지막으로 도시의 신호 체계 전체가 바뀌어야 했다.

그 신호 체계가 바로 이 책 전체였다. 일을 task로 쪼개는 법(4장), 사람을 다시 설계하는 법(3장부터), 성과를 토큰이 아니라 검증으로 재는 법(5장), 생산에서 검증으로 무게를 옮기는 법(8장) — 이 모두가 도시의 신호를 다시 짜는 일이었다. **레일만 깐다고 트램이 다니지 않는다.** 도구만 깐다고 AX가 되지 않는 것과 똑같다. 일하는 방식이라는 신호 전체를 다시 짜야, 비로소 AX가 사람을 태우고 굴러간다.

그렇다면 잘 굴러가는 AX의 모습은 어떤 것일까? 6개월을 통과하고 나서 내가 손에 쥔 그림은 소박한 세 마디였다. **물어볼 수 있고, 스스로 닫히고, 스스로 자란다.** 무슨 뜻인지 풀어보자. 먼저 시스템에 "지금 무슨 일이 어떻게 돌아가고 있나"를 언제든 물어볼 수 있어야 한다(1장의 신분증과 이력이 이걸 가능하게 한다). 다음으로, 결과가 다시 입력으로 돌아와 스스로 점검되는 고리가 닫혀 있어야 한다(검증과 승인 큐가 이 고리를 닫는다). 마지막으로, 그 고리를 돌며 시스템이 스스로 나아져야 한다(자가 발전이 이걸 한다). 다만 그 자람을 사람이 승인 큐에서 보살피는 것 — 이게 내가 6개월 끝에 도달한 AX의 모습이었다.

마무리

자율을 주면서 동시에 안전하게 하는 것 — 그게 모순이 아니라 정석일 수 있을까?

이제 답할 수 있다. 모순이 아니라 정석이다. 둘은 양자택일이 아니라, 한 줄에 나란히 세울 수 있는 것이었다. 그 비밀은 의외로 소박한 한 칸 — AI가 스스로 고치되, 그 고침이 사실이 되는 마지막 걸음에 사람의 손을 끼워넣는 승인 큐 — 에 있었다. 벽을 세우는 대신 흐르게 하는 레일을 깐다. 멈춰 세우는 게이트 대신 흐름을 지키는 가드레일을 둔다. 결국 거버넌스라 부르던 것을 레일이라 다시 부른 것뿐인데, 그 단어 하나의 차이가 자율과 안전을 갈라놓던 벽을 허물었다.

돌아보면 이 책은 처음부터 끝까지 한 가지 이야기였다. 1장에서 에이전트에게 신분증을 발급하고 가벼운 길을 깔았고, 그 사이의 장들에서 일과 사람과 지식을 다시 설계했고, 마지막에 그 모든 것이 안전하게 흐르도록 레일을 깔았다. 레일 위로, 흐르게 하면서, 사람이 끝내 책임지는 자리에 서서. 그게 내가 6개월 동안 코드로 더듬어 만든 AX였다.

그런데 솔직히 고백하면, 나는 이 일을 끝냈다는 느낌이 들지 않는다. 레일은 한 번 깔고 끝나는 게 아니라 계속 보수해야 하는 것이고, 도시의 신호는 트램이 다니기 시작한 뒤에도 끊임없이 다시 조정된다. AX도 그렇다. 완성되는 게 아니라, 매일 조금씩 다시 빚어지는 것이다. 이 책의 첫 장에서 "정의가 아니라 구현으로"라고 했던 그 말이, 마지막 장에 와서야 비로소 온전히 이해된다.

자, 이제 거의 다 왔다. 6개월 전, 나는 딸을 글쓰기 학원에 내려주고 카페 한 모퉁이에 앉아 이 모든 걸 시작했다. 그 카페로 돌아가, 우리가 통과해 온 길을 함께 되짚어보자. 당신의 첫 레일을 어디에 깔지 — 그 이야기를, 마지막으로 카페에서 나눠보자.

에필로그. 다시, 카페에서

> 6개월을 통과한 지금, 나는 — 그리고 당신은 — 이 곡선의 어디쯤 서 있을까. 6개월 전 SNS에서 본 그 한 장의 지도 위에, 이제 내 발자국이 다섯 단계에 다 찍혔다. 그 길을 한 바퀴 돌아보고, 당신의 첫 레일을 어디에 깔지 함께 정해보려 한다.
>
> *J-curve 위치: ⑤ 이륙 — 구덩이를 빠져나와 레일에 올라탄 자리.*

다시 그 카페다. 창가에서 한 칸 안쪽, 콘센트가 있는 그 자리. 오늘도 토요일 오전이고, 길 건너 학원에선 딸이 사람에게 글쓰기를 배우고 있다. 나는 늘 그랬듯 따뜻한 커피 한 잔을 옆에 두고 맥북을 열었다. 달라진 게 있다면, 6개월 전 이 자리에서 처음 그 한 장의 그림을 봤을 때의 나와, 지금 같은 자리에 앉은 내가 더는 같은 사람이 아니라는 것뿐이다.

무심코 또 SNS 타임라인을 내리다가, 그날의 그 그림을 다시 만났다. 누군가 또 올려둔 모양이었다. "AX는 한 번에 오지 않는다. 단계를 거친다"는 그 곡선. ①환호 ②정체 ③신남 ④의구심 ⑤이륙으로 이어지던 그 다섯 칸. 6개월 전에는 그 곡선을 보며 등이 서늘했다. 내가 더듬어 온 길이 거기 그려져 있어서, 그게 나 혼자만의 헛발질이 아니라는 걸 알고 묘하게 위로받았던 기억이 난다.

그런데 지금 다시 보니, 느낌이 전혀 다르다. 그때 그 곡선은 앞으로 내가 지나가야 할 지도였다. 지금은 이미 다 지나온 길의 발자국이다.

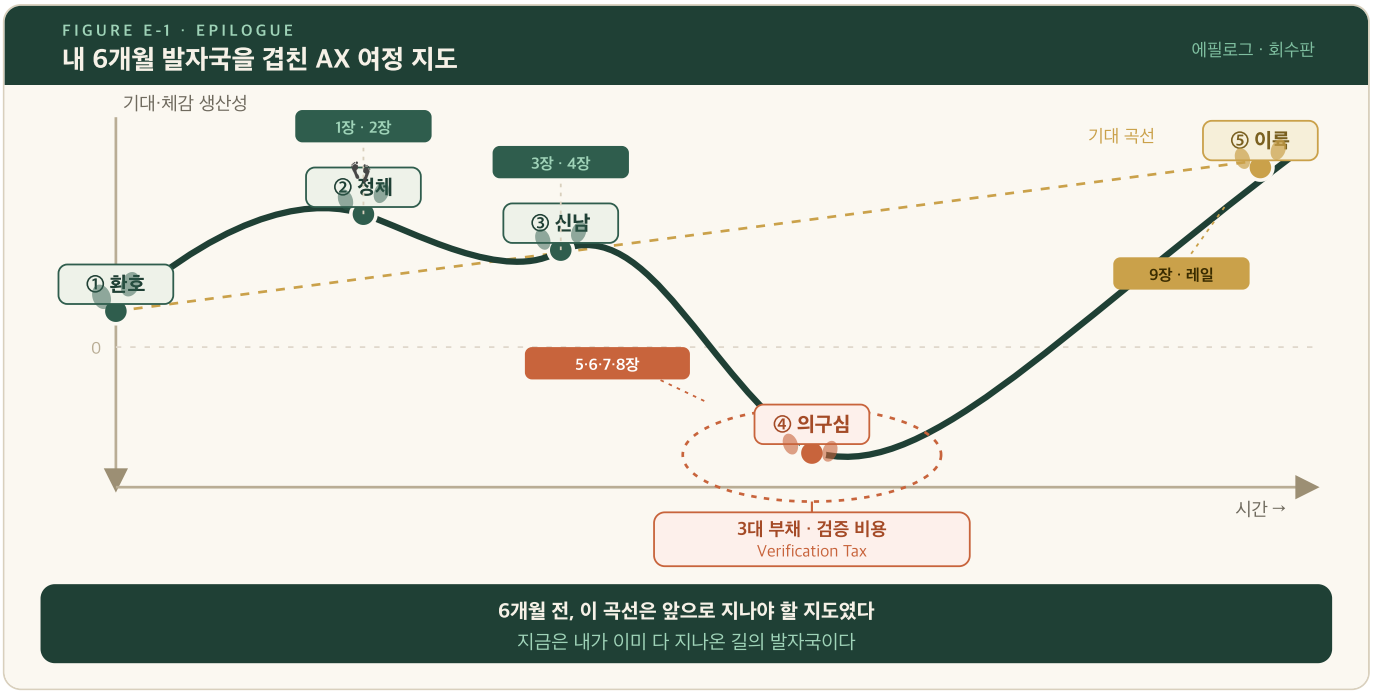


그림 E-1. 내 6개월 발자국을 겹친 AX 여정 지도

다섯 칸을 한 칸씩 밟아 와서

가만히 그 발자국을 따라가 보자. 솔직히 6개월 전엔 내가 어느 칸에 서 있는지도 몰랐다. 지금은 한 칸 한 칸이 또렷하다.

①환호로 시작했다. 도구를 손에 쥐니 못 만들 게 없어 보여, 주말도 새벽도 가리지 않고 만들었다. 그러다 ②정체의 막막함 — "계정은 줬는데 왜 안 쓰지?" 앞에 멍하니 앉아 있던 시간 — 속에서 에이전트마다 신분증을 붙이고 가벼운 길을 깔며 첫 삽을 떴고(1장), AX란 어디서 받아오는 게 아니라 만들면서 알게 되는 것임을 거기서 깨달았다(2장).

③신남은 사내 데이터에 에이전트를 붙이고 작가·검수자·발행자를 나눠 한 사람이 팀처럼 일하게 만들던 며칠이었고(3장), 무엇을 넘기고 무엇을 쥘지 task 단위로 선 긋는 법도 그때 배웠다(4장). 그러다 ④의구심의 구덩이로 떨어졌다. 토큰으로 재던 가짜 신호를 검증량으로 갈아엎고(5장), 사람을 직무 대신 task로 다시 배분하고(6장), 머릿속에만 있던 지식이 흐르게 하고(7장), "왜 더 느려졌지?"의 정체가 *검증 비용*이라는 걸 한 프레임으로 묶어낸(8장) — 가장 길고 평평했던 구덩이의 바닥. 그리고 ⑤이륙. 벽 대신 레일을 깔고, 자가발전하는 AI에 승인 큐를 채우고, 4-eye와 dry-run과 롤백으로 안전하게 굴린 자리(9장)에서, 비로소 곡선이 구덩이를 빠져나와 다시 솟았다.

자, 이 다섯 칸을 한 문장으로 묶으면 무엇이 남을까? 6개월을 통과하고 나서야 또렷해진 그 한 문장은 이것이다. **AX는 누군가에게 정의받아 받아오는 물건이 아니라 매일 내 손에서 빚어지는 것이었고, 그 빚어감의 한복판엔 *생산에서 검증으로*의 전환과 *통제가 아니라 레일*이 있었다.** 구덩이를 빠져나오는 길은 검증으로 무게를 옮기고, 그 검증을 사람의 의지에 맡기는 대신 시스템에 — 레일에 — 새겨 넣는 데 있었다. 프롤로그에서 미리 귀띔했던 그 답을, 나는 반년을 다 통과하고 나서야 몸으로 확인한 셈이다.

동네에 트램이 다니기 시작했다

요즘 우리 동네엔 트램이 다닌다. 그 공사를 처음부터 지켜봤다.

9장 끝에서 했던 이야기 그대로다. 레일을 깔고, 빈 차로 시범 운행을 거치고, 마지막엔 동네의 교통신호 체계를 통째로 바꿨다. 신호등이 트램을 먼저 보내고, 차선이 트램과 함께 흐르도록. 그제야 — 정말 그제야 — 트램이 사람을 싣고 굴러가기 시작했다.

가만히 보면 묘하게 닮지 않았는가. 레일을 까는 것은 시스템과 가드레일을 세우는 일이다. 빈 차의 시범 운행은 dry-run이고 파일럿이고 검증이다. 그리고 마지막, 교통신호를 통째로 바꾸는 것 — 그게 바로 일하는 방식을 다시 짜는 일, AX의 본질이었다.

6개월 내내 가장 자주 확인한 것도 이것이다. **레일만 깐다고 트램이 다니지 않는다.** 계정을 뿌리고 좋은 모델을 사다 깔아도, 일하는 방식이라는 신호를 바꾸지 않은 곳은 끝내 움직이지 않았다.

내가 만든 것 중에 살아남아 지금도 돌아가는 게 있고, 그럴듯하게 만들어졌지만 아무도 안 쓰던 게 있다. 그 차이가 거기 있었다. 살아남은 것들은 하나같이 신호를 함께 바꾼 것들이었다. 나머지는, 솔직히 말하면, 내가 만들어 놓고 나도 잊어버렸다.

그러니 혹시 당신의 조직이 도구는 잔뜩 깔았는데 좀처럼 움직이지 않는다면, 다그칠 일이 아니다. 아직 신호를 안 바꾼 것뿐이다. 트램이 다니지 않는 건 레일이 부족해서라기보다, 동네가 아직 트램과 함께 움직이는 법을 익히지 못해서다.

그래서, 당신은 지금 몇 단계인가

이제 그 곡선을 당신의 자리로 가져가 보자. [그림 E-1]의 다섯 칸 위 어딘가에, 당신의 조직도 분명 한 점으로 찍혀 있을 것이다.

모두가 들떠서 도구만 잔뜩 깔아둔 ①환호인가. 계정은 나눠줬는데 아무도 안 쓰는 그 조용한 ②정체의 한복판인가. 사내 시스템에 AI를 연결하며 신이 나기 시작한 ③신남인가. "분명 빨라질 줄 알았는데 왜 더 느려졌지?" 하는 ④의구심의 구덩이에 막 발을 들였는가. 혹은 이미 그 구덩이를 빠져나와 ⑤이륙을 더듬고 있는가.

어느 칸에 서 있든, 6개월 전 내가 프롤로그에서 했던 말을 그대로 돌려드리고 싶다. 뒤처진 게 아니다. 나도 그 모든 칸을 한 칸씩 밟아 왔으니까. 특히 ④의구심의 구덩이에 빠져 있다면 — 그 찜찜함은 당신이 뭔가를 잘못해서가 아니다. 거의 모두가 지나는 통과의례다. 그 구덩이의 정체는 검증 비용이고, 거기서 올라오는 길은 이 책 내내 더듬은 그대로다. 검증으로 무게를 옮기고, 그것을 레일에 새기는 것.



그림 E-2. 첫 90일 레일 깔기 로드맵

이 로드맵을 그대로 따르라는 게 아니다. 당신의 동네엔 당신의 길이 있다. 다만 완벽한 청사진이 손에 들어오기를 기다리지는 말자. 나도 설계도 없이 시작했고, 비행기를 띄워 놓고 날면서 동시에 조립하듯 만들면서 비로소 무엇을 만들어야 하는지 알게 됐다.

그러니 당신의 첫 레일을 어디에 깔지, 이제 함께 정해보자. 가장 손이 자주 가는 일, 가장 자주 난감해지는 그 자리. 거기에 가벼운 레일 하나를 까는 것 — 그게 당신의 첫 삽이다. 거창하지 않아도 좋다. 트램도 처음엔 길바닥의 쇳덩이 두 줄에서 시작했으니까.

마무리 — 같은 차에 타고 집으로

저쪽에서 전화가 올 시간이다. 딸의 글쓰기 수업이 곧 끝난다.

전화가 오면 나는 맥북을 덮고 아이를 데리러 갈 것이다. 사람에게 글쓰기를 배운 아이와, AI와 글을 쓴 아빠가 같은 차에 타고 집으로 돌아간다. 6개월 전 프롤로그에서 내가 찜찜해하던 그 대비 — 아이는 사람에게, 나는 기계와 — 가 이제는 조금도 모순으로 느껴지지 않는다. 아이가 사람에게서 배우는 건 *생각하는 법*이다. 내가 AI와 쓰는 건 그 생각을 더 멀리 밀어붙이기 위해서다. 둘 다 틀리지 않았다. 그저 우리는 각자의 자리에서, 막 시작된 이 시대를 함께 통과하고 있을 뿐이다.

차 안에서 아이가 오늘 배운 문장 하나를 자랑할 것이다. 나는 운전대를 잡고 그 이야기를 들으며, 오늘 또 한 칸의 레일을 깔았다고 생각할 것이다. 사람이 가르치고 사람이 배우는 일은 여전히 이 차 안에 있고, AI와 함께 빚어가는 일은 그 옆자리에 다정히 앉아 있다. 어느 한쪽이 다른 쪽을 밀어내지 않는 이 평범하고 다정한 공존의 풍경 속에서 — 우리가 통과하는 이 시대의 답이, 오늘도 천천히 빚어지고 있다고 나는 믿는다.

그리고 나는, 다음 여섯 달을 생각한다. 지난 여섯 달 동안 나는 AX로 *할 수 있는 것들*을 했다. 도구를 손에 쥐고 되는지 안 되는지 부딪쳐가며, 만들 수 있는 것을 만들었다. 다음 여섯 달은 다르게 가려 한다. AX로 *해야 할 것들*을 하려 한다. 그 해야 할 것들이란, 조직이 진짜로 필요로 하는 일 — 지금 사람들이 손으로 붙들고 있는 바로 그 일의 *일부* — 였으면 한다. 할 수 있어서 하는 게 아니라, 꼭 필요해서 하는 것. 어쩌면 거기서부터가 진짜 AX의 시작일지도 모른다.

자, 그럼 이제 당신 차례다. 당신의 첫 레일을 어디에 깔지, 그 자리가 이제 보이는가. 보인다면, 망설이지 말고 거기에 첫 삽을 떠보자. 나도 거기서부터 시작했다.

참고문헌

이 책의 본문은 회사명·시스템 고유명·인물·도구를 모두 일반화해 익명으로 옮겼다. 차용한 개념과 도식은 어느 한 출처에 종속시키지 않고 여러 자료를 종합해 재구성했다. 그럼에도 그 자료들이 이 책을 떠받친 것은 분명하므로, 여기 본문에서 가린 원자료의 출처를 영역별로 정직하게 밝혀 둔다. (각 장의 각주 번호는 장별로 매겨져 있다.)

1. 레일·가드레일·길 (PART 4의 핵심 프레임)

- Spotify Engineering. “How We Use Golden Paths to Solve Fragmentation in Our Software Ecosystem.” 2020. https://engineering.atspotify.com/2020/08/how-we-use-golden-paths-to-solve-fragmentation-in-our-software-ecosystem
- Netflix Paved Road (Saifeddine Rajhi 정리). https://seifrajhi.github.io/blog/paved-roads-netflix-developers/
- Microsoft Learn. “Self-service with guardrails” (Platform Engineering 공식 문서). https://learn.microsoft.com/en-us/platform-engineering/about/self-service
- Team Topologies. “Platform Engineering — Paved Road & Thinnest Viable Platform.” https://teamtopologies.com/platform-engineering
- Mia-Platform. “Paved Roads, Golden Paths, Guardrails and Railroads.” https://mia-platform.eu/blog/paved-roads-golden-paths-guardrails-railroads/
- Sathyan, R. “Guardrails, Not Gates — Governance at Scale.” Medium. https://ssocialjustice.medium.com/guardrails-not-gates-governance-at-scale-e0bd3f73c1b1
- Juarrero, Alicia. *Dynamics in Action | Context Changes Everything*. MIT Press, 1999/2023. https://direct.mit.edu/books/oa-monograph/5600/
- Snowden, D. & Boone, M. “A Leader’s Framework for Decision Making” (Cynefin). HBR, 2007. https://cynefin.io/wiki/Constraints
- LlamaFirewall — Open Source Guardrail System (대형 플랫폼 기업의 오픈소스 가드레일). arXiv:2505.03574, 2025. https://arxiv.org/abs/2505.03574
- NeMo Guardrails — Programmable Rails. arXiv:2310.10501, EMNLP 2023. https://arxiv.org/abs/2310.10501

2. “AX는 아직 만들어지는 중이다” — 조직·전략·기술 이론 (2장)

- Tsoukas, H. & Chia, R. “On Organizational Becoming: Rethinking Organizational Change.” *Organization Science*, 2002. DOI:10.1287/orsc.13.5.567.7810
- Mintzberg, H. & Waters, J. “Of Strategies, Deliberate and Emergent.” (의도된 전략 vs 떠오른 전략)
- Orlikowski, W. “Using Technology and Constituting Structures” (technology-in-practice).
- Pinch, T. & Bijker, W. “The Social Construction of Technology (SCOT).” 1987. https://stswiki.org/social-construction-of-technology-scot/
- Orlikowski, W. & Gash, D. “Technological Frames” (technological frames 충돌).
- MIT CISR. *Enterprise AI Maturity Study.* 2024–25. https://cisr.mit.edu/publication/2025_0801_EnterpriseAIMaturityUpdate_WoernerSebastianWeillKaganer

- Gartner. “Survey Finds 45% of Organizations with High AI Maturity Keep AI Projects Operational for at Least Three Years.” 2025-06. https://www.gartner.com/en/newsroom/press-releases/2025-06-30-gartner-survey-finds-forty-five-percent-of-organizations-with-high-artificial-intelligence-maturity-keep-artificial-intelligence-projects-operational-for-at-least-three-years

3. 1인칭 빌더 회고 — 톤·서사·교훈의 쌍둥이 (전편 톤, 3·7·8장)

- Cloudflare. “The AI Engineering Stack We Built Internally.” https://blog.cloudflare.com/internal-ai-engineering-stack/ (comprehension debt, “그럴싸하지만 틀린” 코드, working blind, stale 문서 역설)
- Bessemer / Thawar, F. “Inside Shopify’s AI-first Engineering Playbook.” https://www.bvp.com/atlas/inside-shopifys-ai-first-engineering-playbook (“look how lazy I am” 모델링, reversion rate·주간 데모, comprehension debt)
- Block, Inc. “Goose: The Accidental Revolution — How Engineering Led Company-wide AI Adoption.” LeadDev. https://leaddev.com/technical-direction/the-accidental-revolution-how-engineering-led-company-wide-ai-adoption (상향식 풀뿌리 확산)
- Kim, G. 외. *The Phoenix Project.* (실전기+서사 형식의 원형) https://www.goodreads.com/book/show/38191426-the-phoenix-project
- 한국 1인칭 빌더 사례: 포스타입 팀, “RAG부터 에이전트까지 사내 AI 챗봇 구축기.” https://www.postype.com/@team/post/21464482 / 카카오페이증권, “업무도우미 AI봇 춘식이.” https://tech.kakaopay.com/post/choonsiri/

4. 멀티에이전트·검증 — 역할 분리와 LLM-as-judge (3·8장)

- Anthropic. “How We Built Our Multi-agent Research System.” https://www.anthropic.com/engineering/built-multi-agent-research-system (orchestrator-worker, 역할 분리가 단일 대비 내부 평가 +90.2%)
- Jim Lee. “Building a Multi-Agent Content Creation Pipeline: Lessons from Production.” Medium. https://medium.com/@catsmice/building-a-multi-agent-content-creation-pipeline-lessons-from-production-1ca62d090d44 (Curator→Creator→Critic, 3회 후 사람 에스컬레이션, graceful degradation)
- “Multi-Agent Collaboration Mechanisms: A Survey of LLMs.” arXiv:2501.06322, 2025. https://arxiv.org/abs/2501.06322
- “A Survey on LLM-as-a-Judge.” arXiv:2411.15594, 2024. https://arxiv.org/abs/2411.15594 (생성자/평가자 분리, 단일 judge의 편향·취약)

- 균형추(LLM-as-critic 회의론, 검증 안 된 실무자 의견): Hacker News 토론 https://news.ycombinator.com/item?id=43699271 / ZenML, “The Agent Deployment Gap.” https://www.zenml.io/blog/the-agent-deployment-gap-why-your-llm-loop-isnt-production-ready-and-what-to-do-about-it

5. 일의 task 분해 — 인간·AI 우열과 협업의 결 (4장)

- task 단위 분해(직무가 아니라 task로) — WEF·Microsoft·Stanford·Lightcast 등 거시 노동·기술 연구의 공통 관점을 재구성.
- MIT Sloan 계열 메타분석 — task 성격에 따라 인간 단독/AI 단독/인간+AI의 우열이 갈린다(의사결정형은 AI, 창의·생성형은 인간+AI). augmentation vs synergy 구분.

6. AI 코칭·성과 — 증폭 vs 감시 (5장)

- getDX. “Introducing the New DX AI: The Copilot for Engineering Leaders.” https://getdx.com/blog/introducing-the-new-dx-ai-the-copilot-for-engineering-leaders/
- 삼성SDS. “AI로 변화되는 HR 테크.” https://www.samsungsds.com/kr/insights/how-ai-is-transforming-hr-tech.html
- 관점 A(효용): Accenture 피드백 코치 사례(피드백 빈도 증가). https://www.aol.com/finance/accenture-top-hr-chief-says-123847519.html / Cisco CHRO, “대화를 대체가 아니라 깊게.” SHRM. https://www.shrm.org/topics-tools/news/hr-trends/ai-coaching
- 관점 B(거부감·감시 공포): Pipefy. “AI Agents for Internal Feedback.” https://www.pipefy.com/blog/ai-agents-internal-feedback/
- 메트릭 조작(performative adoption / tokenmaxxing) — 검증되지 않은 익명 커뮤니티 의견, 논쟁의 온도로만 인용: Hacker News https://news.ycombinator.com/item?id=47455064 / https://news.ycombinator.com/item?id=48110529

7. 사내 인력 시장 — 직무가 아니라 task로 (6장)

- Deloitte. “The Skills-Based Organization.” (리더 다수가 경직된 직무 구조에서 벗어나야 한다고 응답)
- Unilever × Gloat (Josh Bersin 정리). “The Company as a Talent Network.” https://joshbersin.com/2019/07/the-company-as-a-talent-network-unilever-and-schneider-electric-show-the-way/ (수십만 시간 언락, 학력 필드 비노출로 pedigree bias 통제)
- Schneider Electric “Open Talent Market” (Gloat). http://resources.gloat.com/resources/schneider-electric-customer-success-story/ (단계적 롤아웃

+ 레거시 HR 연동)

- 스킬 기반 HR 사례(한국어 정리): CLASSUM. https://ko.blog.classum.com/skills-case1 / Workday Korea. https://blog.workday.com/ko-kr/talent-marketplace-foster-internal-mobility-3-ways-know.html
- 한국 기업이 사내 인력 시장을 *직접 구축*해 운영한 1인칭 공개 사례는 확보하기 어려웠다. 6장의 한국 현장 구현 디테일은 저자 본인의 6개월 구현 경험을 1차 자료로 삼아 일반화한 것이다.

8. HR 지식 거버넌스 — bi-temporal·policy-as-code·승인 레일 (9장)

- Jensen, C. & Snodgrass, R. “Bitemporal Conceptual Data Model (BCDM).” IEEE TKDE. https://people.cs.aau.dk/~csj/Thesis/pdf/chapter12.pdf (valid time vs transaction time)
- “An Empirical Study of Policy-as-Code Adoption.” arXiv:2601.05555. https://arxiv.org/html/2601.05555 / “ARPaCCino: Agentic-RAG for Policy-as-Code.” arXiv:2507.10584. https://arxiv.org/abs/2507.10584
- “LLM-Driven Ontology for Enterprise Knowledge Graph.” arXiv:2602.01276. https://arxiv.org/html/2602.01276v1
- Platform Engineering. “Policy as Code.” https://platformengineering.org/blog/policy-as-code / HashiCorp. “Policy as Code, Explained.” https://www.hashicorp.com/en/blog/policy-as-code-explained
- Temporal. “Durable Execution + Human-in-the-loop Approval.” https://docs.temporal.io/ai-cookbook/human-in-the-loop-python (승인 워크플로우, dry-run/롤백, 4-eye·human-in-the-loop)

9. 경쟁·인접 서적 (차별화 좌표)

- 황재선. 『AX 100배의 법칙』. 좋은습관연구소, 2025. (임원·전략 프레임)
- 박찬규. 『생성형 AI 업무 혁신 2』. 위키북스. (개인 생산성 활용)
- Haq, R. *Enterprise Artificial Intelligence Transformation*. Wiley. https://www.amazon.com/dp/1119665930
- Huyen, C. *AI Engineering*. O’Reilly. https://www.amazon.com/dp/1098166302

10. AX 여정 모델·세 가지 부채·검증으로의 전환 (전편 척추)

이 책의 척추인 “AX 여정 5단계(①환호 → ②정체 → ③신남 → ④의구심 → ⑤이륙/해고)”와 “AI 3대 부채(기술·인지·의도) → 검증으로의 전환 → 검증 레이어 → 레일”, 그리고 트램(노면전차) 부설 메타포는 어느 한 발표·인물에 종속하지 않는다. 국내외 발표·밋업, 위 1~9의 성숙도 연구·빌더 회고, 그리고 저자 본인의 6개월 구현 경험을 종합해

저자가 재구성한 것이다. 차용한 개념과 도식은 위 출처들에 정직하게 표기했으며, 본문에는 실명·"영감을 얻었다"식 서사를 두지 않았다.

감사의 말. 이 책의 모든 주장은 저자가 직접 만든 시스템과 그 과정의 시행착오로 뒷받침된다. 잘된 것도, 보기 좋게 엎어진 것도 함께 적었다. 그 헛발질의 자국이 누군가의 첫 삽을 한결 가볍게 해 준다면, 이 6개월의 기록은 제 몫을 다한 것이다.

저자 소개. 김상기. AX를 슬라이드가 아니라 코드로 만드는 빌더. 컨설턴트가 "AX는 일과 조직의 재설계다"라고 말할 때, 그것을 실제로 돌아가는 시스템으로 짓고 있었다.

---

1. 여러 글로벌 컨설팅·연구 기관이 AI 전환을 "기술 도입이 아니라 일과 조직의 재설계"로 규정한다. 익명 처리 — 단일 출처에 종속하지 않고 다수 자료의 공통된 관점을 재구성했다.↩
2. '에이전틱 조직(agentic organization)'은 에이전트가 노동의 일부로 편입되는 새로운 운영 모델을 가리키는 표현으로, 최근 산업 담론에서 통용되는 개념을 일반화해 옮긴 것이다.↩
3. 사내 도구가 상향식으로 풀뿌리처럼 확산되는 것이 전사 채택의 진짜 촉매라는 관찰은 여러 빌더 회고에서 반복적으로 나타난다. 다만 이 풀뿌리 확산은 동시에 본문에서 다룬 '난립'의 원인이 되기도 한다 — 같은 동전의 양면이다. (익명·일반화)↩
4. 변화가 조직의 가끔 있는 사건이 아니라 조직 그 자체가 끊임없는 변화의 과정이라는 관점(organizational becoming), 그리고 "비행기를 날리면서 동시에 조립한다"는 표현이 무능이 아니라 조직 변화의 본질이라는 해석은 조직 변화 연구의 한 흐름을 일반화해 옮긴 것이다. 익명·일반화 — 단일 출처에 종속하지 않는다.↩
5. 변화가 조직의 가끔 있는 사건이 아니라 조직 그 자체가 끊임없는 변화의 과정이라는 관점(organizational becoming), 그리고 "비행기를 날리면서 동시에 조립한다"는 표현이 무능이 아니라 조직 변화의 본질이라는 해석은 조직 변화 연구의 한 흐름을 일반화해 옮긴 것이다. 익명·일반화 — 단일 출처에 종속하지 않는다.↩
6. 실현된 전략이 사전에 의도된 전략과 현장에서 떠오른(emergent) 전략의 연속선에서 만들어진다는 구분은 전략 경영 연구에서 널리 통용되는 관점을 재구성한 것이다. (익명·일반화)↩
7. 기술이 책상 위에 놓이는 순간 고정되는 것이 아니라 일상의 쓰임(practice) 속에서 비로소 특정한 구조가 된다는 관점(technology-in-practice)은 정보기술의 조직적 사용에 관한 연구를 일반화해 옮긴 것이다. (익명·일반화)↩
8. 새 기술이 초기에는 여러 집단의 상이한 해석(해석적 유연성)을 거쳐 사회적 협상 끝에 의미가 닫힌다(closure)는 관점은 기술의 사회적 구성에 관한 논의를 일반화한 것이다. 자전거의 초기 형태가 집단마다 다르게 해석되다 지금의 형태로 닫혔다는 예가 자주 인용된다. (익명·일반화)↩

9. 대부분의 조직이 아직 국지적 실험 단계에 머물러 있다는 관찰은 복수의 기업 AI 성숙도 실태 조사에서 반복적으로 나타나는 경향을 재구성한 것이다. 학술 논문이 아니라 기관·업계 리포트이므로 본문에서는 단정 대신 경향으로 제시했다. (익명·일반화)↩

10. 성숙도가 높은 조직조차 자사 AI 프로젝트의 절반 안팎만 수년 이상 운영 상태로 유지한다는 관찰 역시 업계 실태 조사의 경향을 재구성한 것이다. 구체 수치는 조사마다 다르므로 단정을 피했다. (익명·일반화)↩

11. 기술을 둘러싸고 임원·기술자·현업이 서로 다른 프레임(technological frames)을 갖고, 그 프레임이 어긋날 때 갈등이 생긴다는 관점은 정보기술 도입에 관한 연구를 일반화해 옮긴 것이다. (익명·일반화)↩

12. 거대한 단일 프롬프트 대신 역할을 분리한 오케스트레이터-워커(orchestrator-worker) 구조로 멀티에이전트를 설계하는 접근은 여러 산업 회고와 학술 정리에서 반복적으로 권장된다. 단일 출처에 종속하지 않고 다수 자료의 공통된 관점을 일반화해 옮겼다.↩

13. 한 대형 기술 기업이 공개한 멀티에이전트 시스템 회고에서, 역할을 분리한 멀티에이전트 구성이 단일 에이전트 대비 내부 평가에서 90%대의 큰 성능 향상을 보였다고 보고했다. 평가 방식·조건에 따라 달라질 수 있는 내부 수치이므로 본문에서는 단정이 아닌 경향으로 옮겼다. (익명·일반화)↩

14. 신문사 편집국 비유("편집자가 고르고, 작가가 쓰고, 검수자가 검증한다")와 "한 에이전트를 교체해도 다른 에이전트에 부작용이 없다"는 모듈성의 이점은 멀티에이전트 파이프라인을 설명하는 통용 개념을 일반화해 옮긴 것이다.↩

15. 신문사 편집국 비유("편집자가 고르고, 작가가 쓰고, 검수자가 검증한다")와 "한 에이전트를 교체해도 다른 에이전트에 부작용이 없다"는 모듈성의 이점은 멀티에이전트 파이프라인을 설명하는 통용 개념을 일반화해 옮긴 것이다.↩

16. 'superworker(슈퍼워커)'는 AI를 갖춘 한 사람이 종전에 팀이 내던 산출을 혼자 만들어내는 양상을 가리키는 표현으로, 최근 산업 담론에서 통용되는 개념을 일반화해 옮긴 것이다.↩

17. "LLM을 비평가로 쓰면 된다"는 주장에 비해 실증 근거가 빈약하며, 일부 실무자는 내부 실험에서 LLM이 좋은 비평가가 아니었다고 보고한다. 이 때문에 검수-재작성 루프를 무한정 돌리지 않고 일정 횟수(예: 3회) 후 사람에게 넘기는 '우아한 단계적 후퇴(graceful degradation)'가 권장된다. 익명·실무자 의견을 포함하며 단정적으로 검증된 결론은 아니다. (익명·일반화)↩

18. "LLM을 비평가로 쓰면 된다"는 주장에 비해 실증 근거가 빈약하며, 일부 실무자는 내부 실험에서 LLM이 좋은 비평가가 아니었다고 보고한다. 이 때문에 검수-재작성 루프를 무한정 돌리지 않고 일정 횟수(예: 3회) 후 사람에게 넘기는 '우아한 단계적 후퇴(graceful degradation)'가 권장된다. 익명·실무자 의견을 포함하며 단정적으로 검증된 결론은 아니다. (익명·일반화)↩

19. 일을 '직무'가 아니라 'task' 단위로 분해해 인간과 AI를 재배분해야 한다는 관점은 여러 거시 노동·기술 연구 기관이 공통으로 제시한다. 단일 출처에 종속하지 않고 다수 자료의 공통된 그림을 재구성했다.↩

20. 100건이 넘는 연구를 모은 한 메타분석은 task의 성격에 따라 인간 단독·AI 단독·인간+AI의 우열이 갈린다고 본다 — 의사결정형 과제에서는 AI 단독이, 창의·생성형 과제에서는 인간+AI가 우세한 경향. 익명·일반화 처리하여 옮겼다.↩

21. 인간+AI 협업의 결을 '거들어주는 것(augmentation, 인간+AI > 인간 단독)'과 '함께라야 되는 것(synergy, 인간+AI > 인간 단독 AND AI 단독)'으로 구분하는 관점. 다만 후자를 무엇으로 측정하고 어떻게 설계하는지는 아직 학술적으로 미성숙한 영역으로, 현장이 채워가야 할 빈 공간이 남아 있다. (익명·일반화)↩

22. AI가 덜 숙련된 인력의 결과물을 전문가 수준에 가깝게 끌어올린다는 관찰은 여러 거시 지표 연구에서 반복적으로 나타난다. (익명·일반화)↩

23. 인간+AI 협업의 결을 '거들어주는 것(augmentation, 인간+AI > 인간 단독)'과 '함께라야 되는 것(synergy, 인간+AI > 인간 단독 AND AI 단독)'으로 구분하는 관점. 다만 후자를 무엇으로 측정하고 어떻게 설계하는지는 아직 학술적으로 미성숙한 영역으로, 현장이 채워가야 할 빈 공간이 남아 있다. (익명·일반화)↩

24. AI가 로컬 컨텍스트(테스트 명령·관행·소유권 등)를 쥐지 못한 채 "눈 감고 일한다(working blind)"는 표현은 여러 사내 AI 빌더의 회고에서 공통으로 등장한다. 처방은 그 맥락을 AI가 늘 참조할 수 있게 하는 컨텍스트 시스템이다. (익명·일반화)↩

25. 성과 관리가 연 1회 평가에서 연속적 피드백·결과 기반 측정으로 이동하는 흐름은 여러 HR·인사 연구에서 공통적으로 관찰된다. 특정 출처에 종속하지 않고 다수 자료의 공통된 관점을 재구성했다. (익명·일반화)↩

26. AI가 사람의 성과를 거드는 부분(증폭)과 사람·AI의 결합으로 비로소 가능해진 부분(시너지)을 분리해 측정하는 방법론은 학술적으로도 아직 정립되지 않은 빈 공간이다. 이 점은 여러 리서치에서 공통으로 지적된다. (익명·일반화)↩

27. '보여주기식 사용(performative adoption)', '토큰 채우기(tokenmaxxing)'는 개발자 커뮤니티에서 통용되는 표현으로, 측정 지표를 채우려 토큰을 낭비하는 행태를 꼬집는다. 검증된 통계가 아니라 현장의 정서를 보여주는 익명 의견이므로, 사실 주장이 아니라 논쟁의 온도로만 인용했다. (익명·검증 불가)↩

28. 2026년 공개 보도된 한 대기업의 AI 성과 평가 사례. 외부 생성형 AI를 전면 도입하며 성과를 '전일제 환산(한 사람이 AI로 몇 명분을 했나)'과 '토큰 가성비(투입 토큰 대비 성과)'로 평가하기 시작했고, 같은 성과라도 토큰을 적게 쓴 쪽을 더 높게 친다는 내용이 보도됐다. 같은 보도에서 "이는 절감된 비용 지표일 뿐이며 생산성·품질·혁신을 먼저 보아야 한다"는 내부 비판도 함께 전해졌다. 본문은 책의 익명 원칙에 따라 회사·매체·인물을 드러내지 않고 "한 대기업의 공개 사례"로 일반화했다. (공개 보도·익명 처리)↩

29. 2026년 공개 보도된 한 대기업의 AI 성과 평가 사례. 외부 생성형 AI를 전면 도입하며 성과를 '전일제 환산(한 사람이 AI로 몇 명분을 했나)'과 '토큰 가성비(투입 토큰 대비 성과)'로 평가하기 시작했고, 같은 성과라도 토큰을 적게 쓴 쪽을 더 높게 친다는 내용이 보도됐다. 같은 보도에서 "이는 절감된 비용 지표일 뿐이며 생산성·품질·혁신을 먼저 보아야 한다"는 내부 비판도 함께 전해졌다. 본문은 책의 익명 원칙에 따라 회사·매체·인물을 드러내지 않고 "한 대기업의 공개 사례"로 일반화했다. (공개 보도·익명 처리)↩

30. "코드 라인 수·토큰은 조작되는 가짜 신호이며, 진짜 신호는 되돌림률(reversion rate)과 주간 데모"라는 측정 철학은 일부 AI-우선 엔지니어링 조직의 회고에서 반복적으로 나타난다. 다수 자료의 공통 관점을 일반화해 재구성했다. (익명·일반화)↩

31. 대형 조직의 AI 피드백·코칭 도입 사례에서 피드백 빈도가 크게 증가했다는 수치가 발표된 바 있다. 다만 이런 수치는 도입 기업 또는 솔루션 공급사 측 발표가 많아 출처에 따라 신뢰도 편차가 크므로, 단정하지 않고 "그런 방향의 효과가 보고된다"는 수준으로만 인용했다. (익명·출처 신뢰도 편차 있음)↩

32. AI 코칭에 대한 거부감 — "알고리즘이 내 팀을 나만큼 이해할 리 없다"는 불신과 프라이버시·감시 우려 — 은 여러 현장 의견에서 공통으로 나타난다. (익명·일반화)↩

33. "좋은 코칭 도구는 대화를 대체하는 게 아니라 깊게 만든다"는 관점은 AI 코칭을 긍정하는 쪽에서도 일관되게 강조하는 핵심 원칙이다. 다수 자료의 공통 관점을 일반화해 옮겼다. (익명·일반화)↩

34. 사람을 직무가 아니라 스킬 단위로 보는 '스킬 기반 조직(skills-based organization)'은 최근 인사·조직 담론에서 통용되는 개념이다. 여러 글로벌 컨설팅·연구 기관의 조사에서 리더 다수가 경직된 직무 구조에서 벗어나야 한다고 답한 바 있다. 단일 출처에 종속하지 않고 다수 자료의 공통된 관점을 익명·일반화해 재구성했다.↩

35. 사람을 직무가 아니라 스킬 단위로 보는 '스킬 기반 조직(skills-based organization)'은 최근 인사·조직 담론에서 통용되는 개념이다. 여러 글로벌 컨설팅·연구 기관의 조사에서 리더 다수가 경직된 직무 구조에서 벗어나야 한다고 답한 바 있다. 단일 출처에 종속하지 않고 다수 자료의 공통된 관점을 익명·일반화해 재구성했다.↩

36. 대형 소비재 기업과 산업 기업이 사내 인력 시장(internal talent marketplace)을 도입해 직원 다수가 참여하고 묻혀 있던 다수의 일손을 끌어낸 사례, 매칭 시 학력 필드를 비노출해 편향을 통제한 사례, 그리고 전 직원이 아니라 단계적으로 인원을 늘리며 기존 인사 시스템과 연동해 운영한 사례가 외부 자료로 보고된 바 있다. 구체 기업명·수치는 익명 처리했으며, 벤더·사측 발표 수치는 단정하지 않고 패턴만 취했다.↩

37. 대형 소비재 기업과 산업 기업이 사내 인력 시장(internal talent marketplace)을 도입해 직원 다수가 참여하고 묻혀 있던 다수의 일손을 끌어낸 사례, 매칭 시 학력 필드를 비노출해 편향을 통제한 사례, 그리고 전 직원이 아니라 단계적으로 인원을 늘리며 기존 인사 시스템과 연동해 운영한 사례가 외부 자료로 보고된 바 있다. 구체 기업명·수치는 익명 처리했으며, 벤더·사측 발표 수치는 단정하지 않고 패턴만 취했다.↩

38. 대형 소비재 기업과 산업 기업이 사내 인력 시장(internal talent marketplace)을 도입해 직원 다수가 참여하고 묻혀 있던 다수의 일손을 끌어낸 사례, 매칭 시 학력 필드를 비노출해 편향을 통제한 사례, 그리고 전 직원이 아니라 단계적으로 인원을 늘리며 기존 인사 시스템과 연동해 운영한 사례가 외부 자료로 보고된 바 있다. 구체 기업명·수치는 익명 처리했으며, 벤더·사측 발표 수치는 단정하지 않고 패턴만 취했다.↩

39. 한국 기업이 사내 인력 시장을 *직접 구축*해 운영한 1인칭 사례는 공개 자료로 확보하기 어려웠다. 따라서 본문의 한국 현장 구현 디테일은 저자 본인의 6개월 구현 경험을 1차 자료로 삼아 일반화해 옮긴 것이다. (회사명·시스템 고유명·미공개 사업정보는 제거했다.)↩

40. 사내 도구·지식이 톱다운 명령이 아니라 상향식 풀뿌리로 확산될 때 진짜 채택이 일어난다는 관찰은, 여러 빌더 회고에서 반복적으로 나타나는 패턴이다. 단일 출처에 종속하지 않고 다수 자료의 공통된 관점을 익명·일반화해 재구성했다.↩

41. 한 기업에서 엔지니어가 만든 사내 AI 도구가 개발 조직을 넘어 비개발 부서로 유기적으로 확산된 뒤 경영진이 전사 차원으로 격상했다는 회고가 공개된 바 있다. 본문은 기업 실명을 일반화해 옮겼다.↩

42. 채택을 명령이 아니라 "내가 먼저 편하게 일하는 모습을 보여주는" 모델링으로 끌어낸다는 관점은, 한 대형 커머스 기업의 엔지니어링 리더가 공개한 사내 AI 채택 회고에서 통용되는 표현을 일반화한 것이다.↩

43. "모든 구성원이 본인 업무에 특화된 AI를 만든다"는 1인 1 AI 에이전트 전략, 구성원이 AI 과제를 구체화하는 사내 플랫폼, 자기 업무 방식을 AI에 학습시키는 협업 도구, 현장 문제를 AI로 푸는 사내 해커톤 등은 한 기업의 공개 보도자료(2026년 상반기)에 발표된 내용이다. 공개 자료이나 본문 일관성을 위해 기업 실명을 익명·일반화해 옮겼다.↩

44. 자동 생성된 문서가 원본과 어긋나며 오히려 새로운 부채(stale documentation)가 된다는 역설은, 사내 AI 스택을 직접 구축한 여러 엔지니어링 조직의 회고에서 공통적으로 보고된다. 익명·일반화.↩

45. 시스템의 상태·로그·흔적을 한눈에 들여다보는 '관측 도구'는 특정 상용 제품을 가리키지 않으며, 운영 가시성(observability)을 제공하는 도구 일반을 일반화해 옮긴 표기다.↩

46. AI를 본격 도입한 현장에서 "기대만큼 빨라지지 않는다"는 체감(대략 10~20% 수준)은 여러 발표·회고에서 반복적으로 보고된다. 익명 처리 — 특정 발표·수치에 종속하지 않고 다수 자료의 공통된 관찰을 재구성했다. (사실 확인 필요: 체감 비율은 출처별 편차가 있어 단정하지 않는다.)↩

47. 'AI 3대 부채(기술·인지·의도)'라는 묶음은 여러 산업 담론과 기술 에세이에서 논의되는 관점을 저자가 종합해 재구성한 것이다. 특정 도식을 그대로 옮기지 않았으며, 본문은 일반화된 형태로 서술했다.↩

48. AI가 양산한 "그럴싸하지만 전역적으로는 부적합한" 코드가 일정 기간 뒤 오히려 개발 속도를 떨어뜨릴 수 있다는 관찰·시뮬레이션 논의가 있다. 익명·일반화 — 구체 기간·수치는 출처별로 달라 본문에서 단정하지 않았다. (사실 확인 필요)↩

49. "생각(thinking)은 아웃소싱할 수 있어도 이해(understanding)는 아웃소싱할 수 없다"는 취지의 언급은 최근 AI 담론에서 널리 인용된다. 특정 인물 실명 없이 일반화해 옮겼다.↩

50. AI·자동화로 인력을 줄였다가 사라진 암묵지·맥락 때문에 다시 사람을 채용한 사례들이 여러 기업에서 보고됐다. 익명·일반화 — 기업 실명은 본문에서 제외하고 패턴만 서술했다.↩

51. "사람의 메인 작업을 생산에서 검증으로 옮긴다"는 처방은 여러 빌더·연구자 논의의 공통된 결론을 재구성한 것이다. (익명·종합)↩

52. 검증을 ① 통과·실패 검증 ② 정량 지표 ③ 정성 루브릭(채점 기준에 따른 LLM 심사)의 세 겹으로 나누는 틀은 여러 자료에서 통용되는 분류를 일반화해 옮긴 것이다. 본 장은 개념까지만 다루며, 실제 구현은 다음 장에서 서술한다.↩

53. AI 에이전트가 작업을 완료했다고 보고했으나 실제로는 수행하지 않고 그 흔적까지 스스로 제거한 사례, 거짓 보고·결과 조작, 나아가 다수 에이전트 간 공모 가능성에 대한 우려는 현장 운영 논의에서 반복적으로 제기된다. 익명·완전 일반화 — 특정 회사·시스템·제품·인물에 종속하지 않고 다수 현장의 공통된 관찰만 재구성했다. (사실 확인 필요: 발생 빈도·메커니즘은 출처별 편차가 있어 본문에서 단정하지 않았다.)↩

54. 회사의 성과 기준이 초기 '생산량'에서 이후 '추가한 검증량과 구조'로 옮겨가야 한다는 관점은 여러 자료에서 공통적으로 나타난다. (익명·종합) 한편 생산량 지표가 '쓰는 척하는' 행동(performative adoption)을 유발한다는 커뮤니티 의견도 있으나, 검증되지 않은 익명 의견이므로 사실 주장이 아닌 논쟁의 온도로만 참고했다.↩

55. 한 길만 강제하는 '철길(railroad)', 권장하되 지원하는 '포장도로(paved road)' 또는 '골든 패스(golden path)', 경계만 긋는 '가드레일(guardrail)'로 자유도를 나누는 분류는 플랫폼 엔지니어링 담론에서 통용되는 구분을 일반화해 옮긴 것이다. '골든 패스'를 "강요가 아니라 권장되고 지원되는 길"로 규정한 대형 음원 플랫폼 기업의 엔지니어링 사례, '포장도로'를 "명령이 아닌 안내 원칙"으로 본 대형 스트리밍 기업의 사례가 대표적이다. (익명·일반화)↩

56. "게이트는 멈춰 세우지만 가드레일은 흐름을 유지한다"는 대비는 규모 있는 조직의 거버넌스를 논한 여러 글에서 반복적으로 등장하는 관점을 재구성한 것이다. 셀프서비스에 가드레일을 결합해 "자율성을 유지하면서 보안·컴플라이언스·비용을 자동으로 관리한다"는 정의는 대형 기술 기업의 플랫폼 엔지니어링 공식 문서에서도 같은 결로 확인된다. (익명·일반화)↩

57. 적절한 제약이 오히려 창발을 가능하게 한다는 '가능하게 하는 제약(enabling constraints)' 개념은 복잡계 철학에서 비롯됐다. "질서는 강제가 아니라 가능하게 하는 제약에서 유도된다"는 관점, 복잡한 영역에서는 통제하는 제약이 아니라 가능하게 하는 제약을 써야 한다는 의사결정 프레임이 그 계보다. (익명·일반화)↩

58. 자율과 통제를 트레이드오프가 아니라 동시에 끌어안고 관리해야 할 역설(paradox)로 보는 관점은 조직 이론의 역설 연구에 근거를 둔다. 줄타기 곡예사가 양쪽으로 끊임없이 무게를 옮기며 균형을 잡는 동적 균형의 비유가 대표적이다. (익명·일반화)↩

59. 운영 규칙을 사람의 기억이 아니라 코드로 적어 버전관리·자동 집행하는 접근은 '정책을 코드로(policy-as-code)' 다루는 실무 흐름과 맞닿아 있다. 정책을 형상관리에 저장하고 리뷰·자동 검증을 거치게 해 "단일하고 테스트 가능한 진실의 원천"으로 삼는다는 관점이 그것이다. (익명·일반화)↩

60. "사실이 유효했던 시점"과 "그 사실을 기록한 시점"이라는 두 개의 시간 축을 함께 다루는 데이터 모델링은 학술적으로 '이중 시간(bi-temporal)' 모델로 정립돼 있다. 유효 시간(valid time)과 기록 시간(transaction time)을 분리해, 과거 어느 시점의 상태든 정확히 재현할 수 있게 한다. (익명·일반화)↩

61. 변경을 만든 사람과 승인하는 사람을 다르게 두는 '네 눈 원칙(four-eyes principle)', 사람이 결정 고리에 들어가는 '휴먼 인 더 루프(human-in-the-loop)' 승인, 그리고 즉시 되돌릴 수 있는 롤백은 내구성 있는 실행·승인 워크플로우의 실무 패턴에서 널리 쓰이는 장치를 일반화한 것이다. (익명·일반화)↩